

Vorlesung Datenbanken

Prof. Dr. Dietmar Seipel
Universität Würzburg

Wintersemester 2025/26
October 13, 2025

Inhaltsverzeichnis

1	Grundkonzepte von Datenbanken	1
1.1	Der Datenbankansatz	1
1.1.1	Beispiel: die UNIVERSITY–Datenbank	4
1.1.2	Charakteristika	14
1.2	Das Entity–Relationship–Modell	24
1.2.1	Grundbegriffe	24
1.2.2	Erweiterungen des ER–Modells	46
1.2.3	Beispiel: die COMPANY–Datenbank	59

2	Das Relationenmodell: Sprachen und Systeme	69
2.1	Grundlegende Definitionen	69
2.2	Relationale Algebra	86
2.3	Die relationale Datenbanksprache SQL	101
2.3.1	Datendefinition in SQL	102
2.3.2	Anfragen in SQL	120
2.3.3	Update–Statements in SQL	178
2.3.4	Views (Sichten) in SQL	184
2.3.5	Bedingungen, Trigger und Indexe	193
2.3.6	Embedded SQL	196

3	Datenbankentwurf im Relationenmodell	200
3.1	Informelle Design-Richtlinien und Update-Anomalien	201
3.2	Funktionale Abhängigkeiten	211
3.3	Normalformen	264
3.4	Mehrwertige und Verbund-Abhängigkeiten	⟨ 280 ⟩
3.5	Verlustfreie Zerlegungen	309
3.5.1	BCNF-Dekomposition	320
3.5.2	3NF-Synthese	333

4	Transaktionen und Recovery	373
4.1	Transaktionsverwaltung	375
4.1.1	Das Synchronisationsproblem	376
4.1.2	Operationen, Log–Buch, ACID–Prinzip	385
4.1.3	Transaktionen, Schedules und Serialisierbarkeit	406
4.2	Concurrency Control	462
4.2.1	Das Zwei–Phasen–Sperrprotokoll	467
4.2.2	Deadlocks	479
4.2.3	Zeitstempelverfahren	488
4.2.4	Mehrversionen–Scheduling–Verfahren	496
4.3	Recovery	501
	Anhang	504

Überblick

- Gegenwärtig werden die Datenbestände in größeren Informationssystemen meist in relationalen Datenbanken verwaltet. Verbreitete Systeme sind z.B. DB2 (von IBM), Oracle und die freien Systeme MySQL und POSTGRESQL.
- Die strukturelle Modellierung der Anwendungsdaten erfolgt meist im Entity–Relationship–Modell (ER–Modell), welches sodann ins relationale Modell abgebildet wird. Mit Hilfe von Normalformen wird die Struktur der Relationen bewertet und weiter verbessert.
- Der Datenzugriff im relationalen Modell erfolgt mittels der Structured Query Language (SQL).

Grundlagen

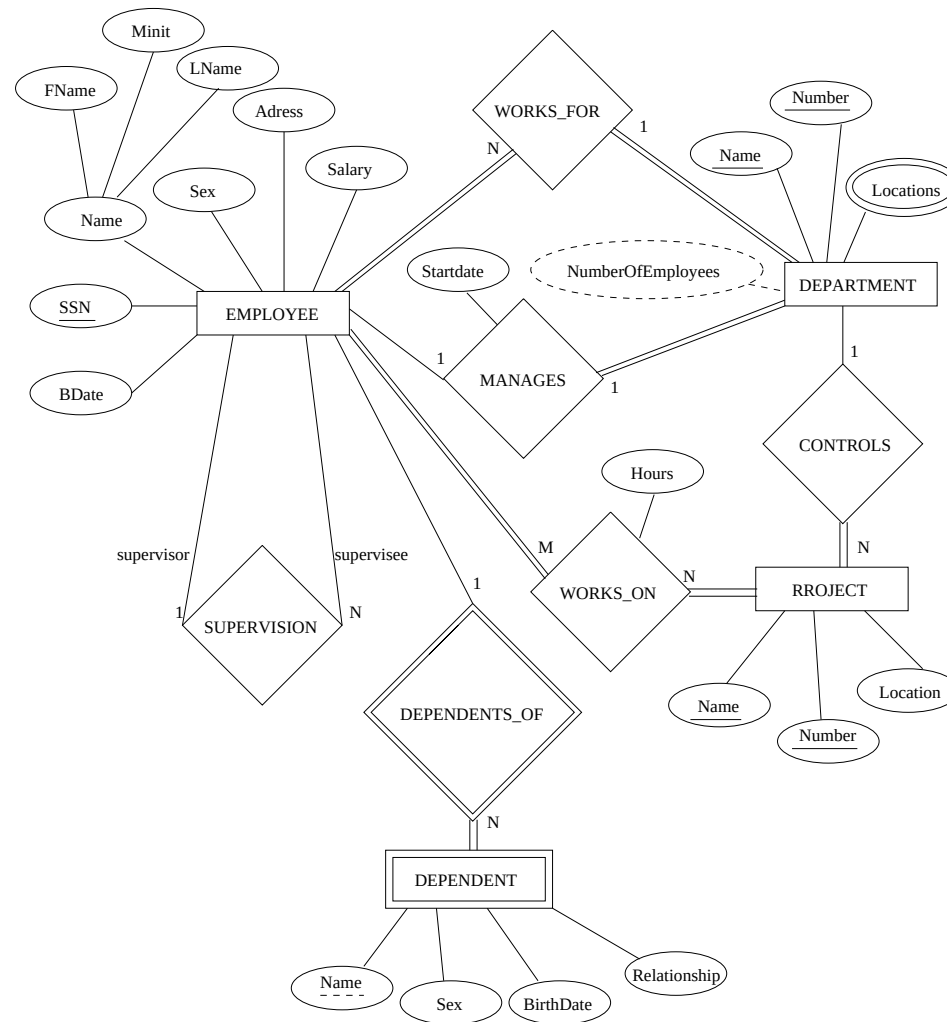
- Eine Datenbank integriert die Daten verschiedener Benutzer (Datenintegration). Das Datenbankmanagementsystem (DBMS) stellt Mechanismen zur Verfügung, die die Eingabe inkorrektur Daten verhindern (Datenkonsistenz).



- Jeder Benutzer kann sich mittels des Systemkatalogs über die Struktur der Datenbank informieren. Der Zugriff auf den Systemkatalog erfolgt genau wie der eigentliche Datenzugriff auf der sehr abstrakten Ebene von Relationen. Die tatsächliche, interne Repräsentation der Daten im DBMS wird verborgen (Datenabstraktion und Datenunabhängigkeit).
- Das DBMS stellt sicher, daß die Konsistenz der Daten beim parallelen Zugriff verschiedener Benutzer erhalten bleibt (Transaktionsverwaltung), und daß bei Systemabstürzen keine Daten verloren gehen (Recovery, Datensicherheit). Außerdem wird über Zugriffsrechte der Datenschutz gewährleistet.
- Die Verwendung von Datenbanken ermöglicht die Datenintegration, und sie verkürzt die Entwicklungszeiten für größere Informationssysteme deutlich.

Datenmodellierung im ER–Modell

- Die erste Modellierung erfolgt im Entity–Relationship–Modell (ER–Modell), welches gleichartige Objekte der realen Welt als Entity–Typen modelliert und die Beziehungen dazwischen als Relationship–Typen. Die Entities und Relationships werden durch Attribute beschrieben.
- Man verwendet hauptsächlich die graphische Veranschaulichung in Form von ER–Diagrammen. Ähnliche Konzepte werden mittlerweile auch bei der objektorientierten Modellierung auf der Basis der Unified Modelling Language (UML) verwendet.
- Es gibt eine generische Abbildung vom ER–Modell ins Relationenmodell, bei der sowohl die Entity–Typen als auch die Relationship–Typen als Relationenschemata zusammen mit Schlüssel– bzw. Fremdschlüsselbedingungen repräsentiert werden.



Datenbankzugriff im Relationenmodell

- Der Zugriff auf die Daten erfolgt über die Structured Query Language (SQL). Die Abarbeitung der SQL-Anfragen erfolgt intern über Operationen auf den Relationen (relationale Algebra).
- Es gibt SQL-Befehle für das Erzeugen und Löschen (**CREATE**, **DROP**) von Datenbanken bzw. Tabellen (Relationen), für das Einfügen und Verändern der Tabellen (**INSERT**, **DELETE**, **UPDATE**) und für den eigentlichen Datenzugriff (**SELECT**).
Beim **SELECT**-Statement kann man verschiedene Tabellen verknüpfen (Join), und man kann Gruppen bilden und über diese aggregieren.
- SQL kann aus Programmiersprachen wie **Java** heraus aufgerufen werden (Embedded SQL, Java Database Connectivity, JDBC).

Die Tabelle (Relation) EMPLOYEE der Datenbank COMPANY enthält Spalten (Attribute) zu den Angestellten (FNAME, . . . , SALARY) einschließlich ihrer Vorgesetzten (SUPERSSN) und Abteilungsnummern (DNO):

EMPLOYEE									
FNAME	MINIT	LNAME	SSN	BDATE	ADDRESS	SEX	SALARY	SUPERSSN	DNO
John	B	Smith	444444444	1955-01-09	731 Fondren, Houston, TX	M	30000	222222222	5
Franklin	T	Wong	222222222	1945-12-08	638 Voss, Houston, TX	M	40000	111111111	5
Alicia	J	Zelaya	777777777	1958-07-19	3321 Castle, Spring, TX	F	25000	333333333	4
Jennifer	S	Wallace	333333333	1931-06-20	291 Berry, Bellaire, TX	F	43000	111111111	4
Ramesh	K	Narayan	555555555	1952-09-15	975 Fire Oak, Humble, TX	M	38000	222222222	5
Joyce	A	English	666666666	1962-07-31	5631 Rice, Houston, TX	F	25000	222222222	5
Ahmad	V	Jabbar	888888888	1959-03-29	980 Dallas, Houston, TX	M	25000	333333333	4
James	E	Borg	111111111	1927-11-10	450 Stone, Houston, TX	M	55000	NULL	1

In SQL kann man Anfragen stellen, die verschiedene Tabellen (Relationen) der Datenbank geeignet verknüpfen.

Datenmodellierung im Relationenmodell

- Ein Relationenschema besteht aus einer Menge von Attributen und einer Menge von strukturellen Bedingungen (Datenabhängigkeiten). Eine Relation (Tabelle) ist eine Menge von Tupeln (Zeilen). Jede Zeile besitzt einen Eintrag pro Attribut (Spalte).
- Bei der Datenmodellierung gibt es informelle Design-Richtlinien, welche die Übersichtlichkeit des Entwurfs, die Vermeidung von Redundanz, Update-Anomalien und Nullwerten, sowie die korrekte Verknüpfung von zerlegten Tabellen betreffen.
- Als formales Konzept werden Normalformen verwendet. Der Entwurf soll möglichst ein Datenbankschema in dritter Normalform (3NF) liefern, eventuell sogar in Boyce-Codd-Normalform (BCNF), da diese Normalformen die informellen Design-Richtlinien sehr gut umsetzen.

- Die Normalformen basieren auf dem Konzept der funktionalen Abhängigkeiten. Die einfachste funktionale Abhängigkeit ist die Schlüsselabhängigkeit; sie besteht darin, daß ein Teil der Attribute, der dann Schlüssel genannt wird, bereits ausreicht um ein Tupel eindeutig zu identifizieren.
- Auf der Basis der bekannten funktionalen Abhängigkeiten kann eine Relation in mehrere Relationen in 3NF bzw. BCNF zerlegt werden, so daß sie aus diesen verlustfrei mit Hilfe von Joins (meist über Fremdschlüssel) rekonstruiert werden kann.
- Zur Zelegung in 3NF bzw. BCNF verwendet man den Synthese– bzw. den Dekompositions–Algorithmus.

Transaktionen und Recovery

Im Straßenverkehr regeln zur Synchronisation des Mehrbenutzerbetriebs Ampeln – und andere Verkehrszeichen – welches Fahrzeug gerade einen Straßenabschnitt befahren darf:



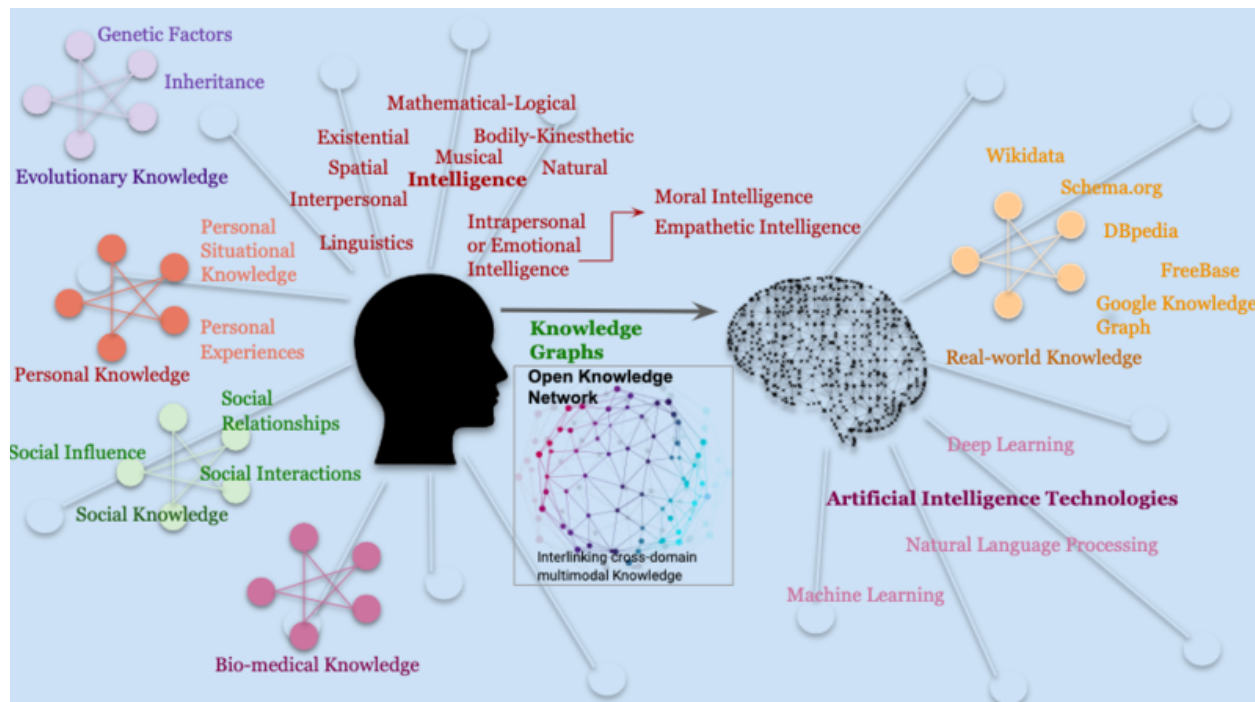
Datenbanksysteme verwenden Sperrprotokolle und Zeitstempelverfahren.

- Für den Mehrbenutzerbetrieb ist es erforderlich, daß die Transaktionen nach einem genau definierten Protokoll auf die Datenbank zugreifen.
- Durch das ACID–Prinzip wird garantiert, daß Transaktionen beim parallelen Datenbankzugriff einen konsistenten Datenbankzustand hinterlassen.
- Das *Zwei–Phasen–Sperrprotokoll* sperrt die von einer Transaktion benötigten Datenbankobjekte geeignet für andere Transaktionen.
- Die *Zeitstempelverfahren* markieren die Datenbankobjekte mit Zeitstempeln, so daß eine Transaktion – im wesentlichen – nur Datenbankobjekte bearbeiten darf, die von älteren Transaktionen markiert wurden.

Weiterführende Themengebiete

- XML–Datenbanken, Semantic Web (RDF, OWL und Ontologien)
- Nicht–Standard–Datenbanken:
deduktive, objektorientierte und aktive Datenbanken
- weitere Datenbankansprachen: DATALOG, SPARQL
- optimierte Auswertung von Datenbankfragen:
Indexstrukturen, Join–Optimierung
- NoSQL–Datenbanken (not only SQL)
- frühere Datenbankstrukturen:
hierarchische Datenbanken und Netzwerk–Datenbanken

Komplexeres Wissen erfordert neue sowie ausdrucks- und leistungsstarke Konzepte für Daten- und Wissensbanken.



Wir untersuchen das Management und die Gewinnung solchen Wissens in den Vorlesungen Advanced Databases und Deduktive Datenbanken.

Daten– und Wissensbanken

Es gibt neuere *Datenmodelle* wie XML, RDF und OWL für komplexe, semistrukturierte Daten und das Semantic Web sowie die dazugehörigen *Anfragesprachen*.

Deduktive Datenbanken sind eine Erweiterung relationaler Datenbanken: sie erlauben zusätzlich regelbasiertes Expertenwissen; auch Anfragen können so flexibler formuliert werden.

Das *Knowledge Discovery in Databases* bietet eine Reihe von Methoden, abstrakteres Regelwissen (Muster) aus Datensammlungen zu gewinnen.

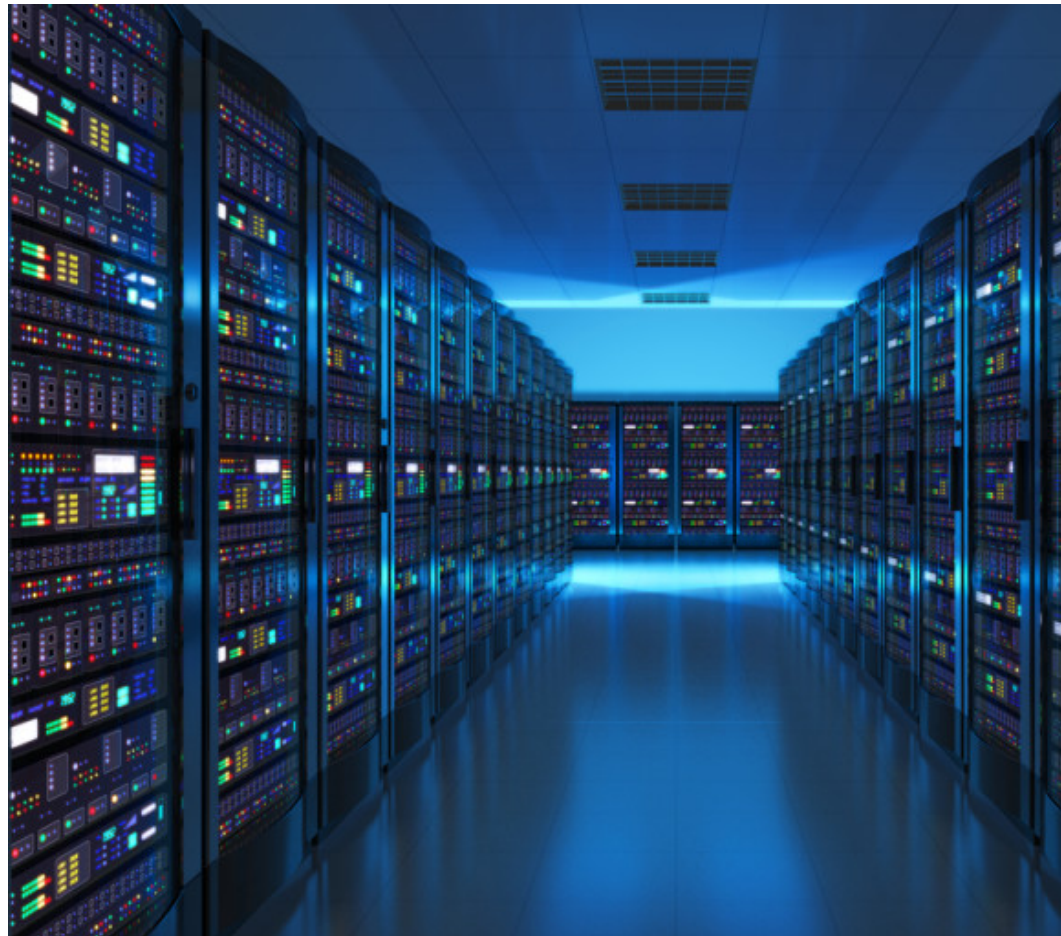
Das Motto ist: wir *ertrinken* in großen Datensammlungen, aber wir *hungern* nach abstrakterem Wissen.

Advanced Databases

Anschlußvorlesung im Sommersemester, Skript in Englisch

Themen

- Knowledge Discovery in Databases, Data Warehouses and Data Mining
- XML Data Modelling
- Deductive and Semantic Web Databases
- Database Access from Programming Languages



1 Grundkonzepte von Datenbanken

1.1 Der Datenbankansatz

Eine *Datenbank* repräsentiert einen Ausschnitt der realen Welt, eine sogenannte “Mini–Welt” (Universe of Discourse), und deren Veränderungen.

Beispiel (UNIVERSITY Datenbank)

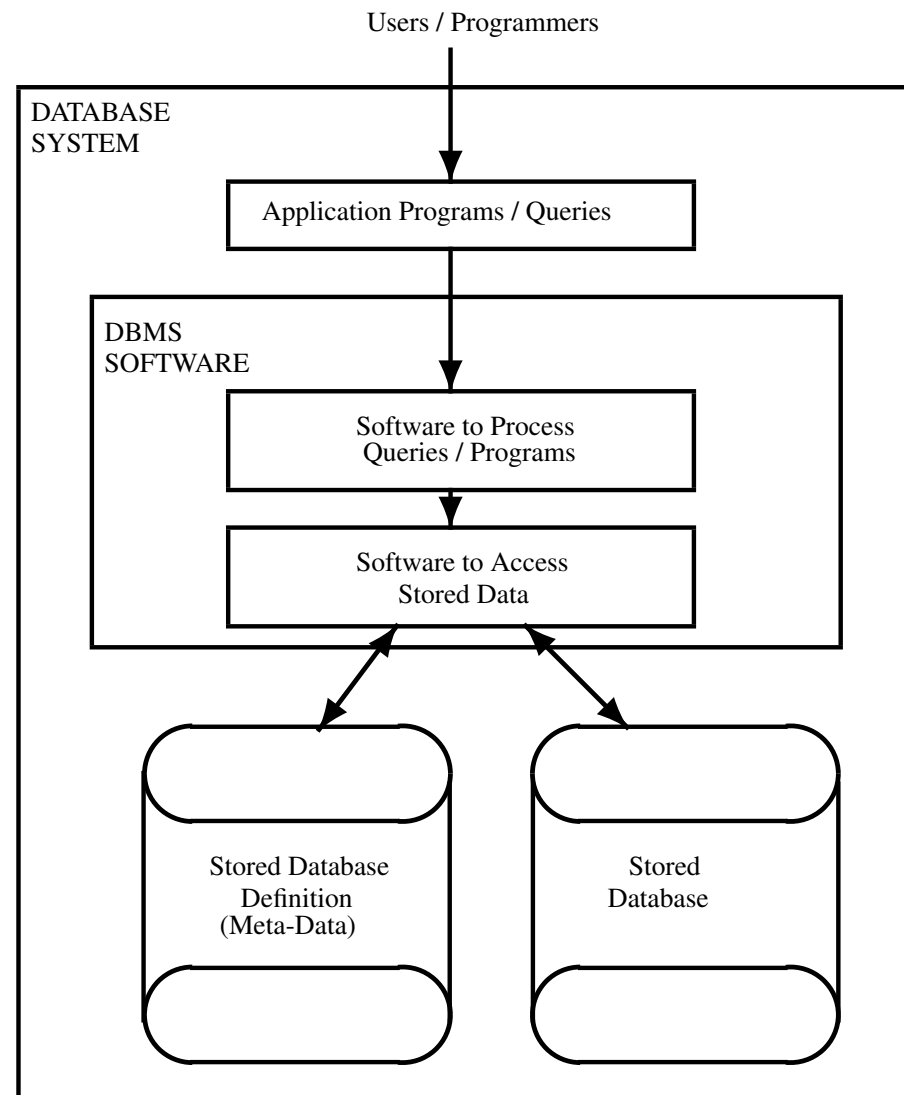
Die Studentenkartei einer Universität enthält Datensätze zu Studierenden, welche unterschiedliche Gesichtspunkte (z.B. Name oder Geburtsdatum) beschreiben.

Die Datenbankbenutzer können die Daten *durchsuchen* (search) / *recherchieren* (retrieve) und *verändern* (update).

Ein *Datenbank–Management–System* (DBMS) ist ein universelles Softwarepaket zur Definition, Konstruktion und Manipulation von Datenbanken für unterschiedliche Anwendungen.

- *Definition* einer Datenbank: Spezifikation der Datentypen, der Datenstrukturen und der Bedingungen (Constraints) an die Daten.
- *Konstruktion* einer Datenbank: Speicherung der Daten auf einem vom DBMS kontrollierten Speichermedium.
- *Manipulation* einer Datenbank: Anfragen an die Datenbank um spezielle Daten zu erhalten, Aktualisierung der Daten bei Änderungen in der Mini–Welt, Generierung von Reports über die Daten.

Die Datenbank und das DBMS formen ein *Datenbanksystem*.



A simplified database system environment.

1.1.1 Beispiel: die UNIVERSITY-Datenbank

Die Datenbank besteht aus 5 Tabellen (Relationen), welche jeweils Datensätze (Zeilen, Tupel) desselben Typs enthalten.

Die Relation STUDENT speichert zu jedem Studierenden den Namen, die Matrikelnummer, das Fachsemester und das Hauptfach.

STUDENT			
Name	Mat.Nr.	Class	Major
Smith	17	1	COSC
Brown	8	2	COSC

Die Relation COURSE speichert zu jedem Kurs den Namen, die Nummer (im Vorlesungsverzeichnis), die Kreditpunkte und das verantwortliche Department.

COURSE			
CourseName	CourseNumber	CreditHours	Department
Intro to Computer Science	COSC1310	4	COSC
Data Structures	COSC3320	4	COSC
Discrete Mathematics	MATH2410	3	MATH
Databases	COSC3380	3	COSC

Da ein Kurs in verschiedenen Semestern von unterschiedlichen Dozenten gehalten werden kann, wird für jedes Semester ein entsprechender Datensatz – mit einem eindeutigen Identifikator – in SECTION gespeichert.

SECTION				
SectionIdentifier	CourseNumber	Semester	Year	Instructor
85	MATH2410	Fall	2005	King
92	COSC1310	Fall	2005	Anderson
102	COSC3320	Spring	2006	Knuth
112	MATH2410	Fall	2006	Chang
119	COSC1310	Fall	2006	Anderson
135	COSC3380	Fall	2006	Stone

Für jeden Studierenden werden seine besuchten Veranstaltungen mit Noten gespeichert. Dabei werden zur Bezeichnung der Studierenden und Veranstaltungen Identifikatoren verwendet, die sich auf die entsprechenden Relationen STUDENT und SECTION beziehen.

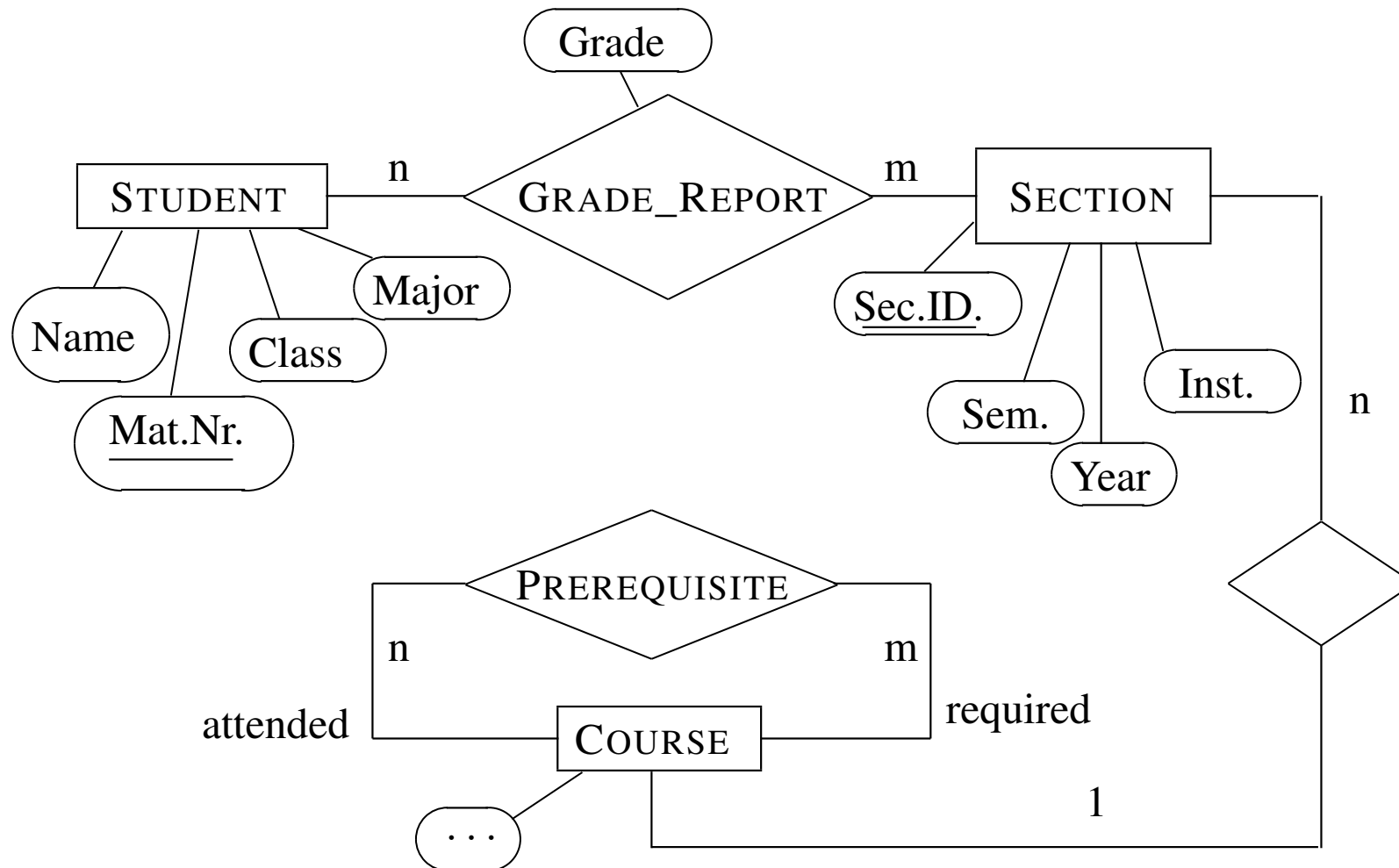
GRADE_REPORT		
Mat.Nr.	SectionIdentifier	Grade
17	112	B
17	119	C
8	85	A
8	92	A
8	102	B
8	135	A

Die Relation PREREQUISITE enthält die Voraussetzungen der Kurse.

PREREQUISITE	
CourseNumber	PrerequisiteNumber
COSC3380	COSC3320
COSC3380	MATH2410
COSC3320	COSC1310

Für jeden Kurs gibt es so viele Tupel wie er Voraussetzungen hat.

In Abschnitt 1.2 werden wir sehen, wie man das Schema der UNIVERSITY–Datenbank durch ein *ER–Diagramm* beschreiben kann:



Die Relationen STUDENT, COURSE, SECTION, GRADE_REPORT und PREREQUISITE ergeben sich aus dem ER-Diagramm:

- STUDENT entspricht dem Entity-Typ STUDENT.
- GRADE_REPORT ergibt sich aus dem gleichnamigen Relationship-Typ, indem man die Schlüssel Mat.Nr. und SectionIdentifier der beteiligten Entity-Typen mit dem Attribut Grade kombiniert.
- Die Attribute CourseName, CourseNumber, CreditHours und Department von COURSE wurden im Bild durch ” . . . ” angezeigt.
- Das Attribut CourseNumber der Relation SECTION ergibt sich aus der Beziehung zu COURSE, indem dessen Schlüssel aufgenommen wird.
- Der rekursive Relationship-Typ PREREQUISITE wird zu einer Relation zwischen zwei Kursen.

Anfragen:

- Gib eine Aufstellung, d.h. eine Liste der Kurse und Noten (Grades), von Smith aus.
- Liste die Namen von Studenten und Noten, welche den Datenbanken–Kurs im Herbst 2006 besucht haben, auf.
- Was sind die Voraussetzungen für den Datenbanken–Kurs?

Updates:

- Ändere die Klasse von Smith auf 2.
- Gib die Note A ein für den Studenten Smith im Datenbanken–Kurs des letzten Semesters.

Aus den vorliegenden Relationen kann man Reports (Sichten) generieren:

TRANSCRIPT					
StudentName	StudentTranscript				
	CourseNumber	Grade	Semester	Year	SectionId
Smith	COSC1310	C	Fall	2006	119
	MATH2410	B	Fall	2006	112
Brown	MATH2410	A	Fall	2005	85
	COSC1310	A	Fall	2005	92
	COSC3320	B	Spring	2006	102
	COSC3380	A	Fall	2006	135

Dabei können z.B. die besuchten Veranstaltungen mit Noten nach Studierenden gruppiert werden (siehe oben),
oder es können die Voraussetzungen der Kurse nach Kursen gruppiert werden (siehe unten).

PREREQUISITES			
CourseName	CourseNumber	PrereqName	PrereqNumber
Databases	COSC3380	Data Structures	COSC3320
		Discrete Math.	MATH2410
Data Structures	COSC3320	Intro to CS	COSC1310

Solche Reports sind wegen der verschachtelten Attributwerte nicht in erster Normalform (1NF). Man nennt sie NF^2 –Relationen (non–first–normal–form).

1.1.2 Charakteristika

Datenintegration und Datenkonsistenz

Integration und konsistente Verwaltung der Daten der unterschiedlichen Benutzergruppen; kontrollierte Redundanz, da sonst Speicherplatzverschwendung und unnötiger Aufwand bei der Wartung des Datenbestandes und eventuelle Inkonsistenzen in den Daten.

Systemkatalog (Data Dictionary)

enthält *Meta-Daten*: Informationen zur Datenstruktur, den Typen und Speicherformaten der einzelnen Datenelemente, sowie Integritätsbedingungen an die Daten (*selbstbeschreibende Natur* eines DBS); die Integritätsbedingungen werden vom Datenbank-Designer definiert und vom DBMS automatisch überprüft; bei traditioneller File-Verwaltung ist die Datendefinition ein Bestandteil der Anwendungsprogramme.

Datenunabhängigkeit und Datenabstraktion

Datenmodelle verbergen Details, die (auf höheren Ebenen) nicht von Interesse sind (3–Ebenen–Architektur);

Unterstützung verschiedener *Sichten* (*Views*) auf die Daten:

eine Sicht ist eine Teilmenge der Datenbank oder sie besteht aus virtuellen Daten, welche aus der Datenbank abgeleitet werden, aber nicht explizit in der Datenbank gespeichert sind.

Datenteilung und Transaktionsverwaltung

Daten von verschiedenen Anwendungen und Benutzern werden in einer gemeinsamen Datenbank integriert.

Concurrency Control–Mechanismen stellen sicher, daß mehrere (eventuell parallele) Datenmanipulationen durch unterschiedliche Benutzer kontrolliert und korrekt abgearbeitet werden können.

Datensicherheit

Backup und Recovery (falls das Computersystem während einer komplexen Transaktion abstürzt)

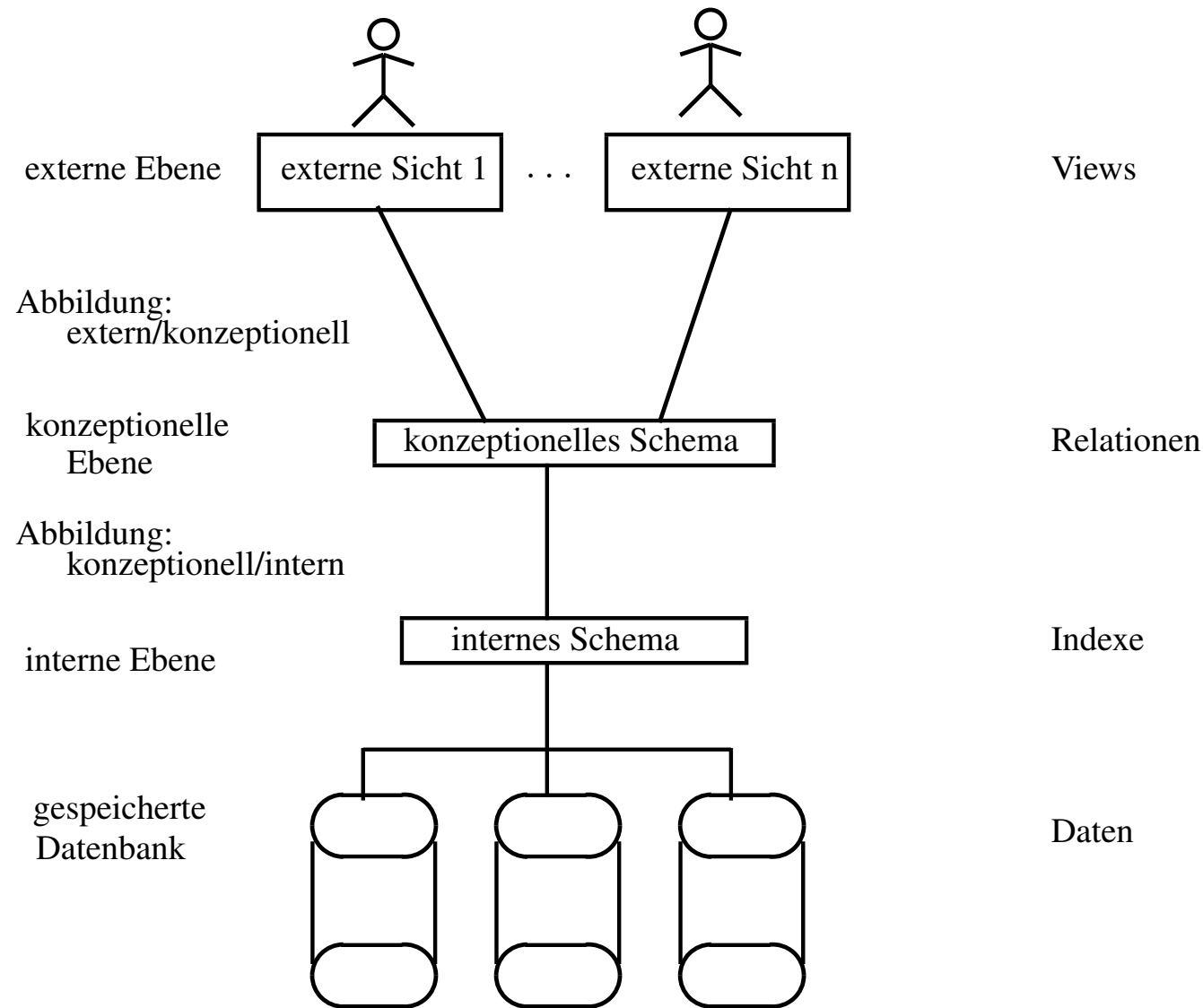
Datenschutz

Zugriffskontrolle über Zugriffsrechte

Datenpersistenz

dauerhafte Speicherung von Daten und Programmen als Erweiterung herkömmlicher Programmiersprachen

Die Drei-Ebenen-Architektur:



Datenbank–Benutzer

Datenbankadministrator (DBA)

- vergibt Zugriffsrechte auf die Datenbank
- koordiniert und überwacht die Benutzung
- beschafft weitere Soft– und Hardware
- ist verantwortlich für die Systemperformance

Datenbank–Designer

- identifiziert die zu speichernden Daten
- spricht mit allen voraussichtlichen Benutzern
- integriert die Sichten der einzelnen Benutzergruppen

Endbenutzer

- erstellt Anfragen, Reports und führt Updates aus

Systemanalytiker und Anwendungsprogrammierer

- Systemanalytiker ermitteln die Anforderungen von Endbenutzern und entwickeln Spezifikationen
- Anwendungsprogrammierer implementieren diese Spezifikationen

unterschiedlichste Benutzerschnittstellen

- Anfragesprachen für gelegentliche Benutzer
- Programmiersprachen–Interfaces für Anwendungsprogrammierer
- Formulare und Kommandocodes für parametrische Benutzer
- menügesteuerte oder natürlichsprachliche Schnittstellen für Einzelbenutzer

Arbeiter hinter den Kulissen

Datenbank–Programmierer

implementieren Module für Systemkatalog, Anfragesprache, Schnittstellenverwaltung, Datenzugriff, Datensicherheit

Tool–Entwickler

entwickeln Softwarepakete für Datenbankentwurf, Performance–Überwachung, natürlichsprachliche oder graphische Schnittstellen, Prototyping, Simulation, Testdatengenerierung

Operateure und Betriebspersonal

verantwortlich für Hardware– und Software–Umgebung des Datenbanksystems

Vorteile und Nachteile des Datenbankansatzes

Vorteile

- Möglichkeit zur Bildung von *Standards* in zentralisierter Datenbankumgebung; dies erleichtert die Kommunikation und Kooperation zwischen verschiedenen Abteilungen, Projekten und Benutzern einer großen Organisation.
- *verkürzte Entwicklungszeiten* für Anwendungen: 15–25% der Entwicklungszeit verglichen mit der Verwendung von Filesystemen
- *Flexibilität*: DBMS erlaubt Veränderungen in der Struktur der Datenbank, ohne die gespeicherten Daten und die existierenden Anwendungsprogramme zu betreffen
- Einsparungen durch zentrale Aktivitäten zur Datenverarbeitung

Nachteile

- *Overhead-Kosten:*
Anfangsinvestition in Hardware, Software und Training;
Allgemeinheit eines DBMS zur Definition und Verarbeitung von Daten;
Kosten für Datensicherheit, Concurrency Control, Recovery,
Integritätsüberprüfung

File-Verwaltung

- ist möglicherweise besser, falls eine der folgenden Situationen vorliegt:
 - die Anwendungen sind einfach, wohldefiniert und nicht dynamisch,
 - es gibt strikte Echtzeit-Anforderungen,
 - es ist keine Mehrbenutzerfähigkeit erforderlich.

1.2 Das Entity–Relationship–Modell

Das Entity–Relationship–Modell (P.P. Chen, 1976) ist ein universelles Datenmodell für die Informationsanalyse und den Entwurf des konzeptionellen Schemas.

1.2.1 Grundbegriffe

Eine *Entity* ist ein in der realen Welt existierendes und eindeutig identifizierbares Objekt (z.B. Ding oder Ereignis).

Entities werden durch Eigenschaften (Attribute) repräsentiert.

Entities gleicher Art werden durch Entity–Typen beschrieben und zu Entity–Mengen zusammengefaßt.

Definition (Entities)

Sei U eine universelle Menge von Attributen, und sei $dom(A)$ der Wertebereich für $A \in U$.

1. Ein Paar $E = (V, K)$, mit $K \subseteq V \subseteq U$, ist ein *Entity-Typ* über der Attributmenge V und der Menge K von Schlüsselattributen.
2. Eine Entity e vom Typ $E = (V, K)$ ist eine Abbildung, welche jedem Attribut $A \in V$ einen Wert $e(A) \in dom(A)$ zuordnet.
Die Projektion $e[X]$ von e auf $X \subseteq V$ ist die Abbildung mit dem eingeschränkten Definitionsbereich X .
3. Eine *Entity-Menge* (Instanz) $I(E)$ vom Typ $E = (V, K)$ ist eine Menge von Entities vom Typ E , die die Schlüsselbedingung erfüllt:
zwei unterschiedliche Entities $e, e' \in I(E)$ können nicht auf K übereinstimmen ($\forall e, e' \in I(E) : e[K] = e'[K] \implies e = e'$).

Beispiel (Entity–Typen, Entities)

STUDENT = ({Name, Mat.Nr., Class, Major}, {Mat.Nr.})

COURSE = ({CourseName, C.Nr., CreditHours, Dept.}, {C.Nr.})

SECTION = ({Sec.ID., Semester, Year, Instructor}, {Sec.ID.})

Bei der obigen Attribut–Reihenfolge entspricht die folgende Entity *st* vom Typ STUDENT dem Tupel (Smith, 17, 1, COSC):

$st(\text{Name}) = \text{Smith}, \quad st(\text{Mat.Nr.}) = 17,$
 $st(\text{Class}) = 1, \quad st(\text{Major}) = \text{COSC}.$

Name	Mat.Nr.	Class	Major
Smith	17	1	COSC

Die Projektion $st[\{\text{Name}, \text{Mat.Nr.}\}]$ entspricht dem eingeschränkten Tupel (Smith, 17).

Durch die Definition von Entities als Abbildungen wird die Zuordnung von Attributen und Werten unabhängig von einer Attribut–Reihenfolge.

Entity–Menge $I(\text{STUDENT})$ vom Typ STUDENT:

Name	Mat.Nr.	Class	Major
Smith	17	1	COSC
Brown	8	2	COSC

Entity se vom Typ SECTION:

SectionIdentifier	Semester	Year	Instructor
112	Fall	2006	Chang

Entity co vom Typ COURSE:

CourseName	CourseNumber	CreditHours	Department
Discrete Mathematics	MATH2410	3	MATH

Eine *Relationship* repräsentiert eine Beziehung zwischen Entities zweier oder mehrerer Entity–Typen; sie kann ebenfalls Eigenschaften haben.
Relationship–Typen beschreiben Relationships gleicher Art,
Relationship–Mengen sind Ausprägungen von Relationship–Typen.

Definition (Relationships)

Seien E_i , $1 \leq i \leq n$, Entity–Typen mit Entity–Mengen $I(E_i)$.

1. $R = (E_1, \dots, E_n, V)$ heißt *Relationship–Typ* auf $\{E_1, \dots, E_n\}$.
 R heißt *binär*, falls $n = 2$, und *informationstragend*, falls $V \neq \emptyset$.
Ein binärer Relationship–Typ mit $E_1 = E_2$ heißt *rekursiv*.
2. Eine Relationship r vom Typ R ist eine Abbildung, welche jedem Entity–Typ E_i eine Entity $r(E_i) \in I(E_i)$ und jedem Attribut $A \in V$ einen Wert $r(A) \in \text{dom}(A)$ zuordnet.
3. Eine *Relationship–Menge* $I(R)$ vom Typ R ist eine Menge von Relationships vom Typ R .

Beispiel (Binäre Relationship–Typen)

GRADE_REPORT = (STUDENT, SECTION, {Grade}),

PREREQUISITE = (COURSE, COURSE, $\emptyset$).

Relationship r vom Typ GRADE_REPORT:

STUDENT	SECTION	Grade
st	se	B

st und se könnten durch ihre Schlüsselattribute Mat.Nr. = 17 bzw. SectionIdentifier = 112 repräsentiert werden:

GRADE_REPORT		
Mat.Nr.	SectionIdentifier	Grade
17	112	B

Der Relationship–Typ PREREQUISITE ist *rekursiv*.

Relationship–Typ

$\text{COURSE_SECTION} = (\text{COURSE}, \text{SECTION}, \emptyset).$

Relationship r vom Typ COURSE_SECTION :

COURSE	SECTION
<i>co</i>	<i>se</i>

co und *se* könnten durch ihre Schlüsselattribute CourseNumber = Math2410 bzw. SectionIdentifier = 112 repräsentiert werden:

COURSE_SECTION	
CourseNumber	SectionIdentifier
Math2410	112

Definition (Funktionalitätsbedingungen)

1. Ist $R = (E_1, E_2, V)$ ein binärer Relationship-Typ, so erfüllt eine *Relationship-Menge* $I(R)$ vom Typ R die Funktionalitätsbedingung $t \in \{1 : 1, 1 : n, n : 1, n : m\}$, falls für alle $r, r' \in I(R)$ gilt:

$$t = 1 : 1: \quad r(E_i) = r'(E_i) \Rightarrow r = r', \text{ für } i = 1, 2,$$

$$t = 1 : n: \quad r(E_2) = r'(E_2) \Rightarrow r = r',$$

$$t = n : 1: \quad r(E_1) = r'(E_1) \Rightarrow r = r'.$$

2. Für $t = n : m$ ist die Funktionalitätsbedingung eigentlich immer erfüllt. Oft wird aber die folgende Annahme gemacht, daß es zwischen zwei Entities höchstens eine Relationship vom Typ R geben kann:

$$t = n : m: \quad (r(E_1) = r'(E_1) \wedge r(E_2) = r'(E_2)) \Rightarrow r = r'.$$

Falls $R = (E_1, E_2, V)$ die Funktionalitätsbedingung $t = 1 : n$ erfüllt, so erfüllt $R = (E_2, E_1, V)$ die Funktionalitätsbedingung $t = n : 1$.

Beispiel (Funktionalitätsbedingungen)

1. Die beiden folgenden Relationship-Typen sind vom Typ $t = n : m$:

GRADE_REPORT = (STUDENT, SECTION, {Grade}),

PREREQUISITE = (COURSE, COURSE, $\emptyset$).

2. Der folgende Relationship-Typ ist vom Typ $t = 1 : n$, da jede SECTION zu genau einem COURSE gehört:

COURSE_SECTION = (COURSE, SECTION, $\emptyset$).

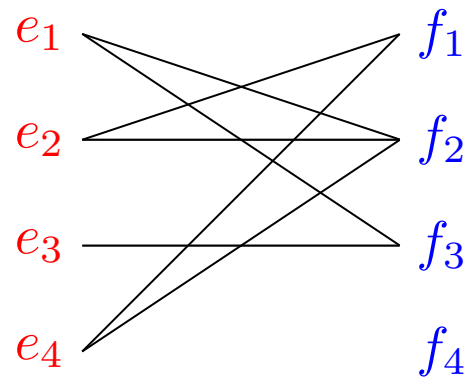
Im relationalen Modell brauchen wir keine Tabelle COURSE_SECTION (siehe oben). Wir nehmen stattdessen das Attribut CourseNumber in SECTION auf.

Beispiel (Funktionalitätsbedingungen)

Die Ausprägungen

$$I(E) = \{ e_1, \dots, e_4 \}, \quad I(F) = \{ f_1, \dots, f_4 \},$$

und die im Bild gezeigte Ausprägung $I(R)$ erfüllen die Funktionalitätsbedingung $t = n : m$:



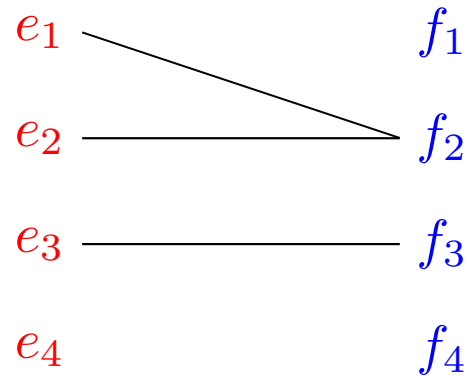
Fast alle Entities $e \in I(E)$ – wie z.B. e_1 – stehen mit mehreren Entities $f \in I(F)$ in Beziehung; Analoges gilt für die Entities $f \in I(F)$.

Deswegen erfüllt $I(R)$ die anderen Funktionalitätsbedingungen nicht.

Die Ausprägungen

$$I(E) = \{e_1, \dots, e_4\}, \quad I(F) = \{f_1, \dots, f_4\},$$

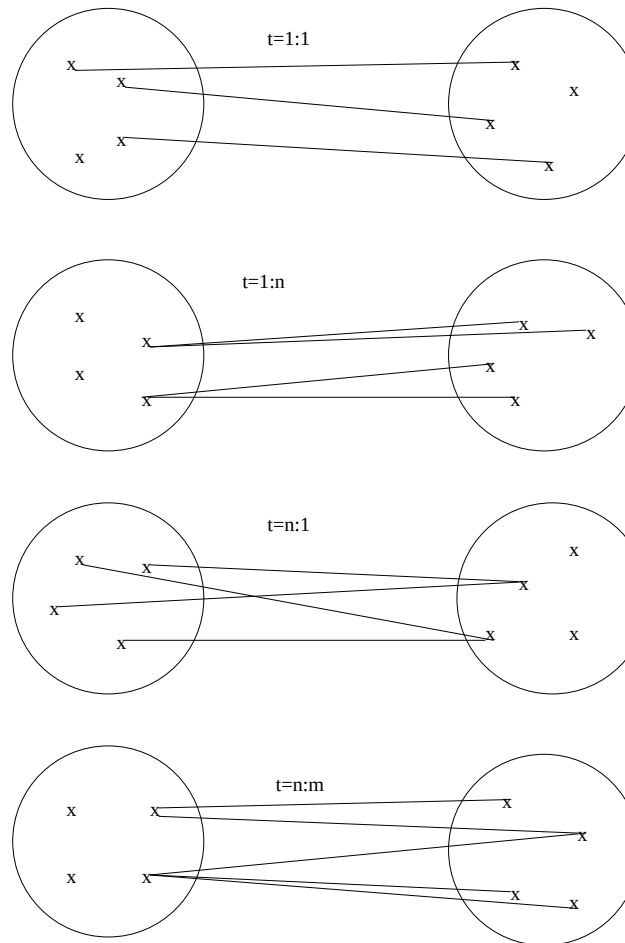
und die im Bild gezeigte Ausprägung $I(R)$ erfüllen die Funktionalitätsbedingung $t = n : 1$:



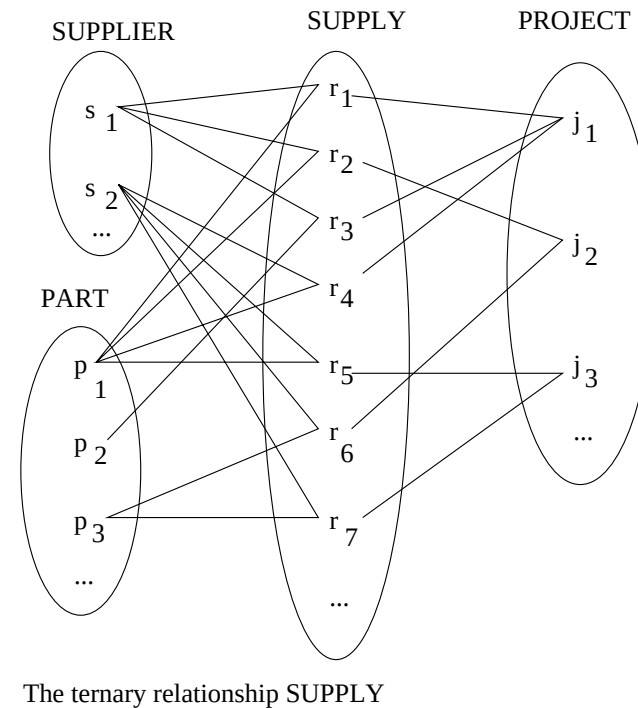
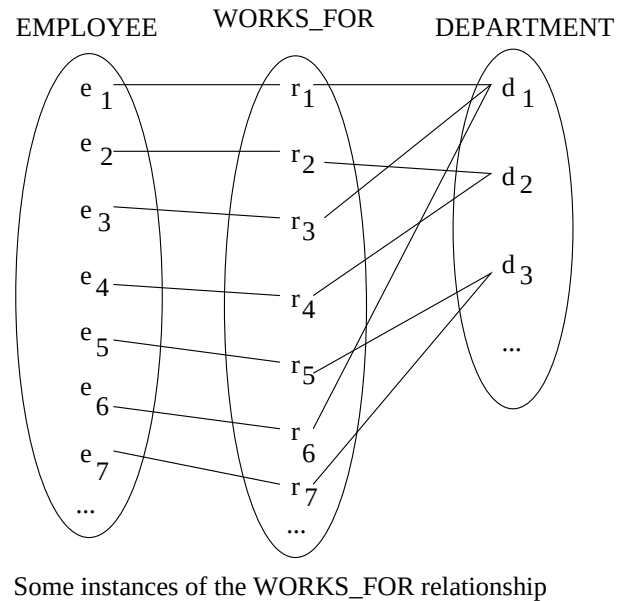
Alle Entities $e \in I(E)$ stehen mit maximal einer Entity $f \in I(F)$ in Beziehung. e_4, f_1, f_4 , stehen mit keiner Entity des anderen Typs in Beziehung.

Da f_2 mit zwei Entities $e \in I(E)$ in Beziehung steht, erfüllt $I(R)$ die Funktionalitätsbedingungen $t = 1 : n$ und $t = 1 : 1$ nicht.

Funktionalitäten von binären Relationship-Typen



Binäre und ternäre Relationship-Mengen



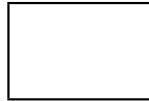
Wenn man die Verbindungskanten zwischen den beteiligten Entities durch die IDs r_i der Relationships ersetzt, dann kann man auch ternäre Relationship-Mengen repräsentieren.

Definition (ER–Datenbank)

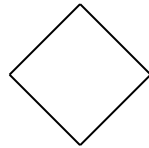
1. Ein *ER–Schema* $ERS = (\mathcal{E}, \mathcal{R})$ besteht aus
 - einer Menge $\mathcal{E}$ von Entity–Typen und
 - einer Menge $\mathcal{R}$ von Relationship–Typen auf $\mathcal{E}$.
2. Eine *ER–Ausprägung* $ERI = (I(\mathcal{E}), I(\mathcal{R}))$ für ERS besteht aus
 - einer Menge $I(\mathcal{E})$ von Entity–Mengen $I(E)$ für die Typen $E \in \mathcal{E}$ und
 - einer Menge $I(\mathcal{R})$ von Relationship–Mengen $I(R)$ auf $I(\mathcal{E})$ für die Typen $R \in \mathcal{R}$.
3. Dann ist $ERDB = (ERS, ERI)$ eine *ER–Datenbank*.

ER-Diagramme

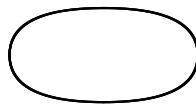
Entity-Typen



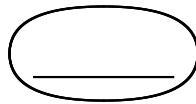
Relationship-Typen



Attribute



beschreibendes Attribut



Schlüsselattribut

Zuordnungen



(von Entity-Typen, Relationship-Typen, Attributen)

Beispiel (ER-Diagramm)

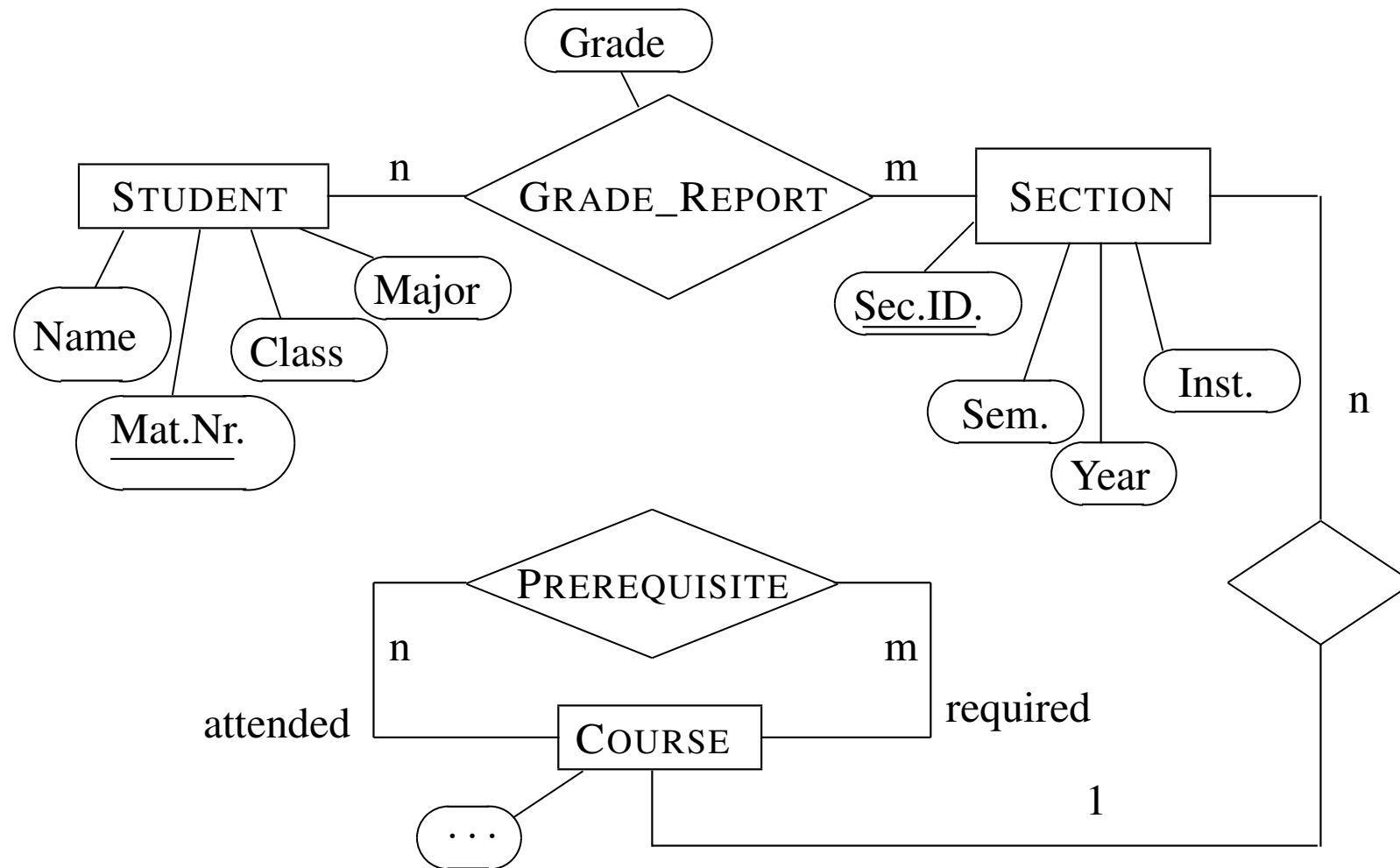
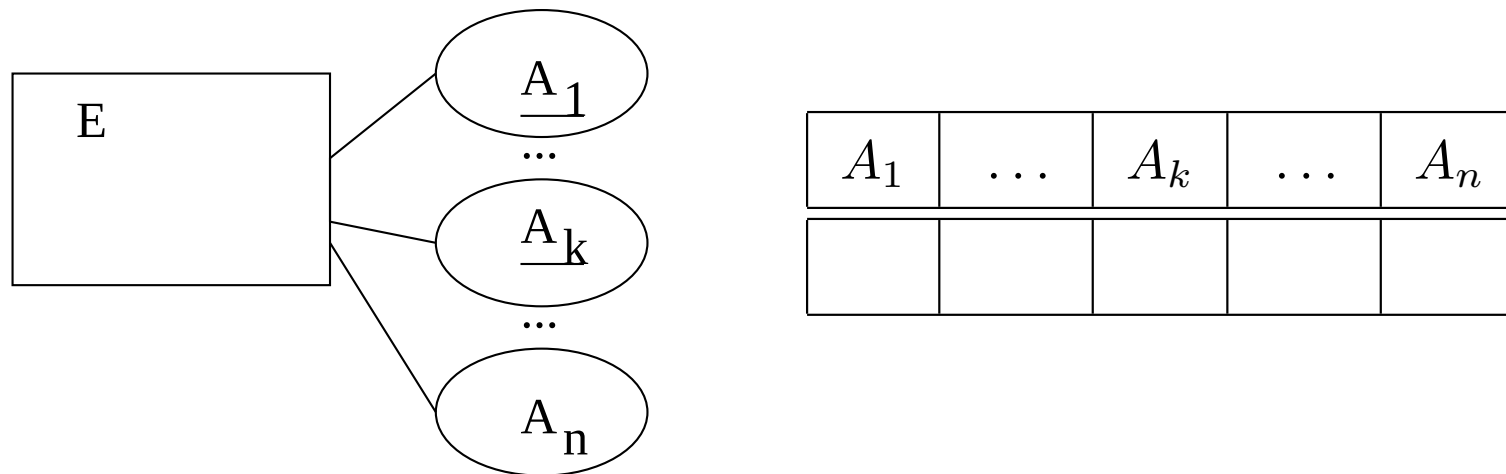


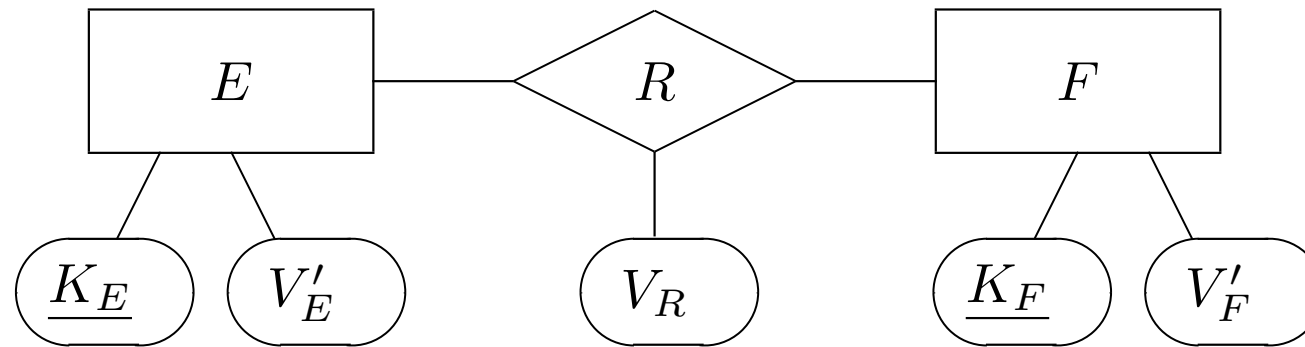
Abbildung ins Relationenmodell

Sowohl Entities als auch Relationships werden in Tabellen (Relationen) gespeichert.

1. Ein Entity-Typ $E = (V_E, K_E)$ mit der Attributmenge $V_E = \{ A_1, \dots, A_n \}$ und dem Schlüssel $K_E = \{ A_1, \dots, A_k \}$, $1 \leq k \leq n$, wird zu einer Tabelle:



2. Ein binärer Relationship–Typ $R = (E, F, V_R)$ kann auch als Tabelle realisiert werden:



Dabei sind E und F Entity–Typen:

- $E = (V_E, K_E)$, mit dem Schlüssel K_E und der weiteren Attributmenge $V'_E = V_E \setminus K_E$.
- $F = (V_F, K_F)$, mit K_F und $V'_F = V_F \setminus K_F$.

K_E			K_F			V_R		
A_1	$\dots$	A_k	B_1	$\dots$	B_l	V_1	$\dots$	V_h

Schlüssel in der Relationship-Tabelle:

$t = 1 : 1 \longrightarrow$ zwei Schlüssel: K_E und K_F

$t = 1 : n \longrightarrow$ ein Schlüssel: K_F

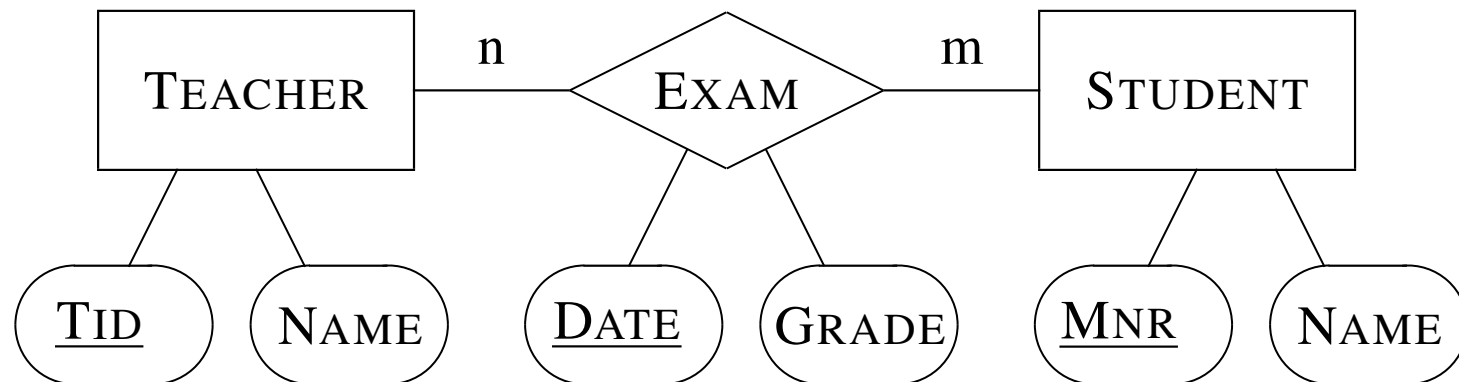
$t = n : 1 \longrightarrow$ ein Schlüssel: K_E

$t = n : m \longrightarrow$ ein Schlüssel: $K_E \cup K_F$

Kontrollierte Redundanz ist aus Effizienzgründen manchmal sinnvoll (z.B. Adresse bei BestellNr. mitführen); dann werden Anfragen schneller, Änderungen aber aufwendiger.

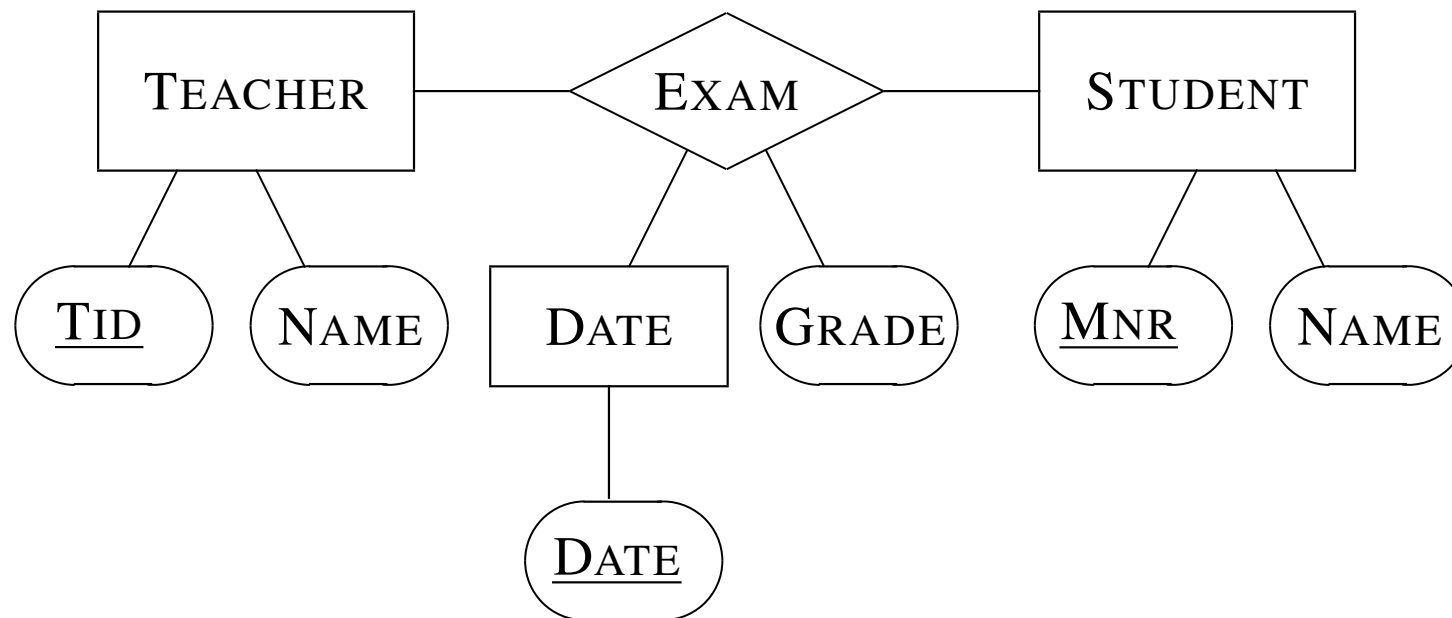
3. Ausnahme:

Die $n : m$ -Beziehung EXAM zwischen STUDENT und TEACHER erfordert eine Erweiterung des Schlüssels, da ein Student mehrere Prüfungen bei demselben Dozenten haben kann.



Wenn man davon ausgeht, daß ein Student an einem Tag bei demselben Dozenten maximal eine Prüfung hat, so kann man eine Tabelle zu EXAM erzeugen mit dem Schlüssel $K_{\text{EXAM}} = \{ \text{TID}, \text{MNR}, \text{DATE} \}$ und dem weiteren Attribut GRADE.

Im klassischen ER-Modell müßte man stattdessen einen weiteren Entity-Typ für das Datum mit nur einem einzigen Attribut DATE einführen. Dann wäre EXAM ein ternärer Relationship-Typ.



Wir können dann keine Funktionalitätsbedingung angeben, da der Relationship-Typ nicht mehr binär ist.

4. Alternative für 1 : n –Relationship–Typen:

den Schlüssel K_E von E in die Tabelle für F aufnehmen

Beispiele:

- in der UNIVERSITY–Datenbank wurde die 1 : n –Beziehung zwischen COURSE und SECTION durch Aufnahme des Schlüssels CourseNumber von COURSE in die Tabelle SECTION realisiert
- in der COMPANY–Datenbank (später) wurde die 1 : n –Beziehung WORKS_FOR zwischen DEPARTMENT und EMPLOYEE durch Aufnahme des Schlüssels DNO von DEPARTMENT in die Tabelle EMPLOYEE realisiert

1.2.2 Erweiterungen des ER-Modells

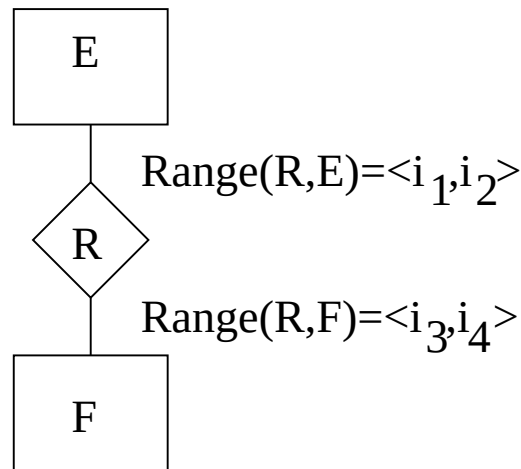
Ziel: Erhöhung der semantischen Modellierungsfähigkeiten

- Kardinalitätsbedingungen (Verallgemeinerung von Funktionalitäten)
- Existenz-Abhängigkeiten
- schwache Entity-Typen
- höhere semantische Datenmodellierungskonzepte:
 - Aggregationshierarchien
 - Generalisierungen

Sei $\langle n, m \rangle$ das Intervall der ganzen Zahlen von n bis m :

$$\langle n, m \rangle = \{ n, n + 1, \dots, m \}.$$

Kardinalitätsbedingung



Eine Ausprägung I von $R = (E, F, V)$ erfüllt die Kardinalitätsbedingung $t = \langle i_1, i_2 \rangle : \langle i_3, i_4 \rangle$, falls für alle $e \in I(E)$ und alle $f \in I(F)$ gilt:

$$|e^+| \in \langle i_1, i_2 \rangle, \text{ wobei } e^+ = \{r \in I(R) \mid r(E) = e\},$$

$$|f^+| \in \langle i_3, i_4 \rangle, \text{ wobei } f^+ = \{r \in I(R) \mid r(F) = f\}.$$

Das Intervall $Range(R, E) = \langle i_1, i_2 \rangle$ gibt die minimale bzw. maximale Anzahl von Relationships $r \in I(R)$ an, die eine Entity $e \in I(E)$ mit Entities $f \in I(F)$ haben kann. Die Menge dieser Relationships ist

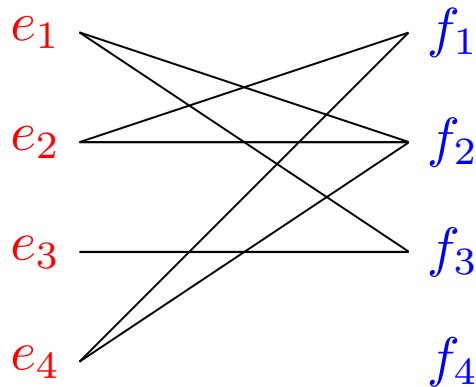
$$e^+ = \{ r \in I(R) \mid r(E) = e \}.$$

In der Regel ist $|e^+|$ die Anzahl der Entities $f \in I(F)$, mit denen e in Beziehung steht. Falls e allerdings mehrere Relationships mit derselben Entity $f \in I(F)$ hat, dann werden diese separat gezählt.

Analoges gilt für $Range(R, F) = \langle i_3, i_4 \rangle$. Die Intervalle geben nur Unter- und Obergrenzen an, die aber in $I(R)$ in der Regel nicht ausgeschöpft sind.

Im folgenden Beispiel gibt es zwischen zwei Entities $e_i \in I(E)$ und $f_j \in I(F)$ nur maximal eine Relationship, die wir mit $r_{i,j}$ bezeichnen. $Range(R, F) = \langle 0, 4 \rangle$ ist nicht ausgeschöpft; das maximale $|f^+|$ ist 3.

Wir betrachten $I(E) = \{ e_1, \dots, e_4 \}$, $I(F) = \{ f_1, \dots, f_4 \}$ und das im Bild gezeigte $I(R)$ mit $\text{Range}(R, E) = \langle 1, 2 \rangle$ und $\text{Range}(R, F) = \langle 0, 4 \rangle$:



$$|f_4^+| = 0, |e_3^+| = 1, |e_1^+| = |e_2^+| = |e_4^+| = |f_1^+| = |f_3^+| = 2, |f_2^+| = 3 :$$

$$\begin{aligned} e_1^+ &= \{ r_{1,2}, r_{1,3} \}, & f_1^+ &= \{ r_{2,1}, r_{4,1} \}, \\ e_2^+ &= \{ r_{2,1}, r_{2,2} \}, & f_2^+ &= \{ r_{1,2}, r_{2,2}, r_{4,2} \}, \\ e_3^+ &= \{ r_{3,3} \}, & f_3^+ &= \{ r_{1,3}, r_{3,3} \}, \\ e_4^+ &= \{ r_{4,1}, r_{4,2} \}, & f_4^+ &= \{ \}. \end{aligned}$$

Bemerkung (Funktionalitäts– vs. Kardinalitätsbedingung)

Die größere Zahl und das korrespondierende größere Intervall stehen auf entgegengesetzten Seiten:

$$1 : 1 \quad \equiv \quad \langle 0, 1 \rangle : \langle 0, 1 \rangle,$$

$$1 : n \quad \equiv \quad \langle 0, \infty \rangle : \langle 0, 1 \rangle,$$

$$n : 1 \quad \equiv \quad \langle 0, 1 \rangle : \langle 0, \infty \rangle,$$

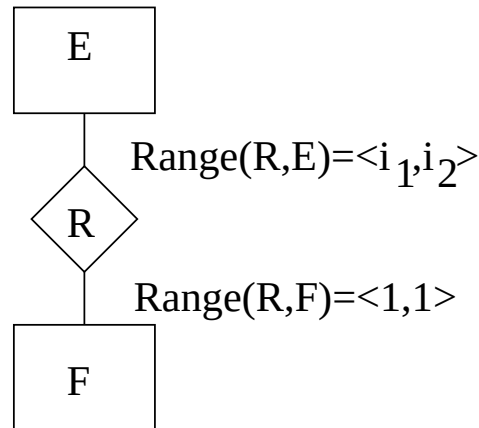
$$n : m \quad \equiv \quad \langle 0, \infty \rangle : \langle 0, \infty \rangle.$$

Kardinalitätsbedingungen müssen beim Update der entsprechenden Relationen überprüft werden.

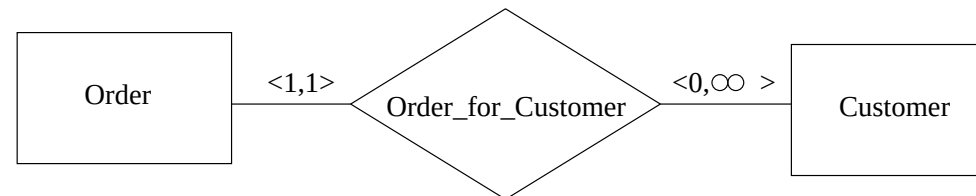
Existenzabhängigkeit

$$t = \langle i_1, i_2 \rangle : \langle 1, 1 \rangle$$

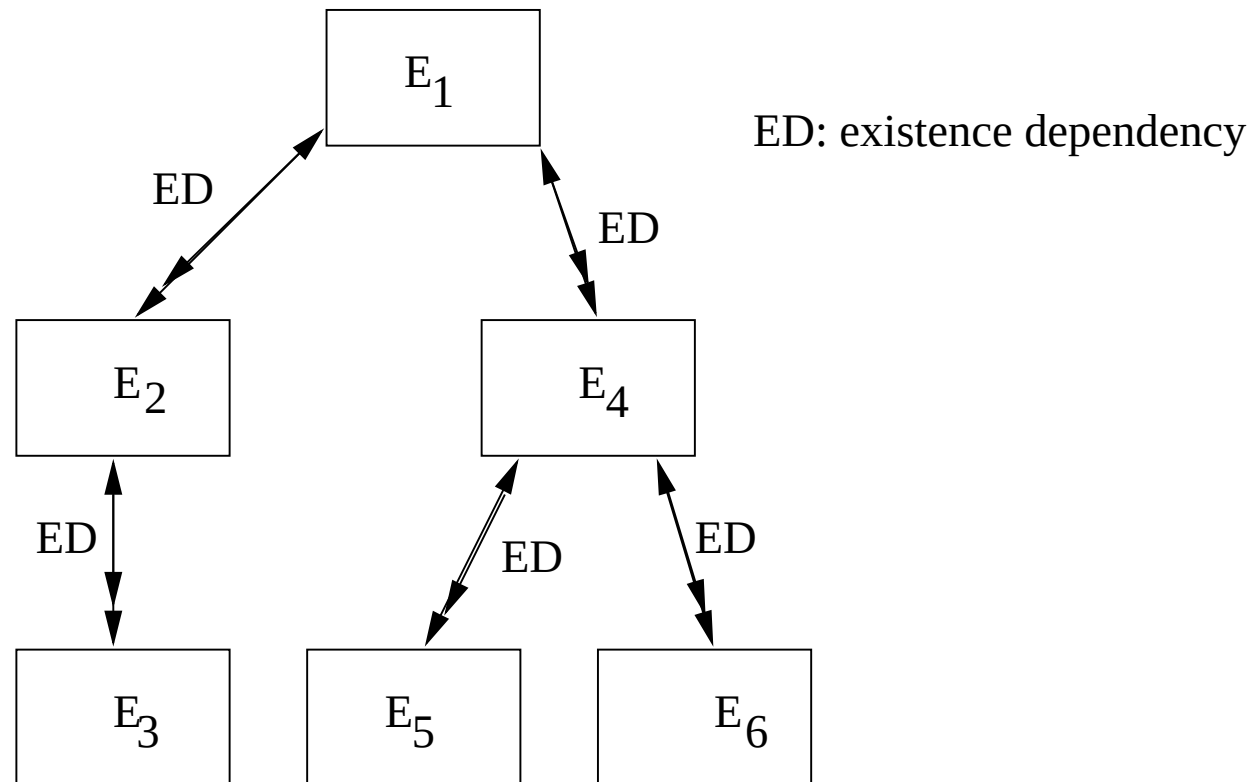
$Range(R, F) = \langle 1, 1 \rangle$ verlangt, daß jede Ziel-Entity $f \in I(F)$ mit genau einer Quellentity $e \in I(E)$ über R in Beziehung stehen muß. $Range(R, E)$ ist beliebig.



Beispiel: Eine Order kann nur existieren, wenn es einen Customer dazu gibt.



Existenzabhängigkeiten sollten möglichst nur *Hierarchien* bilden (keine Zyklen).



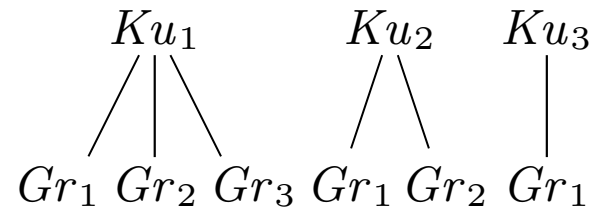
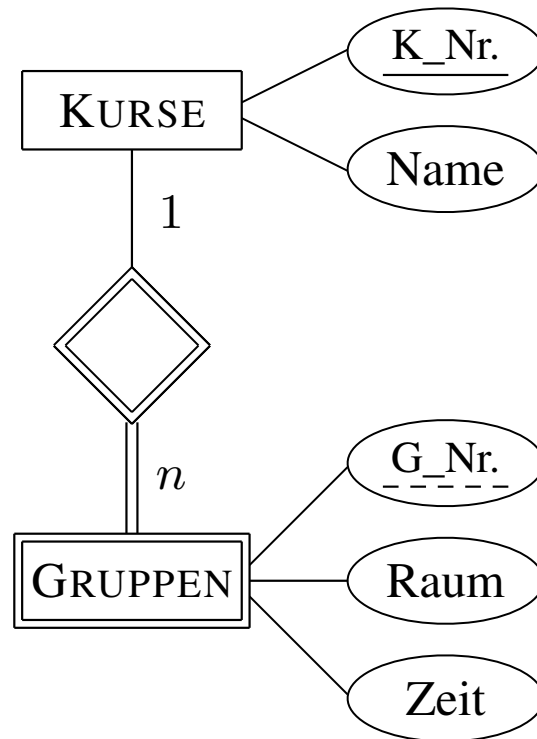
Schwache Entity–Typen

Die bekannte Schlüsselbedingung heißt auch *globale* oder *starke Schlüsselbedingung*. Ein Entity–Typ mit globaler Schlüsselbedingung heißt auch *starker Entity–Typ*. Sein Schlüssel heißt auch *globaler Schlüssel*.

Ein *schwacher* Entity–Typ F tritt zusammen mit einem definierenden 1:n–Relationship–Typ $R = (E, F, V)$ und einer Existenzabhängigkeit auf:

- Jede schwache Entity f von Typ F muß mit genau einer Entity e vom Entity–Typ E in Beziehung stehen.
- Der Schlüsselwert von f ist nur lokal eindeutig innerhalb aller Entities von Typ F , die mit diesem e in Beziehung stehen.
- Der global eindeutige Schlüssel ergibt sich erst durch Verkettung der Schlüssel von e und f .

Beispiel (Übungsgruppen)



Ku_1 : ADS, Gruppe 1–3

Ku_2 : Logik, Gruppe 1–2

Ku_3 : Softwaretechnik, Gruppe 1

Die gleiche Gruppennummer Gr_j ist für unterschiedliche Kurse sinnvoll.

Die Gruppennummer ist *lokal eindeutig* bezüglich eines Kurses.

Die *globale* Eindeutigkeit wird durch die *Verbindung* der Kursnummer Ku_i und der Gruppennummer erzielt.

Relationale Repräsentation:

KURSE	
<i>K_Nr.</i>	<i>Name</i>
<i>Ku₁</i>	<i>ADS</i>
<i>Ku₂</i>	<i>Logik</i>
<i>Ku₃</i>	<i>Softwaretechnik</i>

GRUPPEN			
<i>K_Nr.</i>	<i>G_Nr.</i>	<i>Raum</i>	<i>Zeit</i>
<i>Ku₁</i>	<i>Gr₁</i>	...	...
<i>Ku₁</i>	<i>Gr₂</i>		
<i>Ku₁</i>	<i>Gr₃</i>		
<i>Ku₂</i>	<i>Gr₁</i>		
<i>Ku₂</i>	<i>Gr₂</i>		
<i>Ku₃</i>	<i>Gr₁</i>		

Schwache Schlüsselbedingungen können ebenfalls nur *Hierarchien* bilden.

Definition (Schwacher Entity–Typ)

Sei $E = (V_1, K_1)$ ein starker Entity–Typ mit globalem Schlüssel K_1 und $F = (V_2, K_2)$ ein weiterer Entity–Typ, sowie $R = (E, F, V)$ ein Relationship–Typ, so daß F existenzabhängig ist von E bezüglich R .

1. F heißt *schwacher Entity–Typ*, g.d.w. für alle Ausprägungen $I(E), I(F), I(R)$ die folgende *schwache Schlüsselbedingung* gilt:
für alle $r, r' \in I(R)$:

$$(r(E) = r'(E) \wedge r(F)[K_2] = r'(F)[K_2]) \implies r(F) = r'(F).$$

2. K_2 heißt *lokaler Schlüssel* von F .
 $K_1 \cup K_2$ heißt *globaler Schlüssel* von F .
3. Für $r \in I(R)$ und $e = r(E)$, $f = r(F)$, heißt $e[K_1]$ *vererbter Schlüsselwert* von f .

Relationale Repräsentation:

Da insbesondere ein 1:n-Relationship-Typ $R = (E, F, V)$ vorliegt, wird der Schlüssel K_1 von E in die Relation für F integriert.

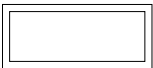
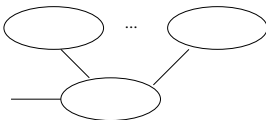
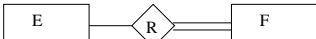
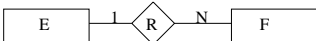
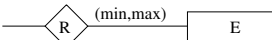
Also erhalten wir zwei Relationen:

- Der starke Entity-Typ $E = (V_1, K_1)$ wird – wie gewohnt – zu einer Relation mit der Attributmenge V_1 und dem Schlüssel K_1 .
- Der schwache Entity-Typ $F = (V_2, K_2)$ wird zu einer Relation mit der Attributmenge $V_2 \cup K_1$ und dem Schlüssel $K_2 \cup K_1$.

Durch die Vereinigung mit dem vererbten Schlüssel K_1 wird der lokale Schlüssel K_2 zu einem globalen Schlüssel $K_2 \cup K_1$.

Der vererbte Schlüssel K_1 realisiert die 1:n-Relationship zwischen den Entities der Typen E und F .

Zusammenfassung der ER-Symbole

<u>Symbol</u>	<u>Bedeutung</u>
	Entity-Typ
	schwacher Entity-Typ
	Relationship-Typ
	identifizierender Relationship-Typ
	Attribut
	Schlüsselattribut
	mehrwertiges Attribut
	zusammengesetztes Attribut
	abgeleitetes Attribut
	zu jedem Element aus F muß es mindestens eines aus E geben
	1:N -Beziehung zwischen E und F
	Constraint (min,max)

1.2.3 Beispiel: die COMPANY-Datenbank

Die COMPANY-Datenbank enthält folgende Entity-Typen:

- Angestellte (EMPLOYEE) und deren Angehörige (DEPENDENT),
- Abteilungen (DEPARTMENT), sowie
- Projekte (PROJECT).

Die Relationship-Typen repräsentieren folgende Beziehungen dazwischen:

- Für alle Angestellten gibt SUPERVISION den Vorgesetzten und DEPENDENTS_OF die Angehörigen an.

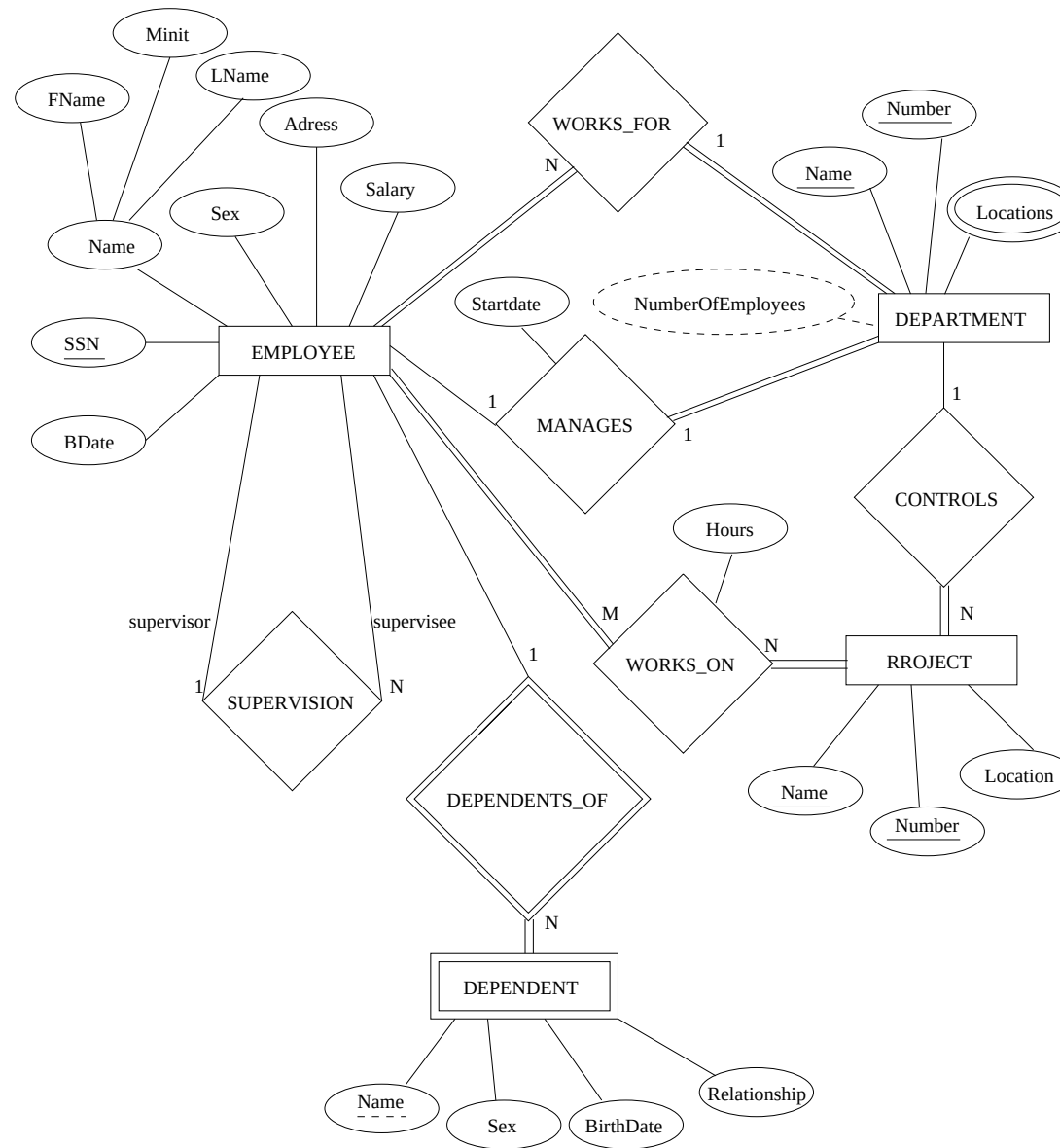
Der schwache Entity-Typ DEPENDENT wird durch den Relationship-Typ DEPENDENTS_OF und den Entity-Typ EMPLOYEE eindeutig gemacht.

- Für alle Abteilungen gibt WORKS_FOR die zugehörigen Angestellten und MANAGES den Manager an.

Die Funktionalitäten und die Existenzabhängigkeiten sagen zusammen aus, daß jeder Angestellte in genau einer Abteilung sein und daß jede Abteilung Angestellte und genau einen Manager haben muß; außerdem kann ein Angestellter maximal eine Abteilung managen.

- Für alle Projekte gibt WORKS_ON die beteiligten Angestellten und CONTROLS die verantwortlichen Abteilungen an.

Die Funktionalitäten und die Existenzabhängigkeiten sagen zusammen aus, daß jeder Angestellte an einem Projekt beteiligt sein und daß jedes Projekt Angestellte und genau eine verantwortliche Abteilung haben muß.

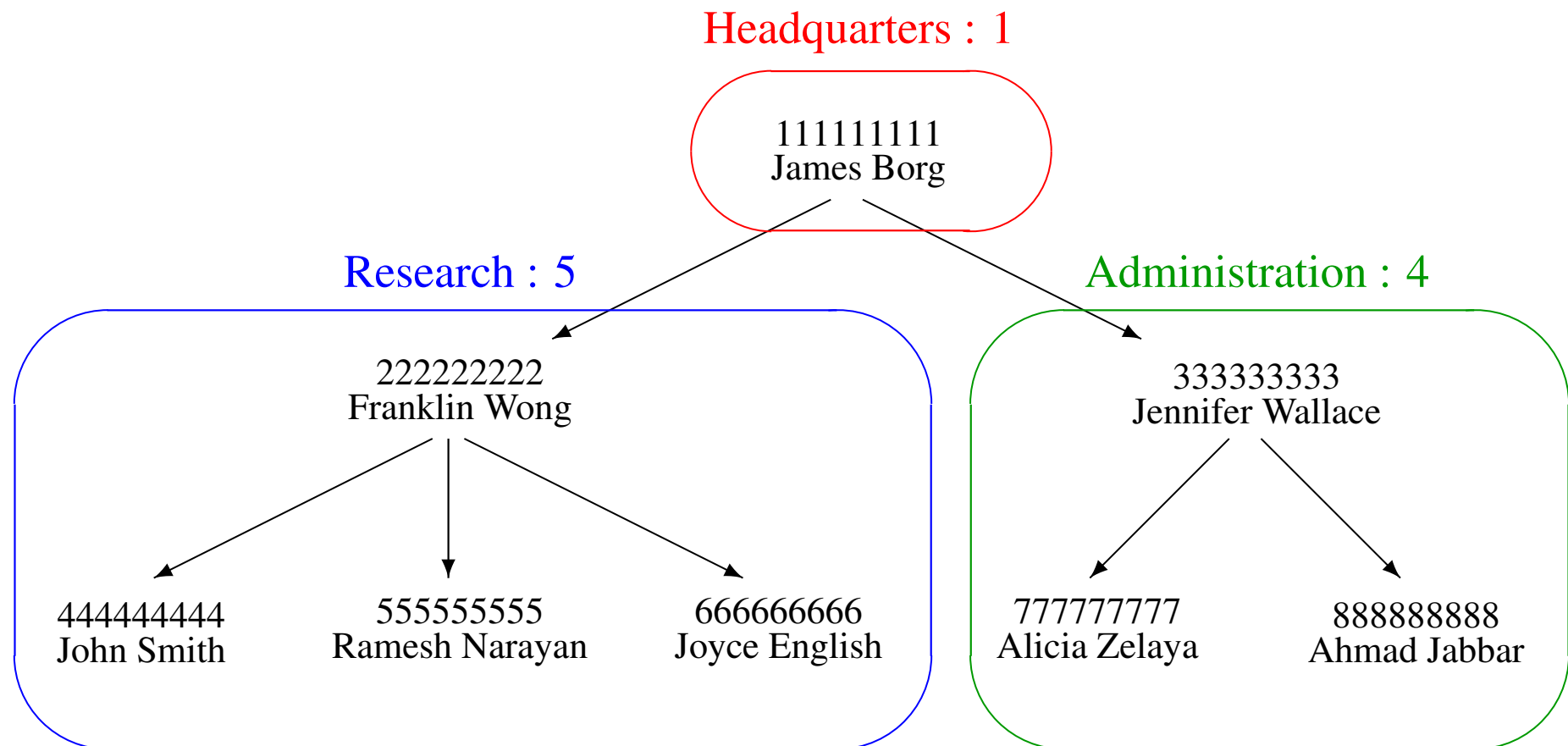


In EMPLOYEE werden die Komponenten des zusammengesetzten Attributs NAME aus dem ER-Diagramm zu 3 eigenständigen Attributen.

Die 1:n-Relationship-Typen WORKS_FOR und SUPERVISION werden als Fremdschlüssel DNO bzw. SUPERSSN in EMPLOYEE integriert.

EMPLOYEE									
FNAME	MINIT	LNAME	SSN	BDATE	ADDRESS	SEX	SALARY	SUPERSSN	DNO
John	B	Smith	444444444	1955-01-09	731 Fondren, Houston, TX	M	30000	222222222	5
Franklin	T	Wong	222222222	1945-12-08	638 Voss, Houston, TX	M	40000	111111111	5
Alicia	J	Zelaya	777777777	1958-07-19	3321 Castle, Spring, TX	F	25000	333333333	4
Jennifer	S	Wallace	333333333	1931-06-20	291 Berry, Bellaire, TX	F	43000	111111111	4
Ramesh	K	Narayan	555555555	1952-09-15	975 Fire Oak, Humble, TX	M	38000	222222222	5
Joyce	A	English	666666666	1962-07-31	5631 Rice, Houston, TX	F	25000	222222222	5
Ahmad	V	Jabbar	888888888	1959-03-29	980 Dallas, Houston, TX	M	25000	333333333	4
James	E	Borg	111111111	1927-11-10	450 Stone, Houston, TX	M	55000	NULL	1

Die Abteilungen, die Angestellten und deren Vorgesetzte:



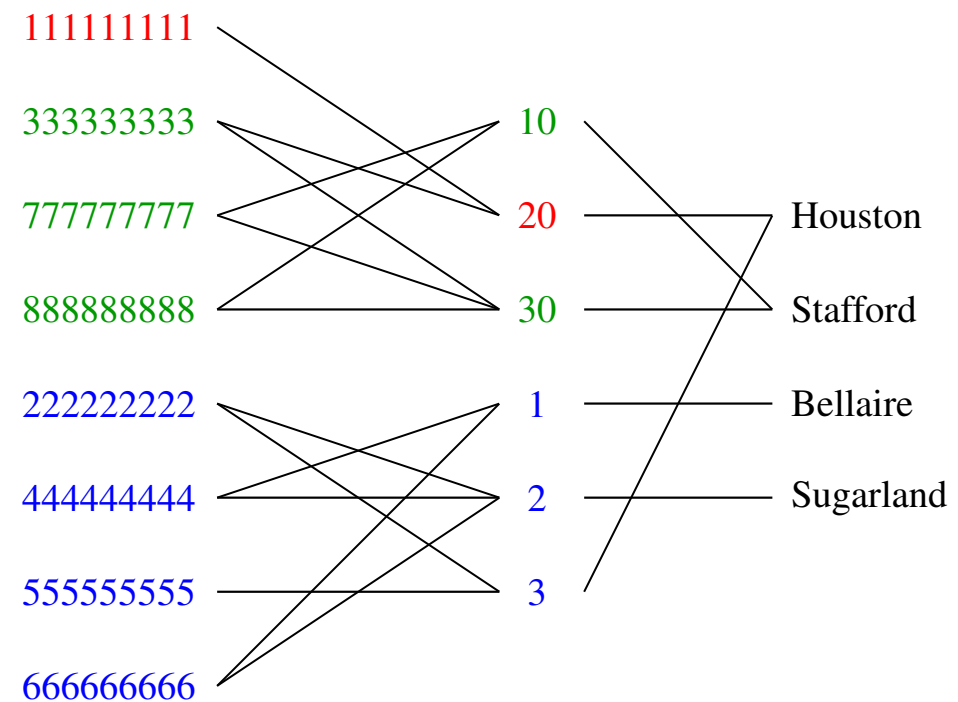
Der 1:1–Relationship–Typ MANAGES wird als Fremdschlüssel MGRSSN zusammen mit dem beschreibenden Attribut MGRSTARTDATE in DEPARTMENT integriert.

Das mehrwertige Attribut Locations des Entity–Typs DEPARTMENT führt zu einer eigenen Tabelle DEPT_LOCATIONS.

DEPARTMENT			
DNAME	DNUMBER	MGRSSN	MGRSTARTDATE
Headquarters	1	111111111	1971-06-19
Administration	4	333333333	1985-01-01
Research	5	222222222	1978-05-22

DEPT_LOCATIONS	
DNUMBER	DLOCATION
1	Houston
4	Stafford
5	Bellaire
5	Sugarland
5	Houston

WORKS_ON		
ESSN	PNO	HOURS
111111111	20	NULL
222222222	2	10.0
222222222	3	10.0
333333333	20	15.0
333333333	30	20.0
444444444	1	32.5
444444444	2	7.5
555555555	3	40.0
666666666	1	20.0
666666666	2	20.0
777777777	10	10.0
777777777	30	30.0
888888888	10	35.5
888888888	30	5.0



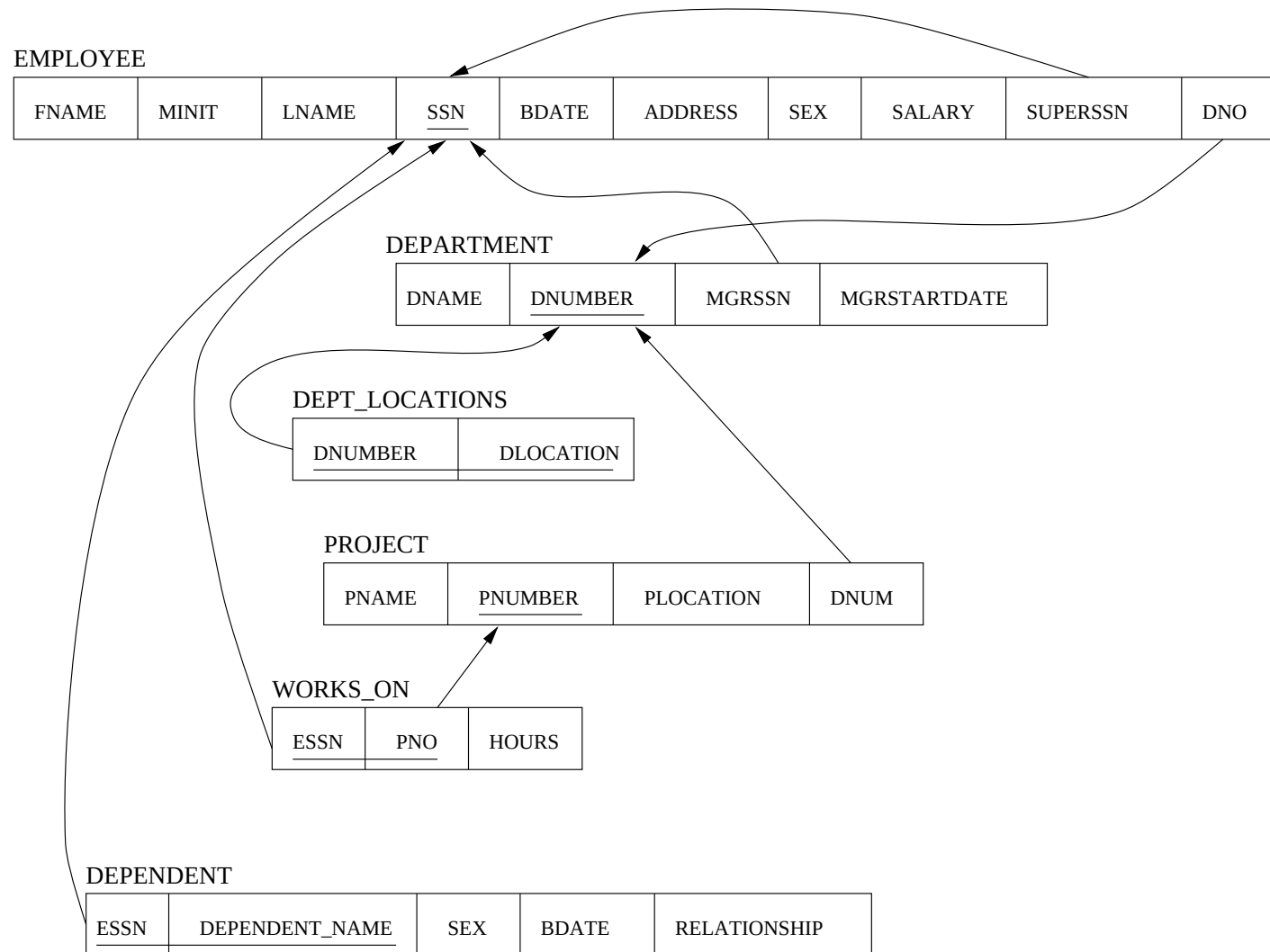
Der 1:n-Relationship-Typ CONTROLS zwischen DEPARTMENT und PROJECT wird als Fremdschlüssel DNUM in PROJECT integriert.

PROJECT			
PNAME	PNUMBER	PLOCATION	DNUM
ProductX	1	Bellaire	5
ProductY	2	Sugarland	5
ProductZ	3	Houston	5
Computerization	10	Stafford	4
Reorganization	20	Houston	1
Newbenefits	30	Stafford	4

Jedes Projekt ist an einem der Standorte der kontrollierenden Abteilung.

Der schwache Entity–Typ **DEPENDENT** wird durch den Fremdschlüssel **ESSN**, der sich auf den Schlüssel **SSN** des Entity–Typs **EMPLOYEE** bezieht, eindeutig.

DEPENDENT				
ESSN	DEPENDENT_NAME	SEX	BDATE	RELATIONSHIP
222222222	Alice	F	1976-04-05	DAUGHTER
222222222	Theodore	M	1973-10-25	SON
222222222	Joy	F	1948-05-03	SPOUSE
333333333	Abner	M	1932-02-29	SPOUSE
444444444	Michael	M	1978-01-01	SON
444444444	Alice	F	1978-12-31	DAUGHTER
444444444	Elizabeth	F	1957-05-05	SPOUSE



Referential integrity constraints displayed on the COMPANY relational database schema

2 Das Relationenmodell: Sprachen und Systeme

E.F. Codd, 1970

2.1 Grundlegende Definitionen

Wir betrachten eine endliche, *universelle Attributmenge* U und eine abzählbare, *universelle Wertemenge* $\mathcal{X}$ mit einem ausgezeichneten *Nullwert* $\text{NULL} \in \mathcal{X}$.

Die Funktion $\text{dom} : U \rightarrow 2^{\mathcal{X}}$ ordne jedem Attribut $A \in U$ einen nichtleeren *Wertebereich* $\text{dom}(A) \subseteq \mathcal{X}$ mit $\text{NULL} \in \text{dom}(A)$ zu.

Tupel und Relationen werden nun analog zu Entities und Entity–Mengen definiert.

Definition (Tupel, Relation)

1. Ein *Tupel* t über $V \subseteq U$ ist eine Abbildung, welche jedem Attribut $A \in V$ einen Wert $t(A) \in \text{dom}(A)$ zuordnet.

$\text{def}(t) := V$ heißt *Definitionsbereich* von t .

Die Projektion $t[X]$ von t auf $X \subseteq V$ ist die Abbildung mit dem eingeschränkten Definitionsbereich X .

2. Eine Menge r von Tupeln über V heißt *Relation* über V .

Beispiel (EMPLOYEE) $V = \{ \text{FNAME}, \text{MINIT}, \text{LNAME}, \dots, \text{DNO} \}$

$t(\text{FNAME}) = \text{'John'}$, $t(\text{MINIT}) = \text{'B'}$, $t(\text{LNAME}) = \text{'Smith'}$,
 $t(\text{SSN}) = \text{'444444444'}$, $\dots$, $t(\text{SUPERSSN}) = \text{'222222222'}$, $t(\text{DNO}) = 5$.

FNAME	MINIT	LNAME	SSN	...	SUPERSSN	DNO
John	B	Smith	444444444	...	222222222	5

Definition (Relationenschema)

1. $R = (V, K, \text{NOTNULL})$ heißt *Relationenschema* mit
$$K \subseteq \text{NOTNULL} \subseteq V \subseteq U$$
2. $\text{def}(R) := V$ heißt auch *Definitionsbereich* von R .
3. K und NOTNULL spezifizieren *intrarelationale Bedingungen* für R :
 - K heißt *Primärschlüssel* von R ,
 - NOTNULL heißt die Menge der NOTNULL -Attribute von R .

Beispiel (EMPLOYEE)

$$\begin{aligned} V &= \{ \text{FNAME}, \text{MINIT}, \text{LNAME}, \text{SSN}, \text{BDATE}, \\ &\quad \text{ADDRESS}, \text{SEX}, \text{SALARY}, \text{SUPERSSN}, \text{DNO} \}, \\ K &= \{ \text{SSN} \}, \\ \text{NOTNULL} &= \{ \text{FNAME}, \text{LNAME}, \text{SSN}, \text{DNO} \}. \end{aligned}$$

Definition (Relation)

Sei $R = (V, K, \text{NOTNULL})$ ein Relationenschema.

Eine *Relation* r für R ist eine Menge von Tupeln über V , die zusätzlich die intrarelationalen Bedingungen von R erfüllen:

1. $\forall t, t' \in r : t[K] = t'[K] \Rightarrow t = t'$,
2. $\forall t \in r \forall A \in \text{NOTNULL} : t(A) \neq \text{NULL}$.

Beispiel (EMPLOYEE)

FNAME	MINIT	LNAME	SSN	...	SUPERSSN	DNO
John	B	Smith	444444444	...	222222222	5
...	...	...	...	...	...	...
James	E	Borg	111111111	...	NULL	1

Definition (Fremdschlüsselbedingung)

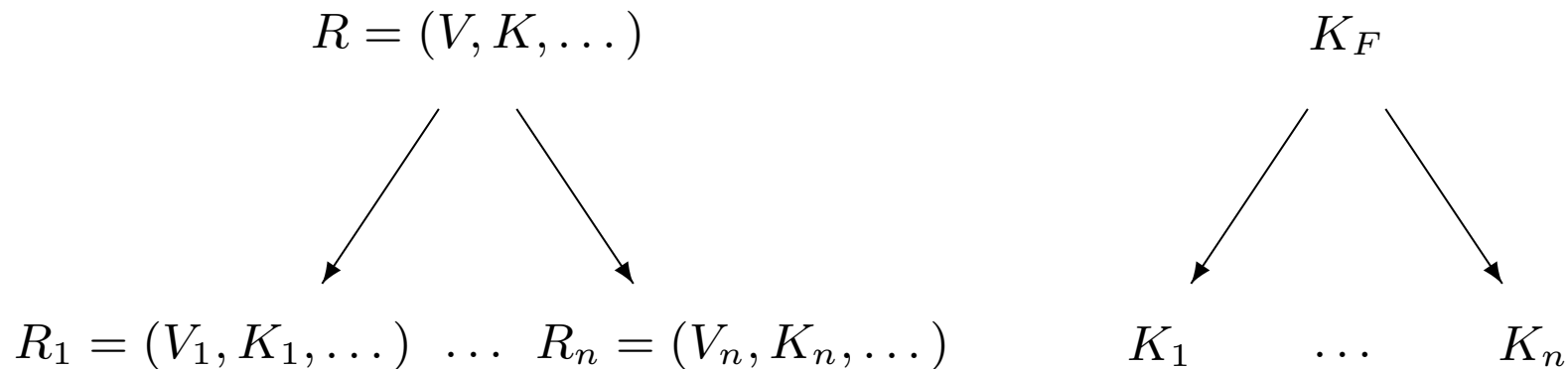
Eine *Fremdschlüsselbedingung* (foreign key constraint)

$$FK = (R, K_F, QFIER, \mathcal{R}, DRULE, URULE)$$

ist gegeben durch

1. ein *referenzierendes* Relationenschema $R = (V, K, \text{NOTNULL})$ mit $K_F \subseteq V$,
2. eine Menge $\mathcal{R} = \{ R_1, \dots, R_n \}$ von *referenzierten* Relationenschemata $R_i = (V_i, K_i, \text{NOTNULL}_i)$,
3. $QFIER \in \{ \textit{exactly one}, \textit{at least one}, \textit{all} \}$,
4. $DRULE, URULE \in \{ \text{NO ACTION}, \text{SET NULL}, \text{SET DEFAULT}, \text{CASCADE} \}$.

Eine Teilmenge $K_F \subseteq V$ der Attributmenge V von R referenziert die Schlüssel K_i der Relationenschemata R_i .

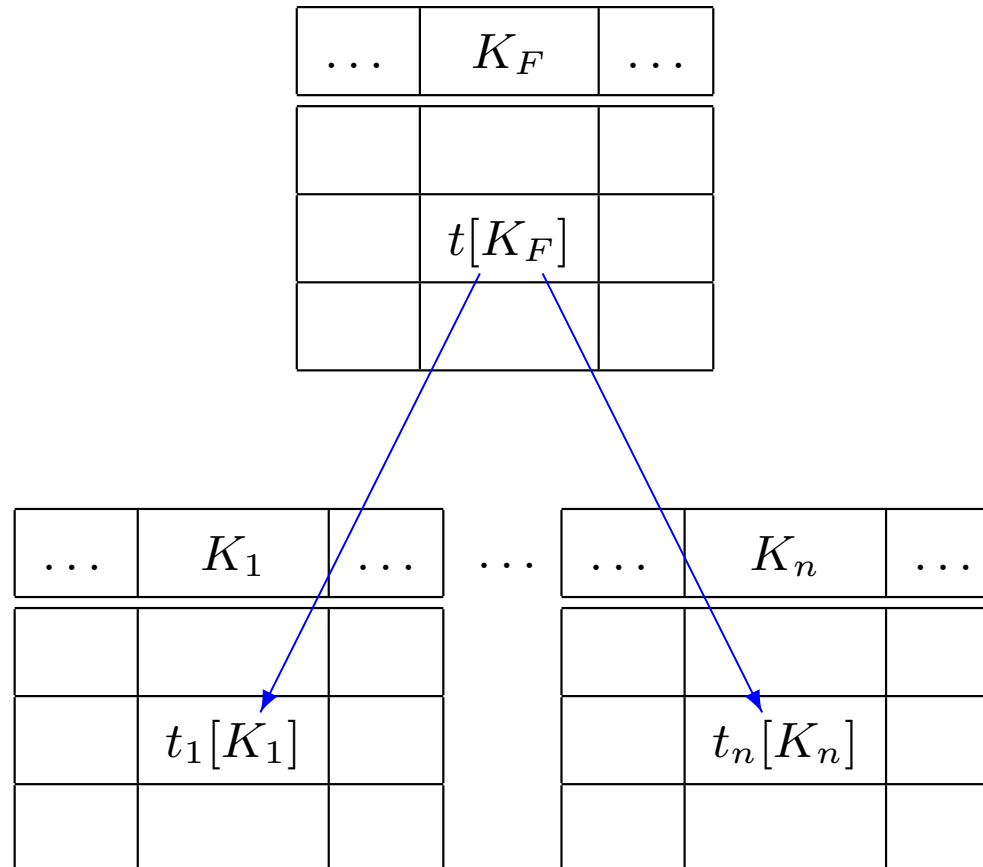


- Sei r eine erlaubte Instanz von R , und
- sei r_i eine erlaubte Instanz von R_i , für $1 \leq i \leq n$.

Ein Tupel $t \in r$ *referenziert* die Relation r_i ,
falls es ein Tupel $t_i \in r_i$ gibt, mit $t[K_F] = t_i[K_i]$.

Da K_i der Schlüssel von R_i ist, gibt es zu jedem $t \in r$ maximal ein solches referenziertes Tupel $t_i \in r_i$.

Ein Tupel $t \in r$ referenziert r_i , falls es ein Tupel $t_i \in r_i$ gibt, mit $t[K_F] = t_i[K_i]$.



Die Fremdschlüsselbedingung FK wird von

- der erlaubten Instanz r von R und
- den erlaubten Instanzen r_i von R_i , für $1 \leq i \leq n$,

erfüllt, falls für alle Tupel $t \in r$ die folgende Bedingung gilt:

entweder ($\forall A \in K_F : t(A) = \text{NULL}$) oder

(falls $QFIER=\text{exactly one}$ ist, so referenziert t genau ein r_i ,

falls $QFIER=\text{at least one}$ ist, so referenziert t mindestens ein r_i ,

falls $QFIER=\text{all}$ ist, so referenziert t alle r_i .)

Der Normalfall ist $n = 1$.

Dann fallen “exactly one”, “at least one” und “all” zusammen.

In SQL können momentan nur Fremdschlüsselbedingungen mit $n = 1$ referenzierten Relationenschemata spezifiziert werden.

Inklusionsabhängigkeit

Eine Fremdschlüsselbedingung

$$FK = (R, K_F, QFIER, \mathcal{R}, DRULE, URULE)$$

mit nur einem referenzierten Relationenschema

$$R_1 = (V_1, K_1, \text{NOTNULL}_1)$$

ist äquivalent zu einer Inklusionsabhängigkeit, denn für die zugehörigen erlaubten Instanzen muß folgendes gelten:

$$\Pi_{K_F}(r) \setminus \{t_{NULL}\} \subseteq \Pi_{K_1}(r_1),$$

wobei t_{NULL} das Tupel über K_F mit lauter Nullwerten ist.

Diese Bedingung nennt man eine Inklusionsabhängigkeit.

Beispiel (COMPANY)

Für die Tabelle EMPLOYEE gilt folgendes:

- Wenn ein Angestellter einen von NULL verschiedenen SUPERSSN–Eintrag hat, dann muß es genau ein referenziertes Tupel in EMPLOYEE geben mit diesem SSN–Eintrag.
Wenn ein Angestellter keinen Vorgesetzten hat (so wie der Chef James E. Borg der Firma), dann kann man NULL eintragen.
- Aufgrund der NOTNULL–Bedingung für DNO muß jeder Angestellte einer Abteilung zugeordnet sein. Die von NULL verschiedene DNO referenziert genau ein Tupel in DEPARTMENT mit demselben DNUMBER–Eintrag.

Da SSN und DNUMBER die Primärschlüssel ihrer Tabellen sind, gibt es jeweils maximal ein referenziertes Tupel.

EMPLOYEE									
FNAME	MINIT	LNAME	SSN	BDATE	ADDRESS	SEX	SALARY	SUPERSSN	DNO
John	B	Smith	444444444	1955-01-09	731 Fondren, Houston, TX	M	30000	222222222	5
Franklin	T	Wong	222222222	1945-12-08	638 Voss, Houston, TX	M	40000	111111111	5
Alicia	J	Zelaya	777777777	1958-07-19	3321 Castle, Spring, TX	F	25000	333333333	4
Jennifer	S	Wallace	333333333	1931-06-20	291 Berry, Bellaire, TX	F	43000	111111111	4
Ramesh	K	Narayan	555555555	1952-09-15	975 Fire Oak, Humble, TX	M	38000	222222222	5
Joyce	A	English	666666666	1962-07-31	5631 Rice, Houston, TX	F	25000	222222222	5
Ahmad	V	Jabbar	888888888	1959-03-29	980 Dallas, Houston, TX	M	25000	333333333	4
James	E	Borg	111111111	1927-11-10	450 Stone, Houston, TX	M	55000	NULL	1

DEPARTMENT			
DNAME	DNUMBER	MGRSSN	MGRSTARTDATE
Headquarters	1	111111111	1971-06-19
Administration	4	333333333	1985-01-01
Research	5	222222222	1978-05-22

Jede DNO-Zelle in EMPLOYEE referenziert genau eine DNUMBER-Zelle in DEPARTMENT mit demselben Attributwert. Analog für SUPERSSN.

Die Relationship–Typen aus dem ER–Diagramm führen zu folgenden Fremdschlüsselbedingungen im Relationenmodell; *QFIER*, *DRULE* und *URULE* werden hier noch vernachlässigt (siehe ”_”):

1. Die 1:N–Beziehung WORKS_FOR, welche als Attribut DNO in die Relation EMPLOYEE integriert wurde, führt zu

$$FK_1 = (\text{EMPLOYEE}, \{ \text{DNO} \}, _, \{ \text{DEPARTMENT} \}, _, _).$$

Das Attribut DNO von EMPLOYEE referenziert den Schlüssel DNUMBER von DEPARTMENT.

2. Die 1:N–Beziehung SUPERVISION, welche als Attribut SUPERSSN in die Relation EMPLOYEE integriert wurde, führt zu

$$FK_2 = (\text{EMPLOYEE}, \{ \text{SUPERSSN} \}, _, \{ \text{EMPLOYEE} \}, _, _).$$

Das Attribut SUPERSSN von EMPLOYEE referenziert den Schlüssel SSN von EMPLOYEE.

3. Die 1:1-Beziehung MANAGES mit Existenzabhängigkeit, welche als Attribut MGRSSN in die Relation DEPARTMENT integriert wurde, führt zu

$$FK_3 = (\text{DEPARTMENT}, \{\text{MGRSSN}\}, _, \{\text{EMPLOYEE}\}, _, _).$$

Das Attribut MGRSSN von DEPARTMENT referenziert den Schlüssel SSN von EMPLOYEE.

4. Die N:M-Beziehung WORKS_ON, welche zu einer eigenen Relation wurde, führt zu je einer Fremdschlüsselbedingung auf die beteiligten Relationen:

$$FK_4 = (\text{WORKS_ON}, \{\text{ESSN}\}, _, \{\text{EMPLOYEE}\}, _, _)$$

$$FK_5 = (\text{WORKS_ON}, \{\text{PNO}\}, _, \{\text{PROJECT}\}, _, _)$$

Das Attribut ESSN von WORKS_ON referenziert den Schlüssel SSN von EMPLOYEE. Das Attribut PNO von WORKS_ON referenziert den Schlüssel PNUMBER von PROJECT.

Fremdschlüsselbedingungen mit mehreren referenzierten Relationenschemas, d.h. $n \geq 2$, treten z.B. auf, wenn ein referenziertes Relationenschema zerlegt wird.

Beispiel (COMPANY)

Falls man das Relationenschema EMPLOYEE disjunkt zerlegt in

- ein Schema E_i mit allen internen und
- ein weiteres Schema E_e mit allen externen Angestellten,

so erhält man anstelle von FK_4 die neue Fremdschlüsselbedingung FK'_4 :

$$FK'_4 = (\text{WORKS_ON}, \{\text{ESSN}\}, \text{exactly one}, \{E_i, E_e\}, _ , _)$$

Das Attribut ESSN von WORKS_ON referenziert den Schlüssel SSN von E_i bzw. E_e .

Fremdschlüssel stellen *inter-relationale* Bedingungen für Datenbankschemata dar.

Definition (Datenbankschema, Datenbank–Instanz)

1. Ein *relationales Datenbankschema* $\mathcal{S} = (\mathcal{R}, \mathcal{FK})$ besteht aus

- einer Menge von Relationenschemata,

$$\mathcal{R} = \{ R_1, \dots, R_n \},$$

- und einer Menge $\mathcal{FK}$ von Fremdschlüsseln auf $\mathcal{R}$.

2. Eine erlaubte *Datenbank–Instanz* von $\mathcal{S}$ ist eine Menge

$$\mathcal{I} = \{ r_1, \dots, r_n \}$$

von Relationen für die Relationenschemata R_i , welche die Fremdschlüsselbedingungen aus $\mathcal{FK}$ erfüllen.

3. Dann ist $\mathcal{DB} = (\mathcal{S}, \mathcal{I})$ eine *relationale Datenbank*.

Die **DELETE**– und **UPDATE**–Modi für Fremdschlüsselbedingungen sind für Operationen (Löschen, Modifizieren) auf den referenzierten Relationen relevant:

NO ACTION: Das Löschen und der Update von referenzierten Tupeln muß abgelehnt werden. Es erfolgt keine Aktion.

SET NULL, SET DEFAULT:

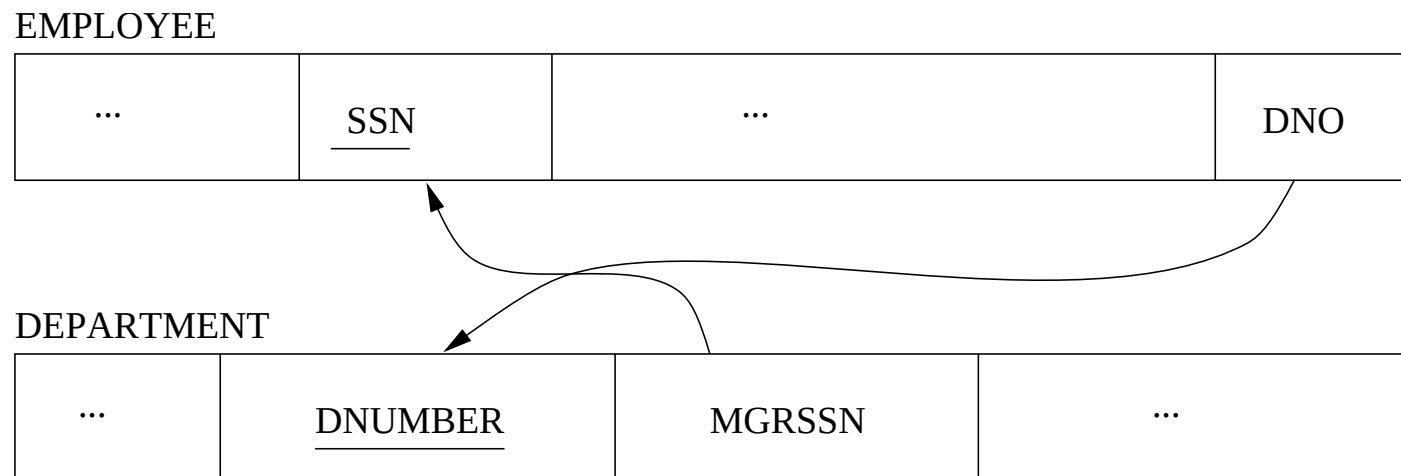
Beim Löschen und Update von referenzierten Tupeln werden die Werte der referenzierenden Attribute auf NULL bzw. auf den DEFAULT–Wert gesetzt. Dies erfolgt nur, soweit keine NOTNULL–Bedingungen verletzt werden; ansonsten muß die Operation abgelehnt werden.

CASCADE:

Beim Löschen von referenzierten Tupeln werden alle referenzierenden Tupel ebenfalls gelöscht. Beim Update werden die Werte der referenzierenden Fremdschlüsselattribute auf die neuen Werte der referenzierten Attribute gesetzt.

Zyklische Fremdschlüsselbedingungen müssen geeignet behandelt werden.

Beispiel (Zyklus)



2.2 Relationale Algebra

äquivalente relationale Anfragesprachen:

- Relationale Algebra
- Relationale Kalküle (Tupelkalküle, Wertebereichskalküle)

Relationale Algebra: 9 Operatoren

- 4 traditionelle Mengenoperatoren:
Vereinigung $\cup$, Schnitt $\cap$, Differenz $\setminus$, kartesisches Produkt $\times$
- 5 spezifische relationale Operatoren:
Selektion σ , Projektion π , Join $\bowtie$, Division $\div$, Umbenennung ρ

Das Ergebnis einer relationalen Operation ist wieder eine Relation, d.h. die Relationale Algebra ist abgeschlossen.

Kartesisches Produkt

1. Für zwei Tupel $t = (t_1, \dots, t_n)$ und $t' = (t'_1, \dots, t'_m)$ ist

$$t \times t' = (t_1, \dots, t_n, t'_1, \dots, t'_m).$$

2. Für zwei Relationen r und s über V_r bzw. V_s betrachten wir das kartesische Produkt $r \times s = \{ t \times t' \mid t \in r, t' \in s \}$ als eine Relation über der Vereinigung $V = V_r \cup V_s$ mit Duplikaten; V ist eine Multimenge. $r \times s$ enthält $|r| \cdot |s|$ Tupel.

r		s		$r \times s$			
A	B	C	D	A	B	C	D
1	2	2	5	1	2	2	5
5	6	2	6	1	2	2	6
		3	7	1	2	3	7
				5	6	2	5
				5	6	2	6
				5	6	3	7

Suchprädikate

- Vergleichsoperatoren:

$$\mathcal{OP} = \{ =, \neq, <, >, \leq, \geq \}$$

- $\theta := A \odot B$, mit $A, B \in U$, $\odot \in \mathcal{OP}$, ist ein Suchprädikat mit $def(\theta) = \{ A, B \}$; z.B. $\theta = (A < B)$.

Für ein Tupel t mit $def(\theta) \subseteq def(t)$ gilt genau dann $\theta(t) = \text{true}$, wenn $t(A) \odot t(B)$.

- $\theta := A \odot a$, mit $A \in U$, $a \in dom(A)$, $\odot \in \mathcal{OP}$, ist ein Suchprädikat mit $def(\theta) = \{ A \}$; z.B. $\theta = (A = 1)$.

Für ein Tupel t mit $def(\theta) \subseteq def(t)$ gilt genau dann $\theta(t) = \text{true}$, wenn $t(A) \odot a$.

- Sind $\theta, \theta_1, \theta_2$ Suchprädikate, dann auch

$$\theta_1 \text{ AND } \theta_2, \quad \theta_1 \text{ OR } \theta_2, \quad \text{NOT } \theta,$$

und es gilt $\text{def}(\theta_1 \text{ AND } \theta_2) = \text{def}(\theta_1 \text{ OR } \theta_2) = \text{def}(\theta_1) \cup \text{def}(\theta_2)$,
 $\text{def}(\text{NOT } \theta) = \text{def}(\theta)$; z.B. $\theta = (A > 4) \text{ AND } (B \neq 1)$.

Für ein Tupel t mit $\text{def}(\theta_1) \cup \text{def}(\theta_2) \subseteq \text{def}(t)$ gilt

$$(\theta_1 \text{ AND } \theta_2)(t) = \theta_1(t) \text{ AND } \theta_2(t),$$

$$(\theta_1 \text{ OR } \theta_2)(t) = \theta_1(t) \text{ OR } \theta_2(t).$$

Für ein Tupel t mit $\text{def}(\theta) \subseteq \text{def}(t)$ gilt

$$(\text{NOT } \theta)(t) = \text{NOT } \theta(t).$$

Für $\theta = (A \leq B) \text{ AND } (C \neq 1)$ und $\{A, B, C\} \subseteq \text{def}(t)$ gilt

$$\theta(t) = (t(A) \leq t(B)) \text{ AND } (t(C) \neq 1).$$

Der Definitionsbereich $\text{def}(\theta)$ eines Suchprädikats θ ist also immer die Menge aller in θ vorkommenden Attribute $A \in U$.

spezifische relationale Operatoren

Sie $\mathcal{R}$ die Menge aller Relationen über einer Attributmenge $V \subseteq U$.

Selektion: $\sigma_\theta : \mathcal{R} \longrightarrow \mathcal{R}$

Falls $\text{def}(\theta) \subseteq \text{def}(r)$, so enthält $\sigma_\theta(r)$ alle Tupel aus r , welche das Suchprädikat θ erfüllen:

$$\sigma_\theta(r) := \{ t \in r \mid \theta(t) = \text{true} \}.$$

Berechnung: $|r|$ Schritte

Projektion: $\pi_V : \mathcal{R} \longrightarrow \mathcal{R}$

Falls $V \subseteq \text{def}(r)$, so erhalten wir $\pi_V(r)$ durch Projektion aller Tupel aus r auf V :

$$\pi_V(r) := \{ t[V] \mid t \in r \}.$$

Berechnung: $|r|$ Schritte

Selektion und Projektion:

r				$\sigma_{A < B}(r)$			$\Pi_A(r)$	
A	B			A	B		A	
1	2	$\Rightarrow$		1	2		1	
4	1			5	6		4	
4	3						5	
5	6							

Bei der Selektion werden Zeilen der Tabelle weggelassen,
bei der Projektion Spalten.

Relationen sind – anders als in SQL – Mengen von Tupeln.

Im obigen Beispiel hat das zweite Tupel (4) in der Projektion $\Pi_A(r)$ zwei Ursachen in r , nämlich (4, 1) und (4, 3); es tritt aber in der Projektion nur einmal auf.

Verbund (Join, Theta-Join): $\bowtie_{\theta}: \mathcal{R} \times \mathcal{R} \longrightarrow \mathcal{R}$

Sei θ ein Suchprädikat, $V_i = \text{def}(r_i)$ und $V = V_1 \cup V_2$ die Vereinigung mit Duplikaten.

Falls $\text{def}(\theta) \subseteq V$, so ist $r_1 \bowtie_{\theta} r_2$ die Relation über V mit allen Tupeln t des kartesischen Produkts $r_1 \times r_2$, welche θ erfüllen:

$$r_1 \bowtie_{\theta} r_2 := \{ t \in r_1 \times r_2 \mid \theta(t) = \text{true} \} = \sigma_{\theta}(r_1 \times r_2).$$

bei lauter Gleichheitsoperatoren in θ : *Equi-Join*.

Berechnung (Nested Loop-Join):

- Finde zu jedem Tupel $t_1 \in r_1$ alle Tupel $t_2 \in r_2$, so daß $t = t_1 \times t_2$ das Suchprädikat θ erfüllt. $\longrightarrow |r_1| \cdot |r_2|$ Schritte
 - Ein Equi-Join über eine Fremdschlüssel/Schlüssel-Beziehung kann den Index über r_2 verwenden. $\longrightarrow |r_1| \cdot \log_2 |r_2|$ Schritte
- Beispiel: $\text{EMPLOYEE} \bowtie_{\text{DNO}=\text{DNUMBER}} \text{DEPARTMENT}$

Equi-Join:

r		s		$\Rightarrow$	$r \bowtie_{B=C} s$			
A	B	C	D		A	B	C	D
1	2	2	5		1	2	2	5
3	4	2	6		1	2	2	6
5	6	3	7		3	4	4	8
		4	8					

Das kartesische Produkt von r und s hat $3 \cdot 4 = 12$ Tupel.

Nur 3 davon kommen in den Equi-Join, denn dafür muß $B = C$ sein.

Natural Join: Join über gemeinsame Attribute

Sie $V_i = \text{def}(r_i)$ und $V = V_1 \cup V_2$ die Vereinigung ohne Duplikate.

Der Natural Join $r_1 \bowtie r_2$ ist ein Equi-Join mit Gleichheitsprädikaten über $V_1 \cap V_2$ gefolgt von einer Projektion zur Elimination doppelter Attribute.

Zwei Tupel $t_i \in r_i$ werden zu einem Tupel $t \in r_1 \bowtie r_2$ verbunden, falls sie auf den gemeinsamen Attributen übereinstimmen, d.h. falls

$t_1[V_1 \cap V_2] = t_2[V_1 \cap V_2] :$

$$t(A) = \begin{cases} t_1(A), & \text{für } A \in V_1 \setminus V_2, \\ t_2(A), & \text{für } A \in V_2 \setminus V_1, \\ t_1(A) = t_2(A), & \text{für } A \in V_1 \cap V_2. \end{cases}$$

Der Natural Join entspricht einem Equi-Join mit Projektion:

$$r \bowtie s' = \Pi_{A,r.B,D}(r \bowtie_{r.B=s'.B} s').$$

r		s'		$r \bowtie_{r.B=s'.B} s'$				$r \bowtie s'$		
A	B	B	D	A	B	B	D	A	B	D
1	2	2	5	1	2	2	5	1	2	5
3	4	2	6	1	2	2	6	1	2	6
5	6	3	7	3	4	4	8	3	4	8
		4	8							

Der Equi-Join $r \bowtie_{r.B=s'.B} s'$ ohne Projektion würde hingegen eine Relation erzeugen, in der das Attribut B doppelt vorkommt.

Wir können kurz $\Pi_A(r)$ für $\Pi_{\{A\}}(r)$ und $\Pi_{A_1,\dots,A_n}(r)$ für $\Pi_{\{A_1,\dots,A_n\}}(r)$ schreiben.

Outer Join:

Tupel aus r bzw. s , die keinen Join-Partner finden, werden durch NULL-Werte aufgefüllt.

A	B	C	D
1	2	2	5
1	2	2	6
3	4	4	8
NULL	NULL	3	7
5	6	NULL	NULL

Durch Projektion des Outer Join u auf die beiden Attributmengen V_r bzw. V_s und Elimination der reinen NULL-Tupel erhält man wieder die ursprünglichen Relationen.

$$r = \sigma_{A \neq \text{NULL} \text{ AND } B \neq \text{NULL}}(\Pi_{A,B}(u)),$$

$$s = \sigma_{C \neq \text{NULL} \text{ AND } D \neq \text{NULL}}(\Pi_{C,D}(u)).$$

Division (Faktorisierung): $\div : \mathcal{R} \times \mathcal{R} \longrightarrow \mathcal{R}$

Falls $\text{def}(s) \subseteq \text{def}(r)$, so definieren wir $r \div s$ als die maximale Relation über $\text{def}(r) \setminus \text{def}(s)$, deren kartesisches Produkt mit s in r enthalten ist:
 $(r \div s) \times s \subseteq r$ (vgl. ganzzahlige Division).

Mathematische Formalisierung (Exkurs):

$$r \div s := \{ t \mid \text{def}(t) = \text{def}(r) \setminus \text{def}(s) \wedge \forall u \in s : t \times u \in r \},$$

wobei $(t_1, \dots, t_n) \times (s_1, \dots, s_m) = (t_1, \dots, t_n, s_1, \dots, s_m)$.

Für $r, s \in \mathbb{N}_+$ ist $t = r \div s$ die maximale natürliche Zahl, so daß $t \times s \leq r$.

Umbenennung (Renaming): $\varrho_\Theta : \mathcal{R} \longrightarrow \mathcal{R}$

$\Theta = \{ A_1 \mapsto B_1, \dots, A_k \mapsto B_k \}$ ist eine Umbenennung der Attribute. Falls $V = \{ A_1, \dots, A_k \} \subseteq \text{def}(r)$, so erhält man die Relation $\varrho_\Theta(r)$, indem jedes Attribut A_i in B_i umbenannt wird; Attribute A von r , mit $A \notin V$, bleiben unverändert.

$$r = \begin{array}{|c|c|c|} \hline A & B & C \\ \hline 7 & 1 & 2 \\ \hline 7 & 3 & 4 \\ \hline 9 & 1 & 2 \\ \hline \end{array} \quad s = \begin{array}{|c|c|} \hline B & C \\ \hline 1 & 2 \\ \hline 3 & 4 \\ \hline \end{array} \Rightarrow r \div s = \begin{array}{|c|} \hline A \\ \hline 7 \\ \hline \end{array}$$

$$r = \begin{array}{|c|c|c|} \hline A & B & C \\ \hline 7 & 1 & 2 \\ \hline 7 & 3 & 4 \\ \hline 9 & 1 & 2 \\ \hline 9 & 3 & 4 \\ \hline \end{array} \quad s = \begin{array}{|c|c|} \hline B & C \\ \hline 1 & 2 \\ \hline 3 & 4 \\ \hline \end{array} \Rightarrow r \div s = \begin{array}{|c|} \hline A \\ \hline 7 \\ \hline 9 \\ \hline \end{array}$$

Für ganze Zahlen gilt z.B. $3 \div 2 = 1$ und $4 \div 2 = 2$.

Beispiel (COMPANY)

Finde die Namen aller Angestellten heraus, die an allen Projekten mitarbeiten, welche von Abteilung Nummer 5 kontrolliert werden:

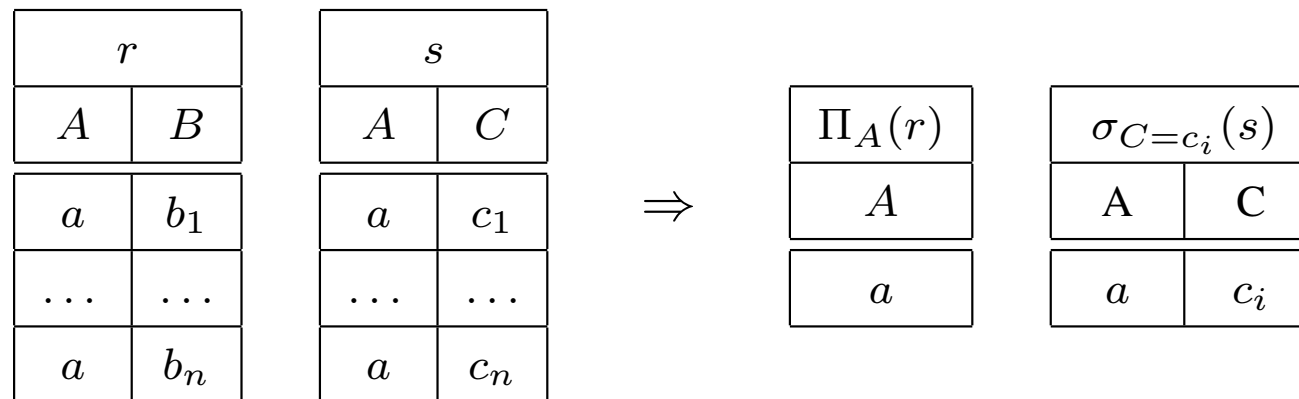
$$\begin{aligned}
 r_1 &= \varrho_{\{PNUMBER \mapsto PNO\}} (\Pi_{PNUMBER}(\sigma_{DNUM=5}(\text{PROJECT}))), \\
 r_2 &= \Pi_{ESSN,PNO}(\text{WORKS_ON}), \\
 r_3 &= \Pi_{FNAME,LNAME}(\text{EMPLOYEE} \bowtie_{SSN=ESSN} (r_2 \div r_1)).
 \end{aligned}$$

<table><tr><th>r_1</th></tr><tr><th>PNO</th></tr><tr><td>1</td></tr><tr><td>2</td></tr><tr><td>3</td></tr></table>	r_1	PNO	1	2	3	<table><tr><th colspan="2">r_2</th></tr><tr><th>ESSN</th><th>PNO</th></tr><tr><td>222222222</td><td>2</td></tr><tr><td>444444444</td><td>1</td></tr><tr><td>444444444</td><td>2</td></tr><tr><td>444444444</td><td>3</td></tr><tr><td>555555555</td><td>3</td></tr><tr><td>666666666</td><td>1</td></tr><tr><td>666666666</td><td>2</td></tr><tr><td>666666666</td><td>3</td></tr><tr><td>666666666</td><td>10</td></tr></table>	r_2		ESSN	PNO	222222222	2	444444444	1	444444444	2	444444444	3	555555555	3	666666666	1	666666666	2	666666666	3	666666666	10	<table><tr><th colspan="2">$r_2 \div r_1$</th></tr><tr><th>ESSN</th><td></td></tr><tr><td>444444444</td><td></td></tr><tr><td>666666666</td><td></td></tr></table>	$r_2 \div r_1$		ESSN		444444444		666666666		<table><tr><th colspan="2">r_3</th></tr><tr><th>FNAME</th><th>LNAME</th></tr><tr><td>John</td><td>Smith</td></tr><tr><td>Joyce</td><td>English</td></tr></table>	r_3		FNAME	LNAME	John	Smith	Joyce	English
	r_1																																													
	PNO																																													
	1																																													
	2																																													
	3																																													
	r_2																																													
	ESSN	PNO																																												
	222222222	2																																												
	444444444	1																																												
444444444	2																																													
444444444	3																																													
555555555	3																																													
666666666	1																																													
666666666	2																																													
666666666	3																																													
666666666	10																																													
$r_2 \div r_1$																																														
ESSN																																														
444444444																																														
666666666																																														
r_3																																														
FNAME	LNAME																																													
John	Smith																																													
Joyce	English																																													

Der Join zur Berechnung von r_3 könnte nach Umbenennung von $r_2 \div r_1$ mittels $\{ESSN \mapsto SSN\}$ auch als Natural Join geschrieben werden.

Einfache Optimierung in der relationalen Algebra

Selektion und Projektion sollten in Ausdrücken der relationalen Algebra frühzeitig angewendet werden, da sie die Anzahl der Tupel einer Relation unter Umständen deutlich verkleinern können:



Der Join $r \bowtie s$ enthält n^2 Tupel, aber

$$s' = \sigma_{C=c_i}(\Pi_{A,C}(r \bowtie s)) = \Pi_{A,C}(r \bowtie \sigma_{C=c_i}(s)) = \Pi_A(r) \bowtie \sigma_{C=c_i}(s)$$

kann aus $\Pi_A(r)$ und $\sigma_{C=c_i}(s)$ berechnet werden, ohne daß man $r \bowtie s$ berechnen muß. Hier enthält s' nur 1 Tupel, denn $s' = \sigma_{C=c_i}(s)$.

2.3 Die relationale Datenbanksprache SQL

Relationenalgebra

Der Benutzer muß beim Formulieren der Anfrage angeben, auf welche Weise das gewünschte Resultat berechnet werden soll.

Deklarative High-Level-Sprachschnittstellen

Der Benutzer gibt auf sehr abstrakte Weise an, wie das gewünschte Resultat der Anfrage aussehen soll. Der Ausführungsplan inklusive der Anfrageoptimierung wird vom System erstellt.

ANSI-Standards

SQL1: 1986, SQL2: 1992

SQL3, SQL:1999 (objektorientierte Erweiterungen)

SQL:2003, SQL/XML:2006

2.3.1 Datendefinition in SQL

Im Großen und Ganzen kann man einen ER–Entwurf in SQL mittels **CREATE TABLE**–Statements umsetzen:

1. Für jedes Attribut einer Tabelle wird auch ein Datentyp angegeben, und möglicherweise eine NOT NULL–Bedingung und ein DEFAULT–Wert.
2. Eine Attributkombination K wird mittels **PRIMARY KEY** (K) zum primären Schlüssel mit automatischer NOT NULL–Bedingung gemacht. Nach diesem werden die Datensätze auf dem Sekundärspeicher organisiert, und es wird automatisch ein Index angelegt.
3. Weitere Attributkombinationen X können mittels **UNIQUE** (X) zu sekundären Schlüsseln gemacht werden. NOT NULL–Bedingungen müssen hierfür bei Bedarf explizit angegeben werden.

4. Fremdschlüsselbedingungen von einer Attributkombination X auf einen Schlüssel K einer Tabelle T werden mittels

FOREIGN KEY (X) REFERENCES $T(K)$

vereinbart.

Erzeugen einer Datenbank

Vor dem erste **CREATE TABLE**–Statement muß man die Datenbank erzeugen und eine Verbindung damit aufbauen:

```
CREATE DATABASE COMPANY;  
USE COMPANY;
```

Alle folgenden SQL–Statements richten sich dann an diese Datenbank.

Erzeugen einer Tabelle

```
CREATE TABLE EMPLOYEE (  
    FNAME      VARCHAR(15) NOT NULL,  
    MINIT      CHAR,  
    LNAME      VARCHAR(15) NOT NULL,  
    SSN        CHAR(9)      NOT NULL,  
    BDATE      DATE,  
    ADDRESS    VARCHAR(30) ,  
    SEX        CHAR,  
    SALARY     DECIMAL(10,2) ,  
    SUPERSSN   CHAR(9) ,  
    DNO        INT          NOT NULL,
```

```
PRIMARY KEY (SSN) ,  
FOREIGN KEY (SUPERSSN)  
    REFERENCES EMPLOYEE (SSN) ,  
FOREIGN KEY (DNO)  
    REFERENCES DEPARTMENT (DNUMBER) ) ;
```

Durch die NOT NULL–Bedingung für DNO wird die Existenzabhängigkeit der Angestellten von den Abteilungen aus WORKS_FOR realisiert.

Die umgekehrte Existenzabhängigkeit kann im CREATE TABLE–Statement nicht realisiert werden.

Mittels UNIQUE (FNAME, MINIT, LNAME, BDATE) könnte man die Kombination aus dem Namen und dem Geburtsdatum zu einem weiteren, sekundären Schlüssel machen.

Syntax

```
CREATE TABLE table-name (  
    column-name type [ NOT NULL | UNIQUE ]  
        [ DEFAULT { literal | NULL } ]  
    [, column-name ... ]  
    [, UNIQUE ( list-of-column-names ) ... ]  
    [, PRIMARY KEY ( list-of-column-names ) ]  
    [, FOREIGN KEY ( list-of-column-names )  
        REFERENCES table-name  
            [ ( list-of-column-names ) ] ... ]  
    [, CHECK ( condition ) ... ] )
```

Fremdschlüsselbedingung mit DELETE- und UPDATE-Modus:

```
FOREIGN KEY ( list-of-column-names )  
REFERENCES table-name  
    [ ( list-of-column-names ) ]  
    [ MATCH { FULL | PARTIAL } ]  
    [ ON DELETE  
        { NO ACTION | CASCADE |  
          SET DEFAULT | SET NULL } ]  
    [ ON UPDATE  
        { NO ACTION | CASCADE |  
          SET DEFAULT | SET NULL } ]
```


Datentypen

1. numerisch:

INT[EGER], SMALLINT

FLOAT, REAL, DOUBLE PRECISION

NUMERIC(I,J), DEC[IMAL](I,J)

precision: I, scale: J (default: J=0),

Beispiel: 1.43 $\Rightarrow$ I=3, J=2

2. Character String:

CHAR[ACTER](N): feste Länge N, der Default ist N=1

VARCHAR(N), CHAR[ACTER] VARYING(N): N ist das Maximum

3. Bit String:

BIT(N) (default: N=1)

BIT VARYING(N)

4. Datum und Zeit:

DATE:	YYYY-MM-DD
TIME:	HH:MM:SS
TIME(I):	HH:MM:SS:F ₁ . . . F _I
TIME WITH TIME ZONE:	08:32:16+01:00
TIMESTAMP:	DATE TIME(6)
TIMESTAMP WITH TIME ZONE	
INTERVAL:	YYYY-MM oder DD TIME

Character Strings sowie Datum und Zeit werden mit Hochkommata angegeben (z.B.: 'John', 'Smith', '1955-01-09'), Zahlenwerte ohne Hochkommata (z.B.: 30000, 5).

Das Erstellen der kompletten COMPANY-Datenbank

```
CREATE TABLE DEPARTMENT (  
    DNAME          VARCHAR(15) NOT NULL,  
    DNUMBER        INT          NOT NULL,  
    MGRSSN         CHAR(9)      NOT NULL,  
    MGRSTARTDATE   DATE,  
    PRIMARY KEY (DNUMBER) ,  
    UNIQUE (DNAME) ,  
    FOREIGN KEY (MGRSSN)  
        REFERENCES EMPLOYEE (SSN) ) ;
```

Durch die NOT NULL-Bedingung für MGRSSN wird die Existenzabhängigkeit aus MANAGES realisiert, die besagt, daß eine Abteilung genau einen Manager haben muß.

```
CREATE TABLE DEPT_LOCATIONS (  
    DNUMBER      INT                NOT NULL,  
    DLOCATION     VARCHAR(15) NOT NULL,  
    PRIMARY KEY  (DNUMBER, DLOCATION) ,  
    FOREIGN KEY  (DNUMBER)  
        REFERENCES DEPARTMENT (DNUMBER)  
        ON DELETE CASCADE ON UPDATE CASCADE );
```

Die Fremdschlüsselbedingung ist mit einem Trigger verbunden, der bei Löschungen und Updates in DEPARTMENT aktiv wird.

```
CREATE TABLE PROJECT (  
    PNAME          VARCHAR(15) NOT NULL,  
    PNUMBER        INT          NOT NULL,  
    PLOCATION       VARCHAR(15) ,  
    DNUM           INT          NOT NULL,  
    PRIMARY KEY (PNUMBER) ,  
    UNIQUE (PNAME) ,  
    FOREIGN KEY (DNUM)  
        REFERENCES DEPARTMENT (DNUMBER) ) ;
```

Durch die NOT NULL–Bedingung für DNUM wird die Existenzabhängigkeit aus CONTROLS realisiert, die besagt, daß ein Projekt genau eine verantwortliche Abteilung haben muß.

```
CREATE TABLE WORKS_ON (  
    ESSN    CHAR(9)          NOT NULL,  
    PNO     INT              NOT NULL,  
    HOURS   DECIMAL(3,1)    NOT NULL,  
    PRIMARY KEY (ESSN, PNO) ,  
    FOREIGN KEY (ESSN)  
        REFERENCES EMPLOYEE (SSN) ,  
    FOREIGN KEY (PNO)  
        REFERENCES PROJECT (PNUMBER) ) ;
```

Die beiden Existenzabhängigkeiten aus WORKS_ON können in den CREATE TABLE–Statements nicht realisiert werden.

```
CREATE TABLE DEPENDENT (  
    ESSN          CHAR(9) NOT NULL,  
    DEPENDENT_NAME VARCHAR(15) NOT NULL,  
    SEX           CHAR,  
    BDATE         DATE,  
    RELATIONSHIP  VARCHAR(8),  
    PRIMARY KEY (ESSN, DEPENDENT_NAME),  
    FOREIGN KEY (ESSN)  
        REFERENCES EMPLOYEE(SSN) );
```

Namen und Trigger für Constraints

```
CREATE TABLE EMPLOYEE (  
    ...  
    DNO INT NOT NULL DEFAULT 1,  
    CONSTRAINT EMPPK PRIMARY KEY (SSN),  
    CONSTRAINT EMPSUPERFK FOREIGN KEY (SUPERSSN)  
        REFERENCES EMPLOYEE (SSN)  
        ON DELETE SET NULL ON UPDATE CASCADE,  
    CONSTRAINT EMPDEPTFK FOREIGN KEY (DNO)  
        REFERENCES DEPARTMENT (DNUMBER)  
        ON DELETE SET DEFAULT ON UPDATE CASCADE );
```

Beim Löschen einer Abteilung wird die DNO der zugehörigen Angestellten auf den Default-Wert 1 gesetzt, beim Update der DNUMBER in DEPARTMENT wird die DNO der zugehörigen Angestellten aktualisiert.


```
CREATE TABLE DEPARTMENT (  
    ...  
    MGRSSN CHAR(9) NOT NULL DEFAULT '111111111',  
    CONSTRAINT DEPTPK PRIMARY KEY (DNUMBER),  
    CONSTRAINT DEPTSK UNIQUE (DNAME),  
    CONSTRAINT DEPTMGRFK FOREIGN KEY (MGRSSN)  
        REFERENCES EMPLOYEE (SSN)  
        ON DELETE SET DEFAULT ON UPDATE CASCADE );
```

Man kann später bei Änderungen an den Tabellenstrukturen über die Namen auf die Constraints zugreifen.

Typ-Definition

```
CREATE DOMAIN SSN_TYPE AS CHAR(9);
```

Schema-Modifikationen

```
DROP DATABASE COMPANY  
    CASCADE  
    RESTRICTED (nur für leeres Schema)
```

```
DROP TABLE DEPENDENT  
    CASCADE  
    RESTRICTED (nur für nicht referenzierte Tabellen)
```

```
ALTER TABLE table-name
{ ADD [ COLUMN ] column-name data-type
| ALTER [ COLUMN ] column-name
    { SET default-definition | DROP DEFAULT }
| DROP [ COLUMN ] column-name
| ADD [ CONSTRAINT constraint-name ]
    { { PRIMARY KEY | UNIQUE }
      ( list-of-column-names )
    | FOREIGN KEY ( list-of-column-names )
      REFERENCES ...
    | CHECK ( condition ) }
| DROP CONSTRAINT constraint-name }
```

Beispiel

```
ALTER TABLE EMPLOYEE  
  ADD JOB VARCHAR(12) ;
```

```
ALTER TABLE EMPLOYEE  
  DROP ADDRESS CASCADE ;
```

```
ALTER TABLE EMPLOYEE  
  DROP CONSTRAINT EMPSUPERFK CASCADE ;
```

```
ALTER TABLE DEPARTMENT  
  ALTER MGRSSN DROP DEFAULT ;
```

```
ALTER TABLE DEPARTMENT  
  ALTER MGRSSN SET DEFAULT '222222222' ;
```

2.3.2 Anfragen in SQL

SELECT-FROM-WHERE-Block:

Verglichen mit der *Relationenalgebra* realisiert der SELECT-Teil eine Projektion auf die interessierenden Attribute, der FROM-Teil realisiert den Join der genannten Tabellen, und der WHERE-Teil enthält die Selektions- und Join-Bedingungen.

SELECT	<attribute list>	→ Resultatsattribute
FROM	<table list>	→ angefragte Tabellen
WHERE	<condition>	→ Bedingung an die Resultatstupel

Joins erfolgen oft über *Schlüssel/Fremdschlüssel*-Beziehungen; dies hilft beim Finden geeigneter Join-Bedingungen. Außerdem können die Join-Partner zu Tupeln aus der Tabelle mit dem Fremdschlüssel in der Tabelle mit dem Schlüssel sehr schnell gefunden werden.

Anfrage 0 (Selektion und Projektion)

Finde das Geburtsdatum und die Adresse des Angestellten mit dem Namen “John B. Smith” heraus.

```
SELECT  BDATE, ADDRESS
FROM    EMPLOYEE
WHERE    FNAME='John' AND MINIT='B' AND LNAME='Smith'
```

Ergebnis:

BDATE	ADDRESS
1955-01-09	731 Fondren, Houston, TX

In der Relationenalgebra:

$$\Pi_{\text{BDATE, ADDRESS}}(\sigma_{\text{FNAME}='John' \wedge \text{MINIT}='B' \wedge \text{LNAME}='Smith'}(\text{EMPLOYEE})).$$

Anfrage (Projektion und Join)

Bestimme für alle Angestellten – gegeben durch Vornamen und Nachnamen – ihre Projekte – gegeben durch die Projektnamen – und die Anzahl der daran gearbeiteten Stunden.

```
SELECT  FNAME, LNAME, PNAME, HOURS
FROM    EMPLOYEE, WORKS_ON, PROJECT
WHERE   SSN=ESSN AND PNO=PNUMBER
```

Jeder Tabellennamen im **FROM**-Teil steht für ein Tupel.

EMPLOYEE				
FNAME	...	LNAME	SSN	...

WORKS_ON		
ESSN	PNO	HOURS

PROJECT		
PNAME	PNUMBER	...

In der Relationenalgebra:

$$\Pi_{\text{FNAME, LNAME, PNAME, HOURS}} ($$

$$\text{EMPLOYEE} \bowtie_{\text{SSN} = \text{ESSN}}$$

$$\text{WORKS_ON} \bowtie_{\text{PNO} = \text{PNUMBER}} \text{PROJECT}).$$

Aus der Tupelsicht

EMPLOYEE				
FNAME	...	LNAME	SSN	...

WORKS_ON		
ESSN	PNO	HOURS

PROJECT		
PNAME	PNUMBER	...

könnte man auch eine entsprechende DATALOG–Sicht gewinnen:

```
e_works_on_p(FNAME, LNAME, PNAME, HOURS) :-  
  employee(FNAME, ..., LNAME, SSN, ...),  
  works_on(SSN, PNO, HOURS),  
  project(PNAME, PNO, ...).
```

Die Join–Bedingungen werden durch gleiche Variablen ausgedrückt.

Für jedes Tripel von Tupeln, welches die **WHERE**-Bedingung erfüllt, kann man durch Projektion auf die Attribute aus dem **SELECT**-Teil ein Resultatstupel bilden.

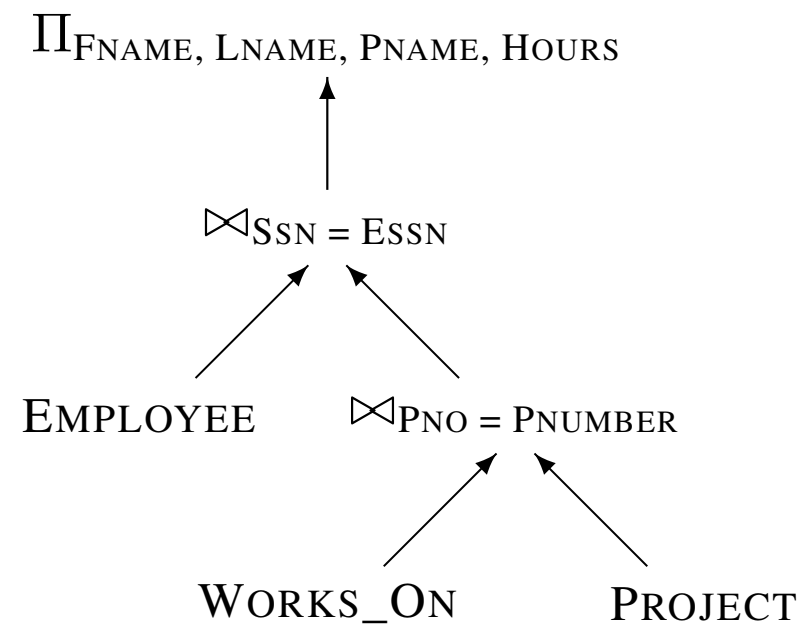
FNAME	LNAME	PNAME	HOURS
John	Smith	ProductX	32.5
John	Smith	ProductY	7.5
Ramesh	Narayan	ProductZ	40.0
Joyce	English	ProductX	20.0
Joyce	English	ProductY	20.0
Franklin	Wong	ProductY	10.0
Franklin	Wong	ProductZ	10.0
Alicia	Zelaya	Newbenefits	30.0
Alicia	Zelaya	Computerization	10.0
Ahmad	Jabbar	Computerization	35.5
Ahmad	Jabbar	Newbenefits	5.0
Jennifer	Wallace	Newbenefits	20.0
Jennifer	Wallace	Reorganization	15.0
James	Borg	Reorganization	NULL

Mit binären Joins in der Relationenalgebra:

$$\Pi_{\text{FNAME, LNAME, PNAME, HOURS}} \left(\text{EMPLOYEE} \bowtie_{\text{SSN} = \text{ESSN}} \left(\text{WORKS_ON} \bowtie_{\text{PNO} = \text{PNUMBER}} \text{PROJECT} \right) \right).$$

Zugehöriger Operatorbaum:

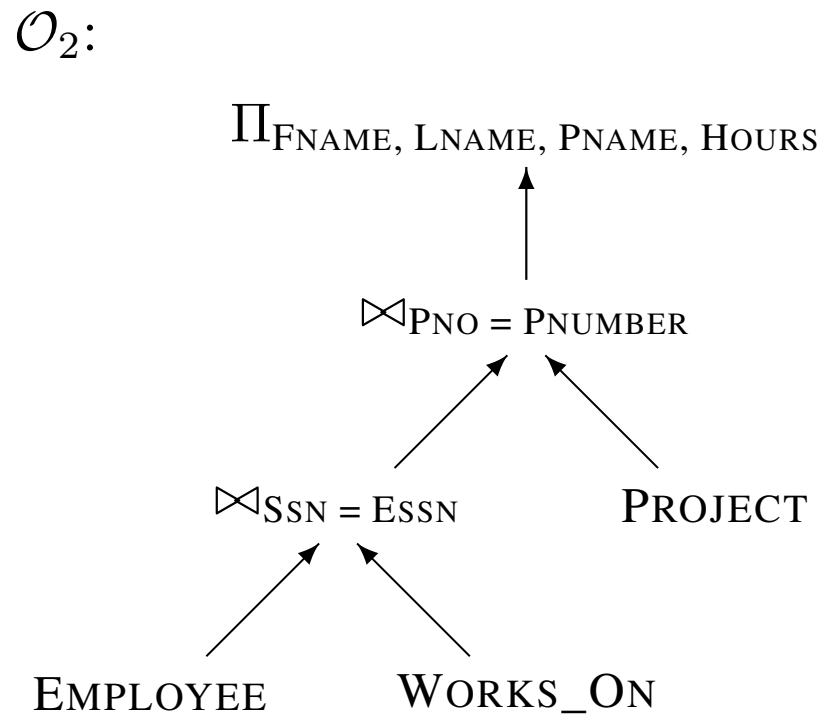
$\mathcal{O}_1$:



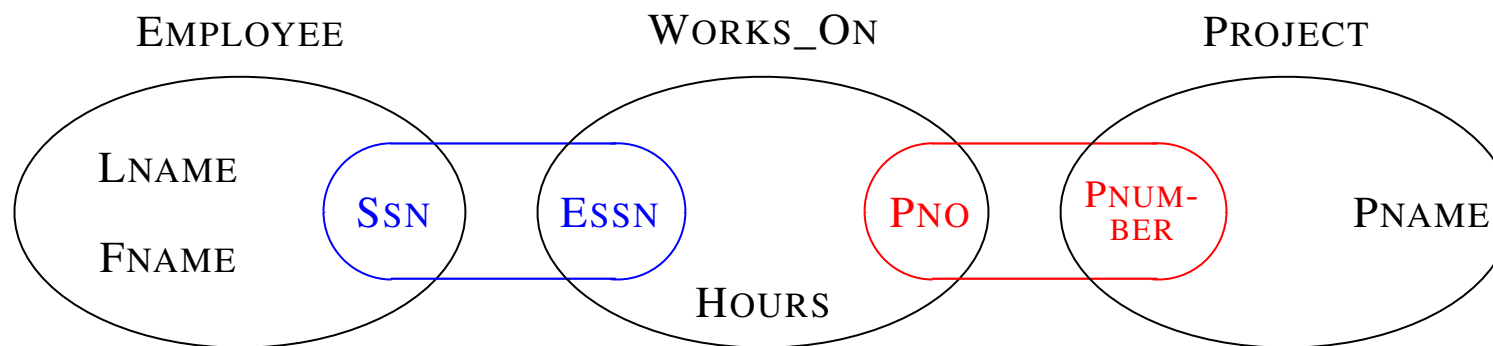
Mit binären Joins in der Relationenalgebra:

$$\Pi_{\text{FNAME, LNAME, PNAME, HOURS}} \left(\left(\text{EMPLOYEE} \bowtie_{\text{SSN} = \text{ESSN}} \text{WORKS_ON} \right) \bowtie_{\text{PNO} = \text{PNUMBER}} \text{PROJECT} \right).$$

Zugehöriger Operatorbaum:



Das folgende *Mengendiagramm* zeigt die Attributmengen der beteiligten Relationen. Es ist angereichert um die Join-Bedingungen (in blau und rot). Es sind nur die Projektions- bzw. Join-Attribute angezeigt.



Der Join sollte immer von der zentralen Relation WORKS_ON ausgehen. Es ist sinnlos, zuerst EMPLOYEE und PROJECT zu joinen, denn dies wäre ein kartesisches Produkt.

Hier erscheint auch ein *ternärer* (Drei-Wege-) Join sinnvoll, bei dem in einem Durchlauf für jedes Tupel aus WORKS_ON indexunterstützt die passenden Partner aus den anderen beiden Relationen gesucht werden.

Anfrage 1 (Selektion, Projektion und Join)

Finde die Namen und Adressen aller Angestellten heraus, die für die “Research”–Abteilung arbeiten.

```
SELECT  FNAME, LNAME, ADDRESS
FROM    EMPLOYEE, DEPARTMENT
WHERE   DNAME='Research' AND DNUMBER=DNO
```

Ergebnis:

FNAME	LNAME	ADDRESS
John	Smith	731 Fondren, Houston, TX
Franklin	Wong	638 Voss, Houston, TX
Ramesh	Narayan	975 Rice, Houston, TX
Joyce	English	5631 Rice, Houston, TX

In der Relationenalgebra:

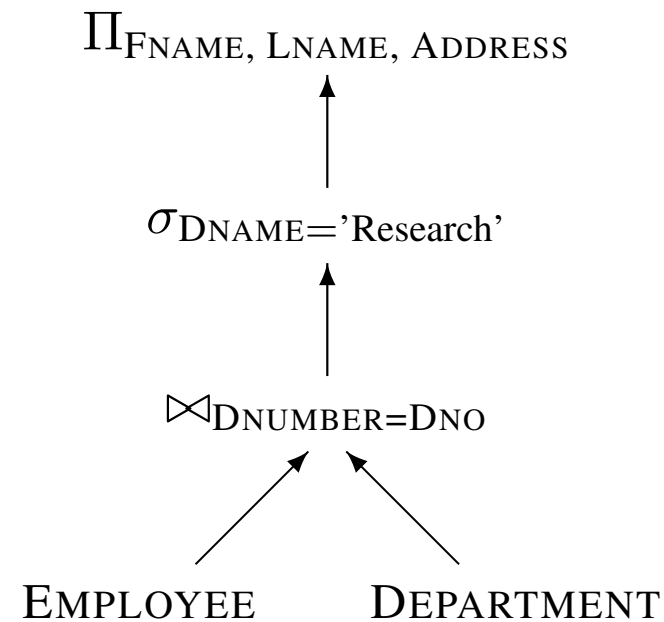
$$\Pi_{\text{FNAME, LNAME, ADDRESS}} \left(\sigma_{\text{DNAME='Research'}} \left(\text{EMPLOYEE} \bowtie_{\text{DNUMBER=DNO}} \text{DEPARTMENT} \right) \right).$$

In der Relationenalgebra:

$$\Pi_{\text{FNAME, LNAME, ADDRESS}} \left(\sigma_{\text{DNAME}=\text{'Research'}} \left(\text{EMPLOYEE} \bowtie_{\text{DNUMBER=DNO}} \text{DEPARTMENT} \right) \right).$$

Zugehöriger Operatorbaum:

$\mathcal{O}_1$:

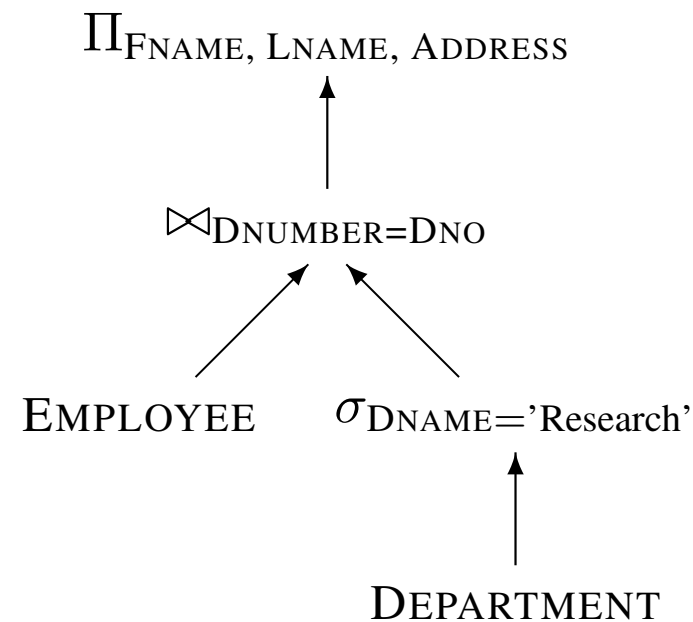


Anfrage–Optimierung: Push Selection into Join

$\Pi_{\text{FNAME}, \text{LNAME}, \text{ADDRESS}}(\text{EMPLOYEE} \bowtie_{\text{DNUMBER}=\text{DNO}} \sigma_{\text{DNAME}='Research'}(\text{DEPARTMENT}))$.

Zugehöriger Operatorbaum:

$\mathcal{O}_2$:



Anfrage 2

Liste die Projektnummer, die Nummer der zugehörigen Abteilung sowie den Nachnamen, die Adresse und das Geburtsdatum des Abteilungsleiters für alle Projekte, die in Stafford angesiedelt sind.

```
SELECT  PNUMBER, DNUM, LNAME, ADDRESS, BDATE
FROM    PROJECT, DEPARTMENT, EMPLOYEE
WHERE    DNUM=DNUMBER AND MGRSSN=SSN
AND      PLOCATION='Stafford'
```

Ergebnis:

PNUMBER	DNUM	LNAME	ADDRESS	BDATE
10	4	Wallace	291 Berry, Bellaire, TX	1931-06-20
30	4	Wallace	291 Berry, Bellaire, TX	1931-06-20

Die Bezeichnungen "Anfrage Nummer"

entsprechen denen aus dem Buch von R. Elmasri und S.B. Navathe.

Zusammenhang mit der Relationenalgebra

SELECT DISTINCT eliminiert doppelte Tupel aus dem Resultat

1. **Projektion:** $\pi_{A_1, \dots, A_k}(R)$, mit $\{A_1, \dots, A_k\} \subseteq \text{def}(R)$:

SELECT DISTINCT $A_1, \dots, A_k$ **FROM** R

2. **Selektion:** $\sigma_{A=B}(R)$, mit $A, B \in \text{def}(R)$:

SELECT DISTINCT *
FROM R
WHERE A = B

3. **Vereinigung | Durchschnitt | Differenz:** $R \cup S$, $R \cap S$, $R \setminus S$

SELECT DISTINCT * FROM R

UNION | INTERSECT | EXCEPT

SELECT DISTINCT * FROM S

4. **Natural Join:** $R \bowtie S$, mit $def(R) \cap def(S) = \{B_1, \dots, B_m\}$:

$$def(R) = \{A_1, \dots, A_n, B_1, \dots, B_m\},$$

$$def(S) = \{B_1, \dots, B_m, C_1, \dots, C_l\}$$

SELECT DISTINCT $A_1, \dots, A_n, R.B_1, \dots, R.B_m, C_1, \dots, C_l$

FROM R, S

WHERE $R.B_1 = S.B_1$ AND ... AND $R.B_m = S.B_m$

Mehrdeutige Attributnamen und Aliasing

Um Mehrdeutigkeiten zu beseitigen, kann man

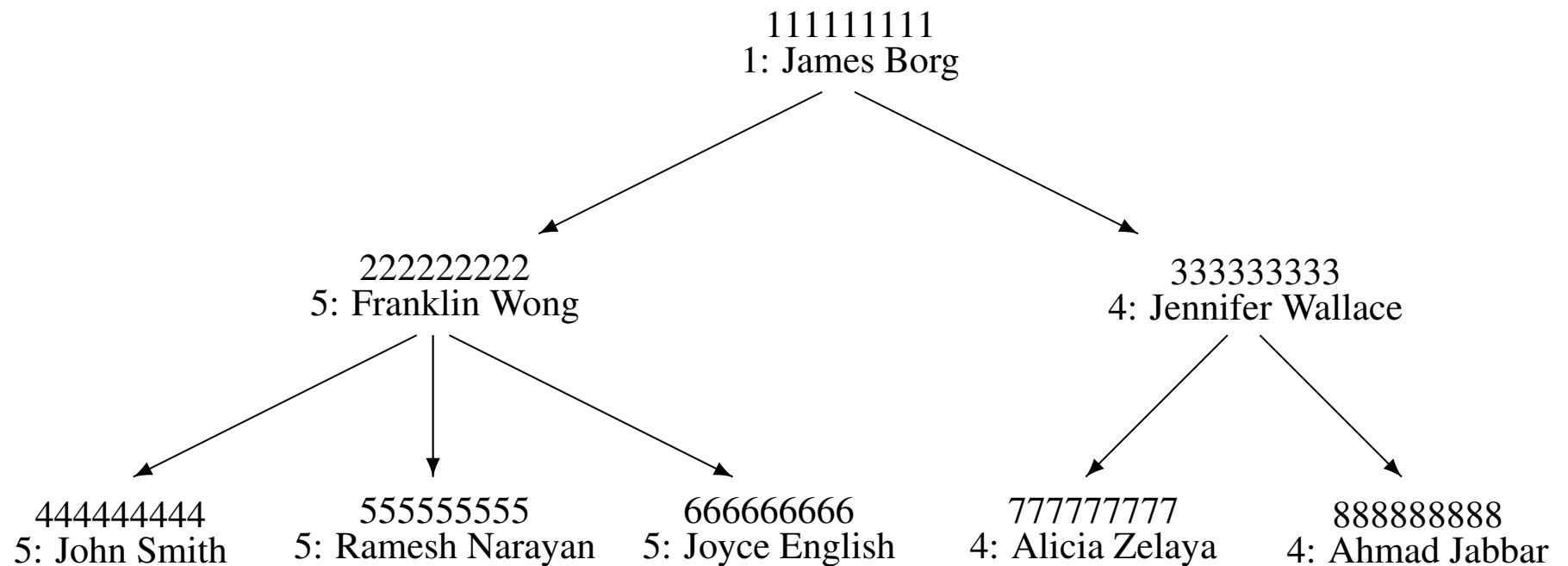
- einen Attributnamen A_Name durch Voranstellen des Relationennamens R_Name eindeutig machen ($R_Name.A_Name$) oder
- ein Alias $Alias$ für einen Relationennamen R_Name vergeben ($R_Name Alias$, z.B. EMPLOYEE E).

Beides ist z.B. erforderlich, wenn man eine Relation mit sich selbst joint.

Anfrage 8

```
SELECT  E.FNAME, E.LNAME, S.FNAME, S.LNAME
FROM    EMPLOYEE E, EMPLOYEE S
WHERE   E.SUPERSSN=S.SSN
```

Die Vorgesetzten–Hierarchie:



EMPLOYEE						
FNAME	MINIT	LNAME	SSN	...	SUPERSSN	...
John	B	Smith	444444444	...	222222222	...
Franklin	T	Wong	222222222	...	111111111	...
...	...	...	...	...	...	...

Ergebnis:

E.FNAME	E.LNAME	S.FNAME	S.LNAME
John	Smith	Franklin	Wong
Franklin	Wong	James	Borg
Alica	Zelaya	Jennifer	Wallace
Jennifer	Wallace	James	Borg
Ramesh	Narayan	Franklin	Wong
Joyce	English	Franklin	Wong
Ahmad	Jabbar	Jennifer	Wallace

Anfragen ohne WHERE-Klausel, Benutzung von '*' (Wildcard)

Anfrage 9 (Projektion)

Suche alle EMPLOYEE SSNs in der Datenbank.

```
SELECT  SSN
FROM    EMPLOYEE
```

Ergebnis:

SSN
444444444
222222222
777777777
333333333
555555555
666666666
888888888
111111111

in der Relationenalgebra: $\Pi_{SSN}(EMPLOYEE)$

Anfrage 1C (Selektion)

```
SELECT  *  
FROM    EMPLOYEE  
WHERE   DNO = 5
```

Ergebnis:

FNAME	MINIT	LNAME	SSN	BDATE	ADDRESS	SEX	SALARY	SUPERSSN	DNO
John	B	Smith	444444444	1955-01-09	. . .	M	30000	222222222	5
Franklin	T	Wong	222222222	1945-12-08	. . .	M	40000	111111111	5
Ramesh	K	Narayan	555555555	1952-09-15	. . .	M	38000	222222222	5
Joyce	A	English	666666666	1962-07-31	. . .	F	25000	222222222	5

in der Relationenalgebra: $\sigma_{DNO=5}(EMPLOYEE)$

Tabellen als Mengen

Anfrage 11

Hole das Gehalt aller Angestellten aus der Datenbank.

```
SELECT  SALARY  
  
FROM    EMPLOYEE
```

```
SELECT  DISTINCT SALARY  
  
FROM    EMPLOYEE
```

Falls es mehrere Angestellte mit dem selben Lohn gibt, so liefert das erste Statement den Lohn mehrfach zurück, während das zweite Statement die einzelnen Lohnstufen jeweils nur einmal auflistet.

Anfrage 4

Erstelle eine Liste aller Projekte, an denen ein Arbeiter mit dem Nachnamen “Smith” arbeitet, oder die einer Abteilung zugeordnet sind, deren Abteilungsleiter den Nachnamen “Smith” trägt.

```
( SELECT  PNO
    FROM    WORKS_ON, EMPLOYEE
    WHERE   ESSN=SSN AND LNAME='Smith' )

UNION

( SELECT  PNUMBER
    FROM    PROJECT, DEPARTMENT, EMPLOYEE
    WHERE   DNUM=DNUMBER
    AND     MGRSSN=SSN AND LNAME='Smith' )
```

Geschachtelte Anfragen und Mengenvergleiche

Finde die ESSN aller Angestellten, die an einem Projekt so viele Stunden arbeiten wie der Angestellte mit der ESSN='4444444444':

```
SELECT  DISTINCT ESSN
FROM    WORKS_ON
WHERE    (PNO, HOURS) IN (
            SELECT    PNO, HOURS
            FROM      WORKS_ON
            WHERE      ESSN='4444444444' );
```

Dabei bezieht sich die innere/äußere Variable ESSN auf das innere/äußere WORKS_ON.

Finde alle Angestellten, die mehr verdienen als alle Angestellten aus Abteilung 5:

```
SELECT  LNAME, FNAME
FROM    EMPLOYEE
WHERE    SALARY > ALL (
            SELECT    SALARY
            FROM      EMPLOYEE
            WHERE      DNO = 5 );
```

Auch möglich mit <, =, <=, >=, <>, SOME, ANY

Anfrage 4A

```
SELECT  DISTINCT PNUMBER
FROM    PROJECT
WHERE    PNUMBER IN (
            SELECT  PNO
            FROM    WORKS_ON, EMPLOYEE
            WHERE    ESSN = SSN AND LNAME = 'Smith' )
OR      PNUMBER IN (
            SELECT  PNUMBER
            FROM    PROJECT, DEPARTMENT, EMPLOYEE
            WHERE    DNUM = DNUMBER
            AND      MGRSSN = SSN AND LNAME='Smith' )
```

Anders als bei Anfrage 4 werden hier keine Duplikate geliefert.

Anfrage 12

Suche die Namen aller Angestellten, die einen Angehörigen mit gleichem Vornamen und Geschlecht haben.

```
SELECT  E.FNAME, E.LNAME
FROM    EMPLOYEE E
WHERE    SSN IN (
            SELECT  ESSN
            FROM    DEPENDENT
            WHERE    SSN = ESSN
            AND      FNAME = DEPENDENT_NAME
            AND      E.SEX = SEX )
```

Das Attribut SEX ohne Präfix E im inneren **SELECT**-Statement bezieht sich auf DEPENDENT.

Eine alternative Formulierung mit einem flachen SELECT–Statement wäre:

```
SELECT  E.FNAME, E.LNAME
FROM    EMPLOYEE E, DEPENDENT D
WHERE   E.SSN = D.ESSN
AND     E.FNAME = D.DEPENDENT_NAME
AND     E.SEX = D.SEX
```

Hier sind zwei Aliase, E und D, erforderlich, um das Attribut SEX korrekt den Relationen EMPLOYEE bzw. DEPENDENT zuzuordnen.

Zur Erinnerung: Die Anfragen orientieren sich an dem Buch von R. Elmasri und S.B. Navathe.

Explizite Mengen

Anfrage 13

Finde die Sozialversicherungsnummern aller Angestellten heraus, die an einem der Projekte 1, 2 oder 3 arbeiten.

```
SELECT  DISTINCT ESSN  
FROM    WORKS_ON  
WHERE    PNO IN (1, 2, 3)
```

Alternativ – aber weniger elegant – könnte man die folgende, etwas längere **WHERE**-Bedingung schreiben:

```
WHERE    PNO = 1  
OR       PNO = 2  
OR       PNO = 3
```

NULL–Werte

Anfrage 14

Finde die Namen aller Angestellten heraus, die keinen Vorgesetzten haben.

```
SELECT  FNAME, LNAME  
FROM    EMPLOYEE  
WHERE    SUPERSSN IS NULL
```

Eigentlich würde bereits “SUPERSSN = **NULL**” eindeutig anzeigen, daß auf den Nullwert getestet wird, da ja keine Hochkommata verwendet werden.

Zur zusätzlichen Unterscheidung muß man aber sogar “**IS**” anstelle von “=” schreiben.

EXISTS

Anfrage 12B

Suche die Namen aller Angestellten, die einen Angehörigen mit gleichem Vornamen und Geschlecht haben.

```
SELECT  E.FNAME, E.LNAME
FROM    EMPLOYEE E
WHERE   EXISTS (
        SELECT  *
        FROM    DEPENDENT
        WHERE   E.SSN = ESSN
        AND     E.SEX = SEX
        AND     E.FNAME = DEPENDENT_NAME )
```

Anfrage 6

Suche die Namen aller Angestellten, die keine Angehörigen haben.

```
SELECT  FNAME, LNAME
FROM    EMPLOYEE
WHERE   NOT EXISTS (
        SELECT  *
        FROM    DEPENDENT
        WHERE   SSN = ESSN )
```

Anfrage 7

Finde die Namen aller Abteilungsleiter heraus, die mindestens einen Angehörigen haben.

```
SELECT  FNAME, LNAME
FROM    EMPLOYEE
WHERE   EXISTS (
        SELECT  *
        FROM    DEPENDENT
        WHERE   SSN = ESSN )
AND     EXISTS (
        SELECT  *
        FROM    DEPARTMENT
        WHERE   SSN = MGRSSN )
```

Eine alternative Formulierung mit einem flachen SELECT-Statement, die zusätzlich noch die Vornamen der jeweiligen Angehörigen ausgibt, wäre:

```
SELECT  FNAME, LNAME, DEPENDENT_NAME
FROM    EMPLOYEE, DEPENDENT, DEPARTMENT
WHERE    SSN = ESSN
AND      SSN = MGRSSN
```

Ergebnis:

FNAME	LNAME	DEPENDENT_NAME
Franklin	Wong	Alice
Franklin	Wong	Joy
Franklin	Wong	Theodore
Jennifer	Wallace	Abner

Falls man DEPENDENT_NAME nicht mit ausgibt, so kann man mittels DISTINCT verhindern, daß derselbe Angestellte mehrmals aufgelistet wird.

Anfrage 3

Finde die Namen aller Angestellten heraus, die an allen Projekten mitarbeiten, welche von der Abteilung mit der Nummer 5 kontrolliert werden.

```
SELECT  FNAME, LNAME
FROM    EMPLOYEE
WHERE   NOT EXISTS (
            SELECT      *
            FROM        PROJECT
            WHERE        DNUM = 5
            AND          NOT EXISTS (
                SELECT      *
                FROM        WORKS_ON
                WHERE        SSN = ESSN
                AND          PNUMBER = PNO ) )
```

In SYSTEM R von IBM war früher auch der Operator **CONTAINS** möglich.

```
SELECT  FNAME, LNAME
FROM    EMPLOYEE
WHERE   ( ( SELECT  PNO
            FROM    WORKS_ON
            WHERE   SSN = ESSN )
        CONTAINS
        ( SELECT  PNUMBER
          FROM    PROJECT
          WHERE   DNUM = 5 ) )
```

Bekanntlich kann diese Anfrage in der Relationenalgebra mittels einer Division ausgewertet werden.

Umbenennung von Attributen und explizite Joins

Anfrage 8A

```
SELECT   E.LNAME As EMPLOYEE,  
           S.LNAME As SUPERVISOR  
  
FROM     EMPLOYEE As E, EMPLOYEE As S  
  
WHERE    E.SUPERSSN=S.SSN
```

Ergebnis:

EMPLOYEE	SUPERVISOR
Smith	Wong
Wong	Borg
Zelaya	Wallace
...	...

Anfrage 1A (Join)

```
SELECT  FNAME, LNAME, ADDRESS
FROM    EMPLOYEE JOIN DEPARTMENT ON DNO=DNUMBER
WHERE   DNAME='Research'
```

Die Join-Bedingung kann in den **FROM**-Teil gezogen werden.

Anfrage 1B (Natural Join)

```
SELECT  FNAME, LNAME, ADDRESS
FROM    EMPLOYEE NATURAL JOIN
        DEPARTMENT AS D(DNAME, DNO, X, Y)
WHERE   DNAME='Research'
```

Die Join-Bedingung $DNO=DNUMBER$ wird beim NATURAL JOIN in 1B dadurch realisiert, daß **DNUMBER** von **DEPARTMENT** mittels **AS** an **DNO** von **EMPLOYEE** angeglichen wird (Renaming).

Anfrage 8B (Outer Join)

Die folgende Anfrage bestimmt wieder die Vorgesetzten:

```
SELECT   E.LNAME As EMPLOYEE,  
           S.LNAME As SUPERVISOR  
  
FROM     EMPLOYEE E LEFT OUTER JOIN EMPLOYEE S  
           ON E.SUPERSSN=S.SSN
```

Der LEFT OUTER JOIN enthält immer Tupel zu den Tupeln der linken Tabelle, selbst wenn es keine passenden Tupel in der rechten Tabelle gibt.

Das bedeutet hier, daß alle Angestellten E aufgelistet werden, selbst wenn sie – wie der Firmenchef James Borg – keinen Vorgesetzten S haben.

Ergebnis:

EMPLOYEE	SUPERVISOR
Smith	Wong
Wong	Borg
Zelaya	Wallace
Wallace	Borg
Narayan	Wong
English	Wong
Jabbar	Wallace
Borg	NULL

Anfrage 2A (Join)

Liste die Projektnummer, die Nummer der zugehörigen Abteilung sowie den Nachnamen, die Adresse und das Geburtsdatum des Abteilungsleiters für alle Projekte, die in Stafford angesiedelt sind.

```
SELECT  PNUMBER, DNUM, LNAME, ADDRESS, BDATE
FROM    ( PROJECT
            JOIN DEPARTMENT ON DNUM=DNUMBER )
            JOIN EMPLOYEE ON MGRSSN=SSN
WHERE    PLOCATION='Stafford'
```

Man kann Join–Spezifikationen auch schachteln; d.h., eine der Tabellen eines Joins kann selbst wieder das Resultat eines Joins sein.

In unserem Falle werden zuerst **PROJECT** und **DEPARTMENT** verbunden; das Resultat wird dann mit **EMPLOYEE** verbunden.

Aggregatsfunktionen und Gruppierung

Anfrage 15

Berechne die Summe, das Maximum, das Minimum und den Durchschnitt der Gehälter aller Angestellten.

```
SELECT    SUM (SALARY), MAX (SALARY),  
           MIN (SALARY), AVG (SALARY)  
FROM      EMPLOYEE
```

Nullwerte (bei SALARY) werden bei der Aggregation nicht berücksichtigt.

Anfrage 18

Bestimme die Anzahl der Mitarbeiter in der Abteilung 'Research'.

```
SELECT    COUNT(*)  
FROM      EMPLOYEE, DEPARTMENT  
WHERE     DNO=DNUMBER AND DNAME='Research'
```

Anfrage 19

Bestimme die Anzahl verschiedener Löhne in der Datenbank.

```
SELECT    COUNT (DISTINCT SALARY)  
FROM      EMPLOYEE
```

Anfrage 20

Bestimme für alle Abteilungen die Abteilungsnummer, die Anzahl der Angestellten und das Durchschnittsgehalt.

```
SELECT      DNO, COUNT(*) AS EMPS, AVG (SALARY)  
FROM        EMPLOYEE  
GROUP BY   DNO
```

Ergebnis von Anfrage 20:

Zunächst werden die Angestellten nach Abteilungszugehörigkeit gruppiert.

FNAME	MINIT	LNAME	SSN	...	SALARY	SUPERSSN	DNO
John	B	Smith	444444444	...	30000	222222222	5
Franklin	T	Wong	222222222	...	40000	111111111	5
Ramesh	K	Narayan	555555555	...	38000	222222222	5
Joyce	A	English	666666666	...	25000	222222222	5
Alicia	J	Zelaya	777777777	...	25000	333333333	4
Jennifer	S	Wallace	333333333	...	43000	111111111	4
Ahmad	V	Jabbar	888888888	...	25000	333333333	4
James	E	Borg	111111111	...	55000	NULL	1

Danach werden die Gruppen zusammengefaßt.

DNO	EMPS	AVG(SALARY)
5	4	33250
4	3	31000
1	1	55000

Anfrage 5

Bestimme die Namen und Vornamen aller Angestellten, die mindestens zwei Angehörige haben.

```
SELECT  LNAME, FNAME
FROM    EMPLOYEE
WHERE   ( SELECT  COUNT(*)
          FROM    DEPENDENT
          WHERE   SSN = ESSN )  >= 2
```

Anfrage 21

Zu jedem Projekt sollen Projektnummer und Projektname sowie die Anzahl der an diesem Projekt arbeitenden Angestellten zurückgegeben werden.

```
SELECT      PNUMBER, PNAME, COUNT(*) AS EMPS
FROM        PROJECT, WORKS_ON
WHERE        PNUMBER = PNO
GROUP BY    PNUMBER, PNAME
```


Gruppierung mit Having

Anfrage 22

Zu jedem Projekt mit mehr als zwei Angestellten, die daran arbeiten, sollen Projektnummer und Projektname sowie die Anzahl der an diesem Projekt arbeitenden Angestellten zurückgegeben werden.

```
SELECT      PNUMBER, PNAME, COUNT(*) AS EMPS
FROM        PROJECT, WORKS_ON
WHERE        PNUMBER=PNO
GROUP BY    PNUMBER, PNAME
HAVING       COUNT(*) > 2
```

Ergebnis von Anfrage 22:

Zunächst wird der Join der Relationen PROJECT und WORKS_ON gebildet, wobei die Bedingung PNUMBR=PNO berücksichtigt wird.

PNAME	PNUMBER	...	ESSN	PNO	HOURS
ProductX	1	...	444444444	1	32.5
ProductX	1	...	666666666	1	20.0
ProduktY	2	...	444444444	2	7.5
ProduktY	2	...	666666666	2	20.0
ProduktY	2	...	222222222	2	10.0
ProductZ	3	...	555555555	3	40.0
ProductZ	3	...	222222222	3	10.0
Computerization	10	...	222222222	10	10.0
Computerization	10	...	777777777	10	10.0
Computerization	10	...	888888888	10	35.0
Reorganization	20	...	222222222	20	10.0
Reorganization	20	...	333333333	20	15.0
Reorganization	20	...	111111111	20	NULL
Newbenefits	30	...	888888888	30	5.0
Newbenefits	30	...	333333333	30	20.0
Newbenefits	30	...	777777777	30	30.0

Dann wird die Bedingung “**HAVING COUNT(*) > 2**” ausgewertet.

PNAME	PNUMBER	...	ESSN	PNO	HOURS
ProduktY	2	...	4444444444	2	7.5
ProduktY	2	...	6666666666	2	20.0
ProduktY	2	...	2222222222	2	10.0
Computerization	10	...	2222222222	10	10.0
Computerization	10	...	7777777777	10	10.0
Computerization	10	...	8888888888	10	35.0
Reorganization	20	...	2222222222	20	10.0
Reorganization	20	...	3333333333	20	15.0
Reorganization	20	...	1111111111	20	NULL
Newbenefits	30	...	8888888888	30	5.0
Newbenefits	30	...	3333333333	30	20.0
Newbenefits	30	...	7777777777	30	30.0

Und schließlich werden noch die Gruppen zusammengefaßt.

PNUMBER	PNAME	EMPS
2	ProductY	3
10	Computerization	3
20	Reorganization	3
30	Newbenefits	3

Anfrage 23

Gib zu jedem Projekt den Projektnamen, die Projektnummer und die Anzahl der Angestellten, die daran arbeiten und Abteilung 5 angehören, an.

```
SELECT      PNUMBER, PNAME, COUNT(*) AS EMPS
FROM        PROJECT, WORKS_ON, EMPLOYEE
WHERE        PNUMBER=PNO AND SSN=ESSN AND DNO=5
GROUP BY    PNUMBER, PNAME
```

Anfrage 24

Es sind alle Abteilungen gesucht, die mehr als 5 Angestellte haben; zu diesen Abteilungen ist auch die Anzahl der Angestellten mit mehr als 40.000 \$ Verdienst gesucht.

```
SELECT      DNAME, COUNT(*) As EMPS
FROM        DEPARTMENT, EMPLOYEE
WHERE       DNUMBER=DNO AND SALARY > 40000
AND         DNO IN (
            SELECT      DNO
            FROM        EMPLOYEE
            GROUP BY    DNO
            HAVING      COUNT(*) > 5 )
GROUP BY    DNAME
```

Anfrage 24A

Es sind alle Abteilungen gesucht, die mehr als 5 Angestellte mit mehr als 40.000 \$ Verdienst haben; zu diesen Abteilungen ist auch die Anzahl dieser Angestellten gesucht.

```
SELECT      DNAME, COUNT(*) AS EMPS
FROM        DEPARTMENT, EMPLOYEE
WHERE       DNUMBER=DNO AND SALARY > 40000
GROUP BY    DNAME
HAVING      COUNT(*) > 5
```

Regeln für **SELECT**-Statements mit **GROUP BY** und **HAVING**

1. Zuerst wird der Join der Relationen aus der **FROM**-Klausel basierend auf den Selektions- und Joinbedingungen der **WHERE**-Klausel berechnet.
2. Dann wird mittels **GROUP BY** die Gruppierung vorgenommen, und die Aggregationsfunktionen werden auf die entstandenen Gruppen angewendet; Nullwerte werden bei der Aggregation ignoriert.
3. Die **HAVING**-Klausel selektiert dann einzelne Gruppen.
4. In der **SELECT**-Klausel sollten nur Attributwerte, nach denen gruppiert wurde, ohne Aggregatsfunktion vorkommen – zusammen mit den berechneten Aggregatswerten **AGG(A)**.
5. Alle nicht aggregierten Attribute aus der **SELECT**-Klausel sollten normalerweise im **GROUP BY** vorkommen.

Vergleich von **SELECT**- und **GROUP BY**-Klausel

In der folgenden Anfrage treten genau die Attributwerte, nach denen gruppiert wird, in der **SELECT**-Klausel ohne Aggregatsfunktion auf.

```
SELECT      DNO, COUNT(*)  
FROM        EMPLOYEE  
GROUP BY   DNO
```

Das Ergebnis zeigt wie sich die 8 Mitarbeiter auf die 3 Abteilungen (DNO=1,4,5) verteilen.

DNO	COUNT(*)
1	1
4	3
5	4

Man kann auch nach DNO gruppieren, ohne daß DNO in der **SELECT**-Klausel auftritt.

```
SELECT      COUNT(*)  
FROM        EMPLOYEE  
GROUP BY   DNO
```

Dann wird für jede Abteilung die Anzahl der Mitarbeiter berechnet, allerdings ohne Zuordnung zur Abteilungsnummer.

COUNT(*)
1
3
4

In MySQL können in der **SELECT**-Klausel sogar Attributwerte, nach denen nicht gruppiert wurde, ohne Aggregatsfunktion auftreten. Dann ist allerdings das Ergebnis meist unsinnig.

```
SELECT  DNO, COUNT(*)  
FROM    EMPLOYEE
```

Diese Anfrage berechnet die Anzahl 8 aller Mitarbeiter der gesamten Firma und gibt sie zusammen mit der Abteilungsnummer 1 aus.

DNO	COUNT(*)
1	8

Teilstring-Vergleiche

Anfrage 25

Gesucht sind alle Angestellten, die in Houston, Texas, leben.

```
SELECT  FNAME, LNAME  
FROM    EMPLOYEE  
WHERE    ADDRESS LIKE '%Houston, TX%'
```

Anfrage 26

Finde alle Angestellten heraus, die in den 50er Jahren geboren wurden.

```
SELECT  FNAME, LNAME  
FROM    EMPLOYEE  
WHERE    BDATE LIKE '__ 5 __ __ __ __'
```

Alternative: BDATE **LIKE** '__ 5%'

Arithmetische Operationen, Ordnung und Formatierung der Ausgabe

Anfrage 27

Zeige die Gehälter, die die Angestellten, die am ProductX arbeiten, im Falle einer 10%–igen Lohnerhöhung bekommen würden.

```
SELECT      FNAME, LNAME, 1.1*SALARY
FROM        EMPLOYEE, WORKS_ON, PROJECT
WHERE        SSN = ESSN
AND          PNO = PNUMBER AND PNAME = 'ProductX'
ORDER BY    FNAME, LNAME
```

Um FNAME absteigend zu sortieren, würde man die **ORDER BY**–Klausel ersetzen durch: **ORDER BY FNAME DESC, LNAME ASC**.

Mittels **SELECT FNAME, ':', LNAME, ':', 1.1*SALARY** könnte man eine ':'–separierte Ausgabe zum Import in MS Excel erzeugen.

Falls nichts angegeben wurde, so ist die Standard–Ordnung aufsteigend.

Anfrage 28

```
SELECT      DNAME, FNAME, LNAME, PNAME
FROM        DEPARTMENT, EMPLOYEE,
              WORKS_ON, PROJECT
WHERE       DNUMBER = DNO
AND         SSN = ESSN
AND         PNO = PNUMBER
ORDER BY    DNAME DESC, LNAME, FNAME
```

2.3.3 Update-Statements in SQL

Zum *Einfügen eines einzelnen Tupels* in eine Relation (**INSERT**) gibt es zwei Möglichkeiten:

1. Man listet die Werte des neuen Tupels in derselben Reihenfolge auf, in der die zugehörigen Attribute beim **CREATE TABLE** angegeben wurden.
2. Man gibt die Attribute zu den Werten des neuen Tupels in entsprechender Reihenfolge explizit an.

In MySQL können auch mehrere Tupel durch Kommata getrennt in einem einzigen **INSERT**-Statement angegeben werden.

Beispiel

```
INSERT INTO  EMPLOYEE
VALUES      ('Richard', 'K', 'Marini', '111222333', '1952-12-30',
            '98 Oak Forest, Katy, TX', 'M', 37000 , '333333333', 4 )
```

```
INSERT INTO  EMPLOYEE(FNAME, LNAME, SSN, DNO)
VALUES      ('Richard', 'Marini', '111222333', 4),
            ('Robert', 'Hatcher', '555666777', 2)
```

Die Überprüfung der *Integritätsbedingungen* ist systemabhängig. Die folgende Operation wird abgelehnt, da kein Wert für “SSN” angegeben ist:

```
INSERT INTO  EMPLOYEE(FNAME, LNAME, DNO)
VALUES      ('Robert', 'Hatcher', 2)
```


Es kann auch eine ganze, mittels eines **SELECT**–Statements erzeugte *Relation* eingefügt werden:

```
CREATE TABLE    DEPT (  
    DNAME          VARCHAR(15),  
    EMPS           INTEGER,  
    TOTAL_SAL      INTEGER );
```

DEPT		
DNAME	EMPS	TOTAL_SAL
Administration	3	93000
Headquarters	1	55000
Research	4	133000

```
INSERT INTO    DEPT  
  
SELECT         DNAME, COUNT(*), SUM(SALARY)  
  
FROM          DEPARTMENT, EMPLOYEE  
  
WHERE         DNUMBER = DNO  
  
GROUP BY      DNAME
```

Das **Löschen** von Tupeln erfolgt mit **DELETE**; auch hier ist die Überprüfung von referentiellen Integritätsbedingungen systemabhängig.

```
DELETE FROM  EMPLOYEE  
WHERE        LNAME='Brown'
```

```
DELETE FROM  EMPLOYEE  
WHERE        DNO IN (  
                SELECT  DNUMBER  
                FROM    DEPARTMENT  
                WHERE   DNAME = 'Research' )
```

```
DELETE FROM  EMPLOYEE
```

Das letzte Statement – ohne **WHERE**-Teil – löscht dabei alle Tupel der Relation EMPLOYEE.

Das **UPDATE**-Kommando

- In einer **WHERE**-Klausel werden die zu modifizierenden Tupel spezifiziert.
- Eine **SET**-Klausel spezifiziert die zu modifizierenden Attribute sowie die zugehörigen Attributwerte.

UPDATE **PROJECT**

SET **PLOCATION='Bellaire', DNUM=5**

WHERE **PNUMBER=10**

Es können mehrere Tupel gleichzeitig mittels einer *Berechnungsregel* modifiziert werden:

```
UPDATE  EMPLOYEE
SET     SALARY = SALARY * 1.1
WHERE   DNO IN (
          SELECT   DNUMBER
          FROM     DEPARTMENT
          WHERE    DNAME = 'Research' )
```

Wie bei Programmiersprachen bezeichnet ein Attributname auf der rechten Seite einer *Zuweisung* den alten Wert, während er auf der linken Seite den neuen Wert bezeichnet.

2.3.4 Views (Sichten) in SQL

Ein View ist eine *virtuelle Tabelle*, welche von anderen Tabellen (gespeicherten Basistabellen oder auch anderen Views) abgeleitet ist. Diese anderen Tabellen werden definierende Tabellen genannt.

Ein View hat einen (virtuellen) Tabellennamen, eine Liste von Attributen, und sein Inhalt wird durch ein **SELECT**-Statement spezifiziert:

```
CREATE VIEW    E_WORKS_ON_P
AS   SELECT    FNAME, LNAME, PNAME, HOURS
          FROM      EMPLOYEE, WORKS_ON, PROJECT
          WHERE     SSN=ESSN AND PNO=PNUMBER
```

E_WORKS_ON_P			
FNAME	LNAME	PNAME	HOURS
John	Smith	ProductX	32.5
John	Smith	ProductY	7.5
Ramesh	Narayan	ProductZ	40.0
Joyce	English	ProductX	20.0
Joyce	English	ProductY	20.0
Franklin	Wong	ProductY	10.0
Franklin	Wong	ProductZ	10.0
Alicia	Zelaya	Newbenefits	30.0
Alicia	Zelaya	Computerization	10.0
Ahmad	Jabbar	Computerization	35.5
Ahmad	Jabbar	Newbenefits	5.0
Jennifer	Wallace	Newbenefits	20.0
Jennifer	Wallace	Reorganization	15.0
James	Borg	Reorganization	NULL

```
CREATE VIEW      DEPT (DNAME, EMPS, TOTAL_SAL)
AS SELECT       DNAME, COUNT(*), SUM (SALARY)
FROM           DEPARTMENT,
                EMPLOYEE
WHERE          DNUMBER=DNO
GROUP BY      DNAME
```

DEPT		
DNAME	EMPS	TOTAL_SAL
Administration	3	93000
Headquarters	1	55000
Research	4	133000

Sobald man in einer Sitzung einen View nicht mehr benötigt, sollte man ihn aus Effizienzgründen löschen:

```
DROP VIEW      E_WORKS_ON_P
```

```
DROP VIEW      DEPT
```

An Views können in üblicher Weise *Anfragen* gestellt werden. Dies erleichtert das *wiederholte* Stellen ähnlicher Anfragen.

```
SELECT  FNAME, LNAME  
FROM    E_WORKS_ON_P  
WHERE    PNAME = 'ProjectX'
```

Ein View ist immer *aktuell*, er reflektiert automatisch Änderungen, welche sich aus Modifikationen der definierenden Tabellen ergeben.

Zur *effizienten Implementierung* gibt es zwei wichtige Strategien:

1. Anfrage–Modifizierung: bildet eine Anfrage an einen View auf Anfragen an die definierenden Relationen ab.
2. View–Materialisierung: bei der ersten Anfrage an einen View wird dieser berechnet; mittels einer inkrementellen Methode werden Änderungen der definierenden Relationen auf den View propagiert.

View–Updates

UPDATE–Operationen auf einem View können (manchmal) auf die definierenden Tabellen propagiert werden.

```
UPDATE  E_WORKS_ON_P
SET     PNAME = 'ProductY'
WHERE    LNAME = 'Smith'
AND      FNAME = 'John'
AND      PNAME = 'ProductX'
```

Leider sind bei Views (wie dem obigen), welche durch Joins aus mehreren definierenden Tabellen abgeleitet sind, die erforderlichen Operationen auf den definierenden Tabellen meist *nicht eindeutig*.

Die erste der folgenden drei Alternativen wäre wohl am sinnvollsten:

1. Alternative: Änderung in WORKS_ON

```
UPDATE  WORKS_ON
SET      PNO = ( SELECT PNUMBER FROM PROJECT
                  WHERE PNAME='ProductY' )
WHERE    ESSN = ( SELECT SSN FROM EMPLOYEE
                  WHERE LNAME='Smith' AND FNAME='John' )
AND      PNO = ( SELECT PNUMBER FROM PROJECT
                  WHERE PNAME='ProductX' )
```

Anstelle von ProductX arbeitet John Smith zukünftig dieselbe Stundenanzahl an ProductY.

2. Alternative: Änderung in PROJECT

```
DELETE  FROM PROJECT
WHERE   PNAME='ProductY'

UPDATE  PROJECT
SET     PNAME='ProductY'
WHERE   PNAME='ProductX'
```

ProductY wird gelöscht, und ProductX wird in ProductY umbenannt.

3. Alternative: Änderung in EMPLOYEE wäre relativ sinnlos.

In den meisten Datenbanksystemen gelten folgende *Regeln*:

- Ein View über einer einzigen definierenden Tabelle ist modifizierbar, falls die View-Attribute den *Primärschlüssel* oder einen anderen *Schlüsselkandidaten* der Basisrelation enthalten, denn dann entspricht jedes virtuelle Tupel des Views genau einem Basis-Tupel.

```
CREATE VIEW  EMP ( LNAME, SSN )  
AS  SELECT  LNAME, SSN  
      FROM  EMPLOYEE  
  
UPDATE  EMP  
SET      LNAME = 'Armstrong'  
WHERE    SSN = '444444444'
```

EMP	
LNAME	SSN
Smith	444444444
Wong	222222222
Zelaya	777777777
Wallace	333333333
Narayan	555555555
English	666666666
Jabbar	888888888
Borg	111111111

- Views, welche mittels *Gruppierung* und *Aggregatsfunktionen* definiert sind, sind nicht modifizierbar.

```
UPDATE   DEPT
SET      TOTAL_SAL = 100000
WHERE    DNAME='Research'
```

DEPT		
DNAME	EMPS	TOTAL_SAL
Administration	3	93000
Headquarters	1	55000
Research	4	133000

- Views über mehreren definierenden Tabellen (mit Joins) sind i.a. nicht modifizierbar.

In SQL2 muß am Ende der View-Definition die Klausel

WITH CHECK OPTION

stehen, wenn der View modifizierbar sein soll.

2.3.5 Bedingungen, Trigger und Indexe

Integritätsbedingungen

```
CREATE ASSERTION SALARY_CONSTRAINT
CHECK NOT EXISTS ( SELECT *
                    FROM EMPLOYEE E, EMPLOYEE M, DEPARTMENT D
                    WHERE E.SALARY > M.SALARY
                    AND E.DNO = D.DNUMBER AND D.MGRSSN = M.SSN ) ;
```

Trigger geben eine *Bedingung* an und eine *Aktion*,
welche ausgeführt werden soll, sobald die Bedingung verletzt wird.

```
DEFINE TRIGGER SALARY_TRIGGER
ON EMPLOYEE E, EMPLOYEE M, DEPARTMENT D:
    E.SALARY > M.SALARY AND
    E.DNO = D.DNUMBER AND D.MGRSSN = M.SSN
ACTION_PROCEDURE INFORM_MANAGER (D.MGRSSN) ;
```

Indexe

Ein Index ist eine *physikalische Zugriffsstruktur*. Er erleichtert den Zugriff auf Tupel über Bedingungen, welche die indexierten Attribute betreffen.

```
CREATE INDEX LNAME_INDEX  
ON EMPLOYEE (LNAME) ;
```

```
CREATE INDEX NAMES_INDEX  
ON EMPLOYEE (LNAME ASC, FNAME DESC, MINIT) ;
```

Zum Primärschlüssel und zu den Sekundärschlüsseln wird automatisch – beim **CREATE TABLE** – ein Index angelegt.

Das Schlüsselwort **UNIQUE** hinter dem **CREATE** erklärt die Indexattribute zum Schlüssel.

```
CREATE UNIQUE INDEX NAMES_INDEX  
ON EMPLOYEE (LNAME, FNAME, MINIT);
```

Eine Basisrelation kann höchstens einen “Clusterindex” – nach diesem sind die Datensätze physisch auf der Festplatte organisiert – haben, aber beliebig viele andere Indexe.

```
CREATE INDEX DNO_INDEX  
ON EMPLOYEE (DNO)  
CLUSTER;
```

Da die Pflege eines Indexes bei Updates der Basisrelation aufwendig ist, sollte man den *Index löschen*, sobald er nicht mehr benötigt wird.

```
DROP INDEX DNO_INDEX;
```


2.3.6 Embedded SQL

SQL kann im Zusammenhang mit herkömmlichen, *prozeduralen Programmiersprachen* wie JAVA, C++ oder C benutzt werden. Diese Sprachen nennt man dann *Hostsprachen*.

```
import java.io.*;
import java.sql.*;
import java.util.*;
import java.text.*;
import java.lang.reflect.*;

class MySQL_Query {

    protected final String dbUser = "seipel";
    protected final String dbPassword = "...";
    protected String jdbcURL = "jdbc:mysql://localhost/company";
    protected String jdbcDriver = "org.gjt.mm.mysql.Driver";
```

```
/* connection to the database */
protected Connection con = null;

public static void main(String args[])
{
    MySQL_Query query = new MySQL_Query();
    query.initConnection();
    query.salary_query();
}

protected void initConnection()
...

private void salary_query()
...

}
```

```
protected void initConnection()
{
    try {
        Class.forName(jdbcDriver).newInstance();
        con = DriverManager.getConnection(
            jdbcURL, dbUser, dbPassword );
    }
    catch (InstantiationException ie) {
        System.out.println(ie + ie.getMessage());
    }
    catch (IllegalAccessException iae) {
        System.out.println(iae + iae.getMessage());
    }
    catch (SQLException sqle) {
        System.out.println(sqle + sqle.getMessage());
    }
    catch (ClassNotFoundException cnfe) {
        System.out.println(cnfe + cnfe.getMessage());
    }
}
```

```
private void salary_query()
{
    try {
        String query =
            "SELECT * FROM employee WHERE salary >= 30000";
        Statement stmt = con.createStatement();
        ResultSet rs = stmt.executeQuery(query);
        while (rs.next()) {
            String fname = rs.getString(1);
            String minit = rs.getString(2);
            String lname = rs.getString(3);
            String salary = rs.getString(8);
            System.out.println( "( " + fname + ", " + minit +
                ", " + lname + ", " + salary + " )" );
        }
    }
    catch (Exception mye){
        System.out.println(mye.toString());
    }
}
```

3 Datenbankentwurf im Relationenmodell

Entwicklung formaler Kriterien, warum eine *Gruppierung von Attributen in Relationen* besser oder schlechter ist als eine andere. Gesichtspunkte:

- *logische Ebene*: wie gut interpretiert der Benutzer die Relationenschemata und ihre Attribute,
- *Zugriffs-/Speicherungs-Ebene*: wie gut sind die Tupel der Basisrelationen abgespeichert/zugreifbar/änderbar.

Zur Beurteilung einzelner Schemata werden *funktionale Abhängigkeiten* und *Normalformen* verwendet.

Zur Beurteilung einer ganzen Menge von Relationenschemata, d.h. von Datenbankschemata, verwendet man die *verlustfreie Verbund*-Eigenschaft und die *Abhängigkeits-Erhaltungs*-Eigenschaft.

3.1 Informelle Design–Richtlinien und Update–Anomalien

Richtlinie 1: Semantik der Attribute eines Relationenschemas

Entwerfe ein Relationenschema so, daß seine Bedeutung leicht zu erklären ist. Es soll einen einfachen Bezug zwischen den Attributen geben: mische nicht Attribute von unterschiedlichen Entity– oder Relationship–Typen in einem einzelnen Schema.

Richtlinie 2: Vermeidung von Redundanz und von Update–Anomalien

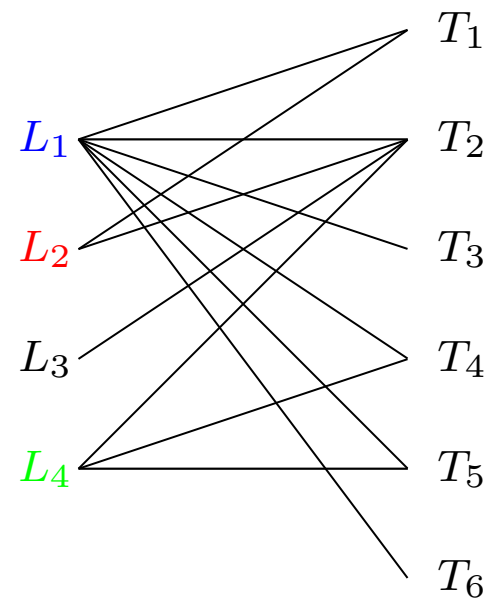
Entwerfe die Basisrelationenschemata so, daß keine Einfüge–, Löscho– oder Modifikationsanomalien auftreten. Unvermeidbare Anomalien sollen gut dokumentiert werden.

Die Vermeidung von Redundanz führt auch zur Minimierung des Speicherplatzbedarfs für die Basisrelationen.

Beispiel (Lieferant)

L: Lieferant, T: Teil, A: Anzahl, O: Ort, K: Entfernung

L	T	A	O	K
L1	T1	300	London	600
L1	T2	200	London	600
L1	T3	400	London	600
L1	T4	200	London	600
L1	T5	100	London	600
L1	T6	100	London	600
L2	T1	300	Paris	450
L2	T2	400	Paris	450
L3	T2	200	Paris	450
L4	T2	200	Brüssel	150
L4	T4	300	Brüssel	150
L4	T5	400	Brüssel	150



LT: n:m-Beziehung

L: Lieferant, T: Teil, A: Anzahl, O: Ort, K: Entfernung



Jeder Lieferant (L) residiert an einem Ort (O) in einer Entfernung (K).
Für jedes gelieferte Teil (T) wird die gelieferte Anzahl (A) gespeichert.

Funktionale Abhängigkeiten und Normalformen dienen zur Analyse der Relationen der Datenbank.

Verlustfreie Zerlegungen dienen zur Verbesserung der Struktur der Datenbank.

Die Tabelle repräsentiert drei Beziehungen:

LTA: zu einem Lieferanten/Teile–Paar wird die gelieferte Teileanzahl angegeben,

LO: zu jedem Lieferanten wird sein Standort angegeben,

OK: zu jedem Ort wird die Entfernung (in km) angegeben.

Die *LO*– und die *OK*–Beziehung sind hochgradig *redundant* gespeichert.

Hier treten *Update–Anomalien* auf:

1. beim Einfügen partieller Tupel zu *LTA* entstehen Nullwerte,
2. beim Löschen des letzten Eintrages eines Lieferanten geht die *LO*–Beziehung verloren,
3. beim Ändern des Ortes zu einem Lieferanten (Umzug) müssen mehrere Tupel geändert werden.

Performance:

Manchmal muß man aus Performance–Gründen gegen einzelne Richtlinien verstoßen, um gewisse Anfragen effizient beantworten zu können. Im allgemeinen sollte man Anomalien aber vermeiden.

Views:

Man sollte Views verwenden, welche mit Hilfe von Joins die gewünschten Attributkombinationen aus verschiedenen Basisrelationen erstellen; diese können dann auch als Module für komplexe Anfragen verwendet werden. Dies bringt Übersichtlichkeit und oft auch einen Performance–Gewinn.

Richtlinie 3: Nullwerte

Vermeide weitestgehend Attribute in Relationen, falls die zugehörigen Attributwerte NULL sein könnten.

Falls eine große Anzahl der Attribute eines Relationenschemas auf viele Tupel der zugehörigen Relation nicht anwendbar sind, so treten viele *Nullwerte* in der Relation auf. Konsequenzen:

- Verschwendung von Speicherplatz
- Joins und Aggregatsfunktionen über Nullwerten sind problematisch
- unterschiedliche Interpretationen für Nullwerte:
 - das Attribut ist auf das Tupel *nicht anwendbar*
 - der Attributwert ist für das Tupel *unbekannt*, oder er ist zwar *bekannt*, aber nicht eingegeben

Richtlinie 4: Überflüssige Tupel

Entwerfe die Relationenschemata so, daß sie mit Hilfe von Joins (über gleiche Attributwerte) korrekte Sichten auf die Daten liefern. Dies ist z.B. gewährleistet, falls die Join-Attribute Schlüssel (Fremdschlüssel) sind.

Falls eine Relation r als ihre *Projektionen* $\pi_{U_1}(r), \dots, \pi_{U_k}(r)$ auf mehrere Attributmengen repräsentiert wird, so gilt für deren Natural Join generell

$$r \subseteq \pi_{U_1}(r) \bowtie \dots \bowtie \pi_{U_k}(r).$$

Es können im Natural Join *überflüssige Tupel* auftreten, welche nicht in r enthalten waren. Gewünscht wäre aber die Gleichheit der beiden Ausdrücke.

Natural Join: $def(r_i) = U_i$

$$r_1 \bowtie r_2 = \{ t \mid def(t) = U_1 \cup U_2 \wedge t[U_1] \in r_1 \wedge t[U_2] \in r_2 \}.$$

Beispiel (Überflüssige Tupel)

1. Für die folgende Relation r und die Projektionen $r_1 = \pi_{AB}(r)$ und $r_2 = \pi_{BC}(r)$ gilt $r \subsetneq \pi_{AB}(r) \bowtie \pi_{BC}(r)$.

r									
A	B	C							
1	3	2							
2	3	1							

 $\rightarrow$

r_1		r_2				
A	B	B	C			
1	3	3	2			
2	3	3	1			

 $\rightarrow$

$r_1 \bowtie r_2$		
A	B	C
1	3	1
1	3	2
2	3	1
2	3	2

Hier sind die Attributmengen der beiden Relationen $U_1 = \{A, B\}$ und $U_2 = \{B, C\}$. Ihr Schnitt ist $U_1 \cap U_2 = \{B\}$.

Das Problem beim Join ist hier, daß es Tupel in r gibt mit demselben Wert für B aber unterschiedlichen Werten für die anderen Attribute.

2. Für dieselbe Relation r und die Projektionen $r_1 = \pi_{AB}(r)$ und $r_2 = \pi_{AC}(r)$ gilt dagegen $r = \pi_{AB}(r) \bowtie \pi_{AC}(r)$.

$r = r_1 \bowtie r_2$			$\leftrightarrow$		r_1		r_2	
A	B	C			A	B	A	C
1	3	2			1	3	1	2
2	3	1			2	3	2	1

Hier sind die Attributmengen der beiden Relationen $U_1 = \{A, B\}$ und $U_2 = \{A, C\}$. Ihr Schnitt ist $U_1 \cap U_2 = \{A\}$.

Da es es keine Tupel in r gibt mit demselben Wert für A aber unterschiedlichen Werten für die anderen Attribute, liefert der Join $r_1 \bowtie r_2$ wieder die Ausgangsrelation r .

3. Für die Lieferantenrelation gilt ebenfalls

$$r = \pi_{LTA}(r) \bowtie \pi_{LO}(r) \bowtie \pi_{OK}(r).$$

$r = r_1 \bowtie r_2 \bowtie r_3$				
L	T	A	O	K
L1	T1	300	London	600
L1	T2	200	London	600
L1	T3	400	London	600
L1	T4	200	London	600
L1	T5	100	London	600
L1	T6	100	London	600
L2	T1	300	Paris	450
L2	T2	400	Paris	450
L3	T2	200	Paris	450
L4	T2	200	Brüssel	150
L4	T4	300	Brüssel	150
L4	T5	400	Brüssel	150

$\leftrightarrow$

$r_1 = \Pi_{LTA}(r)$		
L	T	A
L1	T1	300
L1	T2	200
L1	T3	400
L1	T4	200
L1	T5	100
L1	T6	100
L2	T1	300
L2	T2	400
L3	T2	200
L4	T2	200
L4	T4	300
L4	T5	400

$r_2 = \Pi_{LO}(r)$	
L	O
L1	London
L2	Paris
L3	Paris
L4	Brüssel

$r_3 = \Pi_{OK}(r)$	
O	K
London	600
Paris	450
Brüssel	150

3.2 Funktionale Abhängigkeiten

Funktionale Abhängigkeiten sind intra–relationale Bedingungen für Relationen r über einer Attributmenge U . Sie verallgemeinern den klassischen Schlüsselbegriff.

1. Eine *funktionale Abhängigkeit* (engl.: functional dependency, fd) ist gegeben durch $X \rightarrow Y$ mit $X, Y \subseteq U$. Sie gilt in r , falls

$$\forall t_1, t_2 \in r : t_1[X] = t_2[X] \Rightarrow t_1[Y] = t_2[Y].$$

Y (rechte Seite) heißt dann funktional abhängig von X (linke Seite), und man sagt X *geht nach* Y .

2. Eine fd $X \rightarrow Y$ heißt *trivial*, falls $Y \subseteq X$; sie gilt in allen Relationen.

Falls $X \rightarrow Y$ gilt, so definiert die Projektion $\Pi_{XY}(r)$ von r eine (partielle) Funktion, nach der die Y –Werte von den X –Werten abhängen.

Beispiel (Funktionale Abhängigkeiten)

In der folgenden Relation r ist keines der Attribute alleine ein Schlüssel, aber es gelten funktionale Abhängigkeiten:

r		
A	B	C
1	3	2
1	4	2
2	3	1

- Die fd $\{A\} \rightarrow \{C\}$ gilt, da die beiden ersten Tupel mit demselben Wert für A auch denselben Wert für C haben. Dagegen gilt $\{A\} \rightarrow \{B\}$ nicht, da diese Tupel unterschiedliche Werte für B haben.
- $\{B\} \rightarrow \{A\}$ und $\{B\} \rightarrow \{C\}$ gelten nicht, da das erste und dritte Tupel denselben Wert für B aber unterschiedliche Werte für A und C haben.

Die fd $X \rightarrow Y$ besagt:

$$\forall t_1, t_2 \in r : t_1[X] = t_2[X] \Rightarrow t_1[Y] = t_2[Y];$$

d.h., wenn zwei Tupel $t_1, t_2 \in r$ auf X übereinstimmen, dann müssen sie auch auf Y übereinstimmen.

	...	X	...	Y	...
		...		...	
t_1	...	$t_1[X]$	...	$t_1[Y]$	...
		...		...	
t_2	...	$t_2[X]$	...	$t_2[Y]$	...
		...		...	

Falls $Y = U$ ist, so bedeutet das, daß die beiden Tupel gleich sind.

Dann gibt es keine unterschiedlichen Tupel, die auf X übereinstimmen.

1. Ein *Relationenschema* $R = (U, F)$ ist gegeben durch
 - eine Attributmenge U und
 - eine Menge F von Integritätsbedingungen.
2. Eine Relation r über U , welche alle Integritätsbedingungen aus F erfüllt, heißt erlaubte *Instanz* von R .

Man kann die bekannte *Schlüsseleigenschaft* einer Attributmenge K als funktionale Abhängigkeit $K \rightarrow U$ ausdrücken:

$$t_1[K] = t_2[K] \Rightarrow t_1 = t_2 \Rightarrow t_1[U] = t_2[U].$$

Also wird die bisherige Definition eines Relationenschemas als

$$R = (U, K, NOTNULL)$$

mit Ausnahme der *NOTNULL*-Attribute – die uns in diesem Zusammenhang nicht interessieren – von der neuen Definition subsumiert.

Konventionen:

1. Einzelne Großbuchstaben am Anfang des Alphabets bezeichnen in der Regel Attribute, einzelne Großbuchstaben am Ende des Alphabets bezeichnen in der Regel Attributmengen.
2. Wir können eine Attributmenge $\{ A_1, \dots, A_n \}$ auch kurz als Zeichenkette $A_1 \dots A_n$ schreiben, falls alle A_i nur aus einem Zeichen bestehen.
3. Die Vereinigung $X \cup Y$ zweier Attributmengen X und Y kann als XY abgekürzt werden – wenn klar ist, daß X und Y Attributmengen sind.

Beispiel (Lieferant)

Wir betrachten das Relationenschema $R = (U, F)$ mit

$$U = \{ L, T, A, O, K \},$$

$$F = \{ LT \rightarrow A, L \rightarrow O, O \rightarrow K \}.$$

L: Lieferant, T: Teil, A: Anzahl,

O: Ort, K: Entfernung

- Die bereits bekannte Relation r ist eine erlaubte Instanz von R .
- In r gilt – neben den funktionalen Abhängigkeiten aus F – auch $L \rightarrow K$.

L	T	A	O	K
L1	T1	300	London	600
L1	T2	200	London	600
L1	T3	400	London	600
L1	T4	200	London	600
L1	T5	100	London	600
L1	T6	100	London	600
L2	T1	300	Paris	450
L2	T2	400	Paris	450
L3	T2	200	Paris	450
L4	T2	200	Brüssel	150
L4	T4	300	Brüssel	150
L4	T5	400	Brüssel	150

Die Redundanz und die daraus folgenden Update–Anomalien liegen offenbar an den funktionalen Abhängigkeiten $L \rightarrow O$ und $O \rightarrow K$.

Die fd $LT \rightarrow A$ stellt dagegen kein Problem dar, denn die Beziehung zwischen LT -Paaren und dem gelieferten A ist nicht redundant gespeichert.

In r gilt sogar $LT \rightarrow U$, denn LT ist ein Schlüssel: es gibt keine zwei unterschiedlichen Tupel, die auf LT übereinstimmen.

L	T	A	O	K
L1	T1	300	London	600
L1	T2	200	London	600
L1	T3	400	London	600
L1	T4	200	London	600
L1	T5	100	London	600
L1	T6	100	London	600
L2	T1	300	Paris	450
L2	T2	400	Paris	450
L3	T2	200	Paris	450
L4	T2	200	Brüssel	150
L4	T4	300	Brüssel	150
L4	T5	400	Brüssel	150

In r gilt auch die fd $L \rightarrow K$:

	L	T	A	O	K
t_1	L1	T1	300	London	600
	L1	T2	200	London	600
	L1	T3	400	London	600
	L1	T4	200	London	600
	L1	T5	100	London	600
	L1	T6	100	London	600
	L2	T1	300	Paris	450
	L2	T2	400	Paris	450
	L3	T2	200	Paris	450
	L4	T2	200	Brüssel	150
	L4	T4	300	Brüssel	150
	L4	T5	400	Brüssel	150

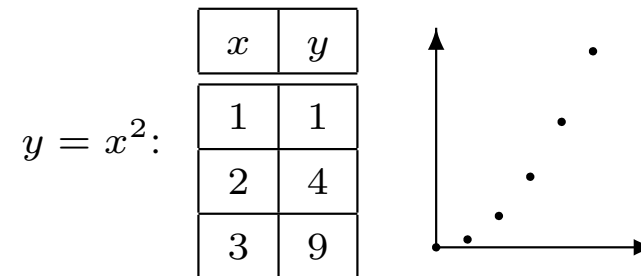
$L \rightarrow K$ folgt aus $L \rightarrow O$ und $O \rightarrow K$ (Transitivität):

Wenn zwei Tupel t_1 und t_2 auf L übereinstimmen, dann müssen sie wegen $L \rightarrow O$ auch auf O übereinstimmen und somit wegen $O \rightarrow K$ auch auf K .

Beispiele

- Die partielle Funktion für $O \rightarrow K$ ist gegeben durch die Projektion $\Pi_{OK}(r)$ von r (Funktionsgraph):

O	K
London	600
Paris	450
Brüssel	150



Die Funktion ist partiell, da sie z.B. für die Stadt “Berlin” nicht definiert ist.

- Die fd $T \rightarrow A$ gilt in r nicht, da es Teile (z.B. T2) gibt, die in unterschiedlichen Anzahlen (200, 400) geliefert werden. Ebenso gilt die fd $L \rightarrow A$ in r nicht.
- Jeder Lieferant liefert jedes Teil allerdings in genau einer Anzahl. Deswegen gilt die fd $LT \rightarrow A$ in r .

- In der erweiterten Relation $r' = r \cup \{t\}$, mit $t = (L1, T1, 400, \text{London}, 600)$, gilt $LT \rightarrow A$ hingegen nicht. r' ist keine erlaubte Instanz von R .
- Man kann die fd $L \rightarrow O$ z.B. mittels des folgenden SQL-Statements überprüfen:

```
SELECT  11.L  
FROM    Lieferant 11, Lieferant 12  
WHERE   11.L = 12.L  
AND     11.O != 12.O
```

Falls die Antwort auf diese Anfrage nicht-leer ist, so ist die fd in der Relation Lieferant verletzt.

Falls eine fd $X \rightarrow Y$ in einer Relation r gilt, so gilt $X \rightarrow Y$ auch in allen Teilmengen $r' \subseteq r$. Das Umgekehrte gilt aber – wie bereits gezeigt – nicht.

Überblick

- Man kann algorithmisch mit Hilfe von Kalkülen *logische Folgerungen* aus funktionalen Abhängigkeiten ziehen.
- Der grundlegende *Kalkül von Armstrong* besteht nur aus drei Ableitungsregeln: Reflexivität, Augmentierung und Transitivität.
- Weitere nützliche Ableitungsregeln – wie Zerlegung / Projektion, Vereinigung, Akkumulation, etc. – kann man daraus herleiten.

Logische Folgerungen aus funktionalen Abhängigkeiten

Sei F eine Menge von fds über U .

1. Eine fd $X \rightarrow Y$ über U *folgt* aus F , falls $X \rightarrow Y$ in *allen* erlaubten Instanzen von (U, F) gilt. Dann schreiben wir $F \models X \rightarrow Y$.
2. Zwei fd-Mengen F und G über U heißen *äquivalent*, falls (U, F) und (U, G) dieselben erlaubten Instanzen haben. Dann schreiben wir $F \equiv G$.

Offenbar gilt $F \equiv G$, wenn alle fds in G aus F folgen, und umgekehrt.

Die logische Folgerung ist auf der Basis der obigen Definition nur schwierig zu berechnen, da alle erlaubten Instanzen von (U, F) zu betrachten wären.

Für einfache Beispiele kann man aber die logischen Folgerungen leicht ermitteln.

Es gibt erlaubte Instanzen r' des Lieferantenschemas, in denen neben den fds aus $F = \{ LT \rightarrow A, L \rightarrow O, O \rightarrow K \}$ auch die fd $T \rightarrow A$ gilt, da jedes Teil in maximal einer Anzahl geliefert wird:

r'				
L	T	A	O	K
L1	T1	300	London	600
L1	T2	200	London	600
L1	T3	400	London	600
L1	T4	300	London	600
L1	T5	400	London	600
L1	T6	100	London	600
L2	T1	300	Paris	450
L2	T2	200	Paris	450
L3	T2	200	Paris	450
L4	T2	200	Brüssel	150
L4	T4	300	Brüssel	150
L4	T5	400	Brüssel	150

$T \rightarrow A$ ist aber keine logische Folgerung, da es z.B. in der vorherigen erlaubten Instanz r nicht gilt. $L \rightarrow K$ ist dagegen eine logische Folgerung.

Beispiel (Logische Folgerung, Äquivalenz)

Aus $F = \{ LT \rightarrow A, L \rightarrow O, O \rightarrow K \}$ folgen u.a. die fds

$LT \rightarrow AOK$, $L \rightarrow K$, $L \rightarrow OK$ und $K \rightarrow K$.

Für zwei Tupel $t_1, t_2 \in r$ einer beliebigen erlaubten Instanz r gilt:

1. Aus $t_1[LT] = t_2[LT]$ folgt natürlich $t_1[L] = t_2[L]$.

- Aus $LT \rightarrow A$ folgt $t_1[A] = t_2[A]$.
- Aus $L \rightarrow OK$ folgt $t_1[OK] = t_2[OK]$.

Damit gilt insgesamt $t_1[AOK] = t_2[AOK]$. Dies zeigt $LT \rightarrow AOK$.

2. Aus $t_1[L] = t_2[L]$ folgt wegen $L \rightarrow O$ auch $t_1[O] = t_2[O]$ und wegen $O \rightarrow K$ auch $t_1[K] = t_2[K]$. Dies zeigt $L \rightarrow K$ und $L \rightarrow OK$.

Die fd $L \rightarrow A$ folgt offensichtlich nicht, da sie in der obigen erlaubten Instanz r schon nicht gilt.

Also sind die folgenden fd-Mengen äquivalent: $F \equiv G_1 \equiv G_2 \equiv G_3$.

$$F = \{ LT \rightarrow A, L \rightarrow O, O \rightarrow K \},$$

$$G_1 = F \cup \{ LT \rightarrow O, LT \rightarrow K, L \rightarrow K \},$$

$$G_2 = \{ LT \rightarrow AOK, L \rightarrow OK, O \rightarrow K \},$$

$$G_3 = \{ LT \rightarrow A, L \rightarrow OK, O \rightarrow K \}.$$

G_1 nimmt drei logische Folgerungen zu F hinzu.

G_2 faßt die fds mit den gleichen linken Seiten zusammen und bildet fds aus der Vereinigung der entsprechenden rechten Seiten, denn es gilt

$$\{ L \rightarrow O, L \rightarrow K \} \equiv \{ L \rightarrow OK \},$$

$$\{ LT \rightarrow A, LT \rightarrow O, LT \rightarrow K \} \equiv \{ LT \rightarrow AOK \}.$$

G_3 verkürzt $LT \rightarrow AOK$ zu $LT \rightarrow A$, denn $LT \rightarrow OK$ folgt aus $L \rightarrow OK$.

Ableitungsregeln von Armstrong für funktionale Abhängigkeiten

Die folgenden *Inferenzregeln* (Axiome) stammen von *Armstrong* (1974).

(F1) Reflexivität (triviale fds):

$$\emptyset \vdash X \rightarrow Y, \text{ falls } Y \subseteq X$$

(F2) Augmentierung:

$$\{ X \rightarrow Y \} \vdash XZ \rightarrow YZ$$

(F3) Transitivität:

$$\{ X \rightarrow Y, Y \rightarrow Z \} \vdash X \rightarrow Z$$

Falls $X \rightarrow Y$ mittels des Kalküls von Armstrong aus F abgeleitet werden kann, so schreiben wir $F \vdash X \rightarrow Y$.

Die Reflexivität besagt, daß fds $X \rightarrow Y$, mit $Y \subseteq X$, und insbesondere fds $X \rightarrow X$ trivialerweise immer (d.h. ohne weitere Voraussetzung) gelten.

$$\models \equiv \vdash$$

Logische Folgerung und Ableitung sind äquivalent:
die Inferenzregeln (F1), (F2), (F3) von *Armstrong* bilden
einen *vollständigen* und *korrekten* Kalkül $\vdash$ für $\models$.

Vollständigkeit:

$$F \models X \rightarrow Y \Rightarrow F \vdash X \rightarrow Y$$

(alle Folgerungen können mittels des Kalküls abgeleitet werden)

Korrektheit:

$$F \vdash X \rightarrow Y \Rightarrow F \models X \rightarrow Y$$

(alle mittels des Kalküls abgeleiteten fds folgen)

Durch Anwendung der Inferenzregeln lassen sich also genau alle logischen Folgerungen aus einer fd-Menge systematisch berechnen.

Die *Korrektheit* der Inferenzregeln von Armstrong ist einfach zu beweisen.

Für zwei Tupel $t_1, t_2 \in r$ einer beliebigen erlaubten Instanz r gilt:

(F1) Reflexivität: $X \rightarrow Y$, falls $Y \subseteq X$.

Aus $t_1[X] = t_2[X]$ und $Y \subseteq X$ folgt $t_1[Y] = t_2[Y]$.

(F2) Augmentierung: $\{ X \rightarrow Y \} \vdash XZ \rightarrow YZ$.

Aus $t_1[XZ] = t_2[XZ]$ folgt $t_1[X] = t_2[X]$ und $t_1[Z] = t_2[Z]$.

Wegen $X \rightarrow Y$ gilt dann auch $t_1[Y] = t_2[Y]$.

Insgesamt gilt also $t_1[YZ] = t_2[YZ]$.

(F3) Transitivität: $\{ X \rightarrow Y, Y \rightarrow Z \} \vdash X \rightarrow Z$.

Aus $t_1[X] = t_2[X]$ und $X \rightarrow Y$ folgt $t_1[Y] = t_2[Y]$.

Wegen $Y \rightarrow Z$ gilt dann auch $t_1[Z] = t_2[Z]$.

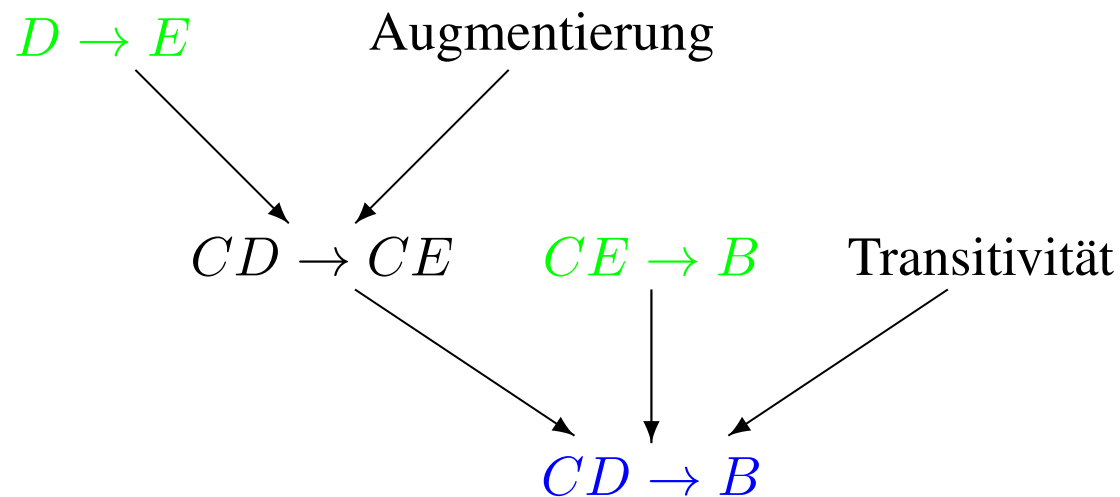
Der *Vollständigkeits*–Beweis, der auf sogenannten Armstrong–Relationen basiert, ist kompliziert und kann hier nicht gezeigt werden.

Beispiel (Inferenzregeln)

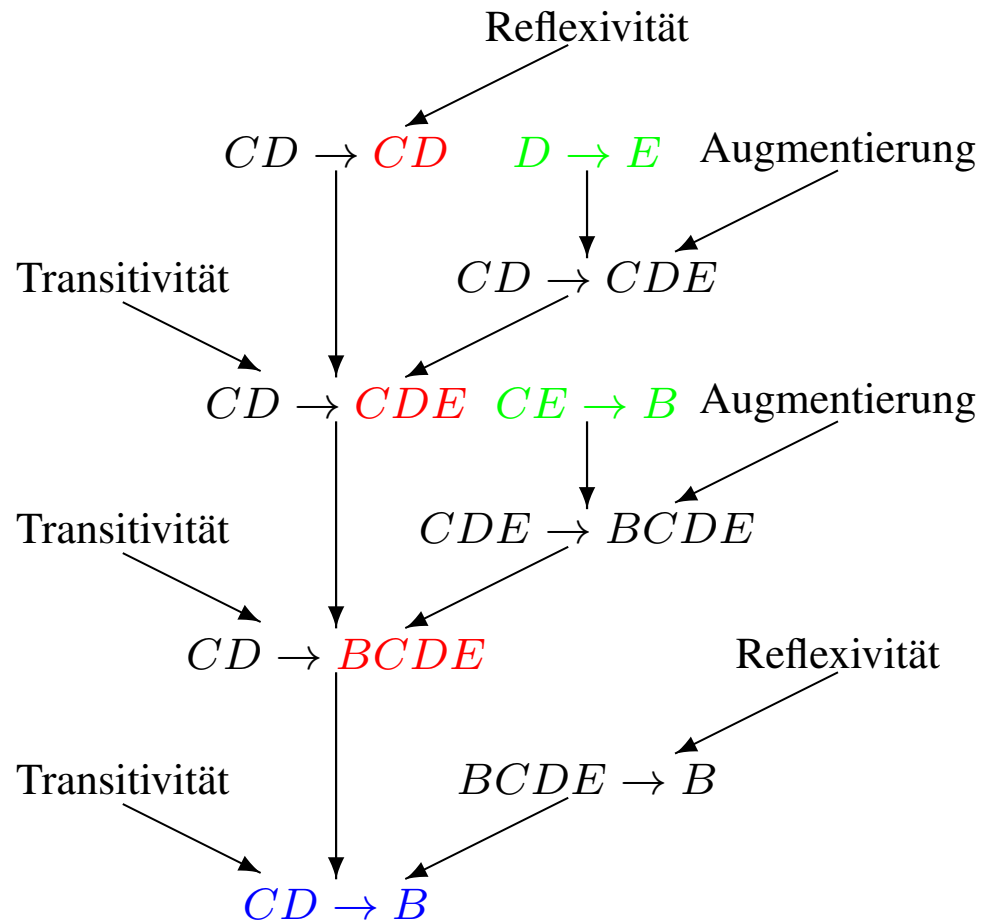
Aus der Menge

$$F = \{ BC \rightarrow D, D \rightarrow E, CE \rightarrow B \}$$

kann man die funktionale Abhängigkeit $CD \rightarrow B$ ableiten:



Alternative Ableitung aufgrund eines generischen Schemas:



Weitere Ableitungsregeln für funktionale Abhängigkeiten

Aus (F1)–(F3) können die folgenden weiteren Regeln abgeleitet werden:

(F4) Zerlegung oder Projektion:

$$\{ X \rightarrow YZ \} \vdash X \rightarrow Y, X \rightarrow Z$$

(F5) Vereinigung:

$$\{ X \rightarrow Y, X \rightarrow Z \} \vdash X \rightarrow YZ$$

(F6) Pseudo-Transitivität:

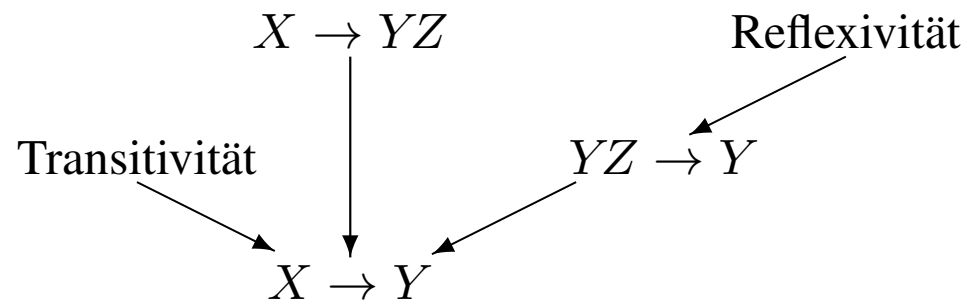
$$\{ X \rightarrow Y, WY \rightarrow Z \} \vdash WX \rightarrow Z$$

(F7) Akkumulation:

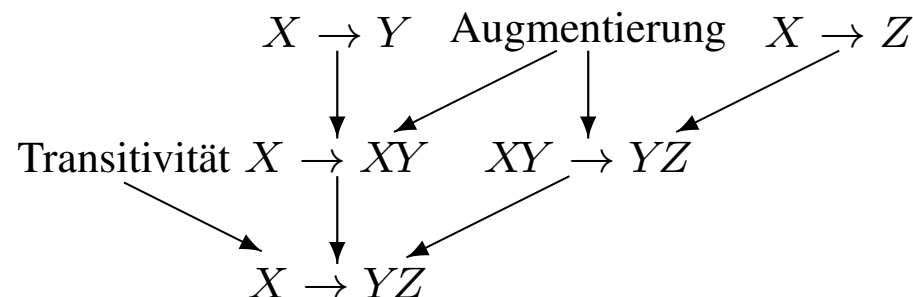
$$\{ X \rightarrow WY, Y \rightarrow Z \} \vdash X \rightarrow WYZ$$

Bei Verwendung dieser zusätzlichen Regeln kann man einfachere Ableitungsbäume erhalten.

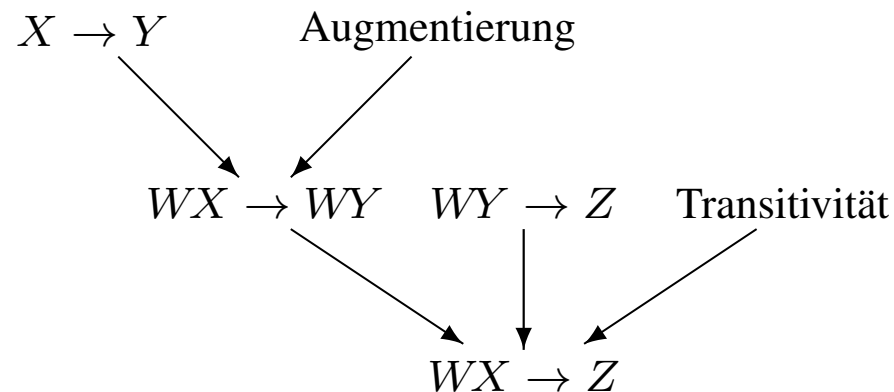
(F4) Die Projektion ergibt sich durch Verknüpfung von Reflexivität und Transitivität: $\{ X \rightarrow YZ \} \vdash X \rightarrow Y$



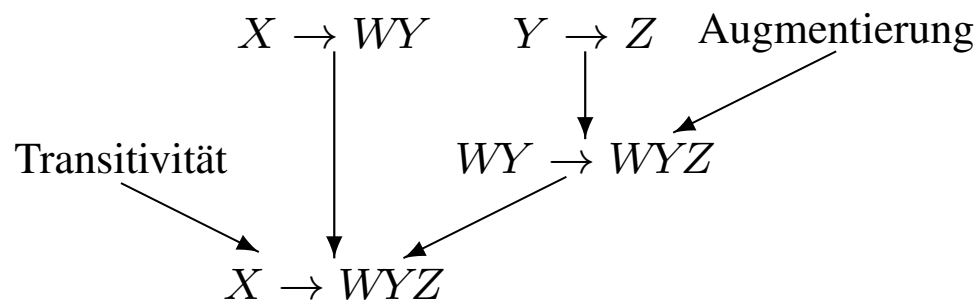
(F5) Die Vereinigung ergibt sich durch Verknüpfung von Augmentierung (zweimal) und Transitivität: $\{ X \rightarrow Y, X \rightarrow Z \} \vdash X \rightarrow YZ$



(F6) Die Pseudo-Transitivität ergibt sich durch Verknüpfung von
 Augmentierung und Transitivität: $\{ X \rightarrow Y, WY \rightarrow Z \} \vdash WX \rightarrow Z$



(F7) Die Akkumulation ergibt sich durch Verknüpfung von Augmentierung
 und Transitivität: $\{ X \rightarrow WY, Y \rightarrow Z \} \vdash X \rightarrow WYZ$

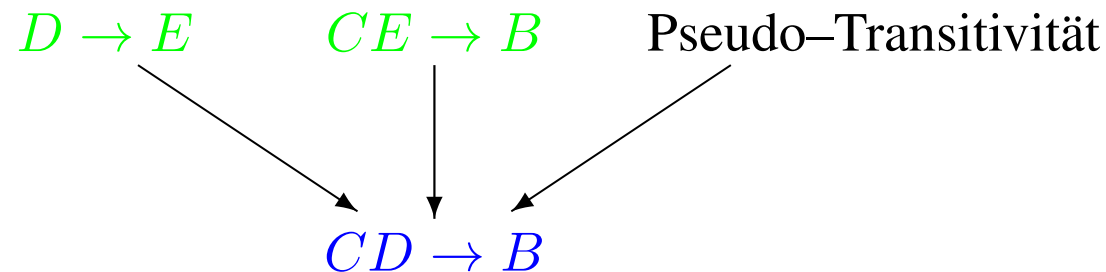


Beispiel (Inferenzregeln)

Aus der Menge

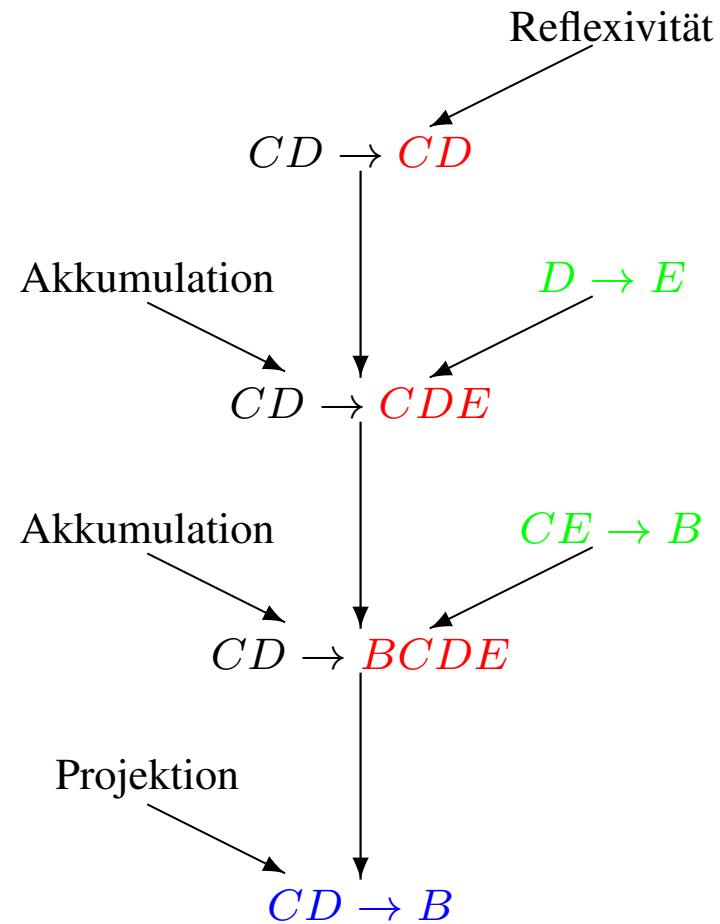
$$F = \{ BC \rightarrow D, D \rightarrow E, CE \rightarrow B \}$$

kann man die funktionale Abhängigkeit $CD \rightarrow B$ nun einfacher ableiten:



Anstelle von Augmentierung und Transitivität (2 Schritte, siehe weiter oben) verwenden wir nun einfach die Pseudo-Transitivität.

Auch die alternative Ableitung aufgrund des generischen Schemas können wir vereinfachen:



Beispiel (Inferenzregeln)

Sei $R = (U, F)$ mit $U = ABCD$ und

$$F = \{ AB \rightarrow C, BC \rightarrow D \}.$$

Aus F folgt die fd $AB \rightarrow D$:

- Mittels Reflexivität gilt $AB \rightarrow B$.
- Durch Vereinigung mit $AB \rightarrow C$ erhalten wir $AB \rightarrow BC$.
- Die transitive Verknüpfung mit $BC \rightarrow D$ liefert nun $AB \rightarrow D$.

Die fd $AB \rightarrow D$ wäre also redundant in der erweiterten Menge

$F' = \{ AB \rightarrow C, BC \rightarrow D, AB \rightarrow D \}$, da sie bereits aus der Teilmenge F folgt.

Einfachere Formulierung der Inferenzregeln

1. Augmentierung:

$\{ X \rightarrow Y \} \vdash X' \rightarrow Y$, falls $X \subseteq X'$ (links vergrößern).

2. Zerlegung oder Projektion:

$\{ X \rightarrow Y \} \vdash X \rightarrow Y'$, falls $Y' \subseteq Y$ (rechts verkleinern).

3. Vereinigung: $\{ X \rightarrow Y, X \rightarrow Y' \} \vdash X \rightarrow Y \cup Y'$.

4. Reflexivität: $\emptyset \vdash X \rightarrow Y$, falls $Y \subseteq X$ (triviale fds).

5. Akkumulation: $\{ X \rightarrow Y, Y' \rightarrow Z \} \vdash X \rightarrow Y \cup Z$, falls $Y' \subseteq Y$.

Als Spezialfall der Akkumulation erhält man für $Y = Y'$ mittels der Projektion die Transitivität: $\{ X \rightarrow Y, Y \rightarrow Z \} \vdash X \rightarrow Z$.

Für die Relation $<$ über den reellen Zahlen gilt die Transitivität z.B. auch:
für alle $X, Y, Z \in \mathbb{R}$ mit $X < Y \wedge Y < Z$ gilt $X < Z$.

Überblick

- Um die *Attributhülle* X_F^+ – die Menge aller von X funktional abhängigen Attribute – zu berechnen, ist ein spezielles generisches Ableitungsschema, die RA*P–Ableitung, richtungsweisend.
- Damit können *Schlüssel* und *Oberschlüssel* X für ein Relationenschema definiert werden.
- fd–Mengen können zu *äquivalenten* fd–Mengen vereinfacht (*minimiert*) werden.
- Darauf aufbauend werden später *Normalformen* für Relationenschemata und entsprechende *Entwurfsalgorithmen* definiert werden.

Generisches Ableitungsschema: RA*P-Ableitung

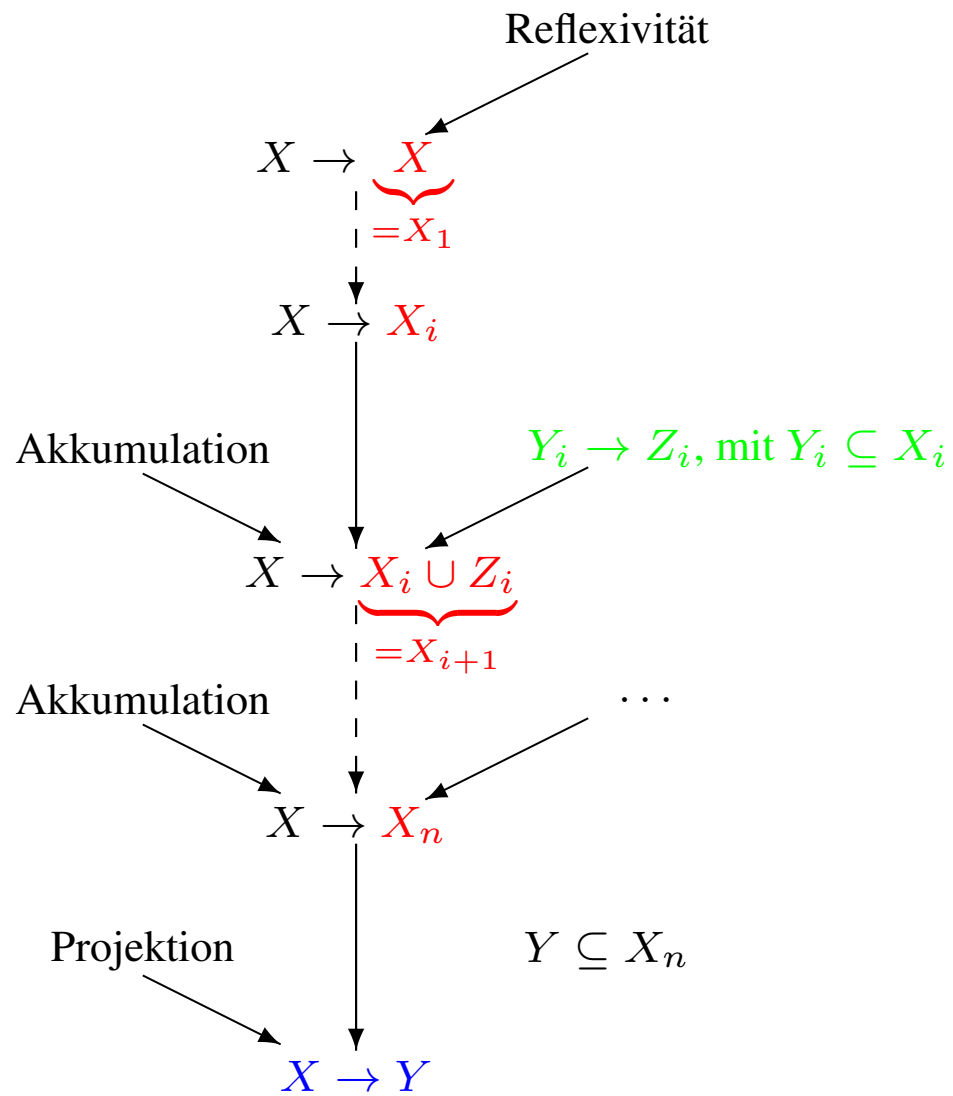
Um eine fd $X \rightarrow Y$ aus einer fd-Menge F herzuleiten, konstruieren wir induktiv eine fd-Folge $X \rightarrow X_i$, für $1 \leq i \leq n$:

(R) Wir setzen $X_1 = X$: $X \rightarrow X$ gilt wegen der Reflexivität.

(A*) $i \rightarrow i + 1$: Falls wir $Y_i \rightarrow Z_i \in F$ finden, mit $Y_i \subseteq X_i$, so kann $X \rightarrow X_i \cup Z_i$ aus $X \rightarrow X_i$ mittels Akkumulation gewonnen werden, und wir setzen $X_{i+1} = X_i \cup Z_i$.

(P) Falls $Y \subseteq X_n$ gilt, dann erhält man aus $X \rightarrow X_n$ mittels Projektion die gewünschte fd $X \rightarrow Y$.

Damit gilt $\{ Y_i \rightarrow Z_i \mid 1 \leq i \leq n \} \vdash X \rightarrow Y$. Am Ende gilt offensichtlich $X_n = X \cup Z_1 \cup \dots \cup Z_n$.



FD–Hülle, Attributhülle

Sei $R = (U, F)$ ein Relationenschema und $X \subseteq U$.

1. Die *FD–Hülle* F^+ von F besteht aus allen fds über U , die aus F folgen:

$$F^+ = \{ X \rightarrow Y \mid F \models X \rightarrow Y \}.$$

2. Die *Attributhülle* X_F^+ von X ist die maximale Attributmenge $Y \subseteq U$, so daß $X \rightarrow Y$ aus F folgt.

Die Attributhülle ist eine Obermenge von X , d.h. $X \subseteq X_F^+$, und sie besteht aus allen Attributen $A \in U$, welche bezüglich F funktional von X abhängig sind, d.h. $X_F^+ = \{ A \in U \mid F \models X \rightarrow A \}$. Es gilt $X \rightarrow X_F^+ \in F^+$.

Am Ende der RA*P–Ableitung gilt $X_n = X_F^+$.

Eigenschaften der Hüllen:

- Die FD–Hülle F^+ besteht aus allen logischen Folgerungen.
Sie ist immer sehr groß, denn sie enthält z.B. auch alle trivialen fds $X \rightarrow Y$, mit $Y \subseteq X$, und sie enthält damit exponentiell viele fds.
- Man kann den Implikationstest auf die Attributhülle X_F^+ stützen ohne die sehr große FD–Hülle F^+ zu berechnen:

$$X \rightarrow Y \in F^+ \iff F \models X \rightarrow Y \iff Y \subseteq X_F^+.$$

- Die Attributhülle X_F^+ kann mit einem einfachen Algorithmus berechnet werden. Sie enthält immer auch X , d.h.

$$X \subseteq X_F^+.$$

Wegen der Reflexivität gilt nämlich für alle $A \in X$ auch $F \models X \rightarrow A$.

Beispiel (Hülle)

Sei $U = \{ L, T, A, O, K \}$ und

$$F = \{ LT \rightarrow A, L \rightarrow O, O \rightarrow K \}.$$

Dann gilt

$$\{ L, T \}^+ = U,$$

$$\{ L \}^+ = \{ L, O, K \} \subsetneq U,$$

$$\{ T \}^+ = \{ T \} \subsetneq U.$$

Also ist $X = \{ L, T \}$ eine minimale Menge von Attributen mit $X^+ = U$.

Für die beiden um ein Attribut kleineren Mengen Y gilt $Y^+ \subsetneq U$.

Weiter gilt

$$\{ O \}^+ = \{ O, K \} \subsetneq U.$$

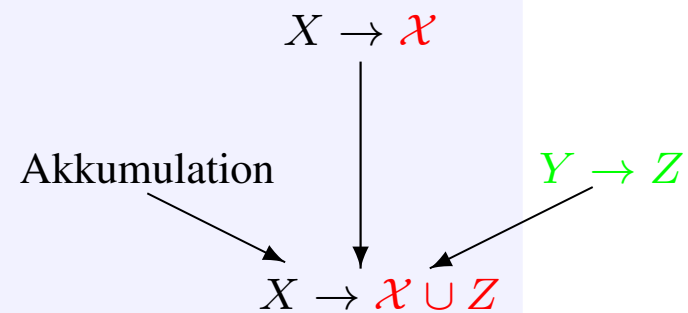
Die Attributhülle X_F^+ kann entsprechend der RA*P–Ableitung effizient berechnet werden, indem man lediglich die rechten Seiten der berechneten fd–Folge betrachtet.

Algorithmus zur Berechnung der Attributhülle

Input: Eine fd–Menge F und eine Attributmenge X

Output: $\mathcal{X} = X_F^+$

```
 $\mathcal{X} := X;$   
REPEAT  
   $\mathcal{X}_{old} := \mathcal{X};$   
  FOR EACH  $(Y \rightarrow Z \in F)$  DO  
    IF  $Y \subseteq \mathcal{X}$  THEN  $\mathcal{X} := \mathcal{X} \cup Z;$   
UNTIL  $\mathcal{X}_{old} = \mathcal{X};$ 
```



Komplexität: $O(|F| \cdot |U|^2 \cdot \log |U|)$

Beispiel (Berechnung der Attributhülle)

Für $F = \{ LT \rightarrow A, L \rightarrow O, O \rightarrow K \}$ und $X = \{ L, T \}$ ist folgende Berechnungsfolge möglich:

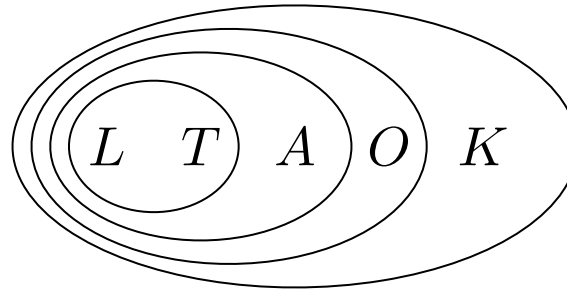
$$\mathcal{X}_1 = \{ L, T \},$$

$$\mathcal{X}_2 = \{ L, T, A \}, \text{ mittels } Y \rightarrow Z = LT \rightarrow A,$$

$$\mathcal{X}_3 = \{ L, T, A, O \}, \text{ mittels } Y \rightarrow Z = L \rightarrow O,$$

$$\mathcal{X}_4 = \{ L, T, A, O, K \} = U, \text{ mittels } Y \rightarrow Z = O \rightarrow K.$$

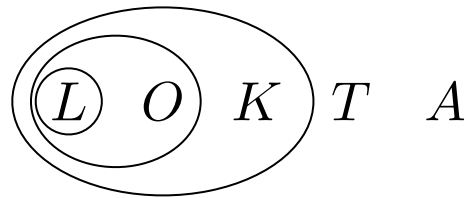
- Die Berechnung erfordert in diesem Falle nur eine Iteration.
- Für die Reihenfolge $LT \rightarrow A, O \rightarrow K, L \rightarrow O$, gibt es zwei Iterationen, da $O \rightarrow K$ erst in Iteration 2 angewendet werden kann, nachdem $L \rightarrow O$ in Iteration 1 angewendet wurde.
- Unabhängig von der Reihenfolge der fds erhält man immer dasselbe Endergebnis, in diesem Falle $\{ L, T \}_F^+ = U$.



Für $\mathcal{X}_1 = X = \{L, T\}$ können wir folgende Berechnungsfolge erhalten:

- Aufgrund der fd $LT \rightarrow A$ kann man $\mathcal{X}_1$ zu $\mathcal{X}_2 = \{L, T, A\}$ erweitern.
- Dieses wird mittels $L \rightarrow O$ zu $\mathcal{X}_3 = \{L, T, A, O\}$.
- Abschließend kann man mittels $O \rightarrow K$ zu $\mathcal{X}_4 = U$ erweitern.
- Also gilt $\{L, T\}_F^+ = U$.

Wir betrachten nun $\mathcal{X}_1 = X = \{ L \}$.



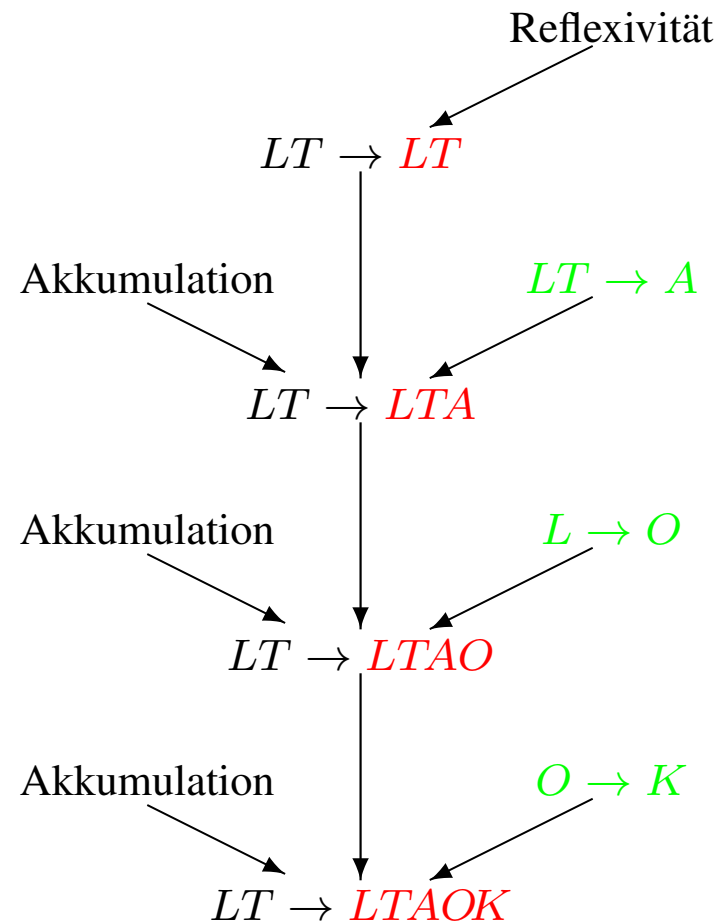
- Aufgrund der fd $L \rightarrow O$ kann man $\mathcal{X}_1$ zu $\mathcal{X}_2 = \{ L, O \}$ erweitern.
- Dieses kann man mittels $O \rightarrow K$ zu $\mathcal{X}_3 = \{ L, O, K \}$ erweitern.
- Also gilt $\{ L \}_F^+ = \{ L, O, K \}$.

Eine fd $X \rightarrow Y$ folgt aus F , genau dann wenn $Y \subseteq X_F^+$ gilt.

- Wegen $\{L, T\}_F^+ = \{L, T, A, O, K\}$ folgen aus F u.a. die fds $LT \rightarrow A, LT \rightarrow O, LT \rightarrow K, LT \rightarrow OK, LT \rightarrow L$.
- Die fd $LT \rightarrow L$ ist trivial, da die rechte Seite eine Teilmenge der linken ist. Diese fd gilt in allen Relationen über der Attributmenge U .
- Wegen $\{L\}_F^+ = \{L, O, K\}$ folgen aus F u.a. die fds $L \rightarrow K, L \rightarrow OK$.
- Die fds $L \rightarrow A$ und $T \rightarrow A$ folgen nicht aus F . Um A zu erhalten braucht man L und T zusammen.

Insbesondere folgen aus F natürlich auch alle fds, die bereits in F enthalten sind.

Ableitung aufgrund des generischen Schemas:



Schlüssel, Oberschlüssel

Sei $R = (U, F)$ ein Relationenschema und $X \subseteq U$.

1. X heißt *Oberschlüssel* von R , falls $X_F^+ = U$.
2. Ein minimaler Oberschlüssel X heißt *Schlüssel* von R (d.h., es gibt keinen kleineren Oberschlüssel $X' \subsetneq X$).

Ein Schlüssel ist also eine minimale Attributmenge X mit $X \rightarrow U$.

Beispiel (Schlüssel, Oberschlüssel)

Sei wieder $F = \{ LT \rightarrow A, L \rightarrow O, O \rightarrow K \}$.

- L und T müssen in jedem Schlüssel vorkommen, da sie in keiner fd auf der rechten Seite vorkommen.
- $X = \{ L, T \}$ ist wegen $X_F^+ = U$ bereits ein Oberschlüssel, und somit der *eindeutige Schlüssel* von R .

Algorithmus KEY

Input: Ein Relationenschema $R = (U, F)$ mit $U = \{ A_1, \dots, A_m \}$.

Output: Ein Schlüssel $K \subseteq U$ für R

```
 $K := U;$   
FOR ( $i := 1$  to  $m$ ) DO  
   $K' := K \setminus \{A_i\};$   
  IF ( $K'$  ist Oberschlüssel von  $R$ )  
    THEN  $K := K';$ 
```

Unterschiedliche Reihenfolgen $A_1, \dots, A_m$ der Attribute können verschiedene Schlüssel liefern.

Jedes Attribut, das nicht in der rechten Seite einer fd auftritt, ist in jedem Schlüssel enthalten.

Beispiel (Algorithmus KEY)

Sei $R = (U, F)$ mit

$$U = \{ A, B, C \},$$

$$F = \{ AB \rightarrow C, C \rightarrow B \}.$$

Es gibt zwei Schlüssel: $K_1 = \{ A, C \}$, $K_2 = \{ A, B \}$. Offensichtlich kann man A nicht weglassen, da es nicht auf der rechten Seite einer fd auftritt.

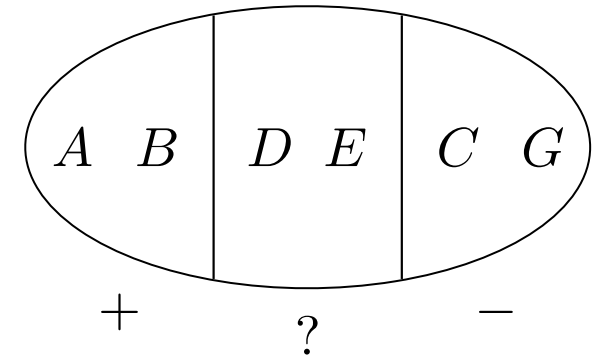
- Für die Reihenfolge $A_1 = A$, $A_2 = B$, $A_3 = C$, wird der Schlüssel K_1 berechnet, da A_1 nicht weggelassen werden kann, A_2 dann aber schon, und A_3 dann wieder nicht.
- Für die Reihenfolge $A_1 = A$, $A_2 = C$, $A_3 = B$, wird der Schlüssel K_2 durch Weglassen von C berechnet.

Ebenso hat $R = (U, F)$ mit $U = \{ A, B \}$ und $F = \{ A \rightarrow B, B \rightarrow A \}$ zwei Schlüssel: $K_1 = \{ A \}$, $K_2 = \{ B \}$.

Sei $R = (U, F)$ mit

$$U = \{ A, B, C, D, E, G \},$$

$$F = \{ AB \rightarrow C, ACD \rightarrow EG, ACE \rightarrow DG \}.$$



Offensichtlich sind A und B in jedem Schlüssel enthalten, da sie nicht auf der rechten Seite einer fd auftreten. Wegen $C \in \{ A, B \}_F^+$ ist C in keinem Schlüssel enthalten. Auch G ist in keinem Schlüssel enthalten, da es auf der rechten Seite von fds und nur da auftritt.

Also kann man A und B nicht weglassen, C und G muß man dagegen weglassen. Wir setzen also immer $A_1 = A, A_2 = B, A_3 = C, A_4 = G$.

- Für $A_5 = D, A_6 = E$ wird der Schlüssel $K_1 = \{ A, B, E \}$ berechnet.
- Für $A_5 = E, A_6 = D$ wird der Schlüssel $K_2 = \{ A, B, D \}$ berechnet.

In beiden Fällen kann A_5 weggelassen werden, und danach A_6 nicht.

Vereinfachung von fd-Mengen

Manchmal können fd-Mengen F durch Verkleinerung der rechten bzw. linken Seiten vereinfacht werden.

Definition (Rechts- und links-minimale fds)

Eine fd $X \rightarrow Y \in F$ heißt

1. rechts-minimal, falls Y einelementig ist, d.h. $Y = \{A\}$.
2. links-minimal, falls es keine echte Teilmenge $X' \subsetneq X$ mit $X' \rightarrow Y \in F^+$ gibt.

Beispiel (Minimalität)

Alle fds in $F = \{ LT \rightarrow A, L \rightarrow O, O \rightarrow K \}$ sind rechts- und links-minimal.

Beispiel (Minimalität von Fds)

1. Aufgrund der Zerlegungs- und der Vereinigungsregel ist die Menge F äquivalent zur Menge G von *rechts-minimalen* fds:

$$F = \{ X \rightarrow A_1 \dots A_m \},$$

$$G = \{ X \rightarrow A_1, \dots, X \rightarrow A_m \}.$$

2. Man darf also die rechte Seite einer fd aufsplitten:

$X \rightarrow AB$ ist äquivalent zu zwei *rechts-minimalen* fds

$X \rightarrow A$ und $X \rightarrow B$.

3. Die *linke Seite* einer fd darf man aber nicht aufsplitten:

$AB \rightarrow Y$ ist schwächer als $A \rightarrow Y$ und $B \rightarrow Y$.

Sowohl aus $A \rightarrow Y$ als auch aus $B \rightarrow Y$ folgt $AB \rightarrow Y$,
aber nicht umgekehrt.

4. Manchmal kann man die *linke Seite* einer fd im Kontext einer fd-Menge *verkleinern*:

Die fd $AB \rightarrow C$ ist nicht *links-minimal* in

$$F = \{ AB \rightarrow C, A \rightarrow B \}.$$

Sie kann zu $A \rightarrow C$ verkleinert werden, da $A \rightarrow B$ gilt.

Die Menge

$$G = \{ A \rightarrow C, A \rightarrow B \}$$

von *links-minimalen* fds ist äquivalent zu F .

Alle fds in F und G sind *rechts-minimal*.

Zu jeder fd-Menge F gibt es (mindestens) eine äquivalente Menge G von *rechts-* und *links-minimalen* fds.

Links-Minimalität

Analog zur Schlüsselberechnung kann man eine fd $X \rightarrow Y \in F$ mit

$$X = \{ A_1, \dots, A_m \}$$

wie folgt links-minimal machen:

```
 $\mathcal{X} := X;$   
FOR ( $i := 1$  to  $m$ ) DO  
   $\mathcal{X}' := \mathcal{X} \setminus \{A_i\};$   
  IF  $F \models \mathcal{X}' \rightarrow Y$   
  THEN  $\mathcal{X} := \mathcal{X}';$ 
```

Am Ende ist $\mathcal{X} \rightarrow Y \in F^+$ links-minimal und $\mathcal{X} \subseteq X$.

Genau wie bei der Schlüsselberechnung können wir – je nach der Reihenfolge der A_i – unterschiedliche Ergebnisse erhalten.

Beispiel (Links–Minimalität)

Sei $R = (U, F)$ mit

$$U = \{ A, B, C \},$$

$$F = \{ AB \rightarrow C, A \rightarrow B, B \rightarrow A \}.$$

Die fd $AB \rightarrow C$ ist nicht links–minimal.

- Für die Reihenfolge $A_1 = A, A_2 = B$ wird die links–minimale fd $B \rightarrow C$ berechnet.
- Für die Reihenfolge $A_1 = B, A_2 = A$ wird die links–minimale fd $A \rightarrow C$ berechnet.

In beiden Fällen kann A_1 weggelassen werden, und A_2 dann aber nicht.

Projektion von Relationenschemas

Definition (Projektion)

Die *Projektion* $\Pi_V(R) = (V, \Pi_V(F))$ eines Relationenschemas $R = (U, F)$ auf eine Attributmenge $V \subseteq U$ ist gegeben durch

$$\Pi_V(F) = \{ X \rightarrow Y \in F^+ \mid XY \subseteq V \}.$$

Eigenschaften (Projektion)

1. Falls F in einer Relation r über U gilt, so gilt $\Pi_V(F)$ in $\Pi_V(r)$.
2. $X \rightarrow Y$ gilt in R , falls X ein Oberschlüssel von $\Pi_{XY}(R)$ ist.

Die Projektion ist beim Schemaentwurf für den Synthese–Algorithmus nicht erforderlich, wohl aber für den Dekompositions–Algorithmus.

Beispiel (Projektionen von Lieferant)

In der Projektion $\Pi_{LK}(r)$ der Lieferantenrelation r gilt die fd $L \rightarrow K$. Man erhält sie durch transitive Verknüpfung von $L \rightarrow O$ und $O \rightarrow K$.

$\Pi_{LK}(r) :$

L	K
L1	600
L2	450
L3	450
L4	150

L ist ein Oberschlüssel von $\Pi_{LK}(R) = (LK, \Pi_{LK}(F))$. In R ist L kein Oberschlüssel, da ein Lieferant in der Regel mehrere Teile liefert.

Es gilt $\Pi_{LK}(F) \equiv \{ L \rightarrow K \}$.

Berechnung (Projektion)

- Im schlechtesten Fall ist die Berechnung von $\Pi_V(F)$ sehr aufwendig, und zwar genau so komplex wie die von F^+ .

In der Praxis ist sie aber meist nicht so schwierig.

- Um eine zu $\Pi_V(F)$ äquivalente Menge G zu erhalten, genügt es z.B., zu jedem $X \subseteq V$, das Obermenge der linken Seite einer fd aus F ist, die fd $X \rightarrow Y$ zu berechnen, mit

$$Y = (X_F^+ \setminus X) \cap V.$$

Falls $Y \neq \emptyset$, so nehmen wir $X \rightarrow Y$ in G auf. Andernfalls gibt es in G nur triviale Folgerungen aus X .

Beispiel (Projektionen von Lieferant)

$$U = \{ L, T, A, O, K \},$$

$$F = \{ LT \rightarrow A, L \rightarrow O, O \rightarrow K \}.$$

3 Projektionen

$$\Pi_{LTA}(F) \equiv \{ LT \rightarrow A \},$$

$$\Pi_{LO}(F) \equiv \{ L \rightarrow O \},$$

$$\Pi_{LK}(F) \equiv \{ L \rightarrow K \}.$$

Hier wurden triviale fds weggelassen, da sie redundant sind.

- Sei $V = LTA$. Für $X = LT$ gilt $X_F^+ = U$ und $(X_F^+ \setminus X) \cap V = A$.
Für $X = L$ gilt $X_F^+ = LOK$ und $(X_F^+ \setminus X) \cap V = \emptyset$.
- Sei $V = LK$. Wir erhalten die fd $L \rightarrow K$ aus $L \rightarrow O$ und $O \rightarrow K$.

Die fd $O \rightarrow K$ ist in keiner der drei Projektionen erhalten. Man könnte sie aber im Join der beiden Projektionen auf LO bzw. LK überprüfen.

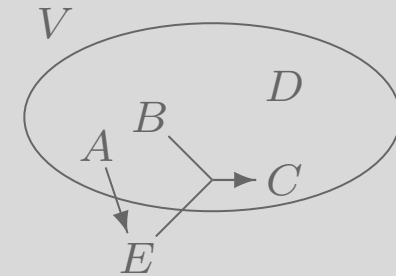
Beispiel (Projektion einer fd-Menge)

$$U = \{ A, B, C, D, E, I \},$$

$$F = \{ A \rightarrow E, BE \rightarrow C \}.$$

Für $V = \{ A, B, C, D \}$ erhält man die Projektion

$$\Pi_V(F) \equiv \{ AB \rightarrow C \}.$$



Es genügt die Menge $X = \{ A, B \} \subseteq V$ zu betrachten:

- Zur Ableitung einer fd aus $\Pi_V(F)$ muß man mit einer fd starten, deren linke Seite in V liegt. Die einzige solche fd ist $A \rightarrow E$.
- Wenn man $\{A\}_F^+ = \{A, E\}$ um B erweitert, so kann man die zweite fd $BE \rightarrow C$ anwenden.
- Man erhält $\{A, B\}_F^+ = \{A, B, C, E\}$, d.h. $AB \rightarrow C \in \Pi_V(F)$.

Um $AB \rightarrow C$ abzuleiten muß man zwischenzeitlich V verlassen.

3.3 Normalformen

Definition (1NF)

Ein Relationenschema $R = (U, F)$ ist in *erster Normalform* (1NF), falls U nur atomare Attribute enthält, d.h. keine mehrwertigen oder zusammengesetzten Attribute.

Ein Relationenschema, das nicht in 1NF ist, und eine Ausprägung davon:

DEPARTMENT			
DNAME	<u>DNUMBER</u>	DMGRSSN	DLOCATIONS
			
DEPARTMENT			
DNAME	<i>DNUMBER</i>	DMGRSSN	DLOCATIONS
Headquarters	1	11111111	{ Houston }
Administration	4	33333333	{ Stafford }
Research	5	22222222	{ Bellaire, Sugarland, Houston }

Normalisierung auf 1NF

Relation in 1NF mit Redundanz:

DEPARTMENT			
DNAME	<i>DNUMBER</i>	DMGRSSN	<i>DLOCATION</i>
Headquarters	1	111111111	Houston
Administration	4	333333333	Stafford
Research	5	222222222	Bellaire
Research	5	222222222	Sugarland
Research	5	222222222	Houston

Der eindeutige Schlüssel ist $K = \{ DNUMBER, DLOCATION \}$.

Es gilt die fd $\{ DNUMBER \} \rightarrow \{ DNAME, DMGRSSN \}$, obwohl die linke Seite kein Oberschlüssel ist. Deswegen sind die Werte zu den Attributen aus der rechten Seite redundant gespeichert.

Wir werden später sehen, daß man diese Relation verlustfrei in die folgenden zwei Relationen zerlegen kann, um die Redundanz zu beseitigen:

DEPARTMENT			DEPT_LOCATIONS	
DNAME	<i>DNUMBER</i>	DMGRSSN	<i>DNUMBER</i>	<i>DLOCATION</i>
Headquarters	1	111111111	1	Houston
Administration	4	333333333	4	Stafford
Research	5	222222222	5	Bellaire
			5	Sugarland
			5	Houston

In der Tabelle DEPT_LOCATIONS werden lediglich die Werte des Attributs *DNUMBER* repliziert (z.B. der Wert 5), das die Verbindung zum Schlüssel der Tabelle DEPARTMENT herstellt (Fremdschlüsselbedingung).

Weitere Normalformen

Definition (Schlüsselattribut)

Sei $R = (U, F)$ ein Relationenschema.

1. $A \in U$ heißt *Schlüsselattribut*, bzw. *prim*, falls es einen Schlüssel K für R gibt mit $A \in K$.
2. Andernfalls heißt A *Nichtschlüsselattribut* (NSA).

Beispiel (Schlüssel von Lieferant)

Der eindeutige Schlüssel für das Lieferantenschema $R = (U, F)$ mit

$$U = \{ L, T, A, O, K \},$$

$$F = \{ LT \rightarrow A, L \rightarrow O, O \rightarrow K \},$$

ist $\mathcal{K} = \{ L, T \}$. Deswegen sind die Attribute L und T prim, und A , O und K sind Nichtschlüsselattribute.

Definition (2NF)

Ein 1NF–Relationenschema $R = (U, F)$ ist in *zweiter Normalform* (2NF), falls für jedes NSA $A \in U$ und jeden Schlüssel K für R gilt, daß die fd $K \rightarrow A$ *links–minimal* ist.

Beispiel (2NF)

Das Lieferantenschema

$$U = \{ L, T, A, O, K \},$$

$$F = \{ LT \rightarrow A, L \rightarrow O, O \rightarrow K \},$$

ist nicht in 2NF, da die fd $LT \rightarrow O$ zum eindeutigen Schlüssel $\mathcal{K} = \{ L, T \}$ wegen $L \rightarrow O$ nicht links–minimal ist, und da O ein NSA ist.

Definition (Normalformen: 3NF, BCNF)

Sei $R = (U, F)$ ein Relationenschema in 1NF.

1. Eine nicht-triviale fd $X \rightarrow Y$ heißt *3NF-verletzend*, falls X kein Overschlüssel von R ist, d.h. falls $X_F^+ \neq U$ gilt, und falls es ein Nichtschlüsselattribut $A \in Y \setminus X$ gibt.
2. R ist in *dritter Normalform* (3NF), falls es keine 3NF-verletzende fd $X \rightarrow Y \in F$ gibt.
3. Eine nicht-triviale fd $X \rightarrow Y$ heißt *BCNF-verletzend*, falls X kein Overschlüssel von R ist, d.h. falls $X_F^+ \neq U$ gilt.
4. R ist in *Boyce-Codd-Normalform* (BCNF), falls es keine BCNF-verletzende fd $X \rightarrow Y \in F$ gibt.

Da jede 3NF-verletzende fd auch BCNF-verletzend ist, folgt aus der BCNF die 3NF. Später zeigen wir auch, daß aus der 3NF die 2NF folgt.

Beispiele (Normalformen: Vergleich und Abgrenzung)

1. Sei $R = (U, F)$ mit $U = ABCD$ und $F = \{ AB \rightarrow C, BC \rightarrow D \}$.

Dann ist AB der eindeutige Schlüssel.

- Die fds $AB \rightarrow C$ und $AB \rightarrow D$ zu den NSAs sind *links-minimal*,
- aber $BC \rightarrow D$ ist 3NF-verletzend.

Deshalb ist R zwar in 2NF, aber nicht in 3NF.

2. Sei $R = (U, F)$ mit $U = ABC$ und $F = \{ AB \rightarrow C, C \rightarrow B \}$.

Dann gibt es die zwei Schlüssel AB und AC .

- Folglich sind alle Attribute *prim*.
- Aber $C \rightarrow B$ ist BCNF-verletzend.

Deshalb ist R zwar in 3NF, aber nicht in BCNF.

Definition (Transitive Abhängigkeit) $X \xrightarrow{\neq} Y \rightarrow A$

Sei $R = (U, F)$ ein Relationenschema und $X \rightarrow A \in F^+$, mit $A \in U$.

1. $X \rightarrow A$ heißt *transitiv*, falls es eine nicht-triviale fd $Y \rightarrow A \in F^+$ gibt, so daß $X \rightarrow Y \in F^+$ und $Y \rightarrow X \notin F^+$. Dann heißt A auch *transitiv abhängig* von X , und $X \xrightarrow{\neq} Y \rightarrow A$ heißt *transitive Kette*.
2. Ansonsten heißt die fd $X \rightarrow A$ *direkt*.

Beispiel (Lieferant, Department)

1. $LT \rightarrow A$ ist direkt.
 $LT \xrightarrow{\neq} L \rightarrow O$ ist eine transitive Kette.
 $LT \xrightarrow{\neq} O \rightarrow K$ ist eine transitive Kette.
2. $\{ \text{DNUMBER}, \text{DLOCATION} \} \xrightarrow{\neq} \{ \text{DNUMBER} \} \rightarrow A$
 ist für $A = \text{DNAME}$ und $A = \text{DMGRSSN}$ eine transitive Kette.

Die Menge F^* aller rechts-minimalen fds $X \rightarrow A_i$ zu den fds $X \rightarrow \{A_1, \dots, A_n\} \in F$ ist äquivalent zu F .

BCNF-verletzende fds aus F^* und transitive Ketten korrelieren:

1. Bei einer transitiven Kette $X \xrightarrow{\neq} Y \rightarrow A$ ist die fd $Y \rightarrow A$ BCNF-verletzend, sie muß aber nicht in F^* liegen.

Es gibt aber eine fd $Y' \rightarrow A \in F^*$, so daß $Y' \subseteq Y_F^+$, z.B. diejenige fd, die bei der Hüllenberechnung von $Y_F^+ = Y_{F^*}^+$ verwendet wird, so daß A hinzukommt. Dann gilt $Y' \rightarrow X \notin F^+$, sonst wäre $Y \rightarrow X \in F^+$.

Also ist $X \xrightarrow{\neq} Y' \rightarrow A$ ebenfalls eine transitive Kette, und $Y' \rightarrow A \in F^*$ ist BCNF-verletzend.

2. Für jede BCNF-verletzende fd $Y \rightarrow A \in F^*$ und jeden Schlüssel X ist $X \xrightarrow{\neq} Y \rightarrow A$ eine transitive Kette.

Redundanz und Update–Anomalien

Sei r eine erlaubte Instanz eines Relationenschemas $R = (U, F)$.

- Eine BCNF–verletzende fd $X \rightarrow Y \in F$ bereitet ein Problem.
Dasselbe gilt auch, falls $X \rightarrow Y$ nur in F^+ enthalten ist.
- Da X kein Overschlüssel ist, kann es in r verschiedene Tupel mit denselben X –Werten geben.
Dann müssen auch die Y –Werte gleich sein, und es kommt zu Redundanz und zu den bekannten Update–Anomalien.
- Falls $Y \setminus X$ allerdings nur Schlüsselattribute enthält, so ist das Problem nicht ganz so groß, da diese weniger häufig geändert werden.
Dann ist die fd nicht 3NF–verletzend.

Beispiel (Lieferant)

$$U = \{ L, T, A, O, K \}, F = \{ LT \rightarrow A, L \rightarrow O, O \rightarrow K \}.$$

Die 3NF-verletzende fd $O \rightarrow K$ führt zu den bekannten Update-Anomalien.

Da O kein Overschlüssel ist, gibt es verschiedene Tupel mit denselben O -Werten, und diese müssen dann auch dieselben K -Werte haben.

L	T	A	O	K
L1	T1	300	London	600
L1	T2	200	London	600
L1	T3	400	London	600
L1	T4	200	London	600
L1	T5	100	London	600
L1	T6	100	London	600
L2	T1	300	Paris	450
L2	T2	400	Paris	450
L3	T2	200	Paris	450
L4	T2	200	Brüssel	150
L4	T4	300	Brüssel	150
L4	T5	400	Brüssel	150

Aus der Korrelation zwischen BCNF/3NF–verletzenden fds und transitiven Ketten ergibt sich der folgende Satz.

Satz (Normalformen)

Sei $R = (U, F)$ ein Relationenschema in 1NF.

1. R ist in 3NF, falls für jedes NSA $A \in U$ und jeden Schlüssel K gilt, daß die fd $K \rightarrow A$ *direkt* ist.
2. R ist in BCNF, falls für jedes Attribut $A \in U$ und jeden Schlüssel K gilt, daß die fd $K \rightarrow A$ *direkt* ist.

Da direkte fds auch links–minimal sind, folgt weiter:

Satz (Normalformen)

Sei $R = (U, F)$ ein Relationenschema in 1NF.

1. Ist R in BCNF, so ist R auch in 3NF.
2. Ist R in 3NF, so ist R auch in 2NF.

Beispiele (Normalformen: Vergleich und Abgrenzung)

1. Sei $R = (U, F)$ mit $U = ABCD$ und $F = \{ AB \rightarrow C, BC \rightarrow D \}$.

Dann ist AB der eindeutige Schlüssel. R ist zwar in $2NF$, aber wegen der folgenden transitiven Kette nicht in $3NF$:

$$AB \not\rightarrow BC \rightarrow D.$$

2. Sei $R = (U, F)$ mit $U = ABC$ und $F = \{ AB \rightarrow C, C \rightarrow B \}$.

Dann gibt es die zwei Schlüssel AB und AC , und alle Attribute sind *prim*. R ist zwar in $3NF$, aber wegen der folgenden transitiven Ketten nicht in $BCNF$:

$$AB \not\rightarrow C \rightarrow B, \quad AC \not\rightarrow C \rightarrow B.$$

Satz (3NF–Test, BCNF–Test)

Sei $R = (U, F)$ ein Relationenschema in 1NF.

Das Problem zu entscheiden,

1. ob ein Attribut $A \in U$ *prim* ist, ist $\mathcal{NP}$ –vollständig.
2. ob R in 3NF ist, ist $\mathcal{NP}$ –vollständig.
3. ob R in BCNF ist, ist in polynomieller Zeit lösbar.

In praktischen Beispielen ist die Anzahl $|U|$ der Attribute und die Anzahl $|F|$ der funktionalen Abhängigkeiten aber nicht sehr groß.

Deswegen stellen die (asymptotischen) Komplexitätsresultate zur $\mathcal{NP}$ –Vollständigkeit kein Problem dar.

Wir werden später sehen, wie wir ein Relationenschema $R = (U, F)$ aufgrund seiner fds verlustfrei in kleinere Relationenschemata in 3NF bzw. BCNF zerlegen können.

Dazu verwenden wir 3NF-kompatible bzw. BCNF-verletzende fds.

Definition (3NF/BCNF-Kompatibilität)

Sei $R = (U, F)$ ein Relationenschema.

Eine fd $X \rightarrow Y$ heißt *3NF-kompatibel* (BCNF-kompatibel), falls das Relationenschema $\Pi_{XY}(R)$ in 3NF (BCNF) ist.

Satz (3NF-Kompatibilität)

Jede *links-minimale* und *rechts-minimale* fd $X \rightarrow A \in F^+$ in einem Relationenschema $R = (U, F)$ ist *3NF-kompatibel*.

Beweis. Angenommen $R' = \Pi_{XA}(R)$ ist nicht in 3NF.

Dann gibt es eine transitive Kette $XA \not\rightarrow Y \rightarrow A'$ zu einem NSA $A' \in XA$, mit $A' \notin Y$. Wegen $Y \not\rightarrow XA$ folgt $YA' \subsetneq XA$.

Wir betrachten zwei Fälle:

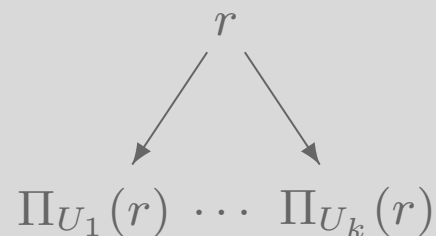
1. $A = A'$: Es folgt $Y \subsetneq X$, und es gilt die fd $Y \rightarrow A$.
Also ist $X \rightarrow A$ nicht links-minimal. $\nrightarrow$
2. $A \neq A'$: In diesem Fall folgt $A' \in X$. Da A' ein NSA und X ein Oberschlüssel von R' ist, existiert ein Schlüssel $K \subsetneq X$ mit $A' \notin K$.
Es gilt dann $K \rightarrow A$, d.h. $X \rightarrow A$ ist nicht links-minimal. $\nrightarrow$

Also ist das Schema R' in 3NF, d.h. $X \rightarrow A$ ist 3NF-kompatibel. □

3.4 Mehrwertige und Verbund-Abhängigkeiten

Die Join-Bedingung $\bowtie (U_1, \dots, U_k)$ besagt, daß man eine Relation r verlustfrei in die Projektionen $\Pi_{U_i}(r)$ zerlegen kann:

$$r = \Pi_{U_1}(r) \bowtie \dots \bowtie \Pi_{U_k}(r)$$



Eine mehrwertige Abhängigkeit ist eine Join-Bedingung mit $k = 2$.

Wir können sie als $X \twoheadrightarrow Y \mid Z$ schreiben, wobei

$$X = U_1 \cap U_2, \quad Y = U_1 \setminus U_2, \quad Z = U_2 \setminus U_1.$$

Eine Menge $\{X \twoheadrightarrow Y_1, \dots, X \twoheadrightarrow Y_m\}$ von mvds mit derselben linken Seite X kann als $X \twoheadrightarrow Y_1 \mid Y_2 \mid \dots \mid Y_m$ abgekürzt werden.

Mehrwertige Abhängigkeiten sind *intra*–relationale Abhängigkeiten, Verbund–Abhängigkeiten sind *inter*–relational.

Falls ein Relationenschema *mehrwertige Attribute* enthält, so gelten in der 1NF–Normalisierung mehrwertige Abhängigkeiten.

Beispiel (Mehrwertige Abhängigkeit)

Wir betrachten ein Relationenschema mit der Attributmenge $U = ABC$ und zwei mehrwertigen Attributen B und C :

	A	B	C
r' :	a_1	$\{ b_1, b_2 \}$	$\{ c_1, c_2, c_3 \}$
	a_2	$\{ b_3 \}$	$\{ c_3, c_4 \}$

1. Dann ist die 1NF–Normalisierung von r' gegeben durch r .

Zwischen B und C besteht in r eigentlich kein *direkter Zusammenhang*.

Dies wird durch die mvd $\{A\} \twoheadrightarrow \{B\} \mid \{C\}$ ausgedrückt.

r :

A	B	C
a_1	b_1	c_1
a_1	b_1	c_2
a_1	b_1	c_3
a_1	b_2	c_1
a_1	b_2	c_2
a_1	b_2	c_3
a_2	b_3	c_3
a_2	b_3	c_4

r_1 :

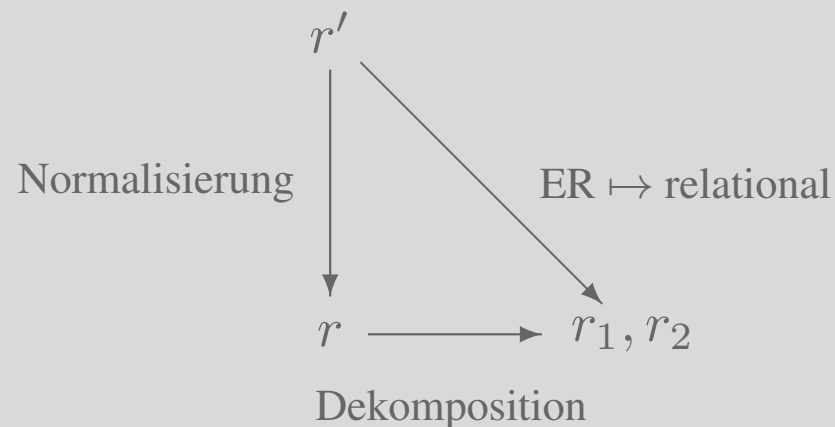
A	B
a_1	b_1
a_1	b_2
a_2	b_3

r_2 :

A	C
a_1	c_1
a_1	c_2
a_1	c_3
a_2	c_3
a_2	c_4

2. Äquivalent kann man r und r' mit den zwei Relationen $r_1 = \Pi_{AB}(r)$ und $r_2 = \Pi_{AC}(r)$ repräsentieren. Man erhält r dann als deren Natural Join: $r = r_1 \bowtie r_2$.

Man kann die Relation r' , die nicht in 1NF ist, zuerst zu einer 1NF-Relation r normalisieren. Dann kann man r mittels einer Dekompositions-Methode, die wir im nächsten Abschnitt kennen lernen werden, verlustfrei in zwei kompaktere Relationen r_1 und r_2 zerlegen:



Alternativ können wir die beiden Relationen r_1 und r_2 auch durch die bereits bekannte Abbildung vom ER-Modell ins relationale Modell erhalten.

Schließlich könnte man r' auch in zwei Relationen zerlegen, die nicht in 1NF sind, $r'_1 = \Pi_{AB}(r')$, $r'_2 = \Pi_{AC}(r')$, und diese dann zu r_1, r_2 normalisieren.

Die verlustfreie Zerlegbarkeit einer Relation r in zwei Projektionen $r_1 = \Pi_{U_1}(r)$ und $r_2 = \Pi_{U_2}(r)$ bedeutet, daß $r = r_1 \bowtie r_2$ gilt.

- In unserem Beispiel sind die beiden mvds $\{A\} \twoheadrightarrow \{B\}$ bzw. $\{A\} \twoheadrightarrow \{C\}$, die in r gelten, verantwortlich für die verlustfreie Zerlegbarkeit. Wir werden sehen, daß die beiden mvds äquivalent sind.
- Wir werden zeigen, daß sich eine Relation genau dann verlustfrei in zwei Projektionen $\Pi_{U_1}(r)$ und $\Pi_{U_2}(r)$ zerlegen läßt, wenn die beiden äquivalenten mvds $U_1 \cap U_2 \twoheadrightarrow U_1 \setminus U_2$ und $U_1 \cap U_2 \twoheadrightarrow U_2 \setminus U_1$ gelten.
- Solche Datenabhängigkeiten nennen wir auch binäre Verbundabhängigkeiten (Join Dependencies, jds), und wir schreiben $\bowtie (U_1, U_2)$.
- In unserem Beispiel gilt die jd $\bowtie (\{A, B\}, \{A, C\})$ in r .

Definition (Mehrwertige Abhängigkeit, mvd)

Sei U eine Attributmenge, sowie $X, Y \subseteq U$ und $Z = U \setminus XY$.

1. Eine *mehrwertige Abhängigkeit* (engl.: multivalued dependency, mvd)

$X \twoheadrightarrow Y$ gilt für eine Relation r über U , falls

$$\begin{aligned} \forall \mu, \nu \in r : (\mu[X] = \nu[X] \Rightarrow \\ (\exists \rho \in r : \rho[X] = \mu[X] = \nu[X] \wedge \\ \rho[Y] = \mu[Y] \wedge \rho[Z] = \nu[Z])). \end{aligned}$$

2. Eine mvd $X \twoheadrightarrow Y$ heißt *trivial*, falls $Y \subseteq X$ oder falls $X \cup Y = U$; sie gilt in allen Relationen.

Logische Folgerung: Sie Σ eine Menge von Datenabhängigkeiten und δ eine einzelne Datenabhängigkeit:

$\Sigma \models \delta$, falls in jeder Relation r , in welcher Σ gilt, auch δ gilt.

- Während eine fd $X \rightarrow Y$ mit $X \cup Y = U$ ausdrückt, daß X ein Oberschlüssel ist, ist eine entsprechende mvd $X \twoheadrightarrow Y$ trivial. Zum Beweis nehme man $\rho = \mu$: dann gilt $\rho[Y] = \mu[Y]$, und da $Z = U \setminus XY = \emptyset$ ist, gilt automatisch auch $\rho[Z] = \nu[Z]$.
- Eine Relation r über U mit einer mvd $X \twoheadrightarrow Y$, wobei $Z = U \setminus XY$, könnte man besser als eine NF^2 -Relation r' repräsentieren,

		X	Y	Z				
r :	μ	x	y_1	z_1	r' :	X	Y	Z
	ν	x	y_2	z_2		x	$\{y_1, y_2\}$	$\{z_1, z_2\}$
	ρ	x	y_1	z_2				
	ρ'	x	y_2	z_1				

oder als zwei Relationen $\Pi_{XY}(r)$ und $\Pi_{XZ}(r)$.

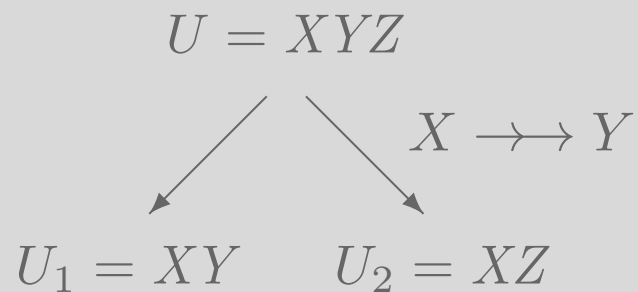
Satz (Dekomposition)

Seien $X, Y \subseteq U$ und $Z = U \setminus XY$.

Dann gilt die mvd $X \twoheadrightarrow Y$ in einer Relation r über U , g.d.w.

$$r = \Pi_{XY}(r) \bowtie \Pi_{XZ}(r).$$

D.h. die Zerlegung in $U_1 = XY$ und $U_2 = XZ$ ist *verlustfrei*:



Der folgende Beweis verwendet wiederholt eine – triviale – Eigenschaft von Tupeln t : $(t[X_1])[X_2] = t[X_2]$, falls $X_2 \subseteq X_1 \subseteq \text{def}(t)$.

Beweis.

“ $\Rightarrow$ ”: Angenommen $X \twoheadrightarrow Y$ gilt in r .

Es genügt zu zeigen: $r \supseteq \Pi_{XY}(r) \bowtie \Pi_{XZ}(r)$.

Sei also $\rho \in \Pi_{XY}(r) \bowtie \Pi_{XZ}(r)$. Dann existieren Tupel

$$\mu' = \rho[XY] \in \Pi_{XY}(r),$$

$$\nu' = \rho[XZ] \in \Pi_{XZ}(r).$$

Somit existieren auch Tupel $\mu, \nu \in r$ mit $\mu' = \mu[XY]$, $\nu' = \nu[XZ]$.

Nun folgt

$$\mu[X] = \mu'[X] = \rho[X] = \nu'[X] = \nu[X],$$

$$\mu[Y] = \mu'[Y] = \rho[Y],$$

$$\nu[Z] = \nu'[Z] = \rho[Z].$$

Aus der Gültigkeit von $X \twoheadrightarrow Y$ in r folgt nun $\rho \in r$.

“ $\Leftarrow$ ”: Angenommen $r = \Pi_{XY}(r) \bowtie \Pi_{XZ}(r)$.

Seien $\mu, \nu \in r$ mit $\mu[X] = \nu[X]$, und sei

$$\mu' = \mu[XY] \in \Pi_{XY}(r),$$

$$\nu' = \nu[XZ] \in \Pi_{XZ}(r).$$

Sei das Tupel ρ gegeben durch

$$\rho[X] = \mu[X] = \nu[X],$$

$$\rho[Y] = \mu[Y],$$

$$\rho[Z] = \nu[Z].$$

Dann gilt $\rho \in \Pi_{XY}(r) \bowtie \Pi_{XZ}(r) = r$.

Also gilt die mvd $X \twoheadrightarrow Y$ in r .



Inferenzregeln für mvds

Das folgende Regelsystem (M1)–(M4) ist vollständig und korrekt für mvds.

Seien $X, Y, Z, W, W' \subseteq U$:

(M1) Komplement:

$$\{ X \twoheadrightarrow Y \} \vdash X \twoheadrightarrow U \setminus Y, X \twoheadrightarrow U \setminus (XY).$$

(M2) Reflexivität (triviale mvd):

$$\emptyset \vdash X \twoheadrightarrow Y, \text{ falls } Y \subseteq X.$$

(M3) Erweiterung:

$$\{ X \twoheadrightarrow Y \} \vdash XW \twoheadrightarrow YW', \text{ falls } W' \subseteq W.$$

(M4) Pseudo-Transitivität:

$$\{ X \twoheadrightarrow Y, Y \twoheadrightarrow Z \} \vdash X \twoheadrightarrow Z \setminus Y.$$

Inferenzregeln für fds und mvds

Seien $X, Y, X', Y' \subseteq U$

(FM1) Replikation:

$$\{ X \rightarrow Y \} \vdash X \twoheadrightarrow Y.$$

(FM2) Koaleszenz:

$$\{ X \twoheadrightarrow Y, X' \rightarrow Y' \} \vdash X \rightarrow Y \cap Y', \text{ falls } X' \cap Y = \emptyset.$$

Satz (Kalkül für fds und mvds)

Sei $\Sigma = F \cup M$ eine Menge von fds und mvds. Dann ist die Regel-Menge

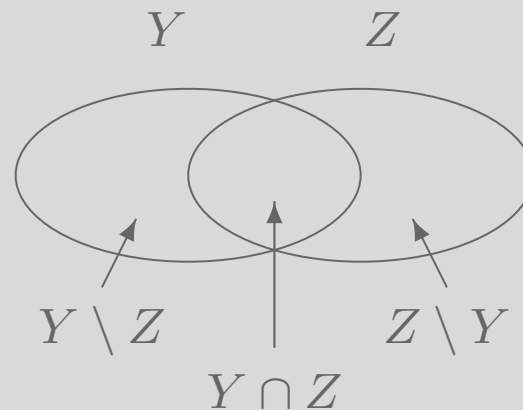
$$\{ (F1),(F2),(F3), (M1),(M2),(M3),(M4), (FM1),(FM2) \}$$

vollständig und korrekt zur Ableitung der aus Σ folgenden fds und mvds.

Weitere Inferenzregeln für mvds, Vergleich mit fds

1. Man kann die rechte und linke Seite einer mvd disjunkt machen, denn aus $X \twoheadrightarrow Y$ folgt $X \twoheadrightarrow Y \setminus X$.
2. Partitions-Regel: Man kann die rechten Seiten von mvds zur selben linken Seite X disjunkt machen, denn aus $X \twoheadrightarrow Y \mid Z$ folgt

$$X \twoheadrightarrow Y \cap Z \mid Y \setminus Z \mid Z \setminus Y.$$



3. Vereinigungs-Regel: Man kann die rechten Seiten von mvds zur selben linken Seite X vereinigen, denn aus $X \twoheadrightarrow Y \mid Z$ folgt $X \twoheadrightarrow Y \cup Z$.

4. Wegen der Vereinigungs–Regel folgt aus

$$X \twoheadrightarrow Y \cap Z \mid Y \setminus Z \mid Z \setminus Y$$

auch $X \twoheadrightarrow Y \mid Z$, denn

$$Y = (Y \cap Z) \cup (Y \setminus Z),$$

$$Z = (Y \cap Z) \cup (Z \setminus Y).$$

5. Also ist $X \twoheadrightarrow Y \mid Z$ äquivalent zu $X \twoheadrightarrow Y \cap Z \mid Y \setminus Z \mid Z \setminus Y$.

Z.B. ist $A \twoheadrightarrow BC \mid CD$ äquivalent zu $A \twoheadrightarrow B \mid C \mid D$, denn für $Y = BC$ und $Z = CD$ gilt $Y \cap Z = C$, $Y \setminus Z = B$, $Z \setminus Y = D$.

6. Die Zerlegungs–Regel gilt für mvds nicht:

$X \twoheadrightarrow Y \cup Z$ ist schwächer als $X \twoheadrightarrow Y \mid Z$.

Dagegen ist eine fd $X \rightarrow Y \cup Z$ wegen der Zerlegungs– und der Vereinigungs–Regel zur Menge $\{ X \rightarrow Y, X \rightarrow Z \}$ von fds äquivalent.

Abhängigkeitsbasis $DEP_G(X)$

Sei G eine Menge von mvds und $X = \{A_1, \dots, A_n\} \subseteq U$.

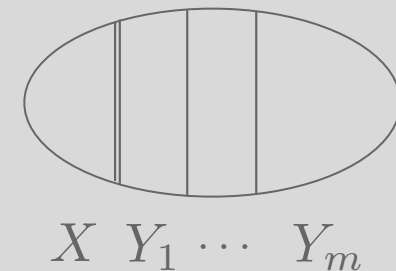
1. Aus der Reflexivitäts- und der Partitions-Regel folgt, daß es eine eindeutige Partition

$$DEP_G(X) = \{ \overbrace{\{A_1\}}^{Y_{m+1}}, \dots, \overbrace{\{A_n\}}^{Y_{m+n}}, Y_1, \dots, Y_m \}$$

von U gibt, so daß $G \models X \twoheadrightarrow Y$ gilt, genau dann wenn

- es Mengen $Y_{i_1}, \dots, Y_{i_k} \in DEP_G(X)$ gibt
- mit $Y = Y_{i_1} \cup \dots \cup Y_{i_k}$.

2. Wir schreiben dann $X \twoheadrightarrow Y_1 \mid Y_2 \mid \dots \mid Y_m$.



Vergleich von fds und mvds

Für eine Attributmenge X entspricht die Abhängigkeitsbasis $DEP_G(X)$ für mvds der Attributhülle X_F^+ für fds:

1. Für eine mvd-Menge G und $X, Y \subseteq U$ gilt

$$\begin{aligned} G \models X \twoheadrightarrow Y &\iff \\ \exists Y_{i_1}, \dots, Y_{i_k} \in DEP_G(X) : \\ Y &= Y_{i_1} \cup \dots \cup Y_{i_k}. \end{aligned}$$

2. Für eine fd-Menge F und $X, Y \subseteq U$ gilt

$$\begin{aligned} F \models X \rightarrow Y &\iff \\ \exists B_{i_1}, \dots, B_{i_k} \in X_F^+ : \\ Y &= \{ B_{i_1}, \dots, B_{i_k} \}. \end{aligned}$$

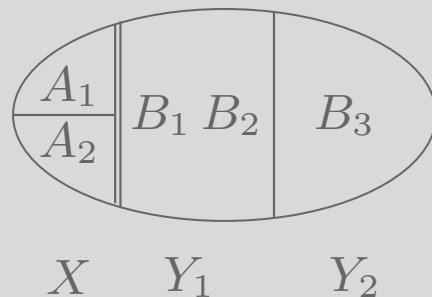
Für $Y_i = \{ B_i \}$ könnte man auch wieder $Y = Y_{i_1} \cup \dots \cup Y_{i_k}$ schreiben.

Beispiel (Abhängigkeitsbasis)

Seien $X = \{A_1, A_2\}$, $Y_1 = \{B_1, B_2\}$, $Y_2 = \{B_3\}$ und $U = X \cup Y_1 \cup Y_2$.

Dann erhalten wir für $G = \{X \twoheadrightarrow Y_1\}$ folgende Abhängigkeitsbasis:

$$DEP_G(X) = \{\{A_1\}, \{A_2\}, Y_1, Y_2\}.$$

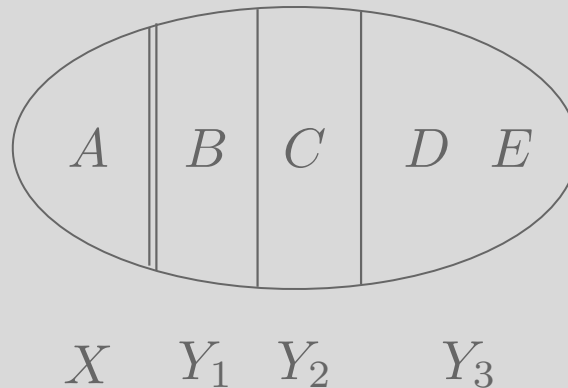


1. Aus G folgen die mvds $X \twoheadrightarrow \{A_1, B_3\}$ und $X \twoheadrightarrow \{B_1, B_2, B_3\}$,
wohingegen $X \twoheadrightarrow \{B_2, B_3\}$ nicht folgt, da B_2 nur zusammen mit B_1
auf der rechten Seite auftreten darf.
2. Insgesamt gibt es genau 16 mvds der Form $X \twoheadrightarrow Y$, die aus G folgen.
Man erhält die Mengen Y als Vereinigung $Y = Y_{i_1} \cup \dots \cup Y_{i_k}$
kompletter Blöcke $Y_{i_j} \in DEP_G(X)$.

Beispiel (Abhängigkeitsbasis)

Für $G = \{ A \twoheadrightarrow BC, DE \twoheadrightarrow C \}$ und $U = ABCDE$, $X = A$ gilt

$$DEP_G(X) = \{ \{A\}, \{B\}, \{C\}, \{D, E\} \}.$$



1. Aus $A \twoheadrightarrow BC$ folgt $A \twoheadrightarrow U \setminus (ABC) = DE$ mittels der Komplement-Regel.
2. Aus $A \twoheadrightarrow DE$ und $DE \twoheadrightarrow C$ folgt $A \twoheadrightarrow C$ mittels der Pseudo-Transitivitäts-Regel.
3. Aus $A \twoheadrightarrow BC$ und $A \twoheadrightarrow C$ folgt $A \twoheadrightarrow (BC) \setminus C = B$.

Definition (Verbundabhängigkeit, Join Dependency, jd)

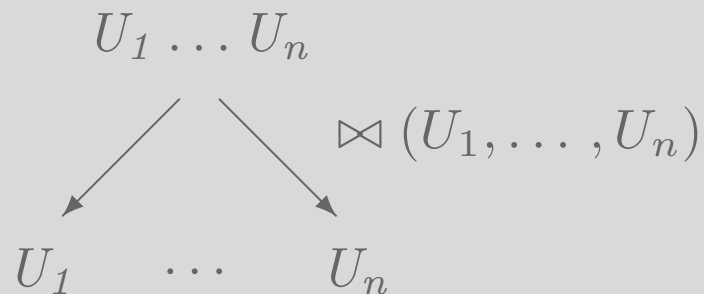
Sei $R = (U, \Sigma)$ ein Relationenschema zu einer Attributmenge $U = \cup_{i=1}^n U_i$ und einer Menge Σ eine Menge von Datenabhängigkeiten über U .

Die *Verbundabhängigkeit*

$$\bowtie (U_1, \dots, U_n)$$

gilt in R , falls für alle erlaubten Instanzen r von R gilt:

$$r = \Pi_{U_1}(r) \bowtie \dots \bowtie \Pi_{U_n}(r).$$



Beispiel (jd)

Wir betrachten die folgenden beiden Relationen r_1 und r_2 ,
und die jd $\delta = \bowtie (AB, BC, CD)$. In der ersten Relation r_1 gilt δ .

$r_1 :$

A	B	C	D
1	1	1	1
1	1	1	2
1	1	2	2
1	2	2	2
2	2	2	2

$r_2 :$

A	B	C	D
1	1	1	1
1	1	1	2
2	1	2	2

In r_2 gilt δ nicht, da $\Pi_{AB}(r_2) \bowtie \Pi_{BC}(r_2) \bowtie \Pi_{CD}(r_2) \supsetneq r_2$ die drei angegebenen überflüssigen Tupel enthält:

A	B
1	1
2	1

B	C
1	1
1	2

C	D
1	1
1	2
2	2

A	B	C	D
1	1	2	2
2	1	1	1
2	1	1	2
.	.	.	.

Folgerung aus dem Dekompositionssatz:

Satz (mvds sind binäre jds)

Es seien $X, Y \subseteq U$, $Z = U \setminus (XY)$ und r eine Relation über U .

Dann gilt die mvd $X \twoheadrightarrow Y$ in r , g.d.w. die jd $\bowtie (XY, XZ)$ in r gilt.

Binäre jds

1. Folglich ist jede mvd über paarweise disjunkten Mengen X, Y, Z äquivalent zu einer binären jd:

$$X \twoheadrightarrow Y \mid Z \iff \bowtie (XY, XZ),$$

2. und umgekehrt:

$$\bowtie (U_1, U_2) \iff U_1 \cap U_2 \twoheadrightarrow U_1 \setminus U_2 \mid U_2 \setminus U_1.$$

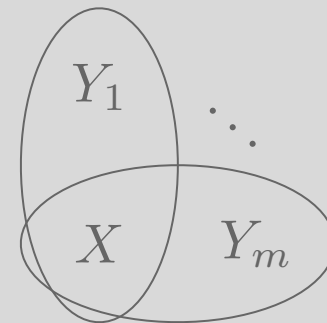
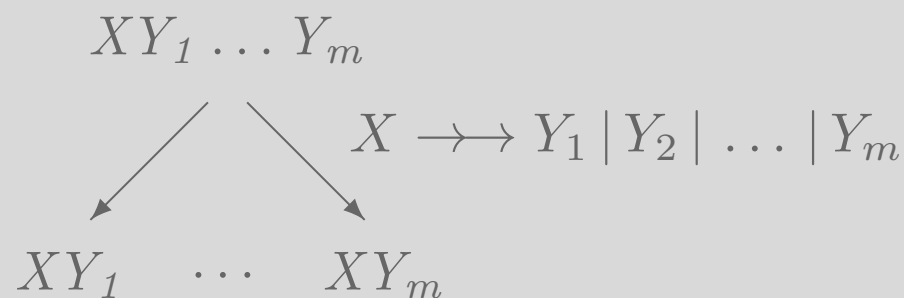
Allgemeine jds

1. Eine mvd über paarweise disjunkten Mengen $X, Y_1, \dots, Y_m$,

$$X \twoheadrightarrow Y_1 \mid Y_2 \mid \dots \mid Y_m,$$

ist ebenfalls äquivalent zu einer (azyklischen) Join-Dependency

$$\bowtie (XY_1, XY_2, \dots, XY_m),$$



aber nicht jede Join-Dependency ist äquivalent zu einer mvd.

2. Nicht jede jd ist äquivalent zu einer Menge von mvds,
denn es gibt Relationen, die verlustfrei in drei (oder mehr) Projektionen zerlegbar sind, aber nicht in zwei.

Definition (4NF, 5NF)

Es sei $R = (U, \Sigma)$ ein Relationenschema in 1NF und Σ eine beliebige Menge von Datenabhängigkeiten.

1. $K \subseteq U$ heißt *Oberschlüssel* von R , falls die fd $K \rightarrow U$ aus Σ folgt.
Ein minimaler Oberschlüssel ist ein *Schlüssel*.
2. $K \rightarrow U$ heißt *Schlüsselabhängigkeit*, falls K ein Schlüssel für R ist.
3. Falls Σ nur fds und mvds enthält, so ist R in *vierter Normalform (4NF)*, falls die linke Seite X jeder nicht-trivialen mvd mit $\Sigma \models X \twoheadrightarrow Y$ ein Oberschlüssel ist, d.h. $\Sigma \models X \rightarrow U$.
4. R ist in *fünfter Normalform (5NF)* bzw. *Projekt-Join-Normalform (PJNF)*, falls jede jd mit $\Sigma \models \bowtie (X_1, \dots, X_n)$ alleine aus den Schlüsselabhängigkeiten $K \rightarrow U$ von R folgt.

Satz (Normalformen)

Sei $R = (U, \Sigma)$ ein Relationenschema:

$$R \text{ ist in } 5\text{NF} \Rightarrow R \text{ ist in } 4\text{NF} \Rightarrow R \text{ ist in BCNF.}$$

Die Datenabhängigkeiten in einem 4NF– bzw. 5NF–Relationenschema sind also äquivalent zu Schlüsselabhängigkeiten.

Deswegen reicht das Konzept der

- Primär– bzw.
- Sekundär–Schlüsselbedingungen

aus SQL (**CREATE TABLE**) zur Datenmodellierung aus.

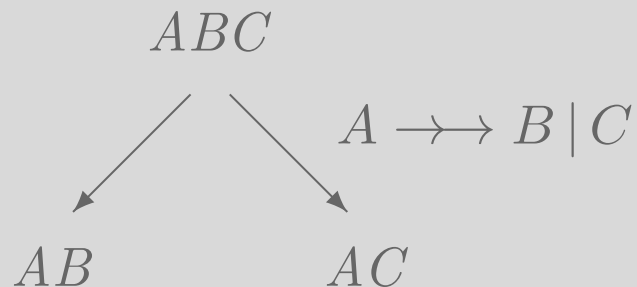
Beispiel (4NF)

Sei $R = (U, \Sigma)$ ein Relationenschema mit $U = ABC$ und

$$\Sigma = \{ A \twoheadrightarrow B, A \twoheadrightarrow C \}.$$

Dann gilt $\Sigma \models A \twoheadrightarrow B$, aber $\Sigma \not\models A \rightarrow U$.

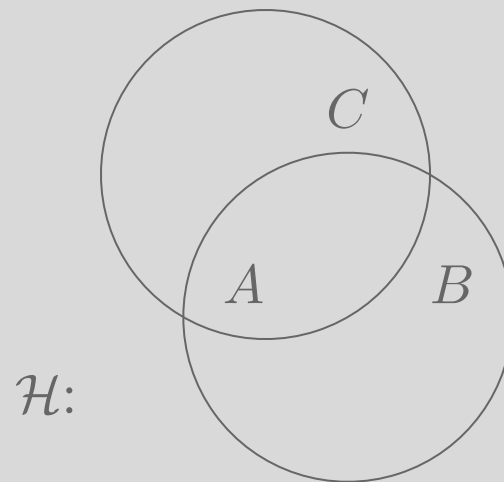
- Deshalb ist R *nicht in 4NF*.
- Man kann R aber anhand von $A \twoheadrightarrow B \mid C$ *verlustfrei* zerlegen in zwei 4NF–Relationenschemata:



Der Hypergraph $\mathcal{H}$ zur mvd $A \twoheadrightarrow B \mid C$ bzw. zur entsprechenden jd

$$\bowtie (AB, AC)$$

ist azyklisch:



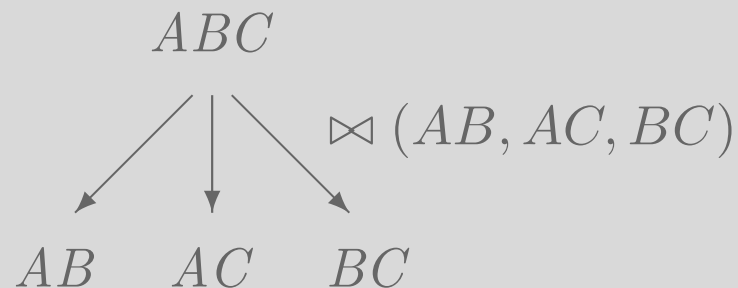
Ein Hypergraph entspricht einem Mengendiagramm.

Beispiel (4NF, 5NF)

Das Relationenschema

$$R = (ABC, \{ \bowtie (AB, AC, BC) \})$$

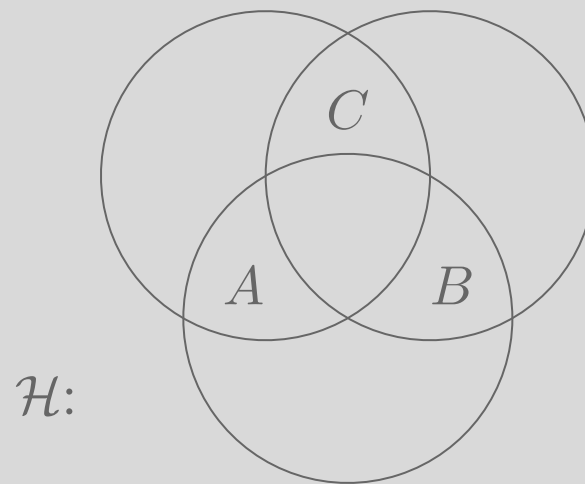
- ist in 4NF, aber nicht in 5NF:
- es gelten weder nicht-triviale fds noch mvds, und
- der einzige Schlüssel ABC impliziert nicht die jd.
- R kann verlustfrei zerlegt werden in drei 5NF-Relationenschemata:



Der Hypergraph $\mathcal{H}$ zur Join Dependency

$$\bowtie (AB, AC, BC)$$

ist zyklisch:



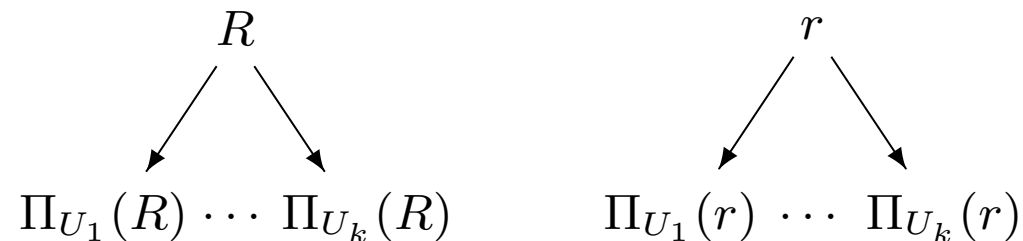
Die jd entspricht keiner Menge von mvds.

3.5 Verlustfreie Zerlegungen

Definition (Relationales Datenbankschema)

1. Ein relationales *Datenbankschema* $\mathcal{D}$ ist eine Menge von Relationenschemata.
2. $\mathcal{D}$ ist in 2NF (3NF, BCNF), falls jedes $R \in \mathcal{D}$ in 2NF (3NF, BCNF) ist.

Zerlegung von R in $\mathcal{D} = \{ \Pi_{U_i}(R) \mid 1 \leq i \leq k \}$:



Eine erlaubte Instanz r von R wird entsprechend zerlegt in eine Datenbank $d = \{ \Pi_{U_i}(r) \mid 1 \leq i \leq k \}$ zu $\mathcal{D}$.

Definition (Zerlegung)

Sei $R = (U, F)$ ein Relationenschema und $\mathcal{D} = \{ (U_1, F_1), \dots, (U_k, F_k) \}$ ein Datenbankschema.

1. $\mathcal{D}$ heißt *Zerlegung* von R , falls gilt:

a) $U = U_1 \cup \dots \cup U_k$, und

b) $F_i \equiv \Pi_{U_i}(F)$, für alle $1 \leq i \leq k$.

2. Eine Zerlegung $\mathcal{D}$ von R heißt

a) *verlustfrei*, falls für R die Join-Bedingung $\bowtie (U_1, \dots, U_k)$ gilt,

d.h., falls für alle erlaubten Instanzen r von R gilt

$$r = \Pi_{U_1}(r) \bowtie \dots \bowtie \Pi_{U_k}(r).$$

b) *unabhängig*, falls $F \equiv F_1 \cup \dots \cup F_k$.

1. Zerlegung: Alle Attribute aus U bleiben in mindestens einem Relationenschema $R_i = (U_i, F_i)$ erhalten, und die fd-Mengen F_i sind äquivalent zu den Projektionen aus F .
2. a) Verlustfreiheit: Jede erlaubte Instanz r zu R kann als Join der Projektionen $r_i = \Pi_{U_i}(r)$ rekonstruiert werden.
b) Unabhängigkeit: Die Einhaltung der fds in den Mengen F_i der einzelnen Relationenschemata $R_i = (U_i, F_i) \in \mathcal{D}$ kann man leicht in den Relationen r_i der erlaubten Datenbankinstanzen $d = \{ r_1, \dots, r_k \}$ zu $\mathcal{D}$ testen.
Falls es aber eine fd $f \in F$ gibt, die nicht aus $F_1 \cup \dots \cup F_k$ folgt, dann können wir f nur aufwendig testen, indem wir vorher den Join mehrerer Relationen aus d berechnen.

Redundanz

- Falls eine Zerlegung $\mathcal{D} = \{ (U_1, F_1), \dots, (U_k, F_k) \}$ zwei Relationenschemata (U_i, F_i) und (U_j, F_j) mit sich enthaltenden Attributmengen $U_j \subsetneq U_i$ enthält, so kann man das Relationenschema (U_j, F_j) zur kleineren Attributmenge aus $\mathcal{D}$ entfernen.
- Aus $U_j \subsetneq U_i$ folgt nämlich für alle Relationen r über U
$$\Pi_{U_i}(r) \bowtie \Pi_{U_j}(r) = \Pi_{U_i}(r).$$
- Deswegen gilt für alle erlaubten Instanzen r über R
$$r = \Pi_{U_1}(r) \bowtie \dots \bowtie \Pi_{U_k}(r) = \bowtie_{1 \leq i \leq k, i \neq j} \Pi_{U_i}(r).$$
- Also ist $\mathcal{D} \setminus \{ (U_j, F_j) \}$ immer noch eine verlustfreie Zerlegung.

Beispiel (Verlustfreie und unabhängige Zerlegung)

Für das Relationenschema $R = (U, F)$ mit $U = \{ A, B, C \}$ und

$$F = \{ A \rightarrow B, B \rightarrow C, A \rightarrow C \}$$

erhalten wir eine Zerlegung für

$$U_1 = \{ A, B \} \text{ und } U_2 = \{ B, C \}.$$

Die projizierten fd-Mengen sind

$$F_1 = \Pi_{U_1}(F) \equiv \{ A \rightarrow B \} \text{ und}$$

$$F_2 = \Pi_{U_2}(F) \equiv \{ B \rightarrow C \}.$$

Die Zerlegung ist verlustfrei – wie wir später sehen werden.

Sie ist auch unabhängig, denn $F \equiv F_1 \cup F_2$.

Die fd $A \rightarrow C$, die weder in F_1 noch in F_2 alleine gilt, folgt aus $F_1 \cup F_2$.

Eine Zerlegung $\mathcal{D}$ ist generell nur interessant, wenn sie verlustfrei ist. Daneben stellt man meist noch die Anforderung, daß alle Relationenschemata in $\mathcal{D}$ auch in einer gewissen Normalform sind (z.B. 3NF oder BCNF).

Wir werden später folgendes sehen:

- Für jedes Relationenschema gibt es eine verlustfreie und unabhängige Zerlegung in 3NF, die durch den Synthese–Algorithmus geliefert wird.
- Für jedes Relationenschema gibt es eine verlustfreie Zerlegung in BCNF, die durch den Dekompositions–Algorithmus geliefert wird.
- Wir werden aber ein Relationenschemata kennen lernen, für das es keine verlustfreie Zerlegung in BCNF gibt, die auch noch unabhängig ist.

Beispiel (Zerlegung von Lieferant)

$R = (U, F)$:

$$U = \{ L, T, A, O, K \},$$

$$F = \{ LT \rightarrow A, L \rightarrow O, O \rightarrow K \}.$$

1. Die Zerlegung in $U_1 = LTA$, $U_2 = LO$, $U_3 = OK$, mit $F_i = \Pi_{U_i}(F)$, ist verlustfrei und unabhängig. Die Zerlegung ist unabhängig, da jede fd aus F in einer der drei Projektionen erhalten ist:

$$\Pi_{U_1}(F) \equiv \{ LT \rightarrow A \},$$

$$\Pi_{U_2}(F) \equiv \{ L \rightarrow O \},$$

$$\Pi_{U_3}(F) \equiv \{ O \rightarrow K \}.$$

2. Die Zerlegung in $U_1 = LTA$, $U_2 = LO$, $U_3 = LK$, mit $F_i = \Pi_{U_i}(F)$, ist verlustfrei. Sie ist aber nicht unabhängig, da die fd $O \rightarrow K$ in keiner der drei Projektionen erhalten ist:

$$\Pi_{U_1}(F) \equiv \{ LT \rightarrow A \},$$

$$\Pi_{U_2}(F) \equiv \{ L \rightarrow O \},$$

$$\Pi_{U_3}(F) \equiv \{ L \rightarrow K \}.$$

Sie folgt auch nicht aus $\cup_{i=1}^3 \Pi_{U_i}(F)$. Um $O \rightarrow K$ zu testen, muß man zuerst den Join

$$r_2 \bowtie r_3 = \Pi_{LO}(r) \bowtie \Pi_{LK}(r)$$

berechnen.

Die Verlustfreiheit werden wir später zeigen.

r :

L	T	A	O	K
L1	T1	300	London	600
L1	T2	200	London	600
L1	T3	400	London	600
L1	T4	200	London	600
L1	T5	100	London	600
L1	T6	100	London	600
L2	T1	300	Paris	450
L2	T2	400	Paris	450
L3	T2	200	Paris	450
L4	T2	200	Brüssel	150
L4	T4	300	Brüssel	150
L4	T5	400	Brüssel	150

1. Zerlegung: $r = r_1 \bowtie r_2 \bowtie r_3$

$r_1 = \Pi_{LTA}(r) :$

L	T	A
L1	T1	300
L1	T2	200
L1	T3	400
L1	T4	200
L1	T5	100
L1	T6	100
L2	T1	300
L2	T2	400
L3	T2	200
L4	T2	200
L4	T4	300
L4	T5	400

$r_2 = \Pi_{LO}(r) :$

L	O
L1	London
L2	Paris
L3	Paris
L4	Brüssel

$r_3 = \Pi_{OK}(r) :$

O	K
London	600
Paris	450
Brüssel	150

2. Zerlegung: $r = r_1 \bowtie r_2 \bowtie r_3$

$r_1 = \Pi_{LTA}(r) :$
wie oben

$r_2 = \Pi_{LO}(r) :$

L	O
L1	London
L2	Paris
L3	Paris
L4	Brüssel

$r_3 = \Pi_{LK}(r) :$

L	K
L1	600
L2	450
L3	450
L4	150

Um $O \rightarrow K$ zu testen, muß man zuerst den Join

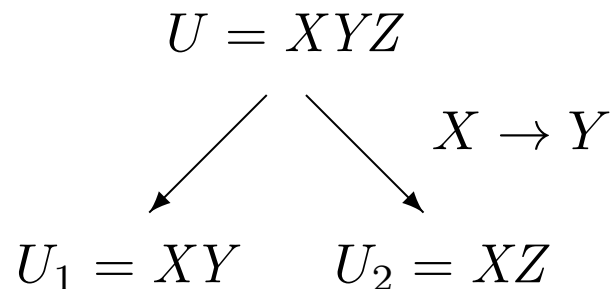
$r_2 \bowtie r_3 = \Pi_{LO}(r) \bowtie \Pi_{LK}(r)$ berechnen.

3.5.1 BCNF–Dekomposition

Satz (Dekomposition)

Es sei $R = (U, F)$ mit $X \rightarrow Y \in F^+$ und $Z = U \setminus (XY)$.

Dann ist die Zerlegung $\mathcal{D} = \{ \Pi_{XY}(R), \Pi_{XZ}(R) \}$ verlustfrei.



Folglich kann man jede erlaubte Instanz r von R aus den Projektionen rekonstruieren als $r = \Pi_{XY}(r) \bowtie \Pi_{XZ}(r)$.

Dieser Satz folgt aus dem Dekompositionssatz für mvds, da jede fd $X \rightarrow Y$ die entsprechende mvd $X \twoheadrightarrow Y$ impliziert.

Beispiel (Dekomposition von Lieferant)

Es sei $R = (U, F)$ mit

$$U = \{ L, T, A, O, K \},$$

$$F = \{ LT \rightarrow A, L \rightarrow O, O \rightarrow K \}.$$

Für $X \rightarrow Y = L \rightarrow O$ erhalten wir $Z = TAK$ und

$$R_1 = (LO, \{ L \rightarrow O \}^+),$$

$$R_2 = (LTAK, \{ LT \rightarrow A, L \rightarrow K \}^+).$$

Die fd $O \rightarrow K \in F$ geht bei der Dekomposition verloren, da die Attribute O und K getrennt werden.

Algorithmus DEKOMPOSITION

Input: ein Universalrelationenschema $R = (U, F)$

Output: eine verlustfreie BCNF–Zerlegung $\mathcal{D}$ von R

```
 $\mathcal{D} := \{ R \};$   
WHILE ( $\mathcal{D}$  ist nicht in BCNF) DO  
{  
  wähle  $S = \Pi_V(R) = (V, \Pi_V(F)) \in \mathcal{D}$   
    mit einer nicht-trivialen fd  $X \rightarrow Y \in \Pi_V(F)$ ,  
    so daß  $XY \subsetneq V$ ;  
  ersetze  $S$  in  $\mathcal{D}$  durch  $\Pi_{XY}(S)$  und  $\Pi_{XZ}(S)$ ,  
  wobei  $Z = V \setminus (XY)$ ;  
}
```

Die Verlustfreiheit zeigt man induktiv aufgrund des Dekompositionssatzes.

Alle Relationenschemata $S \in \mathcal{D}$ sind Projektionen von R .

- Am Anfang gilt $\mathcal{D} = \{ R \} = \{ \Pi_U(R) \}$.
- Später gilt $\Pi_W(S) = \Pi_W(\Pi_V(R)) = \Pi_W(R)$, denn $W \subseteq V$, für $W = XY$ und $W = XZ$.

Wir wollen eine Zerlegung $\mathcal{D}$ mit möglichst wenigen Relationenschemata erhalten.

1. Deswegen zerlegt man nur solange, bis alle Relationenschemata $S \in \mathcal{D}$ in BCNF sind.
2. Falls bei der Zerlegung Relationenschemata $S_1 = \Pi_{V_1}(R)$ und $S_2 = \Pi_{V_2}(R)$ mit sich enthaltenden Attributmengen $V_1 \subsetneq V_2$ entstehen, so kann man das enthaltene Relationenschema S_1 aus $\mathcal{D}$ entfernen.
3. Außerdem kann man zur Zerlegung *BCNF-verletzende* fds verwenden:
Das sind nicht-triviale fds $X \rightarrow Y \in \Pi_V(F)$ mit $F \not\models X \rightarrow V$.

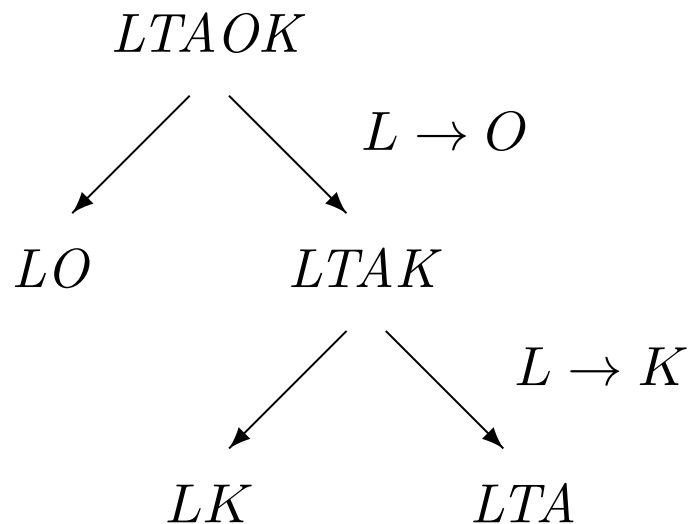
Beispiel (Dekompositionen von Lieferant)

Es sei $R = (U, F)$ mit

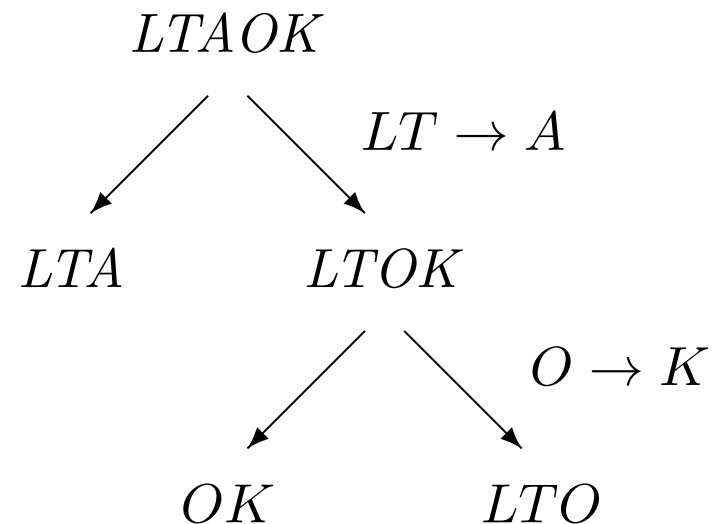
$$U = \{ L, T, A, O, K \},$$

$$F = \{ LT \rightarrow A, L \rightarrow O, O \rightarrow K \}.$$

$\mathcal{D}_1$:



$\mathcal{D}_2$:



Die erzeugten Zerlegungen von $R = (U, F)$ sind *verlustfrei*:

$$\mathcal{D}_1 = \{ \Pi_{LO}(R), \Pi_{LK}(R), \Pi_{LTA}(R) \},$$

$$\mathcal{D}_2 = \{ \Pi_{LTA}(R), \Pi_{OK}(R), \Pi_{LTO}(R) \}.$$

- $\mathcal{D}_1$ ist in BCNF aber nicht unabhängig, denn

$$O \rightarrow K \notin \{ L \rightarrow O, L \rightarrow K, LT \rightarrow A \}^+.$$

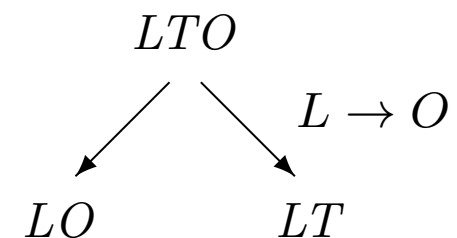
Die fd $O \rightarrow K$ muß in $\Pi_{LO}(r) \bowtie \Pi_{LK}(r)$ getestet werden.

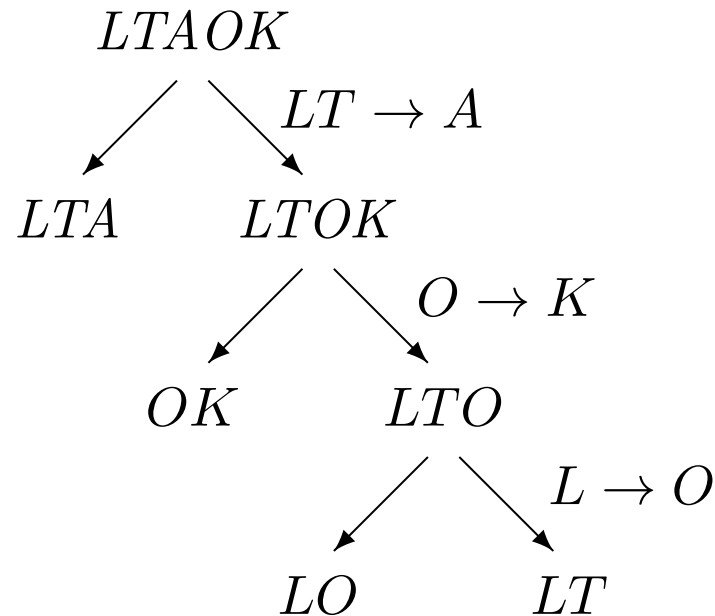
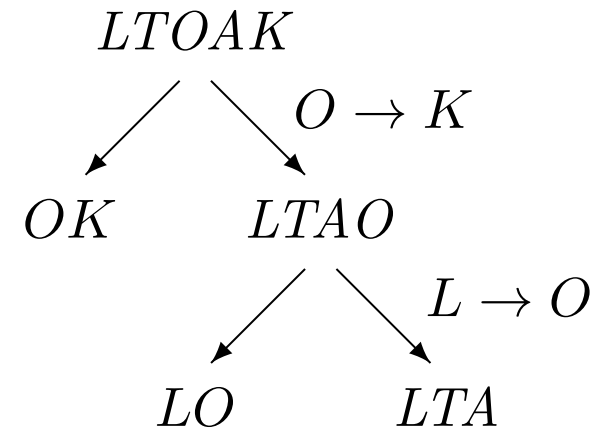
- $\mathcal{D}_2$ ist unabhängig aber nicht in BCNF, denn $L \rightarrow O$ ist BCNF–verletzend in $\Pi_{LTO}(R)$.
- Man kann $\mathcal{D}_2$ mittels $L \rightarrow O$ weiter zu $\mathcal{D}_3$ zerlegen:

$$\mathcal{D}_3 = \{ \Pi_{LTA}(R), \Pi_{OK}(R), \Pi_{LO}(R), \Pi_{LT}(R) \}.$$

$\Pi_{LT}(R)$ ist redundant, da es in $\Pi_{LTA}(R)$ enthalten ist.

$\mathcal{D}_3$ ist in BCNF und unabhängig.



$\mathcal{D}_3$: $\mathcal{D}_4$:

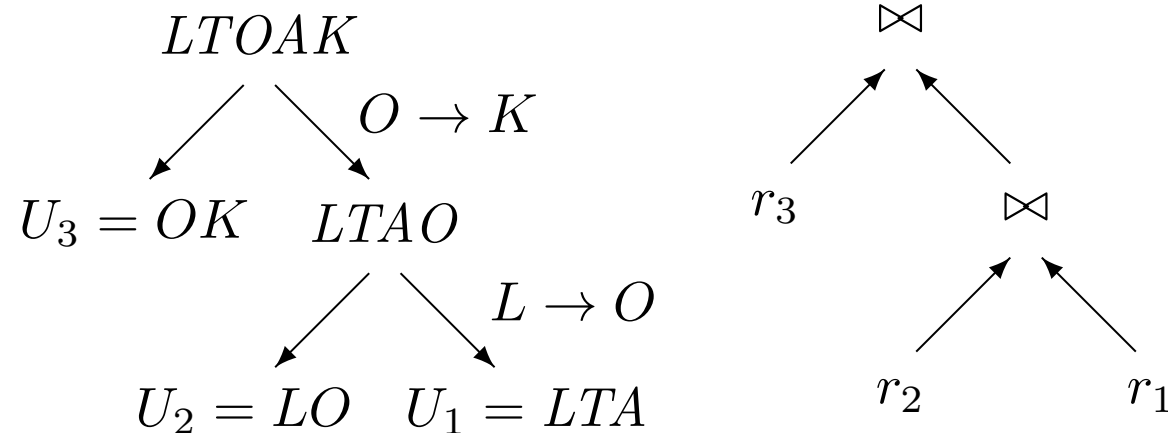
- Die Entfernung von $\Pi_{LT}(R)$ liefert ein Schema

$$\mathcal{D}_4 = \{ \Pi_{LTA}(R), \Pi_{OK}(R), \Pi_{LO}(R) \},$$

das man auch mittels einer anderen Zerlegung direkt erhalten kann.

Es gibt meist viele unterschiedliche Zerlegungsbäume und Zerlegungen.

Wenn man bei der Berechnung des Natural Joins dem Zerlegungsbaum folgt, dann sind je zwei zu verbindende Relationen mittels der Attributmenge X aus der zerlegenden fd $X \rightarrow Y$ verbunden:

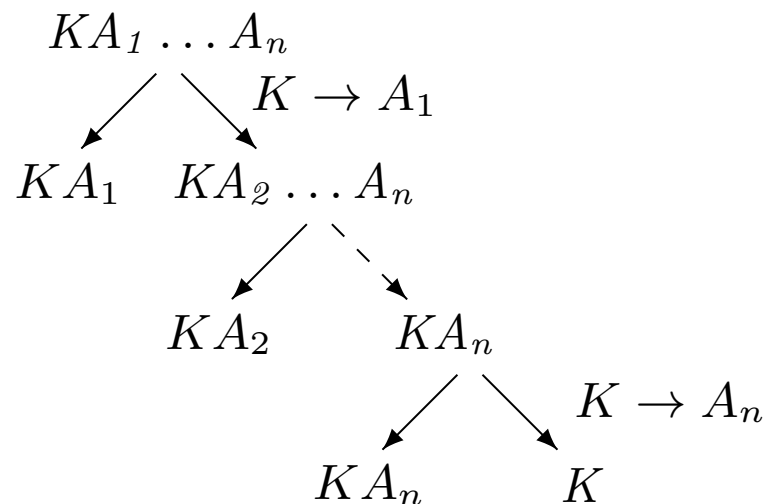


Falls $X \neq \emptyset$, so tritt bei der Berechnung des Natural Joins kein kartesisches Produkt auf. Hier berechnen wir

$$r = r_1 \bowtie r_2 \bowtie r_3 = (r_1 \bowtie r_2) \bowtie r_3.$$

Prinzipiell kann man jede nicht-triviale fd $X \rightarrow Y \in F^+$ zur Zerlegung eines Relationenschemas $R = (U, F)$ heranziehen, falls $X \cup Y \neq U$.

Falls z.B. K ein Schlüssel ist und $U \setminus K = \{A_1, \dots, A_n\}$, so kann man mittels der fds $K \rightarrow A_i$ eine verlustfreie Zerlegung erhalten:



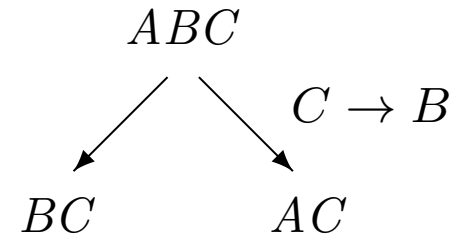
Für einen Schlüssel mit minimaler Kardinalität $|K|$, sind viele der Schemata $\Pi_{KA_i}(R)$ schon in BCNF; die restlichen kann man weiter zerlegen. Im folgenden Beispiel mit $K = AB$ und $A_n = C$ wäre $n = 1$, und $KA_n = U$ müßte weiter zerlegt werden.

Beispiel (Unvereinbarkeit von BCNF und Unabhängigkeit)

Sei $R = (U, F)$ mit

$$U = \{ A, B, C \},$$

$$F = \{ AB \rightarrow C, C \rightarrow B \}.$$



Dann ist R in 3NF, aber nicht in BCNF.

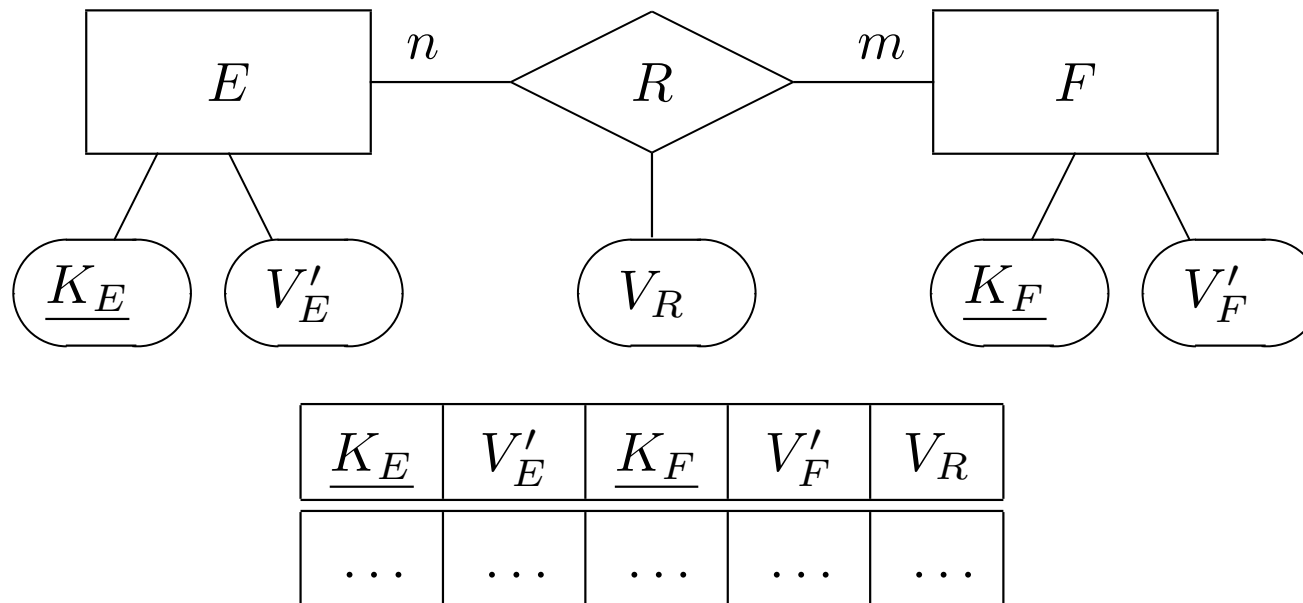
1. Die Dekomposition mittels $AB \rightarrow C$ liefert eine Zerlegung, die wieder $R = \Pi_{ABC}(R)$ enthält (und das darin enthaltene, redundante $\Pi_{AB}(R)$).
2. Die Dekomposition mittels der BCNF-verletzenden fd $C \rightarrow B$ liefert eine *verlustfreie BCNF-Zerlegung* $\mathcal{D}$, die aber *nicht unabhängig* ist:

$$\mathcal{D} = \{ (BC, \{C \rightarrow B\}), (AC, \emptyset) \}.$$

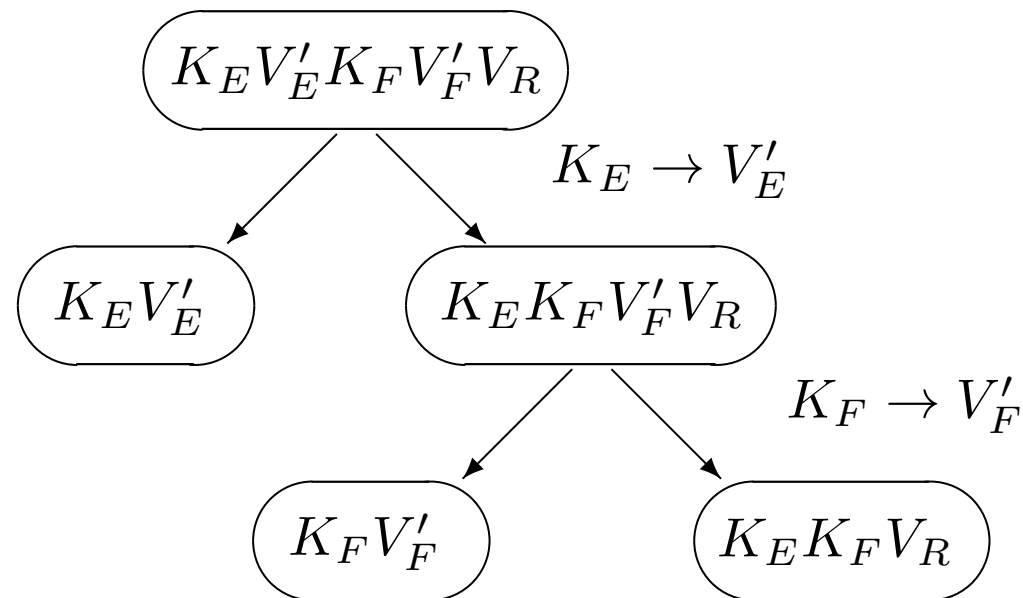
3. Man kann sogar zeigen, daß es für dieses Beispiel *keine unabhängige BCNF-Zerlegung* gibt.

ER-Modellierung

Ein $n : m$ -Relationship-Typ $R = (E, F, V_R)$ zwischen zwei Entity-Typen $E = (V_E, K_E)$ und $F = (V_F, K_F)$ könnte alternativ auch im ersten Schritt als eine Relation mit der Attributmenge $K_E V'_E K_F V'_F V_R$ und den fds $K_E \rightarrow V'_E$, $K_F \rightarrow V'_F$ und – je nach Interpretation – $K_E K_F \rightarrow V_R$ repräsentiert werden, wobei $V'_E = V_E \setminus K_E$ und $V'_F = V_F \setminus K_F$.



Diese Relation könnte dann im zweiten Schritt mittels des *Dekompositionsalgorithmus* in die bekannten Relationen zerlegt werden.



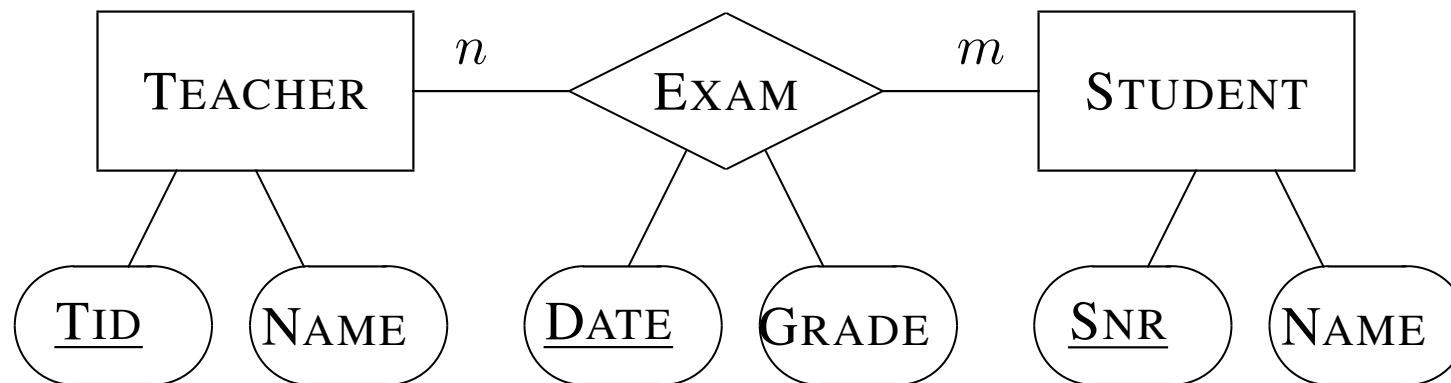
<u>K_E</u>	V' _E
...	...

<u>K_E</u>	<u>K_F</u>	V _R
...	...	...

<u>K_F</u>	V' _F
...	...

Die optionale Bedingung $(r(E) = r'(E) \wedge r(F) = r'(F) \Rightarrow r = r')$ für $n : m$ -Beziehungen entspricht der fd $K_E K_F \rightarrow V_R$.

Im Prüfungsbeispiel mit $K_E = \{ \text{TID} \}$, $V'_E = \{ \text{NAME} \}$, $K_F = \{ \text{SNR} \}$, $V'_F = \{ \text{NAME} \}$, $V_R = \{ \text{DATE}, \text{GRADE} \}$, gilt diese fd nicht, da ein Student mehrere Prüfungen beim selben Dozenten haben kann.



Aber es gilt $\{ \text{TID}, \text{SNR}, \text{DATE} \} \rightarrow \{ \text{GRADE} \}$, falls ein Student beim selben Dozenten an einem Tag nur eine Prüfung haben kann.

Dies wird durch das Unterstreichen von DATE angezeigt.

3.5.2 3NF–Synthese

Bei der 3NF–Synthese wird eine fd–Menge F zuerst zu einer Basis G vereinfacht, und dann wird jeder fd aus G ein Relationenschema zugeordnet.

Definition (Basis)

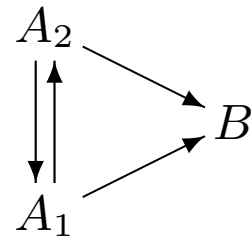
Sei F eine fd–Menge.

1. Eine fd $f \in F$ heißt *redundant*, falls f schon aus $F \setminus \{ f \}$ folgt.
2. Eine *Basis* für F ist eine zu F äquivalente,
 - a) Menge G von rechts– und links–minimalen fds,
 - b) die keine redundanten fds enthält.

Es gibt immer *mindestens eine* Basis, und es gibt manchmal mehrere Basen.

Beispiel (Basis)

1. Sei $G = \{ A \rightarrow B, B \rightarrow C \}$ und $F = G \cup \{ A \rightarrow C \}$.
 - Dann ist die fd $A \rightarrow C$ redundant in F ,
da sie aus der transitiven Verkettung der beiden anderen fds folgt.
 - G ist die eindeutige Basis für F .
2. Sei $G = \{ A_1 \rightarrow A_2, A_2 \rightarrow A_1 \}$ und $F = G \cup \{ A_1 \rightarrow B, A_2 \rightarrow B \}$.
Dann gibt es die zwei Basen $G_i = G \cup \{ A_i \rightarrow B \}$, $1 \leq i \leq 2$, für F .



3. $F = \{ LT \rightarrow A, L \rightarrow O, O \rightarrow K \}$ ist bereits eine Basis.

Algorithmus BASIS

Sei F eine fd-Menge.

Um eine Basis von F zu bestimmen, gehen wir wie folgt vor:

1. Wir zerlegen die fds $X \rightarrow A_1 \dots A_n$ mit mehrelementigen rechten Seiten in einzelne, rechts-minimale fds $X \rightarrow A_i$.
2. Wir verkleinern die linken Seiten der erhaltenen fds $X \rightarrow A_i$, so daß wir links-minimale fds $X' \rightarrow A_i$, mit $X' \subseteq X$, erhalten.
3. Wir entfernen schrittweise redundante fds.

Die erhaltene fd-Menge G ist eine Basis von F .

Algorithmus BASIS

Input: eine fd-Menge F

Output: eine Basis G von F

```
% mache  $F$  rechts-minimal  
 $G := \{ X \rightarrow A \mid X \rightarrow Y \in F, A \in Y \};$   
  
% mache  $G$  links-minimal  
FOR EACH  $X \rightarrow A \in G$  DO  
    ersetze  $X \rightarrow A$  in  $G$  durch  
    eine links-minimale fd  $X' \rightarrow A \in G^+$ , mit  $X' \subseteq X$ ;  
  
% entferne redundante fds aus  $G$   
FOR EACH  $f \in G$  DO  
    IF  $G \setminus \{f\} \models f$  THEN  $G := G \setminus \{f\};$ 
```

Beispiel (Basis)

Sei $F = \{ AB \rightarrow BC \}$.

- Wir zerlegen die fds in F zunächst in zwei rechts-minimale fds $AB \rightarrow B$ und $AB \rightarrow C$.
- Die erste fd $AB \rightarrow B$ ist trivial, und damit auch redundant. Nach unserem obigen Algorithmus ist sie nicht links-minimal, und sie wird zuerst zur – ebenfalls trivialen und redundanten – fd $B \rightarrow B$ verkleinert.
- Die zweite fd $AB \rightarrow C$ ist links-minimal.
- Nach dem Entfernen der redundanten fd $B \rightarrow B$ erhält man die Basis $G = \{ AB \rightarrow C \}$.

Beispiel (Basen mit unterschiedlicher Kardinalität)

$$F = \{ BC \rightarrow D, D \rightarrow E, ACD \rightarrow B, CE \rightarrow BD \}$$

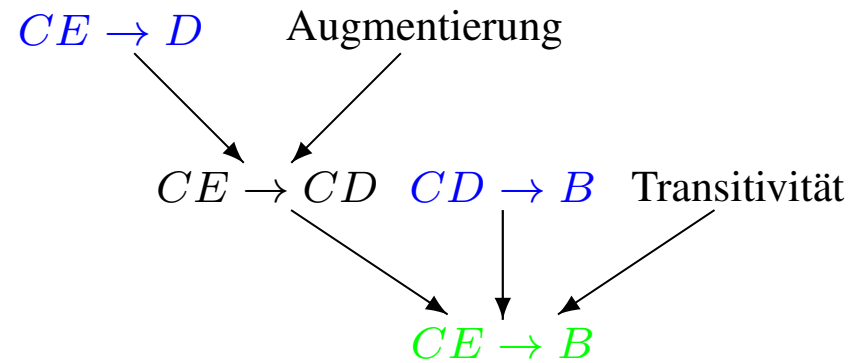
1. Die fd $CE \rightarrow BD$ ist nicht rechts-minimal, sie kann aber in zwei rechts-minimale fds $CE \rightarrow B$ und $CE \rightarrow D$ gesplittet werden.
2. Aus $D \rightarrow E$ und $CE \rightarrow B$ folgt $CD \rightarrow B$. Deshalb ist die fd $ACD \rightarrow B$ nicht links-minimal; ihre linke Seite könnte zu CD verkleinert werden. Wir reduzieren $ACD \rightarrow B$ also zu $CD \rightarrow B$.
3. Dann kann man $CE \rightarrow B$ weglassen, oder man kann $CE \rightarrow D$ und $CD \rightarrow B$ weglassen. Man erhält somit zwei Basen:

$$G_1 = G \cup \{ CD \rightarrow B, CE \rightarrow D \},$$

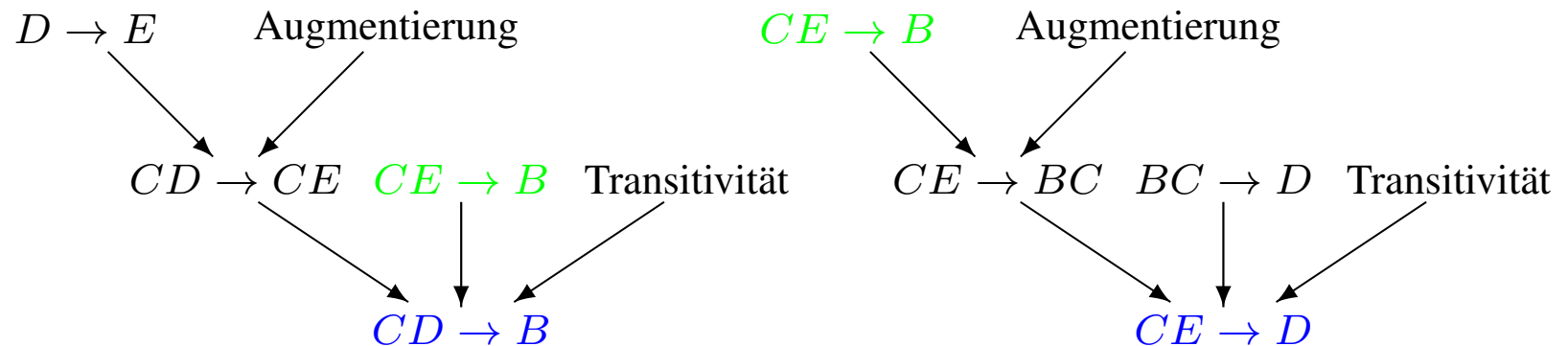
$$G_2 = G \cup \{ CE \rightarrow B \},$$

mit dem gemeinsamen Schnitt $G = \{ BC \rightarrow D, D \rightarrow E \}$.

$$G_1 \models G_2:$$



$$G_2 \models G_1:$$



Verlustfreie und unabhängige 3NF–Zerlegungen

Sei $R = (U, F)$ ein Relationenschema.

1. Wir erzeugen eine Basis G und bilden für jede fd $X \rightarrow A \in G$ ein Relationenschema $\Pi_{XA}(R) = (XA, \Pi_{XA}(F))$.
2. Falls keine der Attributmengen XA ein Overschlüssel von R ist, so bilden wir ein zusätzliches Relationenschema $(K, \emptyset)$ zu einem beliebigen Schlüssel K .

Dann ist das Datenbankschema $\mathcal{D}$, das die gebildeten Relationenschemata enthält, eine verlustfreie und unabhängige 3NF–Zerlegung von R .

Für einen Schlüssel K ist $\Pi_K(F) \equiv \emptyset$.

Algorithmus SYNTHESE

Input: ein Relationenschema $R = (U, F)$

Output: eine verlustfreie und unabhängige 3NF–Zerlegung $\mathcal{D}$ von R

```
 $G := \text{BASIS}(F);$   
 $\mathcal{D} := \{ \Pi_{XA}(R) \mid X \rightarrow A \in G \};$   
IF (für kein  $\Pi_V(R) \in \mathcal{D}$  ist  $V$  ein Overschlüssel von  $R$ )  
THEN  $\mathcal{D} := \mathcal{D} \cup \{ (\text{KEY}(U, F), \emptyset) \};$ 
```

Für einen Schlüssel K ist $\Pi_K(F) \equiv \emptyset$,

- denn es gibt es keine nicht–triviale fd $X \rightarrow Y \in \Pi_K(F)$.
- Sonst würde $K' \rightarrow K$ gelten, für $K' = K \setminus (Y \setminus X)$, denn $X \subseteq K'$.
Wegen $Y \setminus X \neq \emptyset$ wäre $K' \subsetneq K$, und folglich K kein Schlüssel.

Beispiel (Synthese vs. Dekomposition für Lieferant)

Der Synthese-Algorithmus angewandt auf

$$U = \{ L, T, A, O, K \},$$

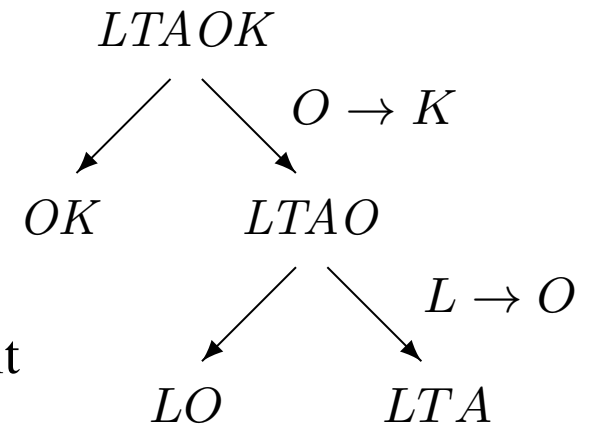
$$F = \{ LT \rightarrow A, L \rightarrow O, O \rightarrow K \},$$

liefert die Basis $G = F$ und die Zerlegung in $\Pi_{U_i}(R)$ mit

$$U_1 = \{ L, T, A \},$$

$$U_2 = \{ L, O \},$$

$$U_3 = \{ O, K \}.$$



Diese Zerlegung könnte man auch durch Dekomposition mit Hilfe der fds $O \rightarrow K$ und $L \rightarrow O$ erhalten.

Datendefinition in SQL

In den Relationenschemata der Zerlegung kann man die fds aus G mit Hilfe von PRIMARY KEY-Klauseln ausdrücken, denn die linken Seiten der entsprechenden fds aus G sind darin Schlüssel.

```
CREATE TABLE LTA (  
    L  VARCHAR(9) ,  
    T  VARCHAR(9) ,  
    A  INT(6) ,  
    PRIMARY KEY (L, T) );
```

Die linke Seite LT der fd $LT \rightarrow A$ war bereits ein Schlüssel im Ausgangsrelationenschema zu $U = \{ L, T, A, O, K \}$.

Die fds $O \rightarrow K$ und $L \rightarrow O$, die man im Relationenschema zu $U = \{ L, T, A, O, K \}$ nicht in SQL ausdrücken konnte, kann man nun ebenfalls in SQL mit Hilfe von PRIMARY KEY-Klauseln ausdrücken:

```
CREATE TABLE LO (  
    L  VARCHAR(9) ,  
    O  VARCHAR(12) ,  
    PRIMARY KEY (L) );
```

```
CREATE TABLE OK (  
    O  VARCHAR(12) ,  
    K  DECIMAL(7, 2) ,  
    PRIMARY KEY (O) );
```

Bezüglich U sind L und O keine Oberschlüssel, bezüglich $U_2 = \{ L, O \}$ und $U_3 = \{ O, K \}$ sind L bzw. O Schlüssel.

Außerdem kann man noch folgende Fremdschlüsselbedingungen – mit Hilfe von FOREIGN KEY–Klauseln – hinzufügen:

- zu LTA:

```
FOREIGN KEY (L)  
REFERENCES LO (L)
```

- zu LO:

```
FOREIGN KEY (O)  
REFERENCES OK (O)
```

Die Tabelle LTA wird nicht referenziert.

Beispiel (Synthese)

In $F = \{ L \rightarrow O, O \rightarrow K, L \rightarrow K \}$ ist die fd $L \rightarrow K$ redundant. $O \rightarrow K$ ist 3NF–verletzend. F ist äquivalent zu seiner Basis $G = \{ L \rightarrow O, O \rightarrow K \}$.

L	O	K
L1	London	600
L2	Paris	450
L5	Prag	600

- Nach dem Synthese–Algorithmus würde man die obige Relation r mittels $\pi_{LO}(r)$ und $\pi_{OK}(r)$ repräsentieren. Eine weitere Relation $\pi_{LK}(r)$ wäre hier redundant.
- Man könnte r aber auch mittels $\pi_{LO}(r)$ und $\pi_{LK}(r)$ repräsentieren.
- Man kann r aber nicht mittels $\pi_{OK}(r)$ und $\pi_{LK}(r)$ repräsentieren, denn $\pi_{OK}(r) \bowtie \pi_{LK}(r)$ enthält das Tupel (L1, Prag, 600) $\notin r$.

Beispiel (Synthese)

Für das Relationenschema $R = (U, F)$ mit $U = \{ A, B, C \}$ und

$$F = \{ A \rightarrow B, B \rightarrow C, A \rightarrow C \}$$

sind der Schlüssel $K = \{ A \}$ und die Basis $G = \{ A \rightarrow B, B \rightarrow C \}$ eindeutig. Es ist wegen $B \rightarrow C$ nicht in 3NF (und somit auch nicht in BCNF).

Die 3NF–Synthese liefert die zugehörige verlustfreie und unabhängige Zerlegung mit $U_1 = \{ A, B \}$ und $U_2 = \{ B, C \}$.

Der Schlüssel K ist in U_1 enthalten.

Die redundante fd $A \rightarrow C \in F$ würde zu einem weiteren Relationenschema mit $U_3 = \{ A, C \}$ führen. Dieses wäre auch in 3NF, da die fd links– und rechts–minimal und somit 3NF–kompatibel ist. Die erweiterte Zerlegung wäre auch verlustfrei und unabhängig. Im laufenden Betrieb wäre es aber unnötig, die weitere Relation zu U_3 zu pflegen.

Beispiel (Synthese vs. Dekomposition)

Die Dekomposition könnte das Relationenschema $R = (U, F)$ mit $U = \{ A, B, C \}$ und

$$F = \{ A \rightarrow B, B \rightarrow C, A \rightarrow C \}$$

auf drei Arten verlustfrei in BCNF (und somit auch 3NF) zerlegen:

- Die Dekomposition mittels $A \rightarrow B$ würde eine Zerlegung mit $U_1 = \{ A, B \}$ und $U_2 = \{ A, C \}$ liefern.
Diese wäre aber nicht unabhängig, da die fd $B \rightarrow C$ verloren geht.
- $A \rightarrow C$ würde die gleiche Zerlegung liefern.
- Die Dekomposition mittels $B \rightarrow C$ würde eine Zerlegung mit $U_1 = \{ B, C \}$ und $U_2 = \{ A, B \}$ liefern.
Diese wäre wie bei der 3NF–Synthese, und somit auch unabhängig.

Beispiel (Synthese vs. Dekomposition)

1. Das Relationenschema $R = (U, F)$ mit

$$U = \{ A, B, C \},$$

$$F = \{ AB \rightarrow C, C \rightarrow B \},$$

ist bereits in 3NF, und es gibt bekanntlich keine verlustfreie BCNF–Zerlegung, die auch noch unabhängig wäre.

- F ist bereits eine Basis.
- Der Synthese–Algorithmus würde U zerlegen in

$$U_1 = ABC \text{ und } U_2 = BC.$$

Das redundante Schema zu U_2 würde man wieder fallen lassen.

- Der Dekompositions–Algorithmus würde U zerlegen in

$$U_1 = BC \text{ und } U_2 = AC.$$

2. Das erweiterte Relationenschema $R = (U, F)$ mit

$$U = \{A, B, C, D\},$$

$$F = \{AB \rightarrow C, C \rightarrow B, C \rightarrow D\},$$

hat die zwei Schlüssel AB und AC und ist wegen $C \rightarrow D$ nicht in 3NF.

- F ist bereits eine Basis.
- Der Synthese-Algorithmus würde U zerlegen in

$$U_1 = ABC, U_2 = BC \text{ und } U_3 = CD.$$

Das redundante Schema zu U_2 würde man wieder fallen lassen.

- Der Dekompositions-Algorithmus könnte U zerlegen in

$$U_1 = BC, U_2 = CD \text{ und } U_3 = AC.$$

Das zugehörige Datenbankschema ist nicht unabhängig.

Für eine Instanz r von R müßte man die fd $AB \rightarrow C$ im Join aus $\pi_{U_1}(r)$ und $\pi_{U_3}(r)$ überprüfen.

Beispiel (Synthese vs. Dekomposition)

Das Relationenschema $R = (U, F)$ mit $U = \{ A, B, C, D, E \}$ und

$$F = \{ BC \rightarrow D, D \rightarrow E, ACD \rightarrow B, CE \rightarrow BD \}$$

mit den zwei Basen

$$G_1 = G \cup \{ CD \rightarrow B, CE \rightarrow D \},$$

$$G_2 = G \cup \{ CE \rightarrow B \},$$

mit dem gemeinsamen Schnitt

$$G = \{ BC \rightarrow D, D \rightarrow E \}$$

hat die drei Schlüssel ACB , ACD und ACE .

Da alle Attribute prim sind, ist R in 3NF.

1. Der Synthese-Algorithmus würde (unnötigerweise) zerlegen in

$$\mathcal{D}_1 = \{ \Pi_{BCD}(R), \Pi_{DE}(R), \Pi_{CDE}(R), \Pi_K(R) \} \text{ oder}$$

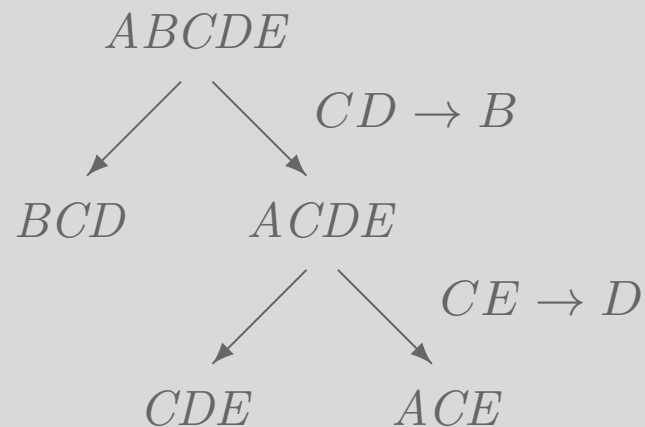
$$\mathcal{D}_2 = \{ \Pi_{BCD}(R), \Pi_{DE}(R), \Pi_{BCE}(R), \Pi_K(R) \},$$

für einen beliebigen Schlüssel K .

2. Die Dekomposition könnte z.B. zwischenzeitlich das reduzierte Schema

$$\mathcal{D}'_1 = \{ \Pi_{BCD}(R), \Pi_{CDE}(R), \Pi_K(R) \}$$

für den Schlüssel $K = ACE$ erzeugen.



$\mathcal{D}'_1$ ist unabhängig. Aber es ist wegen der BCNF–verletzenden fd $D \rightarrow E$ noch nicht in BCNF.

Das Relationenschema $\Pi_{CDE}(R)$ könnte mittels $D \rightarrow E$ weiter zerlegt werden in $\Pi_{DE}(R)$ und $\Pi_{CD}(R)$.

Das redundante Relationenschema $\Pi_{CD}(R)$ könnte man wieder fallen lassen. Insgesamt würde dann das reduzierte Schema

$$\mathcal{D}''_1 = \{ \Pi_{BCD}(R), \Pi_{DE}(R), \Pi_{ACE}(R) \}$$

erzeugt werden.

$\mathcal{D}''_1$ ist nicht unabhängig, da die beiden fds $CE \rightarrow B$ und $CE \rightarrow D$ verloren wurden. Im Vergleich mit den unabhängigen Datenbankschemas $\mathcal{D}_1$ bzw. $\mathcal{D}_2$ fehlen für $K = ACE$ die Relationenschemas $\Pi_{CDE}(R)$ bzw. $\Pi_{BCE}(R)$.

Das Schema zum Schlüssel

Bei der Synthese ist in manchen Fällen – neben den Relationenschemata für die fds aus der Basis – ein zusätzliches Relationenschema zu einem Schlüssel erforderlich.

- Dies kann erforderlich sein, damit alle Attribute aus U in wenigstens einer der Attributmengen U_i der Zerlegung vorkommen – was für jede Zerlegung erforderlich ist.
- Außerdem kann es auch für die Verlustfreiheit der Zerlegung erforderlich sein, z.B. da sonst die Zerlegung sogar unzusammenhängend sein könnte.

Diese beiden Fälle werden durch das folgende Beispiel illustriert.

Beispiel (Synthese)

1. Sei $R = (U, F)$ mit $U = \{ A, B, C, D, E \}$ und

$$F = \{ AB \rightarrow CD, BC \rightarrow D \}.$$

R ist nicht in 3NF wegen $BC \rightarrow D$, und $AB \rightarrow D$ ist redundant.

- Der *eindeutige Schlüssel* von R ist $K = ABE$.
Die *eindeutige Basis* von R ist $G = \{ AB \rightarrow C, BC \rightarrow D \}$.
- Der 3NF–SYNTHESE–Algorithmus liefert eine *unabhängige* und *verlustfreie 3NF–Zerlegung* $\mathcal{D}$ in drei Relationenschemata mit den Attributmengen $U_1 = ABC$, $U_2 = BCD$, $U_3 = ABE$.
- Ohne das Schema zum Schlüssel U_3 wäre das Attribut E nicht in $\mathcal{D}$ enthalten, und $\mathcal{D}$ wäre also keine Zerlegung von R .

2. Sei $R = (U, F)$ mit

$$U = \{ A, B, C, D \},$$

$$F = \{ A \rightarrow B, C \rightarrow D \}.$$

R ist nicht in 3NF wegen $A \rightarrow B$ und wegen $C \rightarrow D$.

- Der *eindeutige Schlüssel* von R ist $K = AC$.
Die *eindeutige Basis* von R ist $G = F$.
- Der 3NF–SYNTHESE–Algorithmus liefert eine *unabhängige* und *verlustfreie 3NF–Zerlegung* $\mathcal{D}$ in drei Relationenschemata
$$R_1 = \Pi_{AB}(R), \quad R_2 = \Pi_{CD}(R), \quad R_3 = (AC, \emptyset).$$
- Ohne das Schema R_3 zum Schlüssel $K = AC$ wäre $\mathcal{D}$ unzusammenhängend, und die Zerlegung wäre nicht verlustfrei.

$r :$

A	B	C	D
a_1	b	c_1	d_1
a_2	b	c_2	d_2

 $\Pi_{AB}(r) :$

A	B
a_1	b
a_2	b

 $\Pi_{CD}(r) :$

C	D
c_1	d_1
c_2	d_2

 $\Pi_{AC}(r) :$

A	C
a_1	c_1
a_2	c_2

 $r \subsetneq \Pi_{AB}(r) \bowtie \Pi_{CD}(r) :$

A	B	C	D
a_1	b	c_1	d_1
a_1	b	c_2	d_2
a_2	b	c_1	d_1
a_2	b	c_2	d_2

 $\Pi_{AB}(r) \bowtie \Pi_{CD}(r)$

ist ein kartesisches Produkt, da die Relationen keine Attribute gemeinsam haben (unzusammenhängend sind).

 $\Pi_{AB}(r) \bowtie \Pi_{AC}(r) \bowtie \Pi_{CD}(r) = r,$

denn AC stellt einen geeigneten Zusammenhang her – BC wäre ungeeignet.

Da der SYNTHES-Algorithmus eine Basis verwendet, enthält die Zerlegung $\mathcal{D}$ möglichst wenige Relationenschemata mit wenigen Attributen.

Wir haben bereits gesehen, daß manchmal einzelne Relationenschemata der Zerlegung redundant sind, da ihre Attributmengen in anderen Relationenschemata enthalten sind.

Ein Relationenschema $R = (U, F)$ ohne fds wird effektiv nicht zerlegt; falls $F = \emptyset$, so ist R schon in 3NF:

- Die Basis ist $G = \emptyset$ und $K = U$ ist der eindeutige Schlüssel.
- Also wird die Zerlegung $\mathcal{D} = \{ (K, \emptyset) \} = \{ (U, F) \}$ erzeugt, d.h. R wird faktisch nicht zerlegt.

Eigenschaften der 3NF–Synthese

Definition (Schema–Tableau)

Wir betrachten ein Datenbankschema

$$\mathcal{D} = \{ R_1, \dots, R_k \} \text{ mit } R_i = (U_i, F_i), \text{ für } 1 \leq i \leq k, \text{ und setzen} \\ U = \cup_{i=1}^k U_i = \{ A_1, \dots, A_m \}.$$

1. Das *Schema–Tableau* $\mathcal{T}_{\mathcal{D}}$ zu $\mathcal{D}$ ist eine Tabelle mit $m = |U|$ Spalten und k Zeilen $w_i = (w_{i,1}, \dots, w_{i,m})$, die den Schemata R_i entsprechen:

$$w_{i,l} = \begin{cases} a, & \text{falls } A_l \in U_i \\ b_i, & \text{falls } A_l \notin U_i \end{cases}$$

Dabei sind $a, b_1, \dots, b_k$ paarweise verschiedene Symbole.

2. w_i heißt *a–Zeile*, falls $w_{i,l} = a$, für alle $1 \leq l \leq m$.

Man schreibt die Attribute $A_1, \dots, A_m$ in eine zusätzliche Kopfzeile.

Tableau-Algorithmus LJP (Lossless Join Property)

Input: eine Zerlegung $\mathcal{D}$ eines Relationenschemas $R = (U, F)$

Output: true, falls $\mathcal{D}$ verlustfrei ist, false, sonst

```
konstruiere  $\mathcal{T}_{\mathcal{D}}$ ;  
WHILE ( $\mathcal{T}_{\mathcal{D}}$  lässt sich verändern) DO  
  FOR EACH ( $X \rightarrow Y \in F$ ) DO  
    FOR EACH ( $w_i, w_j$  in  $\mathcal{T}_{\mathcal{D}}$ ,  $i \neq j$ :  $w_i[X] = w_j[X]$ ) DO  
      identifiziere  $w_i$  und  $w_j$  für alle  $A_l \in Y$  :  
        ◦ ist  $w_{i,l} = a$ , so setze  $w_{j,l} := a$ ,  
        ◦ ist  $w_{j,l} = a$ , so setze  $w_{i,l} := a$ ,  
        ◦ andernfalls setze (o.B.d.A.)  $w_{j,l} := w_{i,l}$   
  IF ( $\exists$  a-Zeile) THEN RETURN true ELSE RETURN false;
```

Satz: $\mathcal{D}$ ist *verlustfrei*, genau dann wenn $\text{LJP}(R, \mathcal{D}) = \text{true}$ gilt.

Beispiel (LJP-Algorithmus)

LJP liefert für die Zerlegung $\mathcal{D}$ von $R = (U, F)$, mit

$$U = \{L, T, A, O, K\},$$

$$F = \{LT \rightarrow A, L \rightarrow O, O \rightarrow K\},$$

in die Attributmengen $U_1 = \{L, T, A\}$, $U_2 = \{L, O\}$ und $U_3 = \{O, K\}$ folgende Tableaus:

	L	T	A	O	K
w_1	a	a	a	b_1	b_1
w_2	a	b_2	b_2	a	b_2
w_3	b_3	b_3	b_3	a	a

Operationen:

1. Die Anwendung von $L \rightarrow O$ auf w_1, w_2 , und danach
2. die Anwendung von $O \rightarrow K$ auf w_1, w_3 ,

liefern eine a -Zeile in w_1 :

	L	T	A	O	K		L	T	A	O	K
w_1	a	a	a	a	b_1	w_1	a	a	a	a	a
w_2	a	b_2	b_2	a	b_2	w_2	a	b_2	b_2	a	b_2
w_3	b_3	b_3	b_3	a	a	w_3	b_3	b_3	b_3	a	a

Die Attributmenge U_1 zu w_1 enthält einen Schlüssel von R . Die fehlenden a 's werden aus den beiden anderen Zeilen geholt.

Diese zeigt die Verlustfreiheit der Zerlegung $\mathcal{D}$.

Beispiel (LJP-Algorithmus)

Anwendung des LJP-Algorithmus auf die Bank-Anwendung:

K = Kunde, N = Kontonummer, S = Saldo, Z = Zweigstelle,

$$F = \{ K \rightarrow S, N \rightarrow SZ \}.$$

Das *Schema-Tableau* zur Zerlegung in $U_1 = KS$ und $U_2 = NSZ$ wird von LJP nicht verändert.

	K	N	S	Z
w_1	a	b_1	a	b_1
w_2	b_2	a	a	a

Die Zerlegung ist *nicht verlustfrei*.

Die Hinzunahme eines Relationenschemas zum Schlüssel $U_3 = KN$ liefert eine *verlustfreie Zerlegung*, denn die Anwendung von $N \rightarrow SZ$ auf w_2, w_3 liefert das zweite Tableau mit einer a -Zeile in w_3 :

	K	N	S	Z		K	N	S	Z
w_1	a	b_1	a	b_1	w_1	a	b_1	a	b_1
w_2	b_2	a	a	a	w_2	b_2	a	a	a
w_3	a	a	b_3	b_3	w_3	a	a	a	a

Die Attributmenge U_3 zu w_3 enthält einen Schlüssel von R . Die fehlenden a 's werden aus der Zeile w_2 geholt.

Das Schema zu $U_1 = KS$ ist also für die Verlustfreiheit der Zerlegung unerheblich. Ohne U_1 wäre die Zerlegung allerdings nicht unabhängig, denn die fd $K \rightarrow S$ ginge verloren.

Satz (3NF–Synthese)

Sei $R = (U, F)$ ein Relationenschema.

Dann liefert $\text{SYNTHESE}(R)$ eine *verlustfreie* und *unabhängige* 3NF–Zerlegung von R .

Beweis.

Sei $\mathcal{D} = \{ R_1, \dots, R_k \}$ mit $R_i = (U_i, F_i)$ die betrachtete Zerlegung.

- *Unabhängigkeit:*

Nach Konstruktion werden alle in der Basis G enthaltenen fds an die Komponenten R_i vererbt. Deshalb gilt

$$F^+ = G^+ = \left(\bigcup_{i=1}^k F_i \right)^+,$$

d.h. die Zerlegung $\mathcal{D}$ ist unabhängig.

- *Verlustfreiheit:*

Nach Konstruktion existiert ein Schema $R_i = (U_i, F_i) \in \mathcal{D}$ mit einem Oberschlüssel U_i , d.h. $U_i \rightarrow U \in F^+$.

Wir wenden nun den Algorithmus LJP auf R und $\mathcal{D}$ an und beobachten speziell die Veränderungen in der Zeile w_i zu R_i .

- Insbesondere interessieren wir uns für die Anwendung jener fds $X \rightarrow Y \in F_j$ auf das Tableau $\mathcal{T}_{\mathcal{D}}$, welche zur Hüllenberechnung von $U = (U_i)_G^+$ verwendet werden.
- Diese fds werden in derselben Reihenfolge, in der sie bei der Hüllenberechnung verwendet werden, auf die Zeilen w_i und w_j des Tableaus angewendet. Eine fd $X \rightarrow Y \in F_j$ wird also auf die zugehörige Zeile w_j sowie die Zeile w_i zum Oberschlüssel U_i angewendet.

Per Induktion kann man zeigen, daß vor der Anwendung von $X \rightarrow Y$ in allen X -Spalten von w_i nur a -Einträge stehen.

Da auch in den X -Spalten von w_j nur a -Einträge stehen, können die a -Einträge der Y -Spalten von w_j nach w_i übertragen werden:

	U_i	$X \setminus U_i$	Y
w_i	$a \dots a$	$a \dots a$	
		$\uparrow$	$\uparrow$
w_j	$a \dots \dots \dots a$	a	$\dots a$

Nach der Anwendung aller fds $X \rightarrow Y$ aus der Hüllenberechnung auf das Tableau gilt folglich:

es stehen nur a -Einträge in den $(U_i)_G^+$ -Spalten von w_i .

Wegen der Unabhängigkeit von $\mathcal{D}$ gilt aber für den Oberschlüssel U_i :

$$(U_i)_G^+ = (U_i)_F^+ = U,$$

d.h. w_i ist eine a -Zeile.

Folglich ist die Zerlegung $\mathcal{D}$ *verlustfrei*.

- *3NF*:

Wir betrachten 2 Fälle für $S \in \mathcal{D}$:

- Falls das Schema $S = (K, \emptyset)$ zu einem Schlüssel K für R explizit in $\mathcal{D}$ aufgenommen wurde, so ist S trivialerweise in 3NF.
- Falls $S = \Pi_{XA}(R)$ ein Schema ist, welches durch eine fd $X \rightarrow A \in G$ aus der Basis G erzeugt wurde, so gilt:
 - * $X \rightarrow A$ ist links-minimal und rechts-minimal, und somit
 - * 3NF-kompatibel.

Also ist auch dieses Schema S in 3NF.

Somit ist die gesamte Zerlegung $\mathcal{D}$ in 3NF.



Algorithmus SYNTHESE'

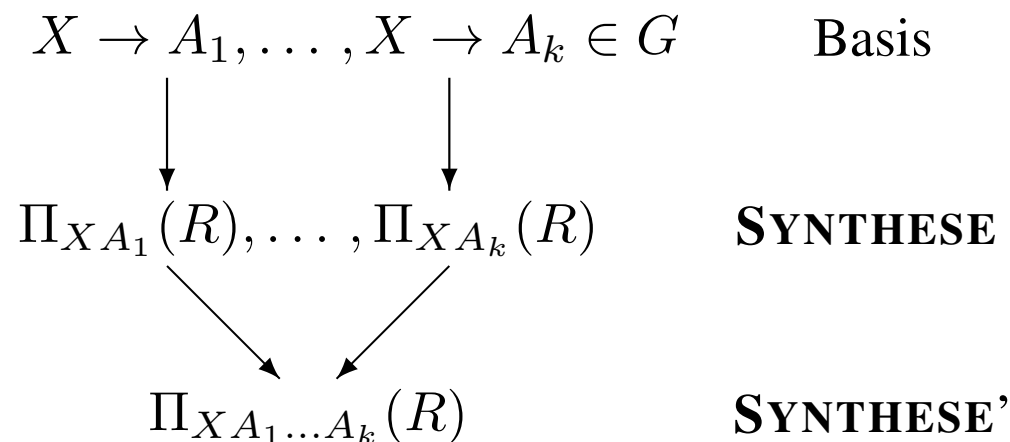
Input: $R = (U, F)$

Output: Eine verlustfreie und unabhängige Zerlegung $\mathcal{D}$ von R

```
 $G := \text{BASIS}(F); \mathcal{D} := \emptyset;$   
FOR EACH (linke Seite  $X$  einer fd aus  $G$ ) DO {  
   $Y := \{ A \in U \mid X \rightarrow A \in G \};$   
   $\mathcal{D} := \mathcal{D} \cup \{ \Pi_{XY}(R) \};$   
IF (für kein  $\Pi_V(R) \in \mathcal{D}$  ist  $V$  ein Oberschlüssel von  $R$ )  
THEN  $\mathcal{D} := \mathcal{D} \cup \{ (\text{KEY}(U, F), \emptyset) \};$ 
```

Auch der Algorithmus SYNTHESE'(R) liefert für alle Relationenschemata R eine *verlustfreie* und *unabhängige 3NF-Zerlegung*.

Da SYNTHESSE' eine Basis verwendet, enthält die Zerlegung $\mathcal{D}$ möglichst wenige Relationenschemata mit wenigen Attributen, die dann zusätzlich noch geeignet zusammengefaßt werden, ohne die 3NF zu verletzen.



Prinzip “one fact in one place”:

Jede linke Seite X einer fd aus der Basis G wird als ein Konzept aufgefaßt und die abhängigen Attribute in $Y = \{ A_1, \dots, A_k \}$ als dessen Eigenschaften; X und Y werden in einem Schema zusammengefaßt.

Beispiel (SYNTHESE vs. SYNTHESE')

Das Relationenschema $R = (U, F)$ mit

$$U = \{ A, B, C, D, E \} \text{ und } F = \{ AB \rightarrow CD \}$$

ist nicht in 3NF. Die *eindeutige Basis* von R ist

$$G = \{ AB \rightarrow C, AB \rightarrow D \},$$

und der eindeutige Schlüssel von R ist $\{A, B, E\}$.

1. Der Algorithmus SYNTHESE liefert die Zerlegung in drei Relationenschemata:

$$R_1 = \Pi_{ABC}(R), \quad R_2 = \Pi_{ABD}(R), \quad R_3 = (ABE, \emptyset).$$

2. Der Algorithmus SYNTHESE' liefert dagegen eine Zerlegung in nur zwei Relationenschemata $R_{1,2}$ und R_3 , in der R_1 und R_2 zusammen ein Schema bilden: $R_{1,2} = \Pi_{ABCD}(R)$.

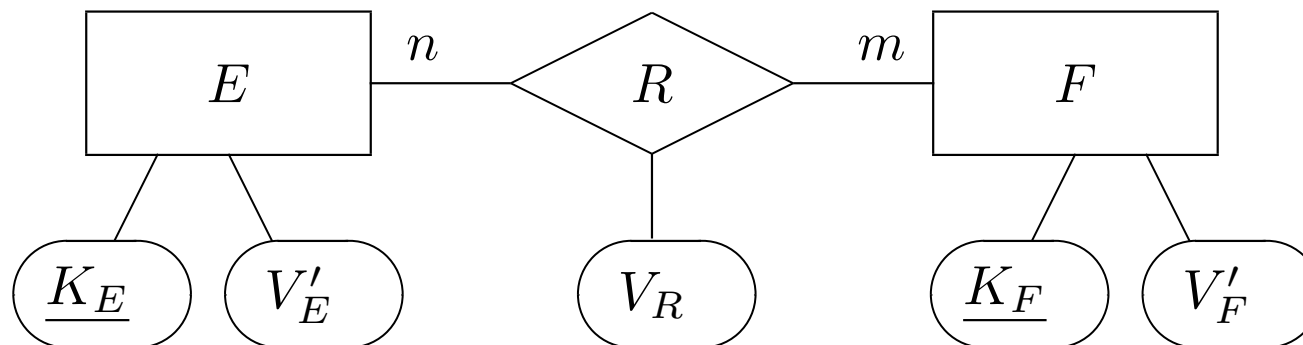
ER-Modellierung

Der Synthese-Algorithmus kann auch für einen $n : m$ -Relationship-Typ $R = (E, F, V_R)$ zwischen $E = (V_E, K_E)$ und $F = (V_F, K_F)$ angewendet werden. Für $V'_E = V_E \setminus K_E$, $V'_F = V_F \setminus K_F$, und $U = K_E V'_E K_F V'_F V_R$ ist

$$G = \{ K_E \rightarrow V'_E, K_F \rightarrow V'_F, K_E K_F \rightarrow V_R \}$$

bereits eine Basis, und wir erhalten auch hier wieder die Zerlegung in

$$U_1 = K_E V'_E, U_2 = K_F V'_F, U_3 = K_E K_F V_R.$$



Dies gilt auch, falls wir die fd $K_E K_F \rightarrow V_R$ nicht annehmen, denn dann kommt U_3 als Relation zum eindeutigen Schlüssel hinzu.

4 Transaktionen und Recovery

Zur Unterstützung des Mehrbenutzerbetriebs und der Datensicherheit, speziell für länger andauernde Transaktionen:

1. In einem *Reisebüro* will ein Kunde eine Kette von Flügen (mit Zwischenstops) buchen, wobei er bereits reservierte Flüge solange blockiert, bis er sich für eine komplette Reise entschieden hat. Gleichzeitig wollen viele andere Kunden und Reisebüros auf dieselbe Datenbank zugreifen.
2. In einer *CAD-Datenbank* werden komplexe Objekte verwaltet, die oft über einen längeren Zeitraum von verschiedenen Benutzern gleichzeitig – aber an unterschiedlichen Stellen – bearbeitet werden.

In beiden Fällen soll jede Transaktion nur einen möglichst kleinen Teil der Datenbank blockieren.

Bei der Flugbuchung greifen *Transaktionen* sehr vieler Beteiligter gleichzeitig auf dieselbe Datenbank zu und machen Änderungen.



Die Reihenfolge der Transaktionen ist egal. Es muß aber am Ende so erscheinen, als wäre jede Transaktion – bestehend aus ihren Teiloperationen – isoliert als atomare Einheit ausgeführt worden. Es soll dauerhaft ein konsistenter Zustand hinterlassen werden (*Serialisierbarkeit, ACID-Eigenschaft*).

4.1 Transaktionsverwaltung

Eine *Transaktion* ist ein Programm, welches auf den Inhalt der Datenbank zugreift oder diesen verändert.

Wir wollen die *verschränkte Ausführung* von Transaktionen und die dabei auftretenden Probleme untersuchen.

Dabei sind vor allem die von einer Transaktion T_i ausgeführten *Lese- und Schreiboperationen* auf der Datenbank von Interesse.

- $r_i(X)$ liest das Datenobjekt X in die Programmvariable X .
- $w_i(X)$ schreibt den Wert der Programmvariablen X auf das Datenobjekt X .

Wir nehmen an, daß Transaktionen nicht geschachtelt werden können.

4.1.1 Das Synchronisationsproblem

Wir betrachten eine *Flugdatenbank*, in der ein Datenobjekt X die Anzahl der reservierten Plätze in Flug X enthält.

Transaktion T_1 storniert N Plätze von Flug X und bucht dafür N Plätze von Flug Y . Transaktion T_2 bucht M Plätze von Flug X .

Für die beiden Transaktionen

$$T_1 = r_1(X); X := X - N; w_1(X); r_1(Y); Y := Y + N; w_1(Y)$$

$$T_2 = r_2(X); X := X + M; w_2(X)$$

erledigen die beiden seriellen Schedules $s_{1,2} = T_1; T_2$ und $s_{2,1} = T_2; T_1$ die Aufgabe korrekt.

Verschränkte Schedules wie s_1 und s_2 können zu Problemen führen:

$s_1 :$		$s_2 :$	
T_1	T_2	T_1	T_2
$r_1(X);$		$r_1(X);$	
$X := X - N;$		$X := X - N;$	
$w_1(X);$			$r_2(X);$
	$r_2(X);$		$X := X + M;$
	$X := X + M;$	$w_1(X);$	
	$w_2(X);$	$r_1(Y);$	
$r_1(Y);$			$w_2(X);$
$Y := Y + N;$		$Y := Y + N;$	
$w_1(Y);$		$w_1(Y);$	

Die dynamische Situation bei der Verarbeitung von Transaktionen kann zu folgenden Problemen führen:

1. Phantom Read, Unrepeatable Read
2. Lost Update
3. Temporary Update, Dirty Read
4. Incorrect Summary

Phantom / Unrepeatable Read

Das *Phantom*– oder *Unrepeatable Read*–Problem besteht darin, daß eine Transaktion ein Datenobjekt zweimal liest, und daß dieses in der Zwischenzeit von einer anderen Transaktion verändert wird.

Lost Update

Das *Lost Update*–*Problem* tritt auf, wenn die Operationen zweier Transaktionen, welche dasselbe Datenobjekt betreffen, verschränkt ausgeführt werden. Dann kann unter Umständen der Wert eines Datenobjekts hinterher falsch sein, da einzelne Update–Operationen verloren gehen.

Beispiel (s_2): In s_2 hat X nach der Ausführung keinen vernünftigen Wert, weil T_2 den Update von T_1 überschreibt, obwohl T_2 den Wert von X vor dem Update durch T_1 gelesen hat.

Temporary Update, Dirty Read

Das *Temporary Update*– oder *Dirty Read*–Problem tritt auf, wenn eine Transaktion ein Datenobjekt verändert und danach aus irgendeinem Grund abgebrochen werden muß.

Dann kann es sein, daß eine andere Transaktion auf diesen temporären Update zugreift, bevor das Datenobjekt auf seinen alten Wert zurückgesetzt wurde. Der temporäre neue Wert wird als *dirty data* bezeichnet.

Beispiel (s_1): Falls Transaktion T_1 in s_1 nach $r_1(Y)$ scheitert, so muß der alte Wert von X wiederhergestellt werden. Inzwischen hat allerdings T_2 schon den temporären, falschen Wert von X gelesen.

Incorrect Summary

Das *Incorrect Summary*–Problem (fehlerhafte Zusammenfassung) tritt auf, wenn eine Transaktion eine *Aggregatsfunktion* berechnet, während andere Transaktionen gleichzeitig die Datenbank verändern.

Dann kann es sein, daß die Aggregatsfunktion auf manche Datenobjekte vor dem Update zugreift und auf manche nach dem Update. Das berechnete Resultat der Aggregatsfunktion entspricht dann keinem *vernünftigen* Datenbankzustand.

Beispiel (s_3): Im folgenden Schedule s_3 berechnet die Transaktion T_3 die Summe der Werte der Datenobjekte X , Y , Z , und liest dabei

- X , nachdem dieses von T_1 um N verringert wurde, und
- Y , bevor dieses um denselben Betrag N erhöht wurde,

woraus eine um N zu niedrige Summe resultiert.

T_1	T_3
$r_1(X);$ $X := X - N;$ $w_1(X);$	$sum := 0;$
$r_1(Y);$ $Y := Y + N;$ $w_1(Y);$	$r_3(X);$ $sum := sum + X;$ $r_3(Y);$ $sum := sum + Y;$
	$r_3(Z);$ $sum := sum + Z;$

SQL-Statements

Die Transaktion T_1 könnte folgenden SQL-Statements entsprechen, welche 10 Reservierungen eines Flugs auf einen anderen Flug tauschen:

```
UPDATE  FLIGHTS
SET      AVAILABLE = AVAILABLE - 10
WHERE    NUMBER = 'DL 003' AND DATE = '2007-12-04';

UPDATE  FLIGHTS
SET      AVAILABLE = AVAILABLE + 10
WHERE    NUMBER = 'LH 007' AND DATE = '2007-12-04';
```

Die Transaktion T_3 könnte ein Teil der Berechnung des folgenden SQL-Statements sein, welches die Anzahl der freien Flüge pro Tag ermittelt:

```
SELECT    DATE, SUM (AVAILABLE)
FROM      FLIGHTS
GROUP BY  DATE
```

Ähnliche Probleme können in der COMPANY-Datenbank entstehen, wenn die Transaktion T_1 Gehaltserhöhungen in EMPLOYEE durchführt, während die Transaktion T_3 die Summe der Löhne pro Abteilung ermittelt.

T_1 könnte folgendem SQL-Statement entsprechen:

```
UPDATE  EMPLOYEE
SET      SALARY = SALARY * 1.1
WHERE    BDATE > '1975-12-24'
```

T_3 entspricht folgendem SQL-Statement:

```
SELECT    DNO, SUM (SALARY)
FROM      EMPLOYEE
GROUP BY  DNO
```

4.1.2 Operationen, Log-Buch, ACID-Prinzip

Das System protokolliert im *Log-Buch* alle Operationen, welche die Werte von Datenobjekten betreffen (d.h. Lese- und Schreiboperationen).

Dadurch kann man bei Transaktionsfehlern immer noch zu einem korrekten Datenbankzustand gelangen ($\rightarrow$ Recovery).

Sicherung des Log-Buchs

- Das Log-Buch wird auf der *Festplatte* gehalten, und es ist somit gegen fast alle Systemfehler resistent.
- Darüberhinaus wird das Log-Buch auch regelmäßig auf externe Speichermedien gesichert, um noch besser vor Datenverlust gefeit zu sein.

Einträge ins Log-Buch

Folgende *Einträge* ins Log-Buch sind für eine Transaktion T möglich:

$start_transaction(T)$: vermerkt den Start der Ausführung von T .

$commit(T)$: vermerkt, daß T erfolgreich beendet wurde, und daß damit der Effekt von T auf die Datenbank permanent gemacht werden kann.

$abort(T)$: vermerkt den Abbruch von T .

$write_item(T, X, V_{old}, V_{new})$: vermerkt, daß T den Wert des Datenobjekts X von V_{old} auf V_{new} verändert hat.

$read_item(T, X)$: vermerkt, daß T den Wert des Datenobjekts X gelesen hat.

Bei einem *Systemzusammenbruch (Crash)* können wir mittels undo bzw. redo wieder einen konsistenten Datenbank–Zustand herstellen:

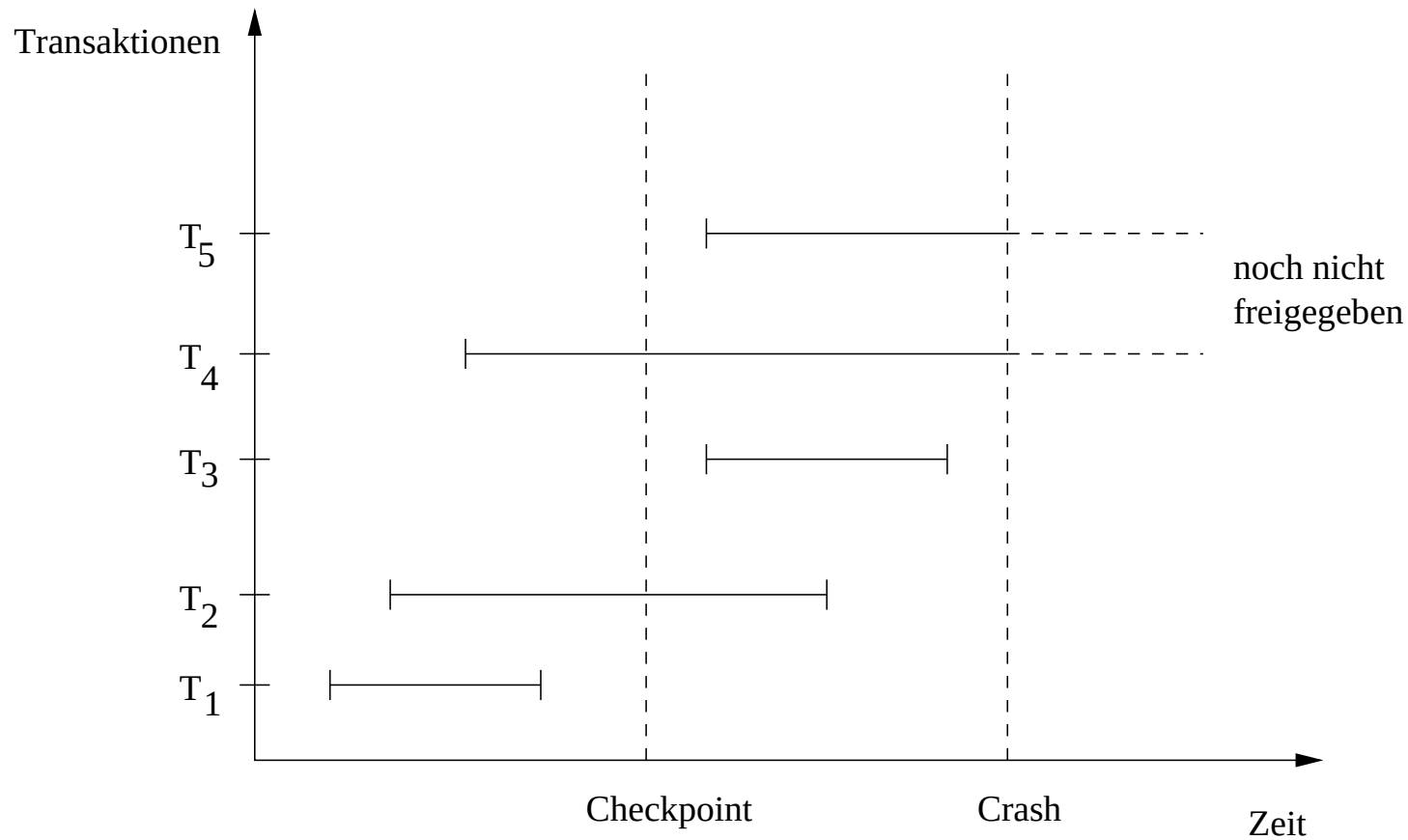
Undo

Falls eine Transaktion abgebrochen werden soll,
so kann man die Schreib–Operationen dieser Transaktion im
Log–Buch rückwärts verfolgen und einzeln zurücknehmen (*undo*),
indem man die Datenobjekte wieder auf ihre alten Werte V_{old} setzt.

Redo

Falls eine Transaktion weiterlaufen soll,
so kann man die Schreiboperationen dieser Transaktion im
Log–Buch vorwärts verfolgen und einzeln wiederholen (*redo*),
indem man die Datenobjekte auf ihre neuen Werte V_{new} setzt.

Relevante Zeitpunkte: Checkpoints, Crashes



T_1 : irrelevant (keine weitere Aktion)

T_2, T_3 : restaurieren (redo) T_4, T_5 : zurücksetzen (undo)

Commit-Punkte der Transaktionen

Eine Transaktion T erreicht ihren *Commit-Punkt*, sobald alle ihre Zugriffsoperationen auf die Datenbank erfolgreich ausgeführt wurden (und der Effekt der Operationen im Log-Buch protokolliert ist).

Danach heißt T *freigegeben* (committed), und der Effekt von T wird als permanent angenommen.

Die Transaktion T schreibt einen Eintrag $commit(T)$ ins Log-Buch.

Vor dem Commit einer Transaktion T müssen alle ihre Log-Buch-Einträge auf Platte gesichert werden (*force writing*).

Bei einem Systemzusammenbruch (Crash) können nur diese Teile des Log-Buches zum Recovery verwendet werden.

Checkpoints

Regelmäßig wird auch ein *checkpoint*–Eintrag ins Log–Buch geschrieben.

Zu diesem Zeitpunkt werden die Updates von freigegebenen Transaktionen in der Datenbank auf Platte durchgeführt – und das *Log–Buch* wird ebenfalls auf Platte gesichert (Sicherungshierarchie).

Folglich müssen alle Transaktionen, welche ihren *commit*(*T*)–Eintrag vor dem *checkpoint*–Eintrag im Log haben, bei einem Systemzusammenbruch nicht wiederholt werden.

Crashes (Systemfehler)

Bei einem Crash wird im Log–Buch rückwärts nach allen Transaktionen T gesucht, welche ihr $start_transaction(T)$ eingetragen haben.

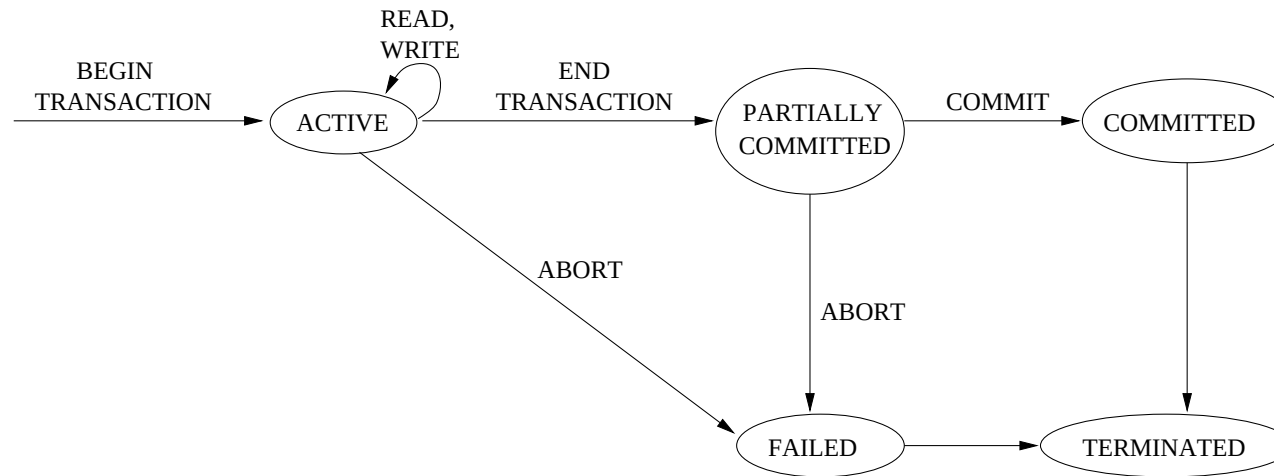
- Die Transaktionen T darunter, die vor dem letzten Checkpoint freigegeben wurden und somit ihr $commit(T)$ eingetragen haben, müssen nicht betrachtet werden; sie sind hier irrelevant; siehe T_1 .
- Transaktionen, welche noch *kein* $commit(T)$ eingetragen haben, müssen während des Recovery mittels *undo*–Operationen rückgängig gemacht werden (*roll–back*); siehe T_4 und T_5 .
- Transaktionen mit $commit(T)$ –Eintrag können mittels *redo* der Schreib–Operationen aus dem Log–Buch wiederholt werden; siehe T_2 und T_3 .

Bei einem Crash müssen alle laufenden und noch nicht freigegebenen Transaktionen zurückgesetzt werden. Dies ist unabhängig davon, ob die Transaktionen vor (T_4) oder nach (T_5) dem letzten Checkpoint gestartet wurden.

Die vor dem letzten Checkpoint bereits freigegebenen Transaktionen (T_1) leiden nicht unter dem Crash, da ihre Effekte und Log–Buch–Einträge bereits auf Platte gesichert wurden – sie sind für unsere Betrachtung hier also irrelevant.

Nur die zwischen dem letzten Checkpoint und dem aktuellen Crash freigegebenen Transaktionen müssen restauriert werden. Dies ist unabhängig davon, ob die Transaktionen vor (T_2) oder nach (T_3) dem letzten Checkpoint gestartet wurden.

Transaktions-Status-Diagramm



Sobald eine Transaktion T alle Datenzugriffe erfolgreich durchgeführt hat, ist sie *partially committed*.

Vor dem endgültigen *commit* müssen Concurrency Control- und Recovery Control-Checks sicherstellen, daß T keine unerwünschten Wechselwirkungen mit anderen Transaktionen hatte.

Dieses würde, genau wie ein Fehler bei den Datenzugriffen, zu einem Abbruch (*abort*) führen, und T wäre fehlgeschlagen (*failed*).

Das ACID–Prinzip

Zum Zwecke einer korrekten Synchronisation und der Gewährleistung von Fehlertoleranz muß die Transaktionsverwaltung folgende vier Eigenschaften sicherstellen:

Atomarität (Atomicity)

Eine Transaktion wird aus der Sicht des Benutzers vollständig oder gar nicht ausgeführt. Wird eine Ausführung vor ihrem eigentlichen Ende abgebrochen, so erscheint die Datenbank hinterher so, als wäre diese Ausführung nie gestartet worden.

Konsistenz (Consistency)

Alle Integritätsbedingungen der Datenbank werden eingehalten, d.h. eine Transaktion hinterläßt stets einen *konsistenten* Zustand, falls sie in einem solchen gestartet wurde.

Isolation

Jede Transaktion läuft isoliert von anderen Transaktionen ab. Sie ist aber nicht nur hinsichtlich der Behandlung des Datenbankpuffer–Bereichs, sondern auch hinsichtlich ihrer Datenbankbehandlung von anderen Transaktionen unabhängig.

Persistenz (Durability)

Falls die Transaktion dem Benutzer als *erfolgreich beendet* gemeldet wird, so überleben die von ihr in der Datenbank erzeugten Effekte jeden danach auftretenden Hardware– (Hauptspeicher) oder Softwarefehler, und zwar sogar dann, wenn sie sich zum Zeitpunkt des Fehlers noch im Datenbankpuffer (und nicht schon in der Datenbank selbst) befinden.

Beispiel (Konsistenz)

1. Vor und nach einer Transaktion T müssen alle Integritätsbedingungen (Primär– und Fremdschlüssel–Bedingungen) erfüllt sein.

Während der Transaktion können einzelne Updates von T die Konsistenz zeitweise verletzen.

2. Der folgende Schedule s entspricht keiner isolierten Ausführung der beiden Transaktionen T_1 und T_2 , denn T_2 liest das Resultat “ $X - 1$ ” von T_1 , und T_1 liest das Resultat “ $2 \cdot Y$ ” von T_2 .

s erzeugt in jedem Falle eine Resultats–Datenbank, welche die Integritätsbedingung “ $X + Y = 0$ ” verletzt, auch wenn die Ausgangs–Datenbank diese Bedingung erfüllt.

T_1	T_2
$r_1(X);$ $X := X - 1;$ $w_1(X);$	$r_2(X);$ $X := 2 \cdot X;$ $w_2(X);$ $r_2(Y);$ $Y := 2 \cdot Y;$ $w_2(Y);$
$r_1(Y);$ $Y := Y + 1;$ $w_1(Y);$	

Für die neuen Werte X' und Y' gilt nämlich $X' = 2 \cdot (X - 1)$ und $Y' = 2 \cdot Y + 1$. Falls $X + Y = 0$ gilt, so folgt daraus

$$X' + Y' = 2 \cdot (X - 1) + 2 \cdot Y + 1 = 2 \cdot (X + Y) - 1 = -1.$$

Wenn ein Schedule dagegen zuerst alle Aktionen von T_1 und dann alle Aktionen von T_2 enthält (1) oder umgekehrt (2), dann bleibt die Integritätsbedingung “ $X + Y = 0$ ” erfüllt:

$$(1) \quad X' + Y' = 2 \cdot (X - 1) + 2 \cdot (Y + 1) = 2 \cdot (X + Y) = 0,$$

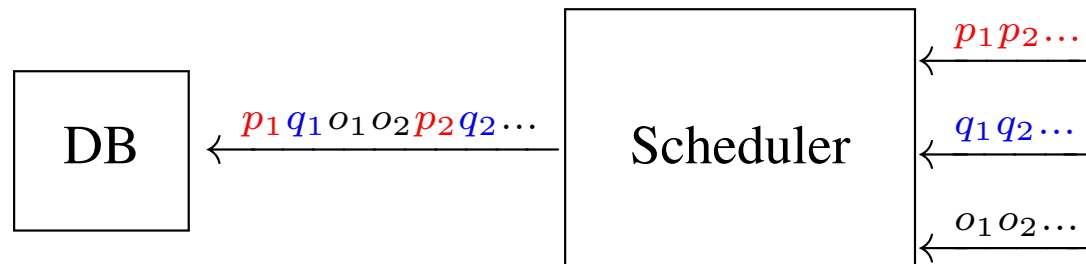
$$(2) \quad X' + Y' = 2 \cdot X - 1 + 2 \cdot Y + 1 = 2 \cdot (X + Y) = 0.$$

Transaktionen und Schedules

1. Zeitlich verzahnte Ausführungsreihenfolgen von Operationen (read, write) verschiedener Transaktionen werden modelliert durch *Schedules*.
2. Die relative Ausführungsreihenfolge der Operationen jeder Transaktion muß im Schedule erhalten bleiben.
3. Es liegt eine *dynamische Situation* vor:
 - Die zu verarbeitenden Transaktionen sind dem DBMS a priori nicht bekannt.
 - Der Scheduler muß jederzeit bereit sein, neu eintreffende Operationen zu verarbeiten und insbesondere mit bereits laufenden zu synchronisieren.

4. Vergleich mit dem *Einfädeln* lassen:

Wenn man die Transaktionen wie die Spuren einer Autobahn bei einer Streckenverengung auf eine einzige Spur und die Operationen wie die Fahrzeuge betrachtet, dann regelt der Scheduler das Einfädeln lassen.



Dabei müssen nicht immer alle Spuren gleichmäßig zum Zuge kommen.

Synchronisationsprobleme im Straßenverkehr werden beim Einfädeln durch das Reißverschlußverfahren gelöst:



Datenbanksysteme regeln die *Serialisierung* meist mittels Sperrprotokollen und Zeitstempelverfahren.

Syntaktische und semantische Äquivalenz, Serialisierbarkeit

1. Wir interessieren uns für Bedingungen, unter denen ein solcher Schedule als *korrekt* bezeichnet werden kann.

Diese Bedingungen sollen *leicht* testbar sein.

Die Theorie der Korrektheit von Schedules soll außerdem *ohne* die Kenntnis *semantischer* Information auskommen.

2. Zur Analyse der Korrektheit von Operationenfolgen werden nur *syntaktische* Kriterien herangezogen, nämlich die Folge der *Lese– und Schreiboperationen*.

Die Korrektheit eines Schedules hängt mit der *Äquivalenz* zu einem *seriellen Schedule* zusammen.

Diese garantiert dann die korrekte Synchronisation beliebiger Operationenfolgen.

Beispiel (Syntaktische und semantische Äquivalenz)

- Die beiden Schedules s_1, s_2 liefern unterschiedliche Ergebnisse ($X - 1$ bzw. $X - 2$) und sind semantisch nicht äquivalent:

$s_1 :$		$s_2 :$	
T_1	T_2	T_1	T_2
$r_1(X);$		$r_1(X);$	
	$r_2(X);$	$X := X - 1;$	
$X := X - 1;$		$w_1(X);$	
$w_1(X);$			$r_2(X);$
	$X := X - 1;$		$X := X - 1;$
	$w_2(X);$		$w_2(X);$

2. Die beiden Schedules s'_1, s'_2 liefern dasselbe Ergebnis und sind semantisch äquivalent für boolesche Werte $X \in \{0, 1\}$, denn

$$X - 1 = X^2 - 1 \Leftrightarrow X \in \{0, 1\},$$

sonst liefern sie unterschiedliche Ergebnisse.

$s'_1 :$		$s'_2 :$	
T_1	T_2	T_1	T_2
$r_1(X);$		$r_1(X);$	
	$r_2(X);$	$X := X^2;$	
$X := X^2;$		$w_1(X);$	
$w_1(X);$			$r_2(X);$
	$X := X - 1;$		$X := X - 1;$
	$w_2(X);$		$w_2(X);$

Nur die Datenoperationen unterscheiden sich im Vergleich zu 1.

Zur Analyse der Korrektheit von Schedules werden nur *syntaktische* Kriterien basierend auf der Folge der *Lese*– und *Schreib*–Operationen herangezogen.

$$\begin{aligned}s_1 &\hat{=} s'_1 \hat{=} r_1(X) r_2(X) w_1(X) w_2(X), \\ s_2 &\hat{=} s'_2 \hat{=} r_1(X) w_1(X) r_2(X) w_2(X).\end{aligned}$$

Die Schedules s_1 und s'_1 sind syntaktisch äquivalent, ebenso s_2 und s'_2 , da sie sich nur in den Daten–Operationen unterscheiden.

- Wegen der unterschiedlichen Reihenfolgen der Lese– und Schreib–Operationen sind s_1 und s_2 aber syntaktisch nicht äquivalent, ebenso s'_1 und s'_2 .
- Wir hatten bereits gesehen, daß s_1 und s_2 semantisch nicht äquivalent sind, s'_1 und s'_2 aber schon.

4.1.3 Transaktionen, Schedules und Serialisierbarkeit

Im Folgenden sei eine *Datenbank* eine Menge $D = \{ X, Y, Z, \dots \}$ von unteilbaren und disjunkten Datenobjekten (z.B. Datenseiten), welche in einem Schritt zwischen Sekundär- und Hauptspeicher bzw. Datenbankpuffer transportiert werden können.

Für die korrekte Synchronisation parallel auszuführender Benutzeraufträge an eine Datenbank ist es wichtig, deren

- lesende (read, r) und
- schreibende (write, w) Datenzugriffe

zu kontrollieren.

Definition (Transaktion)

Eine Transaktion ist eine endliche Folge

$$t_i = p_1 \dots p_n$$

von $n \in \mathbb{N}$ Lese– und Schreib–Operationen

$$p_j \in \{ r_i(X_j), w_i(X_j) \}, \text{ mit } X_j \in D, \text{ für } 1 \leq j \leq n.$$

Wir machen die Annahme, daß in jeder Transaktion jedes Datenobjekt höchstens einmal gelesen oder geschrieben wird.

In Gegenwart mehrerer Transaktionen $t_1, \dots, t_m$ kann man die Schritte der verschiedenen Transaktionen durch ihren *Index* eindeutig zuordnen.

Beispiel (Transaktion)

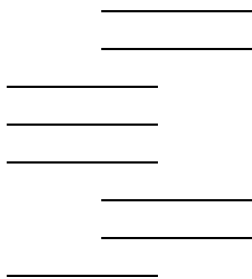
$$t_1 = r_1(X) w_1(X) r_1(Y) w_1(Y)$$

Definition (Shuffle-Produkt)

Das *Shuffle-Produkt* $shuffle(T)$ einer (endlichen) Menge $T = \{t_1, \dots, t_n\}$ von Transaktionen ist die Menge aller Folgen $s = p_1 \dots p_m$ von Schritten, welche folgende Bedingungen erfüllen:

- die Folgen $t_i \in T$ kommen in s als Teilfolgen vor,
- sonst kommen in s keine weiteren Schritte vor.

Das englische *to give the cards a shuffle* bedeutet im Deutschen *die Karten mischen*. Dabei werden zwei Kartenstapel – diese entsprechen den Transaktionen – unter Beibehaltung ihrer relativen Reihenfolge verschränkt.



Definition (Schedule, vollständig)

Sei $T = \{ t_1, \dots, t_n \}$ eine (endliche) Menge von Transaktionen.

1. Ein *vollständiger Schedule* s für T entsteht aus einer Folge $s' \in \text{shuffle}(T)$ durch Hinzunahme der Pseudo-Schritte

- c_i (commit) und
- a_i (abort)

für jede Transaktion $t_i \in T$ nach folgenden Regeln:

- entweder c_i oder a_i ist in s , aber nicht beide: $c_i \in s \Leftrightarrow a_i \notin s$,
- c_i bzw. a_i steht in s irgendwo hinter dem letzten Schritt von t_i .

2. $\text{shuffle}_{ac}(T)$ bezeichnet die Menge aller vollständigen Schedules für T .
3. Ein *Schedule* ist ein Präfix eines vollständigen Schedules.

Definition (Schedule, seriell)

Ein vollständiger Schedule s für eine (endliche) Menge $T = \{t_1, \dots, t_n\}$ von Transaktionen heißt *seriell*, falls der

1. die Transaktionen in einer beliebigen Reihenfolge hintereinander ausführt (d.h. ohne Verschränkung) und dabei
2. jede Transaktion mit einem der Pseudo-Schritte $p_i \in \{c_i, a_i\}$, für $1 \leq i \leq n$, abschließt:

Ein serieller Schedule s ist also auch vollständig, und er hat die Form

$$s = t_{\pi(1)} p_{\pi(1)} \dots t_{\pi(n)} p_{\pi(n)}$$

für eine Permutation $\pi : \langle 1, n \rangle \longrightarrow \langle 1, n \rangle$.

Permutationen sind bijektive Abbildungen

$$\pi : \langle 1, n \rangle \longrightarrow \langle 1, n \rangle.$$

Man kann sie durch ihren Funktionsgraphen (Wertetabelle) veranschaulichen:

i	1	2	3
$\pi(i)$	3	1	2

Die zweite Zeile enthält eine Permutation des Intervalls $\langle 1, n \rangle$.

Für die Transaktionsmenge $T = \{ t_1, t_2, t_3 \}$ und die Pseudo-Schritte c_1, a_2, c_3 erhalten wir den seriellen Schedule

$$s = t_{\pi(1)} p_{\pi(1)} t_{\pi(2)} p_{\pi(2)} t_{\pi(3)} p_{\pi(3)} = t_3 c_3 t_1 c_1 t_2 a_2.$$

Da es $n!$ Permutation des Intervalls $\langle 1, n \rangle$ gibt, gibt es für vorgegebene Pseudo-Schritte genauso viele mögliche serielle Schedules zu T .

Beispiel (Schedule)

Gegeben sei die Menge $T = \{ t_1, t_2 \}$ von Transaktionen:

$$t_1 = r_1(X) w_1(X) r_1(Y) w_1(Y),$$

$$t_2 = r_2(X) w_2(X).$$

1. $s_1 = r_1(X) w_1(X) r_2(X) w_2(X) r_1(Y) w_1(Y) a_1 c_2$
ist ein *vollständiger* Schedule.

Das Präfix $s_2 = r_1(X) w_1(X) r_2(X) w_2(X) r_1(Y) w_1(Y)$
von s_1 ist ein Schedule.

2. $s_3 = t_1 a_1 t_2 c_2$ (für $\pi(1) = 1, \pi(2) = 2$) und $s_4 = t_2 c_2 t_1 a_1$ (für $\pi(1) = 2, \pi(2) = 1$) sind *serielle* Schedules.

Der Status einer Transaktion ergibt sich aus einem Schedule.

Definition (Transaktionsstatus)

1. $trans(s)$ enthält alle an einem Schedule s beteiligten Transaktionen, $commit(s)$ alle bereits freigegebenen, $abort(s)$ alle abgebrochenen, und $active(s)$ alle noch aktiven Transaktionen:

$$trans(s) = \{ t_i \mid s \text{ enthält Schritte von } t_i \},$$

$$commit(s) = \{ t_i \in trans(s) \mid s \text{ enthält } c_i \},$$

$$abort(s) = \{ t_i \in trans(s) \mid s \text{ enthält } a_i \},$$

$$active(s) = trans(s) \setminus (commit(s) \cup abort(s)).$$

2. Die Menge aller in s vorkommenden Schritte (einschließlich der Schritte a_i und c_i) wird mit $op(s)$ bezeichnet.

Beispiel (Transaktionen, Schedules)

Gegeben sei die Menge $T = \{ t_1, t_2, t_3 \}$ von Transaktionen:

$$t_1 = r_1(X) w_1(X) r_1(Y) w_1(Y),$$

$$t_2 = r_2(Z) w_2(X) w_2(Z),$$

$$t_3 = r_3(X) r_3(Y) w_3(Z).$$

1. Dann gilt:

$$s_1 = r_1(X) r_2(Z) r_3(X) w_2(X) w_1(X) r_3(Y) \\ r_1(Y) w_1(Y) w_2(Z) w_3(Z) \in \text{shuffle}(T),$$

$$s_2 = s_1 c_1 c_2 a_3 \in \text{shuffle}_{ac}(T),$$

$$s_3 = t_1 c_1 t_3 a_3 t_2 c_2 \in \text{shuffle}_{ac}(T) \text{ ist ein serieller Schedule.}$$

Das Präfix $s_4 = r_1(X) r_2(Z) r_3(X)$ des vollständigen Schedules s_2 ist ein Schedule.

2. Für den vollständigen Schedule $s = s_2$ und den seriellen Schedule $s = s_3$ sind die Transaktionsmengen gleich:

$$\text{trans}(s) = \{ t_1, t_2, t_3 \},$$

$$\text{commit}(s) = \{ t_1, t_2 \},$$

$$\text{abort}(s) = \{ t_3 \},$$

$$\text{active}(s) = \emptyset.$$

Die Operationenmengen sind folgende:

$$\text{op}(s) = \text{op}(t_1) \cup \text{op}(t_2) \cup \text{op}(t_3) \cup \{ c_1, c_2, a_3 \},$$

$$\text{op}(t_1) = \{ r_1(X), w_1(X), r_1(Y), w_1(Y) \},$$

$$\text{op}(t_2) = \{ r_2(Z), w_2(X), w_2(Z) \},$$

$$\text{op}(t_3) = \{ r_3(X), r_3(Y), w_3(Z) \}.$$

Definition (Abschluß eines Schedules)

1. Sei t_0 eine Transaktion, welche alle in einem Schedule s angesprochenen Datenobjekte – in der Reihenfolge des Auftretens in s jedes Datenobjekt genau einmal – liest und dann freigegeben wird.
2. Dann heißt der Schedule $\bar{s}$ der *Abschluß* von s :

$$\bar{s} = s t_0 c_0.$$

Die Transaktion t_0 liest den am Ende von s *aktuellen* Datenbankzustand, den *Netto-Effekt* von s .

Definition (Projektionen eines Schedules)

1. Die *Projektion* $\Pi_T(s)$ von s auf $T \subseteq \text{trans}(s)$ ist der Schedule, der entsteht, wenn man aus s alle Schritte zu den Transaktionen aus $\text{trans}(s) \setminus T$ entfernt.
2. Die *freigegebene Projektion* (committed projection) von s ist
$$CP(s) = \Pi_{\text{commit}(s)}(s).$$
3. Der *bereinigte* Schedule s^* entsteht aus s , indem man alle Operationen zu abgebrochenen Transaktionen wegläßt:

$$s^* = \Pi_{\text{commit}(s) \cup \text{active}(s)}(s).$$

Für einen vollständigen Schedule s gilt $\text{active}(s) = \emptyset$, und folglich $CP(s) = s^*$.

Beispiel (Abschluß und Projektionen eines Schedules)

Wir betrachten den Schedule

$$s = w_1(X) r_2(X) w_2(Y) r_1(Y) w_1(Y) w_3(X) w_3(Y) c_1 a_2.$$

1. Da s die Datenobjekte X und Y anspricht, ist der Abschluß von s gegeben durch

$$\bar{s} = s t_0 c_0, \text{ wobei } t_0 = r_0(X) r_0(Y).$$

2. Für $T = \{ t_1, t_2 \}$ gilt $\text{commit}(s) = \{ t_1 \}$, $\text{active}(s) = \{ t_3 \}$, und:

$$\Pi_T(s) = w_1(X) r_2(X) w_2(Y) r_1(Y) w_1(Y) c_1 a_2, \quad \mapsto t_1, t_2$$

$$CP(s) = w_1(X) r_1(Y) w_1(Y) c_1, \quad \mapsto t_1$$

$$s^* = w_1(X) r_1(Y) w_1(Y) w_3(X) w_3(Y) c_1. \quad \mapsto t_1, t_3$$

Serialisierbarkeit

Äquivalenz zu einem seriellen Schedule.

Definition (Relationen $<_s$ und $\dot{\rightarrow}_s$)

1. Steht in einem Schedule s ein Schritt p vor einem Schritt q , so schreiben wir $p <_s q$.
2. Seien $r_i(X)$ und $w_j(X)$ zwei Operationen von unterschiedlichen Transaktionen t_i bzw. t_j in s auf demselben Datenobjekt X .
 $r_i(X)$ liest X in s von $w_j(X)$, falls $w_j(X)$ die letzte Schreiboperation in s vor $r_i(X)$ ist, so daß t_j vor $r_i(X)$ nicht abgebrochen ist.
3. Dann schreiben wir $w_j \xrightarrow{X}_s r_i$.

Beispiel (Liest–Von)

Für den Schedule $s = w_1(X) w_2(X) r_3(X) a_2 r_4(X)$ gilt

$$w_1(X) <_s r_3(X), w_1(X) <_s r_4(X),$$

$$w_2(X) <_s r_3(X),$$

$$w_2(X) <_s r_4(X), \text{ aber } w_2(X) <_s a_2 <_s r_4(X),$$

und folglich

$$w_1 \xrightarrow{X}_s r_4,$$

$$w_2 \xrightarrow{X}_s r_3,$$

aber $w_1 \xrightarrow{X}_s r_3$ und $w_2 \xrightarrow{X}_s r_4$ gelten nicht.

Die Liest–Von–Relationen

Die folgenden Relationen basieren auf dem Abschluß eines Schedules, um zusätzlich zu berücksichtigen, welche Schreiboperationen den resultierenden Datenbankzustand ausmachen.

Definition (Liest–Von–Relationen)

1. Die Liest–Von–Relation $RF(s)$ basiert auf dem Abschluß $\bar{s}$ eines Schedules s :

$$RF(s) = \{ w_j \xrightarrow{X} r_i \mid w_j \xrightarrow{X}_{\bar{s}} r_i \}.$$

2. Die bereinigte Liest–Von–Relation $RF^*(s)$ basiert auf dem bereinigten Schedule s^* (ohne die abgebrochenen Transaktionen):

$$RF^*(s) = RF(s^*).$$

Für einen Schedule s und $t_i \neq t_j \in \text{trans}(s)$ gilt:

$$\begin{aligned} w_j \xrightarrow{X} r_i \in RF(s) &\iff w_j \xrightarrow{X}_{\bar{s}} r_i \iff \\ &w_j(X) <_{\bar{s}} r_i(X) \wedge a_j \not<_{\bar{s}} r_i(X) \wedge \\ &(w_j(X) <_{\bar{s}} w_k(X) <_{\bar{s}} r_i(X) \Rightarrow a_k <_{\bar{s}} r_i(X)) . \end{aligned}$$

Im Abschluß liest eine Transaktion t_i ein Datenobjekt X ,

- das von einer anderen Transaktion t_j geschrieben wurde,
- wenn alle dazwischen liegenden Schreiboperation nur von Transaktionen t_k stammen, die vor dem Lesen wieder abgebrochen wurden.

Beispiel (Liest–Von–Relation)

1. Für die Schedules $s_{1,2} = w_1(X) w_2(X)$ und $s_{2,1} = w_2(X) w_1(X)$ gilt

$$\overline{s_{1,2}} = s_{1,2} r_0(X) c_0,$$

$$\overline{s_{2,1}} = s_{2,1} r_0(X) c_0,$$

und da es keine abgebrochenen Transaktionen gibt ist

$$RF^*(s_{1,2}) = RF(s_{1,2}) = \{ w_2 \xrightarrow{X} r_0 \},$$

$$RF^*(s_{2,1}) = RF(s_{2,1}) = \{ w_1 \xrightarrow{X} r_0 \}.$$

Der Netto–Effekt der Schedules, der von $r_0(X)$ gelesen wird, wird von $w_2(X)$ bzw. $w_1(X)$ bewirkt.

2. Unter dem Schutze einer weiteren, abschließenden Schreiboperation $w_3(X)$ haben die beiden Schedules

$$\begin{aligned} s_{1,2,3} &= \overbrace{w_1(X) w_2(X)}^{s_{1,2}} w_3(X), \\ s_{2,1,3} &= w_2(X) \underbrace{w_1(X)}_{s_{2,1}} w_3(X) \end{aligned}$$

denselben Netto-Effekt; wir erhalten

$$RF^*(s_{1,2,3}) = RF^*(s_{2,1,3}) = \{ w_3 \xrightarrow{X} r_0 \}.$$

Der Netto-Effekt wird nun von $w_3(X)$ bewirkt.

Wirkung eines Transaktionsabbruchs

1. Für $s = r_1(X) w_1(Y) r_2(Y) w_2(Y)$ und die Schedules

$$s_1 = s \mathbf{c_1} \mathbf{c_2}, \quad s_2 = s \mathbf{c_1} \mathbf{a_2}, \quad s_3 = r_1(X) w_1(Y) \mathbf{c_1} r_2(Y) w_2(Y) \mathbf{c_2}$$

gilt $\bar{s}_i = s_i r_0(X) r_0(Y) c_0$.

a) In $\bar{s}_1$ und $\bar{s}_3$ gilt für das Datenobjekt Y : r_2 liest von w_1 und r_0 von w_2 . In $\bar{s}_2$ liest wegen a_2 hingegen r_0 das Datenobjekt Y von w_1 .

b) Wir erhalten also die folgenden Relationen:

$$\begin{aligned} RF^*(s_1) &= \{ w_1 \xrightarrow{Y} r_2, w_2 \xrightarrow{Y} r_0 \} = RF(s_1), \\ RF^*(s_2) &= \{ w_1 \xrightarrow{Y} r_0 \}, \quad RF(s_2) = \{ w_1 \xrightarrow{Y} r_2, w_1 \xrightarrow{Y} r_0 \}, \\ RF^*(s_3) &= \{ w_1 \xrightarrow{Y} r_2, w_2 \xrightarrow{Y} r_0 \} = RF(s_3). \end{aligned}$$

c) In s_2 wird t_2 abgebrochen. Da a_2 erst nach $r_2(Y)$ erfolgt, erhalten wir $w_1 \xrightarrow{Y} r_2$. Die relevante Schreiboperation $w_1(Y)$ für $r_0(Y)$ stammt dann aber von der nicht abgebrochenen Transaktion t_1 .

2. Für die Schedules $s_1 = w_1(X) r_2(X)$ und

$$s_2 = w_1(X) r_2(X) \textcolor{red}{a}_1,$$

$$s_3 = w_1(X) \textcolor{red}{a}_1 r_2(X),$$

gilt $RF(s_1) = \{ w_1 \xrightarrow{X} r_2, w_1 \xrightarrow{X} r_0 \}$ und

$$RF(s_2) = \{ w_1 \xrightarrow{X} r_2 \},$$

$$RF(s_3) = \emptyset.$$

- In s_2 und s_3 hat $w_1(X)$ keinen Netto-Effekt, da die Transaktion t_1 abgebrochen wird; der Konflikt $w_1 \xrightarrow{X} r_0$ existiert hier nicht.
- In s_3 wird die Transaktion t_1 sogar vor $r_2(X)$ abgebrochen; deswegen existiert auch der Konflikt $w_1 \xrightarrow{X} r_2$ hier nicht.

Die bereinigten Liest–Von–Relationen von s_2 und s_3 sind beide $\emptyset$, da hierfür der Zeitpunkt des Abbruchs von t_1 keine Rolle spielt.

3. Für den Schedule

$$s = w_1(X) r_2(X) a_1$$

sind

$$RF^*(s) = \emptyset, \text{ und}$$

$$RF(s) = \{ w_1 \xrightarrow{X} r_2 \}$$

unterschiedlich, da der Abbruch a_1 erst nach $r_2(X)$ erfolgt.

$w_1 \xrightarrow{X} r_0$ gilt nicht, da in

$$\bar{s} = w_1(X) r_2(X) a_1 r_0(X) c_0$$

der Abbruch a_1 vor $r_0(X)$ erfolgt.

Bei der View-Äquivalenz müssen die vorkommenden Schritte und die bereinigten RF^* -Relationen (d.h. ohne die abgebrochenen Transaktionen) gleich sein.

Definition (View-Äquivalenz, View-Serialisierbarkeit)

1. Zwei Schedules s, s' heißen *view-äquivalent*, in Zeichen $s \approx_v s'$, falls gilt:

$$\begin{aligned} op(s) &= op(s') \text{ und} \\ RF^*(s) &= RF^*(s'). \end{aligned}$$

2. Ein vollständiger Schedule s heißt *view-serialisierbar*, falls er view-äquivalent ist zu einem seriellen Schedule s' (d.h. $s \approx_v s'$).
3. VSR ist die Klasse der view-serialisierbaren Schedules.

Falls eine Schedule n Transaktionen enthält, dann gibt es $n!$ mögliche serielle Schedules, die man auf View-Äquivalenz testen muß.

Satz (Test auf View-Äquivalenz und -Serialisierbarkeit)

1. Das Problem zu entscheiden, ob zwei gegebene Schedules s, s' view-äquivalent sind, ist in Linearzeit lösbar.
2. Das Problem zu entscheiden, ob ein gegebener Schedule s view-serialisierbar ist, ist $\mathcal{NP}$ -vollständig.

Die Verfahren zur Erzeugung view-serialisierbarer Schedules vermeiden diesen Test – genau wie die 3NF-Synthese den $\mathcal{NP}$ -vollständigen 3NF-Test vermeidet.

Beispiel (View–Serialisierbarkeit)

1. Der Schedule

$$s = r_1(X) r_2(Y) w_1(Y) w_2(Y) c_1 c_2$$

ist zu keiner der beiden möglichen Serialisierungen $s_{1,2}$ und $s_{2,1}$ view–äquivalent und damit *nicht view–serialisierbar*:

$$s_{1,2} = r_1(X) w_1(Y) c_1 r_2(Y) w_2(Y) c_2,$$

$$s_{2,1} = r_2(Y) w_2(Y) c_2 r_1(X) w_1(Y) c_1.$$

Es gilt $op(s) = op(s_{1,2}) = op(s_{2,1})$, aber

$$RF^*(s) = RF(s) = \{ w_2 \xrightarrow{Y} r_0 \},$$

$$RF^*(s_{1,2}) = RF(s_{1,2}) = \{ w_1 \xrightarrow{Y} r_2, w_2 \xrightarrow{Y} r_0 \},$$

$$RF^*(s_{2,1}) = RF(s_{2,1}) = \{ w_1 \xrightarrow{Y} r_0 \}.$$

Auf X gab es keine Schreiboperation.

2. Für den leicht variierten Schedule

$$s = r_1(X) r_2(Y) w_1(Y) w_2(Y) a_1 c_2,$$

in dem die Transaktion t_1 abgebrochen wird, gilt:

$$s_{1,2} = r_1(X) w_1(Y) a_1 r_2(Y) w_2(Y) c_2,$$

$$s_{2,1} = r_2(Y) w_2(Y) c_2 r_1(X) w_1(Y) a_1,$$

Dann gilt $op(s) = op(s_{1,2}) = op(s_{2,1})$ und

$$RF^*(s) = RF^*(s_{1,2}) = RF^*(s_{2,1}) = \{ w_2 \xrightarrow{Y} r_0 \}.$$

Also ist s view-äquivalent zu $s_{1,2}$ und zu $s_{2,1}$ und damit *view-serialisierbar*, denn die Aktionen der abgebrochenen Transaktion t_1 – insbesondere die Schreiboperation $w_1(Y)$ – spielen hier keine Rolle.

Konfliktrelationen

Die Konfliktrelationen berücksichtigen Konflikte, bei denen (mindestens) eine Schreiboperation beteiligt ist.

Definition (Konfliktrelation)

Sei s ein Schedule und $t, t' \in \text{trans}(s)$, mit $t \neq t'$.

1. Zwei Operationen $p(X) \in t$ und $q(X) \in t'$ stehen in s in Konflikt, falls sie auf demselben Datenobjekt X operieren und mindestens eine von ihnen schreibt, d.h. $p = w_i$ oder $q = w_j$.

2. Die *Konfliktrelation* $C(s)$ von s ist definiert durch

$$C(s) = \{ p \xrightarrow{X} q \mid p(X) <_s q(X) \text{ und } p(X) \text{ und } q(X) \text{ stehen in } s \text{ in Konflikt} \}.$$

3. Die *bereinigte Konfliktrelation* $C^*(s)$ von s bezieht sich auf s^* und ignoriert somit die abgebrochenen Transaktionen: $C^*(s) = C(s^*)$.

Beispiel (Konfliktrelation)

1. Für den Schedule $s = w_1(X)$ erhalten wir $C^*(s) = C(s) = \emptyset$ und $RF^*(s) = RF(s) = \{ w_1 \xrightarrow{X} r_0 \}$.
2. Der folgende Schedule operiert auf den Datenobjekten X und Y :

$$s = w_1(X) r_2(X) w_2(Y) r_1(Y) w_1(Y) w_3(X) w_3(Y) c_1 a_2.$$

Wir erhalten folgende Konfliktrelationen:

$$C(s) = C^*(s) \cup \{ w_1 \xrightarrow{X} r_2, r_2 \xrightarrow{X} w_3, w_2 \xrightarrow{Y} r_1, w_2 \xrightarrow{Y} w_1, w_2 \xrightarrow{Y} w_3 \},$$

$$C^*(s) = \{ w_1 \xrightarrow{X} w_3, r_1 \xrightarrow{Y} w_3, w_1 \xrightarrow{Y} w_3 \}.$$

Die abgebrochene Transaktion t_2 wird in $C^*(s)$ nicht berücksichtigt.

$C(s)$ enthält transitive Verknüpfungen wie z.B.

$$w_1 \xrightarrow{X} r_2, r_2 \xrightarrow{X} w_3 \text{ und } w_1 \xrightarrow{X} w_3.$$

$RF(s)$ **und** $RF^*(s)$ **vs.** $C(s)$ **und** $C^*(s)$

Die Konfliktrelation $C(s)$ berücksichtigt alle Konflikte

$$w_i \xrightarrow{X} r_j, w_i \xrightarrow{X} w_j, r_i \xrightarrow{X} w_j,$$

	r_j	w_j
r_i		○
w_i	○	○

an denen eine Schreib–Operation beteiligt ist,

während die Liest–Von–Relation $RF(s)$ nur Konflikte $w_i \xrightarrow{X} r_j$ berücksichtigt, bei denen die schreibende Transaktion t_i vor $r_j(X)$ nicht abgebrochen wurde.

Abgebrochene Transaktionen

- werden in $RF^*(s)$ und in $C^*(s)$ überhaupt nicht berücksichtigt,
- während sie in $RF(s)$ nur dann nicht berücksichtigt werden, wenn der Abbruch vor $r_j(X)$ erfolgte.

Für jedes Datenobjekt X erkennt man die letzte wirksame Schreib-Operation $w_i(X)$, mit $t_i \notin \text{abort}(s)$, wie folgt:

- an $w_i \xrightarrow{X} r_0 \in RF(s)$ bzw.
- an den am weitesten rechts stehenden Konflikten $w_i \xrightarrow{X} p$ oder $p \xrightarrow{X} w_i \in C^*(s)$, an denen X beteiligt ist (falls es solche gibt).

Beispiel ($RF(s)$, $RF^*(s)$ vs. $C(s)$, $C^*(s)$)

1. Für den Schedule $s_1 = w_1(X) w_2(X) r_3(X)$ erhalten wir

$$RF(s_1) = \{ w_2 \xrightarrow{X} r_3, w_2 \xrightarrow{X} r_0 \} = RF^*(s_1),$$

$$C(s_1) = \{ w_1 \xrightarrow{X} w_2, w_1 \xrightarrow{X} r_3, w_2 \xrightarrow{X} r_3 \} = C^*(s_1).$$

2. Für $s_2 = w_1(X) w_2(X) a_2 r_3(X)$ und $s_3 = w_1(X) w_2(X) r_3(X) a_2$ erhalten wir $C(s_1) = C(s_2) = C(s_3)$, aber

$$RF(s_2) = \{ w_1 \xrightarrow{X} r_3, w_1 \xrightarrow{X} r_0 \},$$

$$RF(s_3) = \{ w_2 \xrightarrow{X} r_3, w_1 \xrightarrow{X} r_0 \},$$

da der Abbruch a_2 bei C gar nicht und bei RF erst für nachfolgende Lese-Operationen wirksam ist.

3. $RF^*(s_2) = RF^*(s_3) = RF(s_2)$ und $C^*(s_2) = C^*(s_3) = \{ w_1 \xrightarrow{X} r_3 \}$ berücksichtigen t_2 wegen a_2 dagegen gleich (nämlich gar nicht).

Definition (Konflikt-Äquivalenz, Konflikt-Serialisierbarkeit)

1. Zwei Schedules s und s' heißen *konflikt-äquivalent*, in Zeichen $s \approx_c s'$, falls $op(s) = op(s')$ und $C^*(s) = C^*(s')$.
2. Ein vollständiger Schedule s heißt *konflikt-serialisierbar*, falls er konflikt-äquivalent ist zu einem seriellen Schedule s' (d.h. $s \approx_c s'$).
3. CSR ist die Klasse der konflikt-serialisierbaren Schedules.

Satz (View- vs. Konflikt-Serialisierbarkeit)

1. Jeder konflikt-serialisierbare Schedule ist auch view-serialisierbar.
2. Es gibt view-serialisierbare Schedules, die nicht konflikt-serialisierbar sind.
3. Es gilt also $CSR \subsetneq VSR$.

Beispiel (Konflikt-Äquivalenz)

Wir betrachten die folgenden Schedules:

$$s = r_1(X) r_1(Y) w_2(X) w_1(Y) r_2(Z) w_1(X) w_2(Y),$$

$$s' = r_1(Y) r_1(X) w_1(Y) w_2(X) w_1(X) r_2(Z) w_2(Y).$$

s' entstand aus s durch paarweise Vertauschungen von nicht in Konflikt stehenden Operationen. Dann gilt $op(s) = op(s')$ und

$$C^*(s) = C^*(s') = \{ r_1 \xrightarrow{X} w_2, r_1 \xrightarrow{Y} w_2, w_2 \xrightarrow{X} w_1, w_1 \xrightarrow{Y} w_2 \},$$

und folglich $s \approx_c s'$. Hier gilt auch $s \approx_v s'$, denn

$$RF^*(s) = RF^*(s') = \{ w_1 \xrightarrow{X} r_0, w_2 \xrightarrow{Y} r_0 \}.$$

Definition (Konfliktgraph)

Der *Konfliktgraph* $G(s) = (V, E)$ eines Schedules s ist definiert durch

$$V = \text{commit}(s) \text{ und}$$

$$E = \{ (t, t') \in V^2 \mid \exists p(X) \in t, q(X) \in t' : p \xrightarrow{X} q \in C^*(s) \}.$$

Satz (Test auf Konflikt–Serialisierbarkeit)

Für einen vollständigen Schedule s ist $s \in CSR$, g.d.w. $G(s)$ azyklisch ist.

Der Beweis wird später gezeigt.

Der Test $s \in CSR$ kann also in polynomieller Zeit erfolgen.

Beispiel (Konfliktgraph)

Der vollständige Schedule

$$s = r_1(X) r_2(X) w_1(X) r_3(X) w_3(X) w_2(Y) c_3 c_2 w_1(Y) c_1$$

hat die bereinigte Konfliktrelation

$$C^*(s) = \{ r_1 \xrightarrow{X} w_3, r_2 \xrightarrow{X} w_1, r_2 \xrightarrow{X} w_3, w_1 \xrightarrow{X} r_3, w_1 \xrightarrow{X} w_3, w_2 \xrightarrow{Y} w_1 \}.$$

Die beteiligten Transaktionen sind

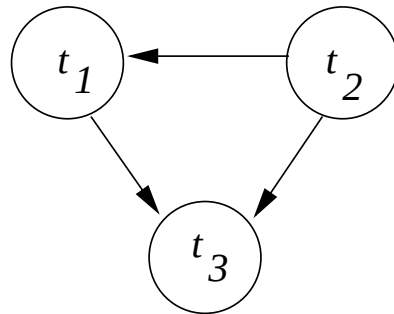
$$t_1 = r_1(X) w_1(X) w_1(Y), \quad t_2 = r_2(X) w_2(Y), \quad t_3 = r_3(X) w_3(X).$$

Wir erhalten den *Konfliktgraphen* $G(s)$ mit

$$V = \{ t_1, t_2, t_3 \},$$

$$E = \{ (t_1, t_3), (t_2, t_1), (t_2, t_3) \}.$$

Dieser ist azyklisch:



Für den seriellen Schedule $s' = t_2 c_2 t_1 c_1 t_3 c_3$ gilt $op(s) = op(s')$ und $C^*(s) = C^*(s')$. Also ist $s \approx_c s'$, und somit ist s *konflikt-serialisierbar*.

Eigenschaften von Konfliktgraphen

Lemma (Konfliktgraphen)

Zwei konflikt-äquivalente Schedules s und s' haben denselben Konfliktgraphen: $s \approx_c s' \implies G(s) = G(s')$.

Beweis.

Sei $G(s) = (V, E)$ und $G(s') = (V', E')$.

Nach Definition der Konflikt-Äquivalenz $s \approx_c s'$ gilt:

- $op(s) = op(s')$,
- $C^*(s) = C^*(s')$.

Aus dem ersten folgt $V = commit(s) = commit(s') = V'$, und aus dem zweitem folgt $E = E'$. Also ist $G(s) = G(s')$. □

Lemma (Konfliktgraphen)

Der Konfliktgraph $G(s)$ eines seriellen Schedules s ist azyklisch.

Beweis.

Sei $s = t_1 p_1 \dots t_n p_n$.

- Für jede Kante $(t_i, t_j) \in E$ gibt es $p \xrightarrow{X} q \in C^*(s)$ mit Operationen $p(X) \in t_i$ und $q(X) \in t_j$.
- Deshalb gilt $i < j$.
- Also können die Knoten von $G(s)$ nach aufsteigenden Indizes topologisch sortiert werden.
- Damit ist $G(s)$ azyklisch. □

Konflikt–neutrale Vertauschungen

Die Relationen $RF^*(s)$ und $C^*(s)$ sind invariant gegenüber gewissen *Vertauschungen* der Operationen von s :

$$\begin{aligned}
 s &= r_1(Y) r_3(V) r_2(Y) w_1(Y) w_1(X) w_2(X) w_2(Z) w_3(X) c_1 c_2 c_3, \\
 &\quad \downarrow \\
 s' &= r_2(Y) r_1(Y) w_1(Y) w_1(X) w_2(X) w_2(Z) r_3(V) w_3(X) c_1 c_2 c_3 \\
 &\quad s' \approx_c s, s' \approx_v s \\
 &\quad \downarrow \\
 s'' &= r_2(Y) w_2(X) w_2(Z) c_2 r_1(Y) w_1(Y) w_1(X) c_1 r_3(V) w_3(X) c_3 \\
 &\quad s'' \approx_v s, s'' \not\approx_c s
 \end{aligned}$$

Die Leseoperationen $r_1(Y)$ und $r_2(Y)$ können vertauscht werden ohne RF^* oder C^* zu ändern. Unter dem Schutze von $w_3(X)$ können $w_1(X)$ und $w_2(X)$ vertauscht werden, ohne RF^* zu ändern; dabei ändert sich aber C^* .

Aktionen unterschiedlicher Transaktionen, die auf unterschiedliche Datenobjekte zugreifen, können natürlich vertauscht werden, ohne RF^* oder C^* zu ändern:

- Da V sonst nicht vorkommt, kann $r_3(V)$ nach hinten wandern.
- $w_2(X)$ kann über $w_1(Y)$ und $r_1(Y)$ hinweg nach vorne wandern.

Schließlich können auch die Pseudo–Schritte c_i and a_i (hier nicht vorhanden) mit Aktionen anderer Transaktionen sowie mit anderen Pseudo–Schritten vertauscht werden – natürlich können sie aber nicht über Aktionen ihrer eigenen Transaktion hinweg wandern.

Der Schedule s ist also *view–serialisierbar*, da s zu dem seriellen Schedule s'' view–äquivalent ist. Wir werden gleich sehen, daß s aber *nicht konflikt–serialisierbar* ist.

Beispiel (View- vs. Konflikt-Serialisierbarkeit)

Der vollständige Schedule von gerade eben,

$$s = r_1(Y) r_3(V) r_2(Y) w_1(Y) \textcolor{red}{w_1(X)} \textcolor{red}{w_2(X)} w_2(Z) \textcolor{green}{w_3(X)} c_1 c_2 c_3,$$

ist view-serialisierbar mit $s \approx_v s'' = t_2 c_2 t_1 c_1 t_3 c_3$:

$$RF^*(s) = RF^*(s'') = \{ w_1 \xrightarrow{Y} r_0, w_2 \xrightarrow{Z} r_0, w_3 \xrightarrow{X} r_0 \}.$$

Wir erhalten aber unterschiedliche Konfliktrelationen

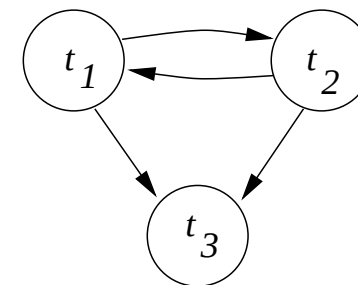
$$C^*(s) = \{ r_2 \xrightarrow{Y} w_1, \textcolor{red}{w_1} \xrightarrow{\textcolor{red}{X}} \textcolor{red}{w_2}, w_1 \xrightarrow{X} w_3, w_2 \xrightarrow{X} w_3 \},$$

$$C^*(s'') = \{ r_2 \xrightarrow{Y} w_1, \textcolor{red}{w_2} \xrightarrow{\textcolor{red}{X}} \textcolor{red}{w_1}, w_1 \xrightarrow{X} w_3, w_2 \xrightarrow{X} w_3 \},$$

und den zyklischen Konfliktgraphen $G(s)$.

Also ist s nicht konflikt-serialisierbar:

$$s \in VSR \setminus CSR.$$



Satz (Test auf Konflikt–Serialisierbarkeit)

Für einen vollständigen Schedule s ist $s \in CSR$, g.d.w. $G(s)$ azyklisch ist.
Der Test $s \in CSR$ kann also in polynomieller Zeit erfolgen.

Beweis.

“ $\Rightarrow$ ” Zu einem Schedule $s \in CSR$ gibt es nach Definition einen seriellen Schedule s' mit $s \approx_c s'$. Für diesen gilt $G(s) = G(s')$.
Da s' seriell ist, ist $G(s')$ azyklisch, und somit auch $G(s)$.

“ $\Leftarrow$ ” Sei $G(s) = (V, E)$ mit $V = \{t_1, \dots, t_n\}$ azyklisch.
Dann kann $G(s)$ topologisch sortiert werden mittels einer Permutation $\pi : \langle 1, n \rangle \longrightarrow \langle 1, n \rangle$.
Sei s'' der Sub–Schedule von s zu den abgebrochenen Transaktionen aus s .

Dann betrachten wir folgenden seriellen Schedule

$$s' = s'' t_{\pi(1)} c_{\pi(1)} \dots t_{\pi(n)} c_{\pi(n)}.$$

Wir zeigen, daß $C^*(s) = C^*(s')$ und somit $s \approx_c s'$ gilt.

Offensichtlich gilt $op(s) = op(s')$, $commit(s) = commit(s')$ und

- Für $p \xrightarrow{X} q \in C^*(s)$, mit $p(X) \in t_i$, $q(X) \in t_j$, gilt

$$(t_i, t_j) \in E \xrightarrow{top. Sort.} t_i \text{ steht vor } t_j \text{ in } s' \implies p \xrightarrow{X} q \in C^*(s').$$
- Für $p \xrightarrow{X} q \in C^*(s')$, mit $p(X) \in t_{\pi(i)}$, $q(X) \in t_{\pi(j)}$, $i < j$, gilt

$$\xrightarrow{top. Sort.} p(X) <_s q(X) \implies p \xrightarrow{X} q \in C^*(s).$$

Also gilt $C^*(s) = C^*(s')$ und somit $s \approx_c s'$. □

Der auf $C^*(s)$ basierende Konfliktgraph $G(s)$ eines vollständigen Schedules s kann dessen Konflikt–Serialisierbarkeit also entscheiden:

- s ist konflikt–serialisierbar, genau dann wenn $G(s)$ *acyklisch* ist.
- Seine *topologische Sortierung* liefert dann einen seriellen Schedule s' , der zu s konflikt–äquivalent ist.

Der entsprechende, auf $RF^*(s)$ basierende Konfliktgraph G kann die View–Serialisierbarkeit dagegen nicht entscheiden:

- Falls er *zyklisch* ist, so ist s sicher nicht view–serialisierbar.
- Falls er *acyklisch* ist, so liefert er eine partielle Ordnung, der potentielle zu s view–äquivalente serielle Schedules s' folgen müssen. Das heißt aber nicht, daß es dann auch einen zu s view–äquivalenten seriellen Schedule s' gibt.

Konfliktgraphen helfen also kaum beim Test auf View–Serialisierbarkeit. Dies ist auch schon deshalb plausibel, da der entsprechende Test $\mathcal{NP}$ –vollständig ist.

Der folgende vollständige Schedule s ist nicht view–serialisierbar, obwohl der entsprechende Konfliktgraph zu $RF^*(s) = \{ w_2 \xrightarrow{Y} r_0 \}$ azyklisch ist:

$$s = r_1(X) r_2(Y) w_1(Y) w_2(Y) c_1 c_2.$$

Vorgehensweise in der Praxis

Solange nur ein Präfix eines Schedules bekannt ist, weiß man nicht, welche der laufenden Transaktionen mit *Commit* bzw. *Abort* beendet werden.

Deswegen geben die *CSR*–basierten Zwei–Phasen–Scheduler Transaktionen erst frei, wenn alle ihre Datenzugriffe in Ordnung waren.

Synchronisationsprobleme

Analyse im Hinblick auf Konflikt-Serialisierbarkeit

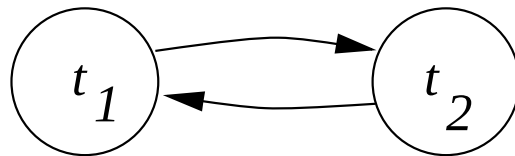
1. Das *Lost-Update-Problem*: Der zugehörige Schedule lautet

$$s = r_1(X) \textcolor{red}{r_2(X)} \textcolor{red}{w_1(X)} r_1(Y) \textcolor{red}{w_2(X)} w_2(Y) c_1 c_2.$$

Es gilt

$$C^*(s) = \{ r_1 \xrightarrow{X} w_2, \textcolor{red}{r_2} \xrightarrow{\textcolor{red}{X}} \textcolor{red}{w_1}, \textcolor{red}{w_1} \xrightarrow{\textcolor{red}{X}} \textcolor{red}{w_2}, r_1 \xrightarrow{Y} w_2 \},$$

und der Konfliktgraph $G(s)$ ist zyklisch:



Folglich ist s *nicht konflikt-serialisierbar*, d.h. $s \notin CSR$.

s ist auch *nicht view-serialisierbar*, d.h. $s \notin VSR$.

2. Das *Dirty-Read-Problem*: Der zugehörige Schedule lautet

$$s = r_1(X) w_1(X) r_2(X) w_2(X) r_1(Y) a_1 c_2.$$

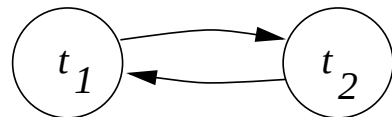
Hier gilt $s \in CSR$, aber die *abgebrochene* Transaktion wird hier nicht berücksichtigt, und die Betrachtung ist nicht adäquat.

Der Abbruch von t_1 führt wegen $w_1 \xrightarrow{X}_s r_2$ auch zum Abbruch von t_2 , d.h. c_2 kann nicht ausgeführt werden.

3. Das Problem der *fehlerhaften Zusammenfassung*: Der zugehörige Schedule lautet

$$s = r_2(A) \dots r_1(X) \textcolor{red}{w_1(X)} \textcolor{red}{r_2(X)} \textcolor{blue}{r_2(Y)} r_1(Y) \textcolor{blue}{w_1(Y)} c_1 c_2.$$

Hier gilt $C^*(s) = \{ \textcolor{red}{w_1} \xrightarrow{\textcolor{red}{X}} \textcolor{red}{r_2}, \textcolor{blue}{r_2} \xrightarrow{\textcolor{blue}{Y}} \textcolor{blue}{w_1} \}$, und der Konfliktgraph $G(s)$ ist zyklisch; s ist also *nicht konflikt-serialisierbar*, $s \notin CSR$:



s ist auch *nicht view-serialisierbar*, d.h. $s \notin VSR$.

Commit–Serialisierbarkeit

Transaktionen können abbrechen, und das System kann im laufenden Zustand unterbrochen werden. Deshalb sollte sich ein Korrektheitskriterium de facto nur auf *freigegebene* Transaktionen beziehen, und bei einem korrekten Schedule sollte auch jedes *Präfix* korrekt sein.

Definition (Abschlußeigenschaften von Schedule–Eigenschaften)

Sei $\mathcal{E}$ eine Schedule–Eigenschaft.

1. $\mathcal{E}$ ist *präfix–abgeschlossen* (PA), genau dann wenn gilt:
 $\mathcal{E}$ gilt für einen Schedule s , g.d.w. $\mathcal{E}$ für jedes Präfix von s gilt.
2. $\mathcal{E}$ ist *commit–abgeschlossen* (CA), genau dann wenn gilt:
 $\mathcal{E}$ gilt für einen Schedule s , g.d.w. $\mathcal{E}$ für den Schedule $CP(s)$ gilt.
3. $\mathcal{E}$ ist *präfix–commit–abgeschlossen* (PCA), genau dann wenn gilt:
 $\mathcal{E}$ gilt für einen Schedule s , g.d.w. $\mathcal{E}$ für den Schedule $CP(s')$ zu jedem beliebigen Präfix s' von s gilt.

Satz (Präfix–Commit–Abgeschlossenheit)

	<i>CA</i>	<i>PA</i>	<i>PCA</i>
<i>VSR</i>	+	–	–
<i>CSR</i>	+	–	+

1. Die *View–Serialisierbarkeit* ist *CA*, aber nicht *PA* und nicht *PCA*.
2. Die *Konflikt–Serialisierbarkeit* ist *CA* und *PCA*, aber nicht *PA*.

Beweis.

1. Für $s = w_1(X) w_2(X) w_2(Y) w_1(Y) c_1 c_2 w_3(X) w_3(Y) c_3$ gilt

$$s \approx_v t_1 c_1 t_2 c_2 t_3 c_3 \approx_v t_2 c_2 t_1 c_1 t_3 c_3,$$

da man $w_1(\chi)$ und $w_2(\chi)$ unter dem Schutz des abschließenden $w_3(\chi)$ vertauschen kann, für $\chi = X, Y$, d.h. $s \in VSR$.

Für das Präfix $s' = w_1(X) w_2(X) w_2(Y) w_1(Y) c_1 c_2$ von s gilt aber $CP(s') = s'$ und $s' \not\approx_v t_1 c_1 t_2 c_2$, $s' \not\approx_v t_2 c_2 t_1 c_1$, d.h. $s' \notin VSR$.

Also ist die V.–S. nicht *PCA*.

2. Für $s \in CSR$ ist $G(s)$ azyklisch.

Der Graph $G(CP(s'))$ zu einem Präfix s' von s ist ein Teilgraph von $G(s)$, und somit ebenfalls azyklisch, d.h. $CP(s') \in CSR$.

Also ist die K.-S. PCA .

3. Die V.-S. und die K.-S. sind beide CA ,

denn für einen vollständigen Schedule s gilt $s^* = CP(s)$,

und folglich $RF^*(s) = RF^*(CP(s))$ und $C^*(s) = C^*(CP(s))$.

4. Jeder serialisierbare Schedule $s \in VSR$ oder $s \in CSR$ muß vollständig sein, aber für einen vollständigen und seriellen – und damit auch serialisierbaren – Schedule s muß nicht jedes Präfix s' vollständig sein.

Z.B. ist für den serialisierbaren Schedule $s = w_1(X) c_1$ das Präfix $s' = w_1(X)$ nicht vollständig und damit auch nicht serialisierbar.

Die V.-S. und die K.-S. sind also beide natürlich nicht PA .



Fehlersicherheit von Schedules

Definition (*RC*, *ACA* und *ST*) Sei s ein Schedule.

RC. s heißt rücksetzbar (recoverable), falls für alle $t_i \neq t_j \in \text{trans}(s)$ gilt:

$$w_j \xrightarrow{X}_s r_i \wedge c_i \in s \Rightarrow c_j <_s c_i.$$

RC bezeichnet die Klasse der rücksetzbaren Schedules.

ACA. s vermeidet kaskadierende Aborts (avoids cascading aborts), falls für alle $t_i \neq t_j \in \text{trans}(s)$ gilt:

$$w_j \xrightarrow{X}_s r_i \Rightarrow c_j <_s r_i(X).$$

ACA bezeichnet die Klasse aller Schedules, welche kaskadierende Aborts vermeiden. Sie vermeiden das Problem des *dirty read*.

ST. s heißt strikt, falls für alle $t_i \neq t_j \in \text{trans}(s)$ und alle $p \in \{r, w\}$ gilt:

$$w_j(X) <_s p_i(X) \Rightarrow a_j <_s p_i(X) \vee c_j <_s p_i(X).$$

ST bezeichnet die Klasse der strikten Schedules.

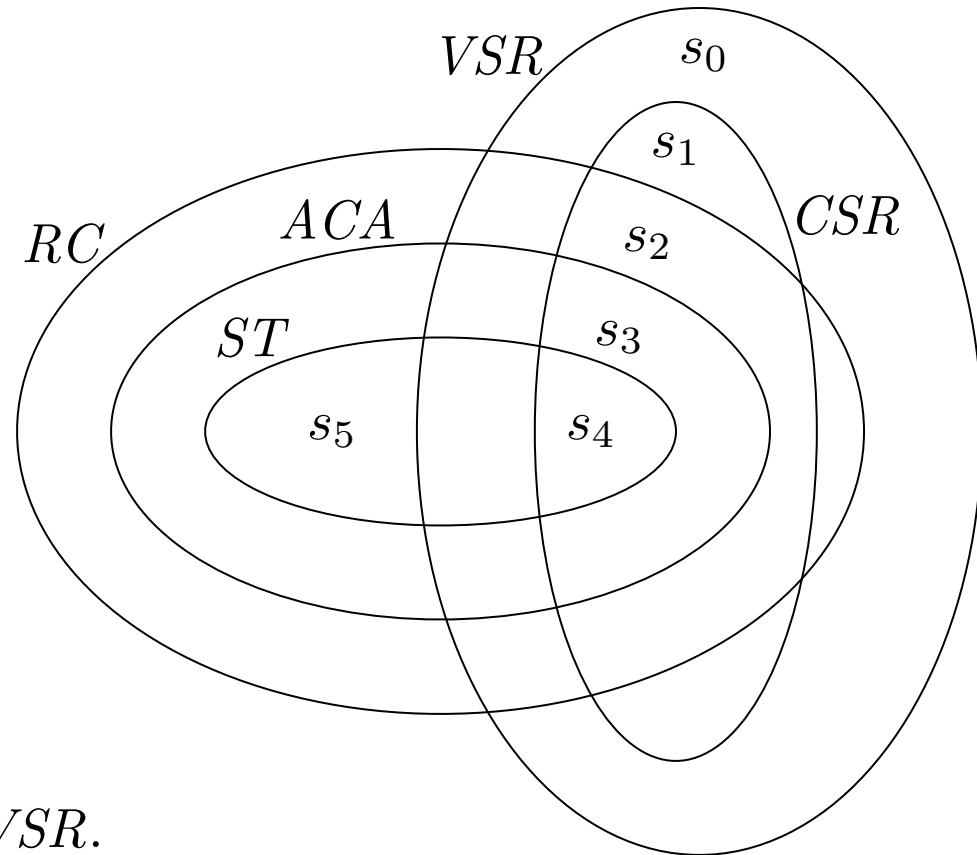
Satz (Klassen von Schedules)

$$ST \subsetneq ACA \subsetneq RC.$$

Später:

1. $\mathcal{E}(2PL) \subseteq CSR$,
2. $\mathcal{E}(S2PL) \subseteq CSR \cap ST$,
3. $\mathcal{E}(TO-Basis) \subseteq CSR$,
4. $\mathcal{E}(TO-Thomas) \subseteq VSR$,
5. $\mathcal{E}(TO-Mehrversionen) \subseteq VSR$.

RC , ACA und ST sind PCA .



Beweis.

$$ST \subseteq ACA.$$

Für $s \in ST$ und $t_i \neq t_j \in trans(s)$ gilt:

$$\begin{aligned} w_j \xrightarrow{X}_s r_i &\implies w_j(X) <_s r_i(X) \wedge a_j \not<_s r_i(X) \\ &\xRightarrow{ST} c_j <_s r_i(X). \end{aligned}$$

$$ACA \subseteq RC.$$

Für $s \in ACA$ und $t_i \neq t_j \in trans(s)$ gilt:

$$\begin{aligned} w_j \xrightarrow{X}_s r_i \wedge c_i \in s &\xRightarrow{ACA} c_j <_s r_i(X) \wedge r_i(X) <_s c_i \\ &\implies c_j <_s c_i. \end{aligned}$$

□

Striktheit nur bezüglich der Lese-Operationen $p_i(X) = r_i(X)$ impliziert offensichtlich auch schon ACA .

Beispiele (RC , ACA , ST)

RC . jede Transaktion wird erst nach der Freigabe aller Transaktionen, von denen sie gelesen hat, freigegeben:

$$s_1 = r_1(X) w_1(X) r_2(X) w_2(X) c_2 c_1 \notin RC$$

$$s_2 = r_1(X) w_1(X) r_2(X) w_2(X) c_1 c_2 \in RC$$

ACA . eine Transaktion darf nur Werte lesen, welche von bereits freigegebenen Transaktionen zuletzt geschrieben wurden:

$$s_2 \notin ACA$$

$$s_3 = r_1(X) w_1(X) w_2(X) c_1 c_2 \in ACA$$

$$s_4 = r_1(X) w_1(X) c_1 r_2(X) w_2(X) c_2 \in ACA$$

ST . eine Transaktion darf nur Werte lesen oder überschreiben, welche von bereits abgeschlossenen Transaktionen zuletzt geschrieben wurden:

$$s_3 \notin ST, s_4 \in ST$$

Beispiel (*VSR* und *CSR* vs. *ST*)

Für den folgenden Schedule gilt $s_5 \in ST \setminus VSR$:

$$s_5 = r_1(X) \textcolor{blue}{w_2(Y)} \textcolor{blue}{w_2(X)} c_2 r_1(Y) c_1.$$

1. $s_5 \notin CSR$, denn $C^*(s) = \{ r_1 \xrightarrow{X} w_2, w_2 \xrightarrow{Y} r_1 \}$.
2. $s_5 \notin VSR$, denn für $s_{1,2} = t_1 c_1 t_2 c_2$ und $s_{2,1} = t_2 c_2 t_1 c_1$ gilt

$$RF(s_{1,2}) = \{ w_2 \xrightarrow{Y} r_0, w_2 \xrightarrow{X} r_0 \},$$

$$RF(s_{2,1}) = \{ w_2 \xrightarrow{Y} r_1, w_2 \xrightarrow{X} r_1 \} \cup RF(s_{1,2}),$$

$$RF(s_5) = \{ w_2 \xrightarrow{Y} r_1 \} \cup RF(s_{1,2}).$$

3. Wegen $w_2(Y) <_{s_5} c_2 <_{s_5} r_1(Y)$ gilt $s_5 \in ST$.

Beispiel ($VSR \setminus CSR$ vs. RC)

Der vollständige Schedule

$$s = r_1(Y) r_3(V) r_2(Y) w_1(Y) w_1(X) w_2(X) w_2(Z) w_3(X) c_1 c_2 c_3,$$

für den bekanntermaßen $s \in VSR \setminus CSR$ gilt, kann mit dem vollständigen Schedule

$$s_{4,5} = \varrho_{\{1 \mapsto 4, 2 \mapsto 5\}}(s_1) = r_4(X) w_4(X) r_5(X) w_5(X) c_5 c_4 \notin RC$$

zu einem Schedule

$$s_0 = s_{4,5} \quad s \in (VSR \setminus CSR) \setminus RC$$

kombiniert werden.

Wir sehen später noch, daß für den Schedule s_1 von oben $s_1 \in \mathcal{E}(2PL)$ gilt:

$$s_1 = r_1(X) w_1(X) r_2(X) w_2(X) c_2 c_1 \notin RC.$$

4.2 Concurrency Control

Wir garantieren die Korrektheit von Schedules durch die Verwendung spezieller Scheduling–Techniken:

1. Beim *Zwei–Phasen–Sperrprotokoll* müssen Datenobjekte vor dem Zugriff durch eine Transaktion gesperrt und nach dem Zugriff wieder freigegeben werden. In der Sperrphase dürfen nur Sperren gesetzt werden. Mit dem ersten Entsperrschritt beginnt die zweite Phase, in der nur noch Sperren freigegeben werden dürfen.
2. Bei *Zeitstempelverfahren* werden Daten beim Zugriff mit Lese– und Schreib–Zeitstempeln versehen. Beim Zugriff auf ein Datenobjekt wird getestet, ob die Zeitstempelordnung verletzt wird.

Überlegungen zum Scheduler-Entwurf

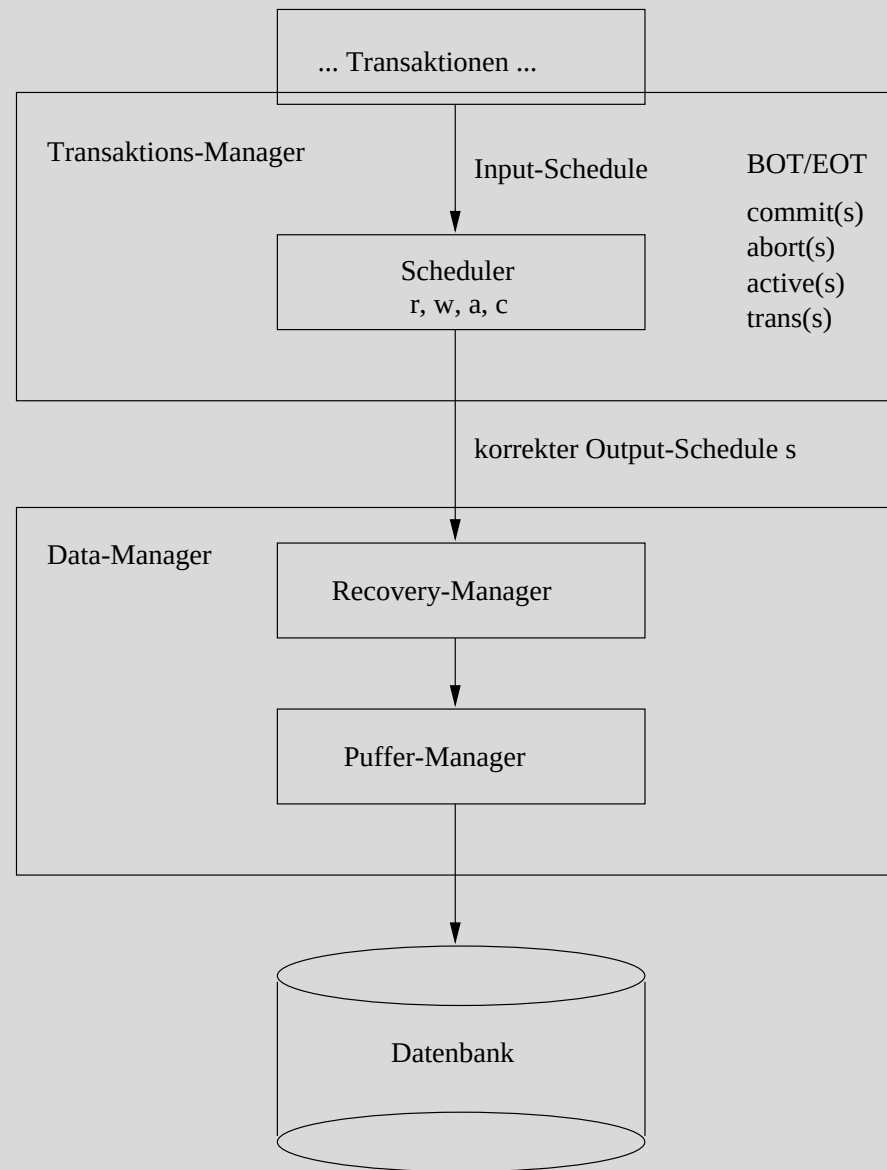
Der *Transaktionsmanager* (TM) nimmt die auszuführenden Transaktionen vom Codegenerator entgegen.

Der *Scheduler* erhält vom TM einen (“zufällig erzeugten”) Input-Schedule mit Schritten vom Typ r , w , a , c . Er hat die Aufgabe, hieraus einen korrekten Output-Schedule zu generieren. Dazu kann er für jeden einzelnen Schritt grundsätzlich eine der drei folgenden Aktionen ausführen:

1. *ausführen* (execute): möglich für r , w , a , c ; der Schritt wird an den Output-Schedule angehängt.
2. *zurückweisen* (reject): möglich für r , w ; die betreffende Transaktion wird abgebrochen, also wird ein a an den Output-Schedule angehängt.
3. *verzögern* (delay): möglich für r , w ; der Schritt wird als derzeit nicht ausführbar an den TM zurückgegeben.

Der *Daten-Manager* (DM) führt die Schritte des (korrekten) Output-Schedules der Reihe nach aus:

- r liest ein Datenobjekt aus der Datenbank in den Puffer.
- w schreibt ein Datenobjekt in die Datenbank oder in den Puffer.
- c macht das Ergebnis der Transaktion permanent.
- a macht die Transaktion durch geeignete Recovery-Maßnahmen ungeschehen.



Komponenten der Transaktionsverarbeitung in einem DBMS.

Definition (aggressiv, konservativ)

Ein Scheduler heißt

1. *aggressiv*, falls er überwiegend durchläßt, d.h. nur selten verzögert.
Hier besteht die Gefahr, daß er irgendwann festhängt (*Deadlock*).
2. *konservativ*, falls er überwiegend verzögert.
Im Extremfall wird der Output–Schedule dabei seriell,
wenn immer alle Transaktionen bis auf eine verzögert werden.

Definition (Sicherheit von Schedules)

Ein Scheduler S heißt

1. *s–sicher*, falls $\mathcal{E}(S) \subseteq CSR$,
2. *f–sicher*, falls $\mathcal{E}(S) \subseteq ST$,
3. *sicher*, falls $\mathcal{E}(S) \subseteq CSR \cap ST$,

wobei $\mathcal{E}(S)$ die Menge aller von S als Output erzeugbaren Schedules ist.

4.2.1 Das Zwei-Phasen-Sperrprotokoll

Zur Synchronisation des Zugriffs verschiedener Transaktionen auf gemeinsam benutzte Daten werden *Sperren* (Locks) verwendet, welche auf Datenobjekte gesetzt bzw. wieder aufgegeben werden können.

Definition (Sperren)

Sei X ein Datenobjekt und t_i eine Transaktion. Dann bezeichnet

$r_i^l(X)$ eine *Lese-Sperre* (read lock) und

$w_i^l(X)$ eine *Schreib-Sperre* (write lock)

von t_i auf X . Das *Rücksetzen* (Aufheben) von Sperren erfolgt mit

$r_i^u(X)$ für Lese-Sperren (read unlock) bzw.

$w_i^u(X)$ für Schreib-Sperren (write unlock).

Ein sperrender Scheduler macht aus jeder Transaktion formal eine um Sperr-Operationen erweiterte Transaktion (locked transaction).

Beispiel (Sperren, Locked Transaction)

Im folgenden Schedule s wurde jede Daten-Operation $p_i(X)$ um entsprechende Sperren zu $p_i^l(X) p_i(X) p_i^u(X)$ zu s' erweitert:

$$\begin{aligned} s &= r_1(X) w_2(X) w_2(Y) c_2 w_1(Y) c_1, \\ s' &= r_1^l(X) r_1(X) \textcolor{red}{r}_1^u(\textcolor{red}{X}) w_2^l(X) w_2(X) \textcolor{blue}{w}_2^u(\textcolor{blue}{X}) \\ &\quad \textcolor{blue}{w}_2^l(Y) w_2(Y) w_2^u(Y) c_2 \textcolor{red}{w}_1^l(\textcolor{red}{Y}) w_1(Y) w_1^u(Y) c_1. \end{aligned}$$

Der resultierende Schedule s' ist nach dem Zwei-Phasen-Sperrprotokoll aber nicht erlaubt, da $\textcolor{red}{r}_1^u(\textcolor{red}{X})$ die Lese-Sperre auf X freigibt bevor $\textcolor{red}{w}_1^l(\textcolor{red}{Y})$ später noch eine Schreib-Sperre auf Y anfordert – analoges gilt für $\textcolor{blue}{w}_2^u(\textcolor{blue}{X})$ bzw. $\textcolor{blue}{w}_2^l(Y)$ (wobei man diese beiden Operationen auch vertauschen könnte)

Definition (Sperrprotokoll)

Ein Scheduler arbeitet nach einem *Sperrprotokoll*, falls für jeden Output s und jede Transaktion $t_i \in \text{trans}(s)$ gilt:

1. $p_i(X) \in s$ für $p \in \{r, w\} \implies p_i^l(X) <_s p_i(X) <_s p_i^u(X)$.
2. Auf jedem Datenobjekt werden Sperren derselben Art von jeder Transaktion höchstens einmal gesetzt,
d.h. s enthält höchstens einen Schritt $r_i^l(X)$ bzw. $w_i^l(X)$.
3. Kein lock/unlock–Schritt ist redundant.
4. Ist ein Datenobjekt X durch zwei verschiedene Transaktionen t_i und t_j an einer Stelle von s gesperrt, so sind diese Sperren nicht in Konflikt.

	$r_i^l(X)$	$w_i^l(X)$
$r_j^l(X)$	+	–
$w_j^l(X)$	–	–

Kompatibilitäts–Matrix:

+ bedeutet kein Konflikt

– bedeutet Konflikt

Beispiele (Sperrprotokoll)

1. Der folgende Schedule s ist nach dem Sperrprotokoll erlaubt:

$$s = r_1^l(X) r_1(X) r_2^l(X) r_2(X) r_2^u(X) w_1^l(Y) r_1^u(X) w_1(Y) w_1^u(Y).$$

Die Lesesperren $r_1^l(X)$ und $r_2^l(X)$ sind kompatibel.

Dadurch kann t_2 verschränkt mit t_1 nach $r_1(X)$ schon $r_2(X)$ ausführen.

Z.B. kann t_2 die Wartezeit ausnutzen, die entsteht falls t_1 noch auf etwas warten muß, bevor es $w_1(Y)$ ausführen kann.

2. Der folgende Schedule s' ist nach dem Sperrprotokoll nicht erlaubt:

$$s' = w_1^l(X) w_1(X) r_2^l(X) r_2(X) r_2^u(X) w_1^l(Y) w_1^u(X) w_1(Y) w_1^u(Y).$$

Die Schreibsperre $w_1^l(X)$ (anstelle von $r_1^l(X)$ verglichen mit oben) und die Lesesperre $r_2^l(X)$ sind nicht kompatibel.

3. Man könnte versuchen, s' zu korrigieren, indem man $w_1^u(X)$ vor $r_2^l(X)$ einschiebt, da dann die Schreibsperr $w_1^l(X)$ nicht mehr aktiv ist, wenn die Lesesperre $r_2^l(X)$ erfolgt:

$$s' = w_1^l(X) w_1(X) w_1^u(X) r_2^l(X) r_2(X) r_2^u(X) w_1^l(Y) w_1(Y) w_1^u(Y).$$

Die entstandene direkte Einklammerung von Aktionen mit den entsprechenden Sperr- und Entsperrschritten ist aber beim $2PL$ -Scheduling nicht erlaubt, da der Sperrschritt $w_1^l(Y)$ erfolgt nachdem mit $w_1^u(X)$ bereits die Entsperrphase von t_1 begonnen hat.

Offensichtlich wäre die angedeutete naive, direkte Einklammerung $p_i^l(X) p_i(X) p_i^u(X)$ von Aktionen $p_i(X)$ mit den entsprechenden Sperr- und Entsperrschritten für jeden Schedule möglich.

Um die bekannten Probleme bei der Verwaltung paralleler Transaktionen zu lösen muß man weitere Bedingungen – siehe $2PL$ -Scheduling – stellen.

In kommerziellen Systemen wird am häufigsten das Zwei-Phasen-Sperrprotokoll (2PL) verwendet.

Definition (*2PL*, Two-Phase Locking)

2PL. Ein Sperrprotokoll heißt *zweiphasig*, falls für jeden erzeugten Schedule s und jede Transaktion $t_i \in \text{trans}(s)$ gilt: nach dem ersten Entsperrschritt $p_i^u(X)$ erfolgt kein weiterer Sperrschritt $q_i^l(Y)$, $p, q \in \{r, w\}$. Ein entsprechender Scheduler heißt *2PL*-Scheduler.

C2PL. Ein *2PL*-Scheduler heißt *C2PL*-Scheduler (konservativer oder statischer *2PL*-Scheduler), falls jede Transaktion vor ihrer ersten Datenoperation $r_i(X)$ oder $w_i(X)$ alle benötigten Sperren setzt.

S2PL. Ein *2PL*-Scheduler heißt *S2PL*-Scheduler (strenger oder dynamischer *2PL*-Scheduler), falls jede Transaktion alle von ihr gehaltenen Sperren erst nach ihrer letzten Datenoperation freigibt.

2 Phasen: Beim *2PL* kann man jede Transaktion unterteilen in

1. eine *Wachstumsphase*, in der Sperren gesetzt, und
2. eine *Schrumpfphase*, in der Sperren freigegeben werden können.

In beiden Phasen können auch Datenoperationen ausgeführt werden.

Bei *C2PL* bzw. *S2PL* werden in der Wachstums– bzw. der Schrumpf–Phase keine Datenoperationen ausgeführt.

Commit: Vor dem erfolgreichen Abschluß einer Transaktion t muß sichergestellt werden, daß alle Transaktionen t' , von denen t gelesen hat, bereits erfolgreich abgeschlossen wurden.

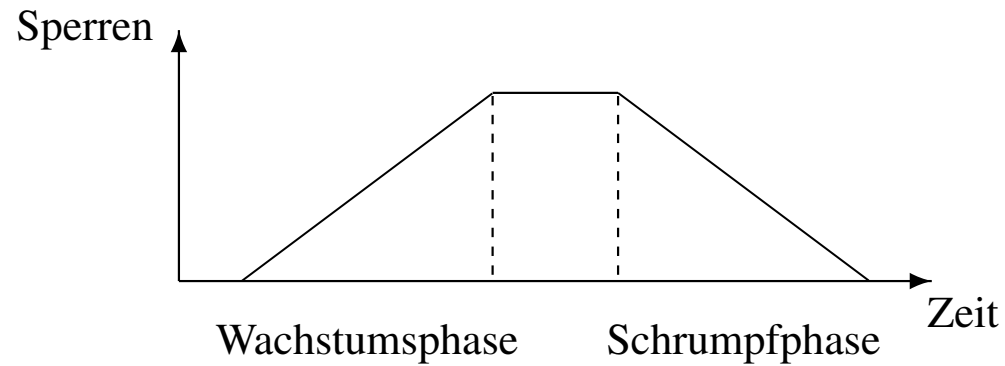
Eigenschaften

2PL erzeugt konflikt–serialisierbare – d.h. s–sichere – Schedules,

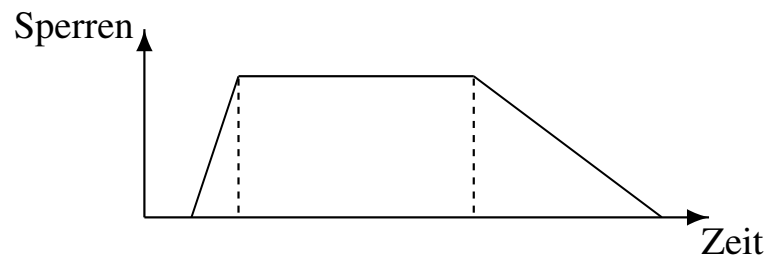
C2PL erzeugt deadlock–freie Schedules,

S2PL erzeugt strikte – und damit auch sichere – Schedules.

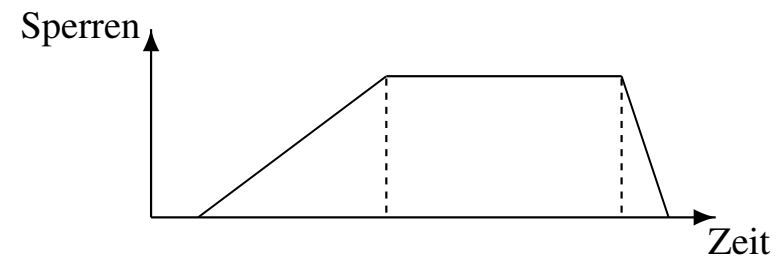
Wachstumsphase und Schrumpfphase beim $2PL$ -Scheduling



$2PL$:



$C2PL$: Sperren ab BOT (Begin of Trans.)



$S2PL$: Sperren bis EOT (End of Trans.)

Beispiele (2PL)

1. Der Schedule s' von oben ist *nicht* zwei-phasig, denn $r_1^u(X) <_{s'} w_1^l(Y)$ und $w_2^u(X) <_{s'} w_2^l(Y)$:

$$s' = r_1^l(X) r_1(X) r_1^u(X) w_2^l(X) w_2(X) w_2^u(X) \\ w_2^l(Y) w_2(Y) w_2^u(Y) c_2 w_1^l(Y) w_1(Y) w_1^u(Y) c_1.$$

In einem zwei-phasigen Schedule kann t_1 den unlock $r_1^u(X)$ erst nach $w_1^l(Y)$ ausführen, und erst dann kann t_2 mit $w_2^l(X)$ starten. Mit $w_2^l(Y)$ muß t_2 bis nach $w_1^u(Y)$ warten.

2. Falls $w_2(Y)$ und $w_1(Y)$ vertauscht wären, so wäre der folgende nicht-serielle, zwei-phasige Schedule möglich:

$$s'' = r_1^l(X) r_1(X) w_1^l(Y) r_1^u(X) w_2^l(X) w_2(X) \\ w_1(Y) w_1^u(Y) c_1 w_2^l(Y) w_2^u(X) w_2(Y) w_2^u(Y) c_2.$$

3. Wir betrachten die beiden Transaktionen $t_1 = r_1(X) w_1(Y)$ und $t_2 = w_2(Y) w_2(X)$. Dann kann der Schedule

$$s'' = r_1^l(X) r_1(X) w_2^l(Y) w_2(Y)$$

von einem $2PL$ -Scheduler nicht fortgesetzt werden.

Hier wartet t_1 auf t_2 wegen Y und t_2 auf t_1 wegen X (*Deadlock*).

Ein $C2PL$ -Scheduler könnte s'' nicht aus t_1 und t_2 erzeugen.

Ein $S2PL$ -Scheduler könnte s'' auch erzeugen.

4. Der Schedule s''' ist zwar zwei-phasig:

$$s''' = r_1^l(X) r_1(X) w_1^l(Y) r_1^u(X) w_1(Y) w_1^u(Y),$$

aber wegen $r_1^u(X) <_{s'''} w_1(Y)$ nicht streng zwei-phasig.

Der Schedule s'''' ist streng zwei-phasig:

$$s'''' = r_1^l(X) r_1(X) w_1^l(Y) w_1(Y) r_1^u(X) w_1^u(Y).$$

Satz (Zwei-Phasen-Scheduler)

1. Ein $2PL$ -Scheduler ist s-sicher: $\mathcal{E}(2PL) \subsetneq CSR$.
2. Ein $S2PL$ -Scheduler ist sicher: $\mathcal{E}(S2PL) \subseteq CSR \cap ST$.

Beispiel (Echte Inklusion)

$$s = w_1(X) r_2(X) c_2 r_3(Y) c_3 w_1(Y) c_1.$$

Dann gilt $s \in CSR$ wegen $s \approx_c t_3 c_3 t_1 c_1 t_2 c_2$.

s ist aber nicht von einem $2PL$ -Scheduler erzeugbar:

$$s \notin \mathcal{E}(2PL).$$

Die beiden Schedules

$$s' = w_1^l(X) w_1(X) w_1^u(X) r_2^l(X) r_2(X) r_2^u(X) c_2 \\ r_3^l(Y) r_3(Y) r_3^u(Y) c_3,$$

$$s'' = w_1^l(X) w_1(X) w_1^l(Y) w_1^u(X) r_2^l(X) r_2(X) r_2^u(X) c_2$$

können von einem *2PL*-Scheduler nicht komplettiert werden.

- Zwischen $w_1(X)$ und $r_2(X)$ müssen $w_1^u(X)$ und $r_2^l(X)$ stehen.
- Wenn man – wie in s' – $w_1^l(Y)$ nicht vor $w_1^u(X)$ ausführt, dann kann man den Lock später nicht mehr bekommen. Deswegen kann $w_1(Y)$ nun nicht ausgeführt werden.

- Wenn man – wie in s'' – $w_1^l(Y)$ vor $w_1^u(X)$ ausführt, dann kann der Lock $r_3^l(Y)$ nicht erfolgen. Da andererseits $w_1^u(Y)$ natürlich nicht vor $w_1(Y)$ erfolgen kann, kann $r_3^l(Y)$ und damit auch $r_3(Y)$ nun nicht ausgeführt werden.

4.2.2 Deadlocks

Definition (Wartegraph, Deadlock)

1. Der *Wartegraph* $WG(s) = (V, E)$ eines Schedules ist gegeben durch

$$V = \text{active}(s),$$

$$E = \{ (t_i, t_j) \in V \mid t_i \text{ wartet darauf, ein Datenobjekt zu sperren,} \\ \text{welches von } t_j \text{ gesperrt ist} \}.$$

2. Am Ende von s liegt ein *Deadlock* vor, falls $WG(s)$ zyklisch ist.

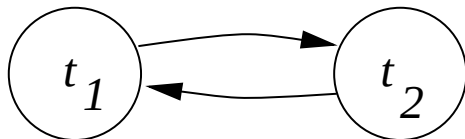
Dann wartet eine Menge $\mathcal{T}$ von Transaktionen wechselseitig auf die Freigabe der von anderen Transaktionen aus $\mathcal{T}$ gehaltenen Sperren.

Beispiel (Wartegraph, Deadlock)

Wir betrachten den Schedule s :

$$s = r_1^l(X) r_1(X) w_2^l(Y) w_2(Y) \underbrace{w_1^l(Y) w_2^l(X)}_{\text{gewünscht}}$$

Dieser hat zwei aktive Transaktionen und den folgenden zyklischen Wartegraphen:



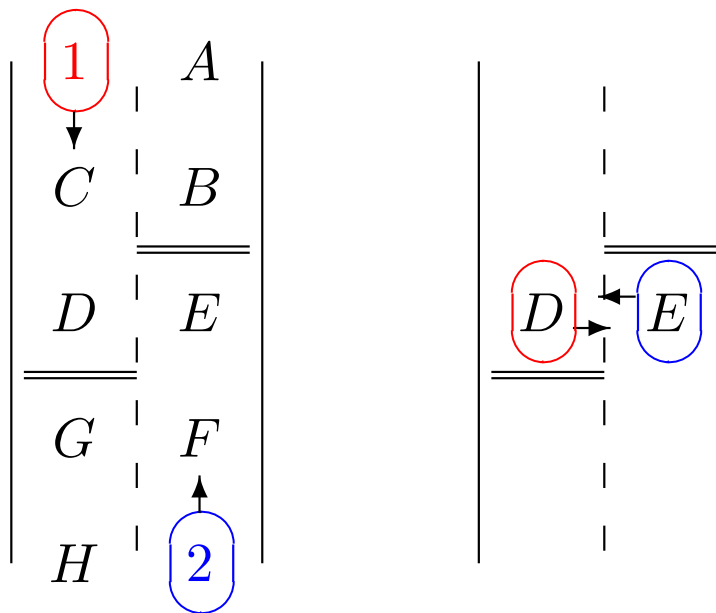
t_1 wartet auf eine Schreibsperre für das Datenobjekt Y , das von t_2 bereits schreibgesperrt wurde, und umgekehrt wartet t_2 auf eine Schreibsperre für das Datenobjekt X , das von t_1 bereits lesegesperrt wurde.

Die beiden Transaktionen wollen ihre gehaltenen Sperren nicht freigeben.

Beispiel (Deadlock im Straßenverkehr)

Das rote Auto 1 möchte über die Route (C, D, E, F, G, H) nach H fahren, das blaue Auto 2 über (F, E, D, C, B, A) nach A .

Sie treffen sich an zwei Hindernissen und blockieren sich gegenseitig: das rote Auto 1 möchte von D nach E fahren, das blaue Auto 2 von E nach D .



Wenn die Autos sich wie beim $2PL$ -Scheduling verhalten, dann möchte keiner zurückfahren:

1. das rote Auto 1 möchte E sperren und kann seine Sperre auf D nicht hergeben, da es nachher noch weitere Sperren auf $F, G, \dots$ benötigt, und
2. das blaue Auto 2 möchte D sperren und kann seine Sperre auf E nicht hergeben, da es nachher noch weitere Sperren auf $C, B, \dots$ benötigt.

Es wäre ein Deadlockzustand erreicht.

Beim konservativen $C2PL$ -Scheduling würde ein Auto dagegen erst losfahren, wenn seine gesamte Fahrtroute für andere Autofahrer gesperrt ist. Damit könnte ein solcher Deadlock nicht auftreten. Der Durchsatz wäre bei einem solchen – im Straßenverkehr unrealistischen – Vorgehen allerdings sehr viel geringer, da auf vielen Routen nur ein Auto unterwegs wäre.

Die Fahrtrouten entsprechen Transaktionen:

$$t_1 = w_1(C) w_1(D) w_1(E) w_1(F) w_1(G) w_1(H),$$

$$t_2 = w_2(F) w_2(E) w_2(D) w_2(C) w_2(B) w_2(A).$$

Zum Zeitpunkt des Aufeinandertreffens könnte ein *2PL*-Scheduler den folgenden, verschränkten Schedule abgearbeitet haben:

$$s = w_1^l(C) w_1(C) w_2^l(F) w_2(F) w_1^l(D) w_1(D) w_2^l(E) w_2(E).$$

Jetzt liegt ein Deadlock vor, da t_1 die Sperre $w_1^l(E)$ setzen möchte, obwohl t_2 bereits E sperrt, und da t_2 die Sperre $w_2^l(D)$ setzen möchte, obwohl t_1 bereits D sperrt.

Ein konservativer *C2PL*-Scheduler würde die Transaktionen dagegen seriell (erst t_1 , dann t_2 , oder umgekehrt) abarbeiten und keinen Deadlock erzeugen.

Deadlock–Verhinderung mit Zeitstempeln

Eine Transaktion t_i erhält einen Transaktionszeitstempel $TS(t_i)$, welcher der Startzeit von t_i entspricht:

falls t_i vor t_j startet, so gilt $TS(t_i) < TS(t_j)$.

Annahme: t_i versucht ein Datenobjekt zu sperren, welches schon von t_j gesperrt ist. Dann gibt es 2 alternative Methoden:

- *wait–die*: Falls $TS(t_i) < TS(t_j)$, d.h. falls t_i älter als t_j ist, dann soll t_i warten dürfen; andernfalls wird t_i abgebrochen.
- *wound–wait*: Falls $TS(t_i) < TS(t_j)$, dann wird t_j abgebrochen (verwundet); andernfalls darf t_i warten.

Beide Methoden begünstigen Transaktion, die schon lange laufen, und von denen man somit annimmt, daß sie bereits viel Arbeit erledigt haben.

Beispiel (Deadlock–Verhinderung im Straßenverkehr)

In unserem Straßenverkehrs–Beispiel würde für den verschränkten Schedule

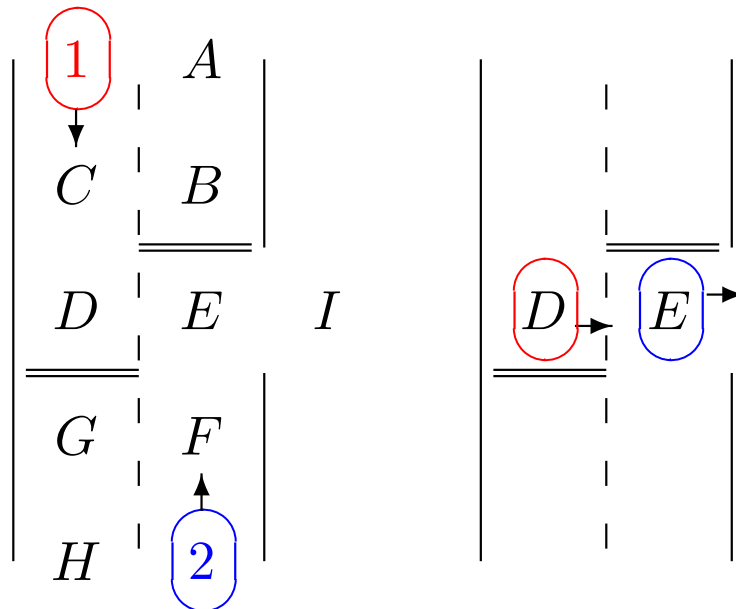
$$s = w_1^l(C) w_1(C) w_2^l(F) w_2(F) w_1^l(D) w_1(D) w_2^l(E) w_2(E)$$

dann mit der Folge $w_1^l(E) w_2^l(D)$ von Sperranfragen so verfahren:

- t_1 versucht das Datenobjekt E zu sperren, welches schon von t_2 gesperrt ist. Die wait–die–Methode würde die Transaktion t_1 zum roten Auto, das früher losgefahren ist, warten lassen.
- Danach versucht t_2 das Datenobjekt D zu sperren, welches schon von t_1 gesperrt ist. wait–die würde die blaue Transaktion t_2 nun abbrechen.
- Die wound–wait–Methode würde die Transaktion t_2 sofort abbrechen.

Somit hätten in unserem konkreten Beispiel beide Methoden denselben Effekt.

Falls das blaue Auto 2 stattdessen in E nach I abbiegen möchte, so steht eigentlich kein Deadlock bevor, und es wäre nicht nötig t_2 abzurechnen:



Das rote Auto 1 möchte über die Route (C, D, E, F, G, H) nach H fahren, das blaue Auto 2 über (F, E, I) nach I .

Sie treffen sich an zwei Hindernissen, und das blaue Auto blockiert das rote eigentlich nur vorübergehend, da es nach I abbiegen möchte.

Der verschränkte Schedule

$$s = w_1^l(C) w_1(C) w_2^l(F) w_2(F) w_1^l(D) w_1(D) w_2^l(E) w_2(E)$$

kann dann mit der Folge $w_1^l(E) w_2^l(I)$ von Sperranfragen so verfahren:

- Jetzt versucht t_1 das Datenobjekt E zu sperren, welches schon von t_2 gesperrt ist, aber nicht umgekehrt.
- Die wait–die–Methode würde die Transaktion t_1 zum roten Auto, das früher losgefahren ist, warten lassen. Da das blaue Auto dann nach I abbiegt, löst sich das Problem auf, sobald t_2 seine Sperren freigibt.
- Die wound–wait–Methode würde die Transaktion t_2 dagegen sofort und unnötigerweise abbrechen.

Somit hätten in unserem modifizierten Beispiel beide Methoden unterschiedliche Effekte.

4.2.3 Zeitstempelverfahren

Ein Zeitstempel $TS(t)$ ist ein eindeutiger Identifikator, welcher vom DBMS kreiert wird für eine Transaktion t .

Die Ordnung der Zeitstempel leitet sich von den *Startzeiten* der Transaktionen ab.

Für jedes Datenobjekt X werden zwei Zeitstempel gehalten:

$read_TS(X)$:

Der Lese–Zeitstempel ist der größte Zeitstempel $TS(t)$ unter allen Transaktionen t , welche X erfolgreich gelesen haben.

$write_TS(X)$:

Der Schreib–Zeitstempel ist der größte Zeitstempel $TS(t)$ unter allen Transaktionen t , welche X erfolgreich geschrieben haben.

Beim *Datenzugriff* auf ein Datenobjekt X wird getestet, ob die Zeitstempelordnung verletzt wird:

1. Falls Transaktion t_i eine *Schreiboperation* $w_i(X)$ ausführt:
 - a) Falls $read_TS(X) > TS(t_i)$ oder $write_TS(X) > TS(t_i)$, so breche t_i ab, denn eine spätere Transaktion hat bereits auf X zugegriffen.
 - b) Ansonsten führe die Schreiboperation aus und setze $write_TS(X) := TS(t_i)$.
2. Falls Transaktion t_i eine *Leseoperation* $r_i(X)$ ausführt:
 - a) Falls $write_TS(X) > TS(t_i)$, so breche t_i ab, denn eine spätere Transaktion hat bereits den gewünschten Wert von X überschrieben.
 - b) Ansonsten führe die Leseoperation aus und setze $read_TS(X) := \max\{ TS(t_i), read_TS(X) \}$.

Diese Vorgehensweise wird als *Basis-TO-Algorithmus* bezeichnet (TO für timestamp ordering).

Wenn zwei Operationen in Konflikt stehen und in der falschen Reihenfolge ausgeführt werden sollen, dann wird die spätere der beiden Operationen zurückgewiesen und die zugehörige Transaktion abgebrochen.

Eigenschaften

1. Der erzeugte Schedule ist *konflikt-serialisierbar*, und der zugehörige serielle Schedule ergibt sich aus der Ordnung der Zeitstempel.
2. Das Verfahren garantiert *nicht* die Vermeidung kaskadierender Aborts (ACA).

Beispiel (Lost Update)

$s_2 :$		
T_1	T_2	
$r_1(X);$ $X := X - N;$		$read_TS(X) = write_TS(X) = 0$ $TS(T_1) = 1, read_TS(X) = 1$
	$r_2(X);$ $X := X + M;$	$TS(T_2) = 3, read_TS(X) = 3$
$w_1(X);$ $r_1(Y);$		$read_TS(X) = 3 > 1 = TS(T_1)$
$Y := Y + N;$ $w_1(Y);$	$w_2(X);$	

Bei der Operation $w_1(X)$ wird die Transaktion T_1 abgebrochen, denn die spätere Transaktion T_2 hat bereits auf X zugegriffen.

Beispiel (Incorrect Summary)

T_1	T_3	
		$read_TS(\dots) = write_TS(\dots) = 0$
	$sum := 0;$	$TS(T_3) = 1$
	$r_3(A);$	$read_TS(A) = 1$
	$sum := sum + A;$	
$r_1(X);$		$TS(T_1) = 4, read_TS(X) = 4$
$X := X - N;$		
$w_1(X);$		$write_TS(X) = 4$
	$r_3(X);$	$write_TS(X) = 4 > 1 = TS(T_3)$
	$sum := sum + X;$	
$\vdots$	$\vdots$	

Bei der Operation $r_3(X)$ wird die Transaktion T_3 abgebrochen, denn die spätere Transaktion T_1 hat bereits den gewünschten Wert von X überschrieben.

TO–Thomas–Algorithmus

Man kann die *Schreibregel von Thomas* verwenden, um mehr Parallelität zu erreichen:

- 1'. Falls Transaktion t_i eine Schreiboperation $w_i(X)$ ausführt:
 - a) Falls $read_TS(X) > TS(t_i)$, so breche die Transaktion ab.
Eine spätere Transaktion hat X bereits gelesen.
 - b) Falls $write_TS(X) > TS(t_i)$, so ignoriere die Schreiboperation.
Eine spätere Transaktion hat X bereits überschrieben, und die Schreiboperation ist somit irrelevant.
 - c) Ansonsten führe die Schreiboperation aus und setze $write_TS(X) := TS(t_i)$.

Hier wird lediglich eine spezielle Art von *View–Serialisierbarkeit* garantiert.

Die $\mathcal{NP}$ –Vollständigkeit des *VSR*–Tests ist hier kein Problem, da *TO–Thomas* einen *VSR*–Schedule erzeugt, ohne ihn testen zu müssen.

Beispiel (View Serialisierbarkeit)

T_1	T_2	
		$read_TS(X) = write_TS(X) = 0$
$r_1(X);$		$TS(T_1) = 1, read_TS(X) = 1$
	$w_2(X);$	$TS(T_2) = 2, write_TS(X) = 2$
$w_1(X);$		$write_TS(X) = 2 > 1 = TS(T_1)$

1. Die Operation $w_1(X)$ wird ignoriert, da die spätere Transaktion T_2 den Wert von X bereits überschrieben hat – die Schreiboperation ist somit irrelevant.
2. Das System führt also den Schedule $s = r_1(X) w_2(X) w'_1(X)$ aus, bei dem $w'_1(X)$ ohne Effekt bleibt.
3. s ist view-äquivalent zum Schedule $s' = r_1(X) w_1(X) w_2(X)$.

Eigenschaften

Zeitstempelverfahren sind *deadlock-frei*.

Allerdings kann es dazu kommen, daß Transaktionen immer wieder abgebrochen und neu gestartet werden ($\rightarrow$ Verhungern, starvation).

Strikte CSR-Schedules

Außerdem kann der Basis-TO-Algorithmus auch so modifiziert werden, daß er *strikte CSR-Schedules* erzeugt:

Führt eine Transaktion t_i eine Schreiboperation $w_i(X)$ oder eine Leseoperation $r_i(X)$ aus mit $TS(t_i) > write_TS(X)$, so wird die Operation bis nach dem Commit bzw. Abort derjenigen Transaktion, welche X zuletzt geschrieben hat, verzögert.

4.2.4 Mehrversionen–Scheduling–Verfahren

Hier werden mehrere Versionen der Datenobjekte gehalten
(→ erhöhter Speicherbedarf).

- Dadurch können Leseoperationen, welche sich auf frühere Versionen beziehen, immer noch ausgeführt werden.
- Man arbeitet hier mit *View–Serialisierbarkeit* und erreicht dadurch mehr Parallelität.
- Die Versionen eines Datenobjekts stellen seine Entwicklungsgeschichte dar.
- Im Extremfall hat man es mit *temporalen* Datenbanken (vgl. Börsencharts) zu tun.

Mehrversionen–Zeitstempelverfahren (kein *ACA*)

Zu jedem Datenobjekt X werden mehrere

Versionen $X_1, \dots, X_k$

gehalten zusammen mit folgenden *Zeitstempeln*:

read _{TS} (X_i):

Der *Lese*–Zeitstempel ist der größte Zeitstempel $TS(t)$ unter allen Transaktionen t , welche X_i erfolgreich gelesen haben.

write _{TS} (X_i):

Der *Schreib*–Zeitstempel ist der größte Zeitstempel $TS(t)$ unter allen Transaktion t , welche X_i erfolgreich geschrieben haben.

Beim Datenzugriff einer Transaktion t_i auf X wird diejenige Version X_m , mit $1 \leq m \leq k$, gewählt, die den höchsten Schreib–Zeitstempel unter allen Versionen mit $write_TS(X_m) \leq TS(t_i)$ hat:

1. Falls t_i eine Schreiboperation $w_i(X)$ ausführt:
 - a) Falls $read_TS(X_m) > TS(t_i)$, so brechen wir t_i ab, da die Transaktion t_j mit dem Zeitstempel $TS(t_j) = read_TS(X_m)$ ansonsten die falsche Version X_m gelesen hätte.
 - b) Andernfalls kreieren wir eine neue Version X_{k+1} von X mit
$$read_TS(X_{k+1}) = write_TS(X_{k+1}) = TS(t_i).$$
2. Falls t_i eine Leseoperation $r_i(X)$ ausführt, so lesen wir den Wert von X_m und setzen
$$read_TS(X_m) := \max\{read_TS(X_m), TS(t_i)\}.$$

Man kann also in jedem Fall vergangene Versionen eines Datenobjekts X lesen. Eine bereits gelesene Version kann man aber nicht mehr überschreiben.

Mehrversionen–Zwei–Phasen–Sperrprotokoll

Zu jedem Datenobjekt X werden *zwei Versionen* X_1 und X_2 gehalten:

1. X_1 enthält immer einen Wert, der von einer freigegebenen Transaktion geschrieben wurde.
2. X_2 wird *kreiert*, wenn eine Transaktion t_i eine Schreib–Sperrung auf X beantragt.

Dann wird X_2 zur lokalen Kopie für t_i , und andere Transaktionen können fortfahren, die freigegebene Version X_1 von X zu lesen.

Das Mehrversionen–Zwei–Phasen–Sperrprotokoll garantiert *ACA*.

Man arbeitet mit folgender *Kompatibilitäts-Matrix*:

	$r_j^l(X)$	$w_j^l(X)$	$c_j^l(X)$
$r_i^l(X)$	+	+	—
$w_i^l(X)$	+	—	—
$c_i^l(X)$	—	—	—

1. Will eine Transaktion t_i freigegeben werden, so muß sie eine *Zertifizierungs-Sperre* (certify lock) $c_i^l(X)$ für alle Datenobjekte X beantragen, für welche sie eine Schreib-Sperre $w_i^l(X)$ hält.
2. Erst wenn t_i alle erforderlichen Zertifizierungs-Sperren erhalten hat, kann t_i freigegeben werden.

Dies vermeidet kaskadierende Aborts, Deadlocks können auftreten.

4.3 Recovery

Das Ziel ist die fehlertolerante Verarbeitung von Transaktionen. Man kann sich 3 Fehlerklassen vorstellen:

1. *Transaktionsfehler*: Eine Transaktion erreicht ihren Commit–Punkt nicht. Gründe dafür können ein Fehler im Programm, aus welchem die Transaktion resultiert, oder die Verwicklung der Transaktion in einen Deadlock mit Auswahl derselben als Opfer sein.
Dann muß die Transaktion rückgängig gemacht werden (roll back).
2. *Systemfehler*: Fehler im DBMS–Code, im Betriebssystem oder in der Rechner–Hardware mit dem Effekt, daß flüchtiger Speicher (Hauptspeicher, Datenbankpuffer) verloren geht.
3. *Mediafehler*: z.B. Headcrashes auf der Platte oder Hardwarefehler im Betriebssystem mit dem Effekt, daß stabiler Speicher (Sekundärspeicher) verloren geht.

Update-Konzepte

1. Beim *verzögerten Update* werden die Änderungen nur im Log-Buch und im lokalen Arbeitsbereich einer Transaktion ausgeführt.

Zum Commit-Zeitpunkt einer Transaktion wird das Log-Buch auf Platte geschrieben (force write), und beim nächsten Checkpoint werden auch die Änderungen aus dem Datenbankpuffer auf Platte (in die Datenbank) geschrieben.

Bei einem Transaktionsfehler ist also kein UNDO nötig.

2. Beim *direkten Update* werden die Änderungen im Log-Buch und in der Datenbank ausgeführt. Hier benötigt man REDO und UNDO.

3. Beim *Shadow-Paging* werden für geänderte Datenseiten zwei Versionen verwaltet: die aktuellen Seiten und die alten Seiten (Schattenseiten). Die Schattenseiten enthalten einen alten, konsistenten Datenbankzustand. Hier benötigt man kein UNDO.

Recovery

Die Wiederherstellung eines konsistenten Datenbankzustandes nach einem Fehler erfolgt mit Hilfe des *Log-Buches*.

1. Nach einem *Systemzusammenbruch* wird eine alte Version der Datenbank eingespielt, und die Aktionen der freigegebenen Transaktionen werden aus dem Log-Buch rekonstruiert (REDO).
2. Bei einem *Transaktionsfehler* werden die vorgenommenen Aktionen der fehlerhaften Transaktionen zurückgenommen (UNDO).

Nur Schreiboperationen müssen zurückgenommen werden:

AFIM (after image) $\rightarrow$ BFIM (before image).

Die Leseoperationen stehen nur im Log-Buch, um Konflikte zu entdecken.

Anhang

SQL: Syntaxzusammenfassung

```
CREATE TABLE <table name> (  
    <column name> <column type> [<attribute constraint>]  
    {, <column name> <column type> [<attribute constraint>]}  
    [<table constraint> {, <table constraint>}] )
```

```
DROP TABLE <table name>
```

```
ALTER TABLE <table name>
```

```
ADD <column name> <column type>
```

```
SELECT [ DISTINCT ] <attribute list>
FROM ( <table name> { <alias> } | <joined table> )
      { , ( <table name> { <alias> } | <joined table> ) }
[ WHERE <condition> ]
[ GROUP BY <grouping attributes>
  [ HAVING <group selection condition> ] ]
[ ORDER BY <column name> [ <order> ]
  { , <column name> [ <order> ] } ]
```

```
<attribute list> ::=
  ( * | ( <column name> |
    <function>( ( [ DISTINCT ] <column name> | * ) ) )
  { , ( <column name> |
    <function>( ( [ DISTINCT ] <column name> | * ) ) ) } )
```

```
<grouping attributes> ::=
  <column name> { , <column name> }
```

<order>::=(ASC | DESC)

INSERT INTO <table name>

[(<column name> { , <column name> })]

VALUES

(<constant value> { , <constant value> })

{ , (<constant value> { , <constant value> }) } |

<select statement>

DELETE FROM <table name>

[WHERE <select condition>]

UPDATE <table name>

SET <column name>=<value expression>

{ , <column name>=<value expression> }

[WHERE <select condition>]

```
CREATE [ UNIQUE] INDEX <index name>  
ON <table name> (<column name> [ <order> ]  
    {, <column name> [ <order> ] })  
[ CLUSTER]
```

```
DROP INDEX <index name>
```

```
CREATE VIEW <view name>  
    [ (<column name> {, <column name> })]  
AS <select statement>
```

```
DROP VIEW <view name>
```


Datenbank–Dumps in MySQL

Mit dem Befehl `mysqldump` kann man die Datenbank `company` in eine Datei `company.sql` speichern, welche aus lauter SQL–Befehlen besteht.

```
% mysqldump --opt company > company.sql
```

```
% echo "create database co; use co;" > h.sql;
```

```
% cat h.sql company.sql > mysql
```

Wenn man die Datei `company.sql` zusammen mit den SQL–Statements `create database co; use co;` auf die Kommandozeile von MySQL schickt, so wird eine neue Datenbank `co` angelegt mit demselben Inhalt wie `company` – die entsprechenden Tabellen einer eventuell bereits vorhandenen Datenbank namens `co` würden überschrieben.

company.sql:

```
-- MySQL dump 8.22
--
-- Host: localhost      Database: company
-----
-- Server version      3.23.52-log

--
-- Table structure for table 'department'
--

DROP TABLE IF EXISTS department;
CREATE TABLE department (
  DNAME varchar(15) NOT NULL default '',
  DNUMBER int(11) NOT NULL default '0',
  MGRSSN varchar(9) NOT NULL default '',
  MGRSTARTDATE date default NULL,
  PRIMARY KEY  (DNUMBER),
  UNIQUE KEY DNAME (DNAME)
) TYPE=MyISAM;

/*!40000 ALTER TABLE department DISABLE KEYS */;
```

```
--  
-- Dumping data for table 'department'  
--  
  
LOCK TABLES department WRITE;  
INSERT INTO department VALUES  
    ('Research',5,'222222222','1978-05-22'),  
    ('Administration',4,'333333333','1985-01-01'),  
    ('Headquarters',1,'111111111','1971-06-19');  
  
/!*40000 ALTER TABLE department ENABLE KEYS */;  
UNLOCK TABLES;
```

Für jede Tabelle gibt es die entsprechenden Statements

- CREATE TABLE
- INSERT INTO

Arbeiten mit MySQL

```
% mysql
```

```
Welcome to the MySQL monitor.  Commands end with ; or \g.
```

```
Your MySQL connection id is .... to server version: 3.23.52-log
```

```
Type 'help;' or '\h' for help. Type '\c' to clear the buffer.
```

```
mysql> use company;
```

```
Reading table information for completion of table and column names
```

```
You can turn off this feature to get a quicker startup with -A
```

```
Database changed
```

```
mysql> show tables;
```

```
+-----+
```

```
| Tables_in_company |
```

```
+-----+
```

```
| department        |
```

```
| dependent          |
```

```
| dept_locations     |
```

```
| employee            |
```

```
| project             |
```

```
| works_on            |
```

```
+-----+
```

```
6 rows in set (0.00 sec)
```

```
mysql> describe department;
```

Field	Type	Null	Key	Default	Extra
DNAME	varchar(15)		UNI		
DNUMBER	int(11)		PRI	0	
MGRSSN	varchar(9)				
MGRSTARTDATE	date	YES		NULL	

```
4 rows in set (0.00 sec)
```

```
mysql> select * from department;
```

DNAME	DNUMBER	MGRSSN	MGRSTARTDATE
Research	5	222222222	1978-05-22
Administration	4	333333333	1985-01-01
Headquarters	1	111111111	1971-06-19

```
3 rows in set (0.00 sec)
```

Literatur

C.J. Date, H. Darwen: *A Guide to the SQL Standard*.

4th Edition, Addison–Wesley, 1997.

R. Elmasri, S.B. Navathe: *Fundamentals of Database Systems*.

3rd Edition, Benjamin Cummings, 2000.

A. Kemper, A. Eickler: *Datenbanksysteme – Eine Einführung*.

6. Auflage, Oldenbourg, 2006.

R. Ramakrishnan, J. Gehrke: *Database Management Systems*.

3rd Edition, McGraw–Hill, 2004.

J.D. Ullman: *Principles of Database and Knowledge–Base Systems*, Volumes I,II.

Computer Science Press, 1989.

G. Vossen: *Datenmodelle, Datenbanksprachen und*

Datenbank–Management–Systeme. 4. Auflage, Oldenbourg, 2000.

Veranstaltungen

Bachelor

- Datenbanken (2–stündig):
ER–Modell, SQL, funktionale Abhängigkeiten, Transaktionen
- Logik (2–stündig):
Aussagenlogik, Prädikatenlogik, Logikprogrammierung

Master

- Datenbanken 2 / Advanced Databases (2–stündig):
Data Warehousing / Mining, XML, RDF, Semantic Web
- DATALOG, PROLOG (beides 2–stündig, früher Deduktive Datenbanken, 4–stündig): Semantik, Auswertungsmethoden, Logikprogrammierung;
Anwendung: Decision Support und Expertensysteme, Semantic Web

Seminar, Praktikum, Bachelor/Master–Arbeit